[University / College Name]

[Faculty / Department]

Elevator Control System with Time Dashboard

Project Report

Submitted by: [Student Name]

Student ID: [ID]

Module: [Module Name and Code]

Module Tutor / Supervisor: [Tutor Name]

Submission date: November 2, 2026

Declaration

I declare that this report and the software it describes are my own work, except where sources are cited in the References. The project has not been submitted for any other award.

Name: [Student Name]

Student ID: [ID]

Signature: _____

Date: _____

Acknowledgement

I thank my module tutor for guidance and feedback throughout the project, my classmates for testing the prototype, and my family for their support. I also acknowledge the open-source MySqlConnector library and the Microsoft .NET documentation that made the database and GUI work possible.

Abstract

In this report, a double elevator control system has been designed using C# WinForms (.NET 10) and MySQL. This double elevator system has animation for elevators, door synchronization, interlocking system, emergency handling, and background events logging. With the help of State and Observer pattern, this report describes system design, implementation, testing, and future improvements.

Contents

Declaration ii
Acknowledgement iii
Abstract iv
1. Introduction 1
1.1 Background of the Project 1
1.2 Scope of the Project 1
1.3 Significance and Real-world Application 1
1.4 Literature Review and Background Study 1
1.5 Report Organization 1
2. Problem Statement 2
2.1 Major Challenges 2
3. Proposed Solution 3
4. Aims and Objectives 4
4.1 Aim 4
4.2 Objectives 4
5. Project Planning and Requirements 5
5.1 Weekly Schedule 5
5.2 Functional Requirements 5
5.3 Non-Functional Requirements 5
6. System / Technical Requirements 7
6.1 Hardware Requirements 7
6.2 Software Requirements 7
7. System Design 8
7.1 Use Case Diagram 8
7.2 Class Diagram 8
7.3 ER Diagram 9
7.4 Activity Diagram 10
8. Tech Stack Implementation 12
8.1 Language Used 12
8.2 Object-Oriented Programming Implementation 12
8.3 Design Patterns Implemented 17
8.4 Database Used 22
8.5 Delegation and Timers 23
8.6 BackgroundWorker 26
8.7 Optimizing the Program 29

8.8 Exception Handling 32
9. Prototype 35
9.1 Main Window 35
9.2 Lift Control Card 35
9.3 Shaft, Cabin and Doors 37
9.4 Elevator Movement and Door Behaviour 38
9.5 Safety Behaviour 39
9.6 Event Log 47
10. Testing and Evaluation 49
10.1 Black Box Testing 49
BB1 50
BB2 52
BB3 53
BB4 55
BB5 57
BB6 58
BB7 59
BB8 61
BB9 63
BB10 63
BB11 64
BB12 65
BB13 65
BB14 65
BB15 66
BB16 66
BB17 67
10.2 White Box Testing 67
WB1 69
WB2 71
WB3 73
WB4 74
WB5 74
WB7 75
WB8 75
WB9 77
WB10 78
WB11 79
WB12 80

WB13 81
WB14 81
11. Critical Thinking 83
11.1 Advantages 83
11.2 Limitations 83
11.3 Future Improvements 83
12. Conclusion 84
13. References 85
References 86
14. Marking Matrix with Self-Assessment 87
15. Appendices 88
Appendix A: GitHub Link to Complete Source Code 88
Source code Screenshot 88
Protoype Screenshot with testing 125

List of Figures

Figure 1 list of abbreviations xi
Figure 2: System architecture: four layers 3
Figure 3: Use case diagram: two actors and eight use cases 8
Figure 4: Class diagram: Elevator, states, observers and repository 9
Figure 5: ER diagram: elevator_log table and its view 10
Figure 6: Activity diagram: request to ready, with five decisions 11
Figure 7 encapsualtion 13
Figure 8 encapsualtion 13
Figure 9 inheritance 14
Figure 10 inheritance 14
Figure 11 Polymorphism 15
Figure 12 Polymorphism 15
Figure 13 Abstraction 16
Figure 14 Abstration 16
Figure 15 Event-driven programming 16
Figure 16 Event-driven programming 17
Figure 17 Resource management 17
Figure 18 Resource management 17
Figure 19 Observer pattern 17
Figure 20 state pattern 18
Figure 21 state pattern 18
Figure 22 state pattern 19
Figure 23 observer pattern 19
Figure 24 observer pattern 19
Figure 25 obserber pattern 20
Figure 26 backgroundpattern 20
Figure 27 background pattern 21
Figure 28 • Separation of responsibilities 21
Figure 29 • Separation of responsibilities 22
Figure 30 jason file 23
Figure 31 exception handling 33
Figure 32 exception handling 33
Figure 33 exception handling 34
Figure 34 main window 35

Figure 35 Lift Control Card 36
Figure 36 Lift control card 37
Figure 37 shaft,cabin and doors 38
Figure 38 shaft cabin and doors 38
Figure 39 interlocks 39
Figure 40 closing the door 40
Figure 41 elevator going up 41
Figure 42 emergency stop 42
Figure 43 opening of door 43
Figure 44 going down 44
Figure 45 emergency stop 45
Figure 46 canin trvaeeling 46
Figure 47 alarm activated 47
Figure 48 event log 48
Figure 49 White box flow diagram: MovingState.Tick 68

List of Tables

Table 1 Weekly projects schedule 5
Table 2 Functional Requirement 5
Table 3 Non-functional requirements 5
Table 4 Minimum hardware requirement 7
Table 5 Software requirement 7
Table 6 Structure of the elevator_log table 10
Table 7 Object-oriented principles and where they appear 12
Table 8 Black box test cases 49
Table 9 White box test cases 68
Table 10 Marking matrix with self-assessment 87

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
DB	Database
ER	Entity Relationship
FR	Functional Requirement
GDI+	Graphics Device Interface Plus
GUI	Graphical User Interface
IDE	Integrated Development Environment
IoT	Internet of Things
NFR	Non-Functional Requirement
OOP	Object-Oriented Programming
PK	Primary Key
SCAN	Elevator disk-scheduling algorithm (serves requests in one direction, then reverses)
SQL	Structured Query Language
UML	Unified Modeling Language

Figure 1 list of abbreviations

1. Introduction

The task will entail making the simulation of the building model which contains two elevators using the computer. This includes the queue request, the moving of the doors, moving of the elevator cabin, among others. The process will also include maintaining logs into the database (MySQL, 2026).

1.1 Background of the Project

The elevator control system is vital for the purpose of security and handling cases of floor choices, door locks, and emergencies. This assignment is about creating a simulation of two elevators on a dashboard without use of any physical machines (ASME, ASME A17.1-2019/CSA B44-19: Safety Code for Elevators and Escalators).

1.2 Scope of the Project

- In Scope: Independent elevators, control, request queue, elevator movement, synchronized doors, safety measures, MySQL logs, and dark customizable dashboard.
- Out of Scope: Hardware, over two levels of floor, multiple buildings control, user account, networking, and IoT.

1.3 Significance and Real-world Application

As per this assignment, there will be practical execution of elevators through requests, door security, and events logging. Apart from this, the other aspects covered by this assignment include OOP, design patterns, and background processing (Gamma, et al., 1994)

1.4 Literature Review and Background Study

- Elevator Control: Manages Requests, Stopping and Safety
- Technology: Latest microprocessor-based Control Technology
- Algorithm: SCAN Algorithm for Requests Management
- Software Tools: C# WinForms, .NET 10, Timer, BackgroundWorker, and MySQL

1.5 Report Organization

Some of the major headings contained within this report include: problem, solution, objective, planning, requirement, design, implementation, prototype, testing, evaluation, conclusion, references, and appendices.

2. Problem Statement

Safety and reliability are essential for the building's elevators with recording events. The present project offers a practical simulation of two elevators that feature door safety and control along with logging.

2.1 Major Challenges

- Smooth flow control for Cab movement with accurate floor alignment.
- Synchronization with Lock status in Door while moving Cab.
- Interlocking, Emergency brake, and Alarms.
- Two Independent Elevators Working.
- Logging Database at background process without freezing UI.
- Clean GUI with state-wise Disable/Enable controls.

3. Proposed Solution

WinForms Dashboard controls two distinct elevators using a common MySQL log.

- UI - Headers, overview, elevator panels, and activities logs.
- Control - Elevator classes and state driven behavior control.
- Database - Log repository keeps events logged in MySQL.
- Background Processing – Logging done through Background Worker to prevent GUI freeze.
- Architecture design - Separation through State and Observer patterns.

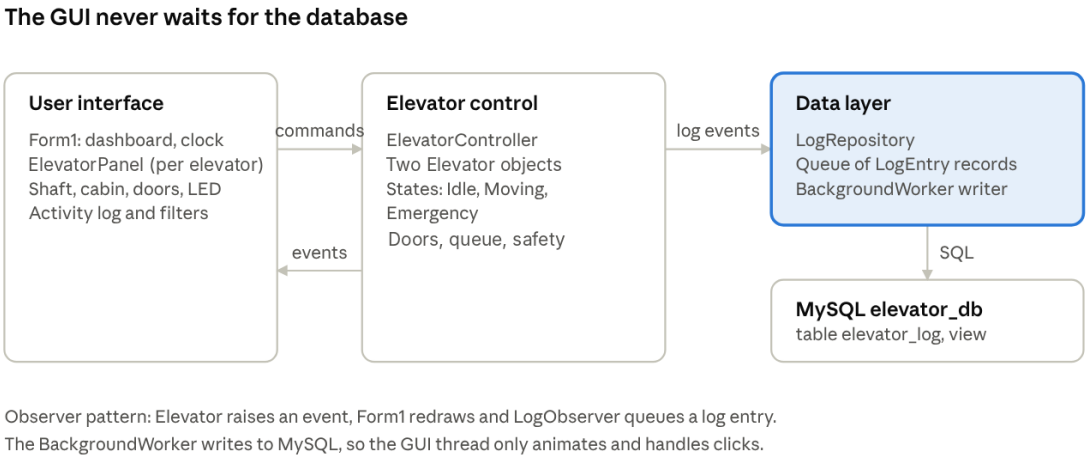


Figure 2: System architecture: four layers

4. Aims and Objectives

4.1 Aim

The aim to design and construct a safe, practical and well-constructed control system simulator for dual elevators with a live dashboard and event log.

4.2 Objectives

1. To model an elevator with floors, a request queue, direction-aware dispatching and a smooth speed profile, and run two elevators independently in one window.
2. To synchronize the inner and landing doors and enforce a safety interlock, emergency stop, alarm and reset.
3. To build a clear, resizable dark GUI with a live clock, status overview and filterable log, and store every event in MySQL on a background thread with graceful handling of database failures.
4. To apply object-oriented principles and the State and Observer design patterns, and verify the system with black box and white box testing.

5. Project Planning and Requirements

5.1 Weekly Schedule

Planned schedule (adjust the weeks to your own timetable).

Table 1 Weekly projects schedule

6-Week Project Plan		
Week	**Activity**	**Output**
1	**Research and requirements**	**Requirement list**
2	**System design and diagrams**	**Design diagrams**
3	**Elevator logic, movement and safety**	**Working core**
4	**GUI and dashboard development**	**Prototype**
5	**MySQL integration and testing**	**Event log and test results**
6	**Fixes, report and submission**	**Final project**

5.2 Functional Requirements

Table 2 Functional Requirement

ID	Requirement
FR1	Accept floor and hall requests.
FR2	Move and serve queued floors.
FR3	Allow manual door control.
FR4	Automate door operation and dwell time.
FR5	Enforce door interlocks and fault stops.
FR6	Emergency stop and reset.
FR7	Activate alarm for 6 seconds or until reset.
FR8	Display elevator status and requests.
FR9	Show time, system status and latest event.
FR10	Store, filter and clear MySQL logs.

5.3 Non-Functional Requirements

Table 3 Non-functional requirements

Quality	**Requirement**
Usability	Invalid buttons are disabled with a hint; colour-coded status; resizable window with scroll bars below about 1260 px.

Quality	Requirement
Performance	20 ms timer tick for smooth animation; database writes never block the GUI.
Reliability	Database errors are reported once and do not crash the simulation; queued entries are flushed on exit.
Safety	Interlock checked on every movement tick.
Maintainability	Separate Models, UI and Data folders; settings in dbsettings.json.
Portability	Runs on Windows with .NET 10 and a local MySQL server.

6. System / Technical Requirements

6.1 Hardware Requirements

Table 4 Minimum hardware requirement

Component	Minimum
Processor	Dual-core 2 GHz
Memory	4 GB RAM
Storage	500 MB free
Display	1280 x 720 or larger (1260 px width avoids scroll bars)
Audio	Optional speaker for the alarm tone

6.2 Software Requirements

Table 5 Software requirement

Software	Purpose
Windows 10 or 11	Operating system (WinForms)
.NET 10 SDK (net10.0-windows)	Build and run
Visual Studio	Development and GUI designer
MySQL Server (127.0.0.1:3306) and MySQL Workbench	Event log database
MySqlConnector 2.4.0 (NuGet)	Database driver

7. System Design

7.1 Use Case Diagram

Passenger and operator are the two participants. Database acts as the supportive system that receives all events.

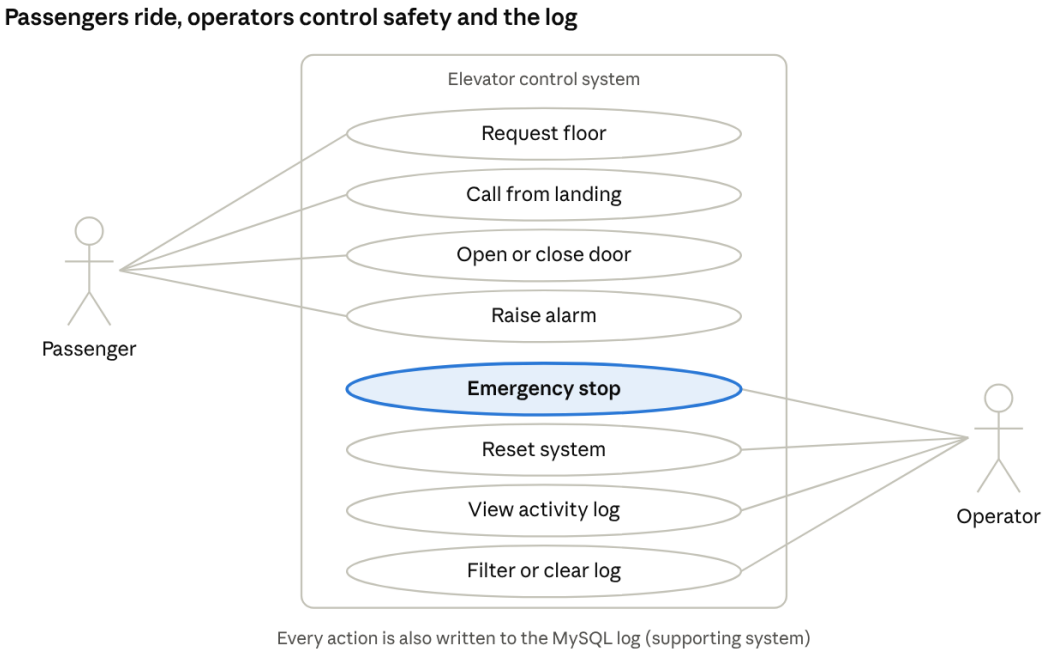


Figure 3: Use case diagram: two actors and eight use cases

7.2 Class Diagram

The Elevator class is the primary one. It contains Door objects, updates the observers, and executes every tick according to the current state.

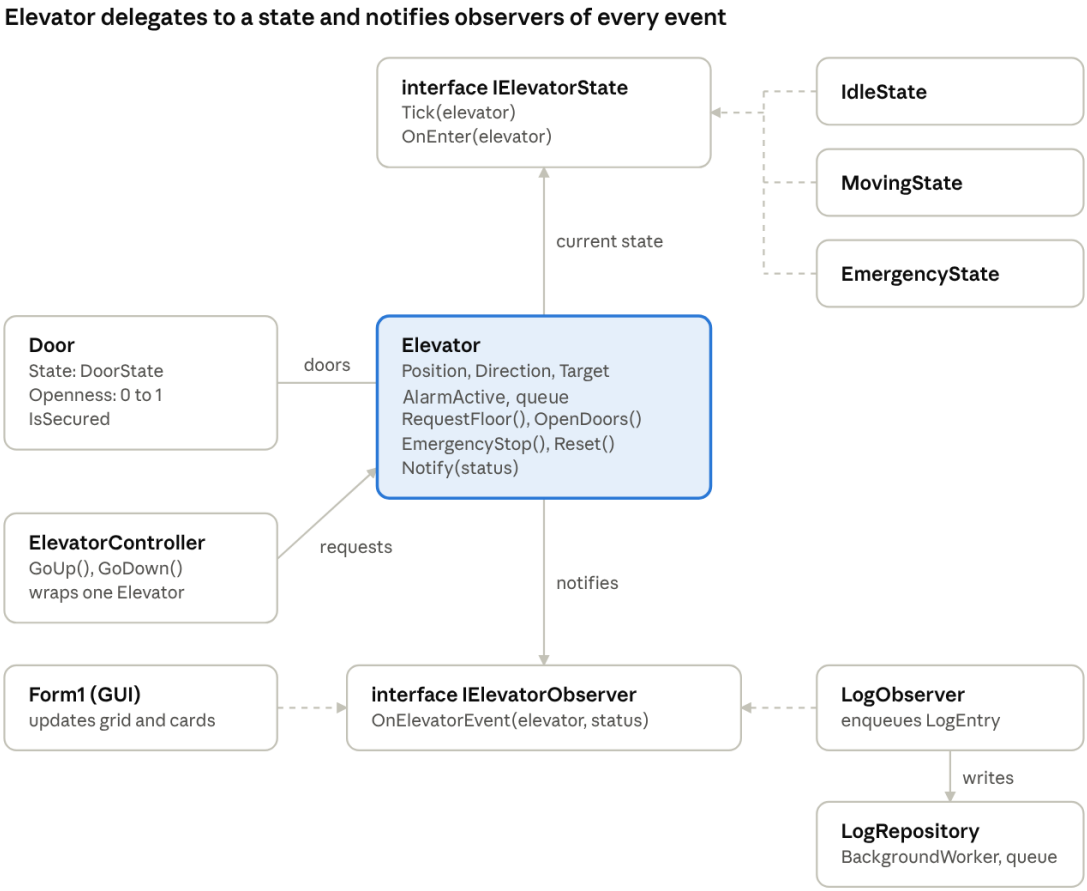


Figure 4: Class diagram: Elevator, states, observers and repository

7.3 ER Diagram

There is one table, elevator_log, in the database. The two elevators use it together, and are distinguished by elevator_name. The view vw_elevator_log organizes data in the same way as the grid in the application.

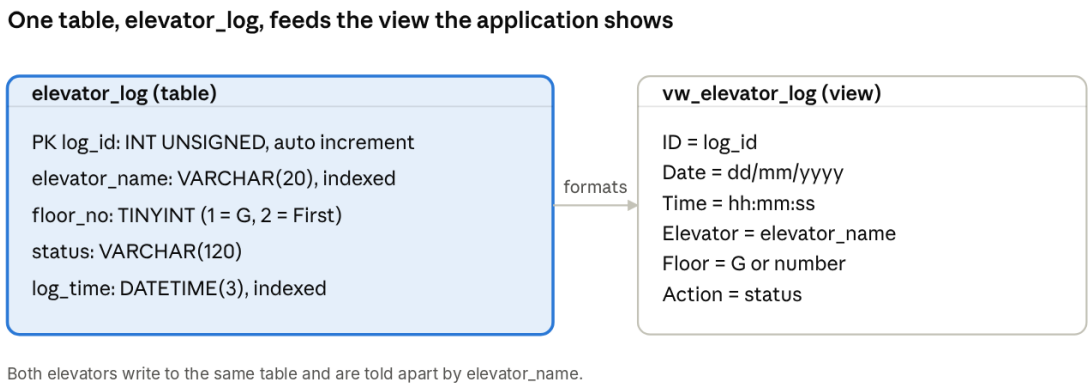


Figure 5: ER diagram: elevator_log table and its view

Table 6 Structure of the elevator_log table

Column	Type	Meaning
log_id	INT UNSIGNED, PK, AUTO_INCREMENT	Record number
elevator_name	VARCHAR(20), NOT NULL	Selected elevator
floor_no	TINYINT, NOT NULL	1 = G, 2 = First floor
status	VARCHAR(120), NOT NULL	What happened
log_time	DATETIME(3), default now	Date and time of the event

Indexes: idx_log_time on log_time, idx_log_elevator on elevator_name. Engine: InnoDB, utf8mb4.

7.4 Activity Diagram

The activity diagram shows what happens from a floor request to a ready elevator, including the interlock and emergency branches.

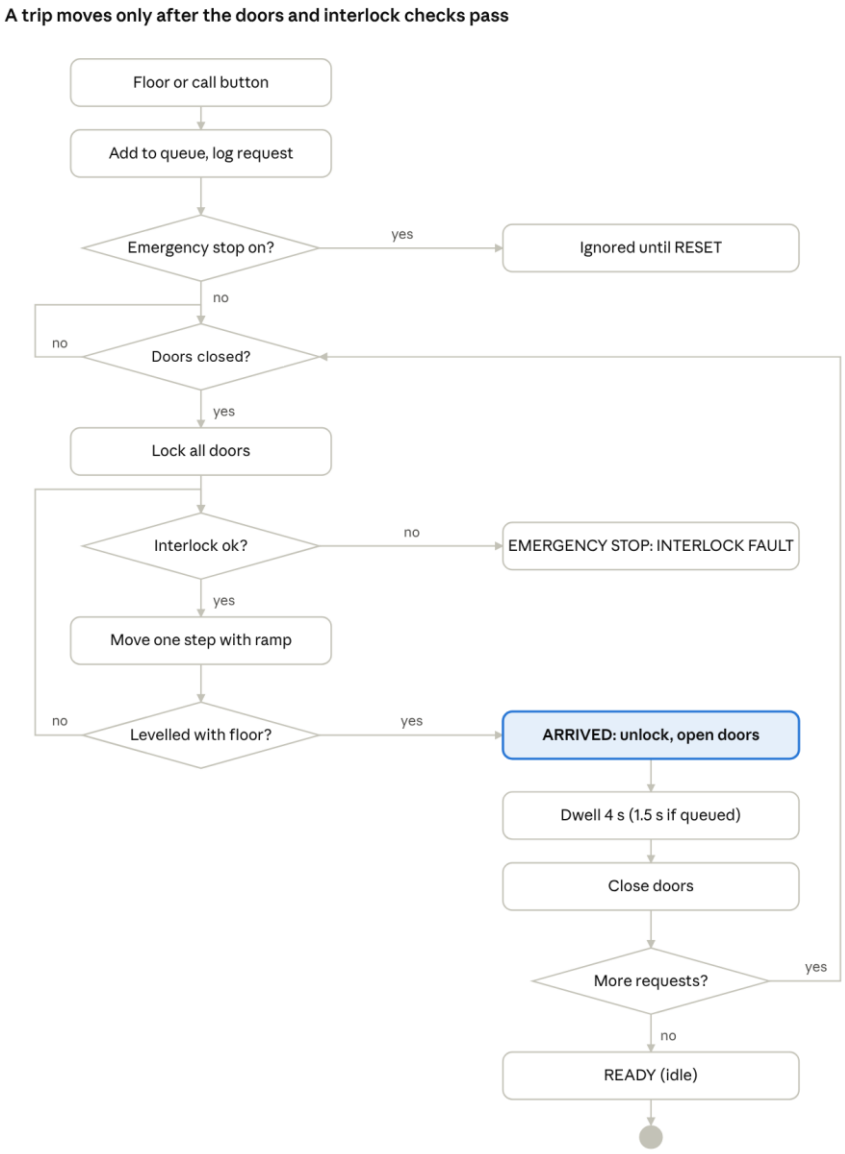


Figure 6: Activity diagram: request to ready, with five decisions

8. Tech Stack Implementation

8.1 Language Used

C# using .NET Framework 10 (net10.0-windows), Windows Forms application. Custom drawn controls (shaft, cabin, doors, LED panel, steel buttons) using GDI+, double buffering.

8.2 Object-Oriented Programming Implementation

Table 7 Object-oriented principles and where they appear

Principle	Where it appears
Encapsulation	Elevator exposes read-only properties (Position, AlarmActive) and changes state only through methods such as RequestFloor, EmergencyStop and Reset; setters are private or internal.
Inheritance	UI classes derive from WinForms controls (Form, Panel, Button, Label) and add the dashboard look.
Polymorphism	IElevatorState is implemented by IdleState, MovingState and EmergencyState; Elevator calls Tick on whichever state is current. IElevatorObserver is implemented by Form1 and LogObserver.
Abstraction	Interfaces hide how states and observers work; Elevator does not know about the GUI or MySQL.
Event-driven programming	PositionChanged and DoorsChanged events and a 20 ms timer tick drive animation; button clicks raise requests.
Resource management	Elevator implements IDisposable; timers are stopped and disposed on close; database connections use using blocks; the log queue is flushed on exit.

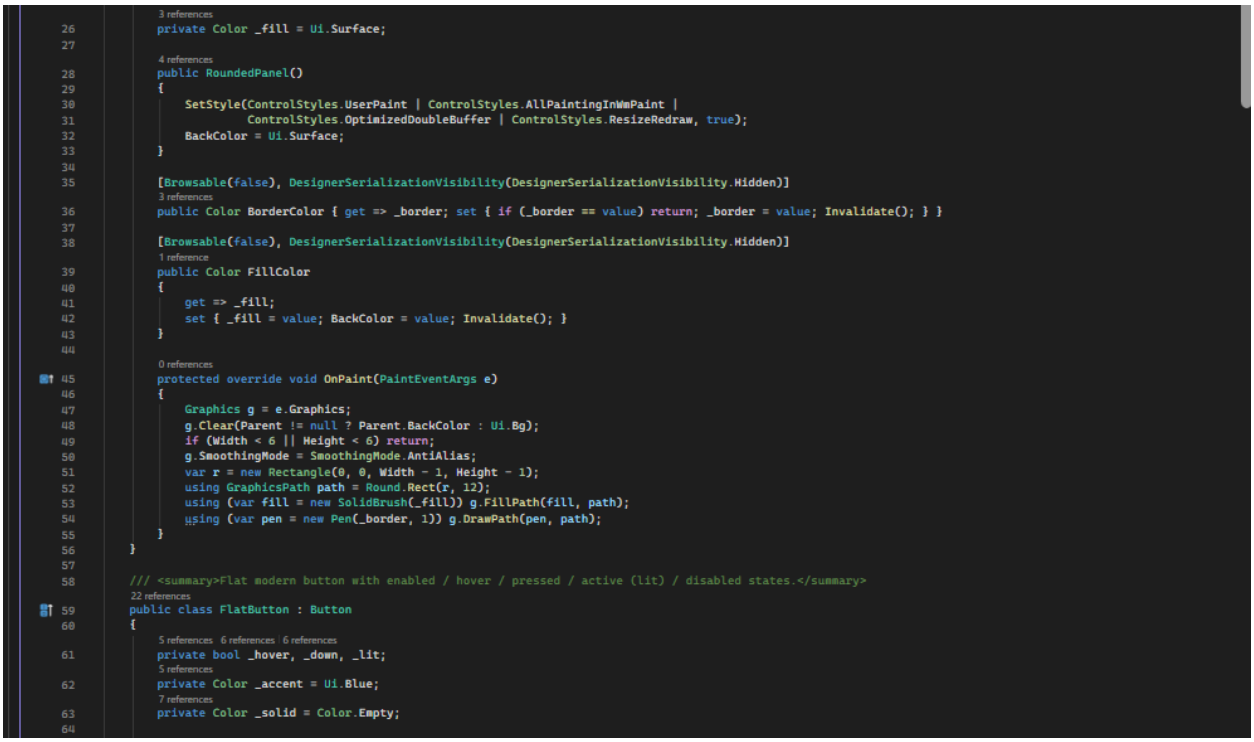
Encapsulation

```
7 references
84      private bool _doorWantOpen;       // true while the doors are opening / open
5 references
85      private int _dwell;               // ticks the doors have been fully open
86
6 references
87      public string Name { get; }
22 references
88      public double Position { get; internal set; }       // 1.0 .. MaxFloor (fractional while moving)
15 references
89      public int Direction { get; internal set; }         // +1 up, -1 down, 0 stopped
2 references
90      public int LastDirection { get; internal set; } = 1;
18 references
91      public int TargetFloor { get; internal set; }       // floor the cabin is travelling to (0 = none)
5 references
92      public IElevatorState State { get; private set; }
11 references
93      public string StatusText { get; internal set; } = "READY";
94
95      /// <summary>The inner cabin door: it is part of the cabin and moves with it.</summary>
23 references
96      public Door Inner { get; }
97
98      /// <summary>The outer landing door of a floor: fixed to the building, never moves.</summary>
2 references
99      public Door GetLanding(int floor) => _landing[floor];
1 reference
100     public double GetLandingOpenness(int floor) => _landing[floor].Openness;
101
9 references
102     public bool AlarmActive { get; private set; }
2 references
103     public DateTime AlarmStarted { get; private set; }
104
2 references
105     public IReadOnlyList<int> Pending => _pending;
3 references
106     public bool IsRequested(int floor) => _pending.Contains(floor);
```

Figure 7 encapsualtion

```
524     // ============================================================================
525     //  Safety
526     // ============================================================================
527
528     /// <summary>Stops the timer at once: cabin and doors freeze, requests are dropped, nothing moves until RESET.</summary>
        2 references
529     public void EmergencyStop()
530     {
531         if (IsEmergency) return;
532         _timer.Stop();
533         _pending.Clear();
534         TargetFloor = 0;
535         StatusText = "EMERGENCY STOP";
536         SetState(new EmergencyState());
537     }
538
539     /// <summary>Clears the emergency state (and a running alarm); the elevator is IDLE again.</summary>
        1 reference
540     public void Reset()
541     {
542         if (IsEmergency)
543         {
544             AlarmActive = false;
545             StatusText = "RESET";
546             SetState(new IdleState());
547             _timer.Start();
548             Notify("RESET – EMERGENCY STOP CLEARED");
549         }
550         else if (AlarmActive)
551         {
552             ClearAlarm("RESET – ALARM CLEARED");
553         }
554     }
555
        1 reference
```

Figure 8 encapsualtion



```
        3 references
        private Color _fill = Ui.Surface;

        4 references
        public RoundedPanel()
        {
            SetStyle(ControlStyles.UserPaint | ControlStyles.AllPaintingInWmPaint |
                     ControlStyles.OptimizedDoubleBuffer | ControlStyles.ResizeRedraw, true);
            BackColor = Ui.Surface;
        }

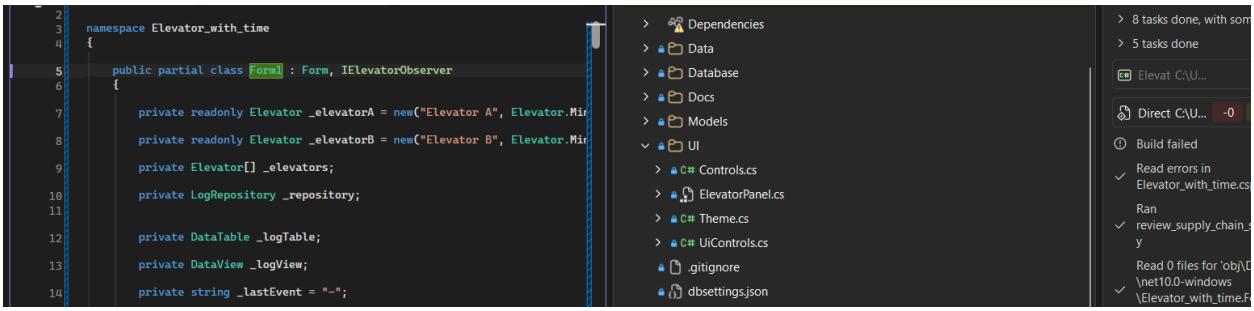
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        3 references
        public Color BorderColor { get => _border; set { if (_border == value) return; _border = value; Invalidate(); } }

        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        1 reference
        public Color FillColor
        {
            get => _fill;
            set { _fill = value; BackColor = value; Invalidate(); }
        }

        0 references
        protected override void OnPaint(PaintEventArgs e)
        {
            Graphics g = e.Graphics;
            g.Clear(Parent != null ? Parent.BackColor : Ui.Bg);
            if (Width < 6 || Height < 6) return;
            g.SmoothingMode = SmoothingMode.AntiAlias;
            var r = new Rectangle(0, 0, Width - 1, Height - 1);
            using GraphicsPath path = Round.Rect(r, 12);
            using (var fill = new SolidBrush(_fill)) g.FillPath(fill, path);
            using (var pen = new Pen(_border, 1)) g.DrawPath(pen, path);
        }
    }

    /// <summary>Flat modern button with enabled / hover / pressed / active (lit) / disabled states.</summary>
    22 references
    public class FlatButton : Button
    {
        5 references  6 references  6 references
        private bool _hover, _down, _lit;
        5 references
        private Color _accent = Ui.Blue;
        7 references
        private Color _solid = Color.Empty;
```

Figure 9 inheritance



```
namespace Elevator_with_time
{
    public partial class Form1 : Form, IElevatorObserver
    {
        private readonly Elevator _elevatorA = new("Elevator A", Elevator.Mir
        private readonly Elevator _elevatorB = new("Elevator B", Elevator.Mir
        private Elevator[] _elevators;
        private LogRepository _repository;

        private DataTable _logTable;
        private DataView _logView;
        private string _lastEvent = "-";
```

```
Dependencies
Data
Database
Docs
Models
UI
  Controls.cs
  ElevatorPanel.cs
  Theme.cs
  UiControls.cs
.gitignore
dbsettings.json
```

```
8 tasks done, with som
5 tasks done
Elevat C:\U...
Direct C:\U...  -0
Build failed
Read errors in Elevator_with_time.cs
Ran review_supply_chain_s y
Read 0 files for 'obj\D \net10.0-windows \Elevator_with_time.F
```

Figure 10 inheritance

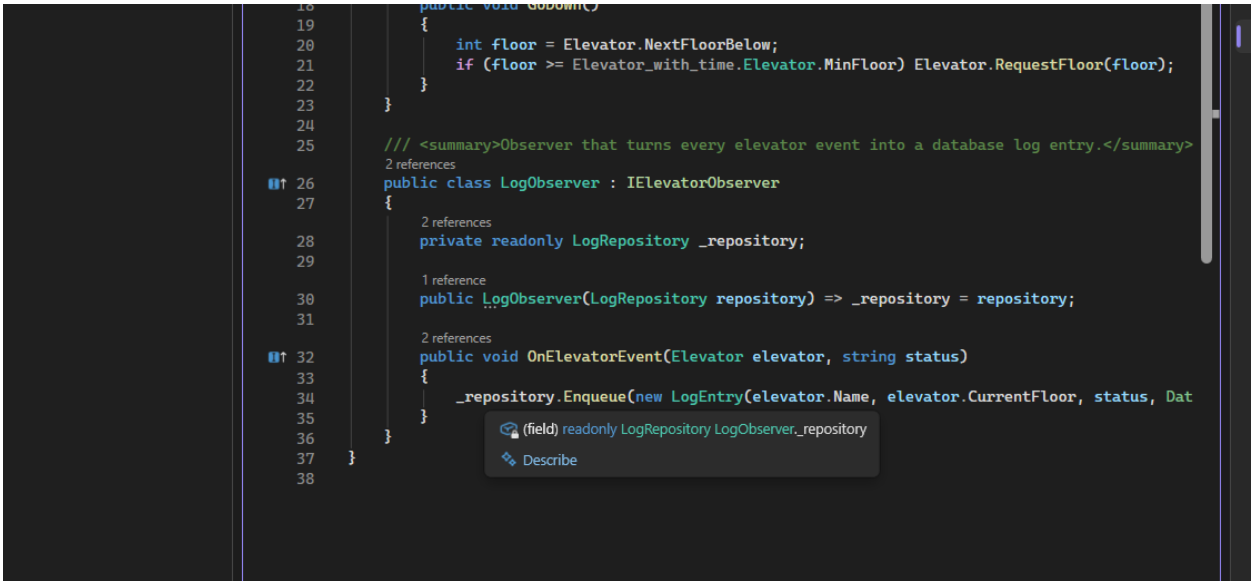


Figure 11 Polymorphism

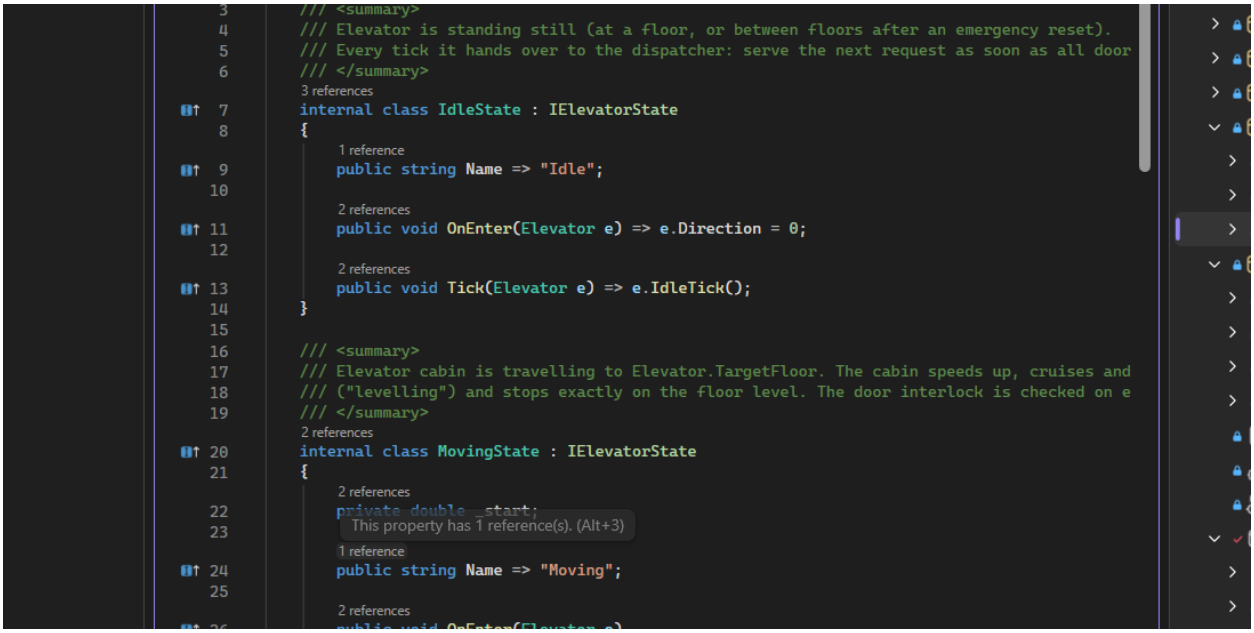


Figure 12 Polymorphism

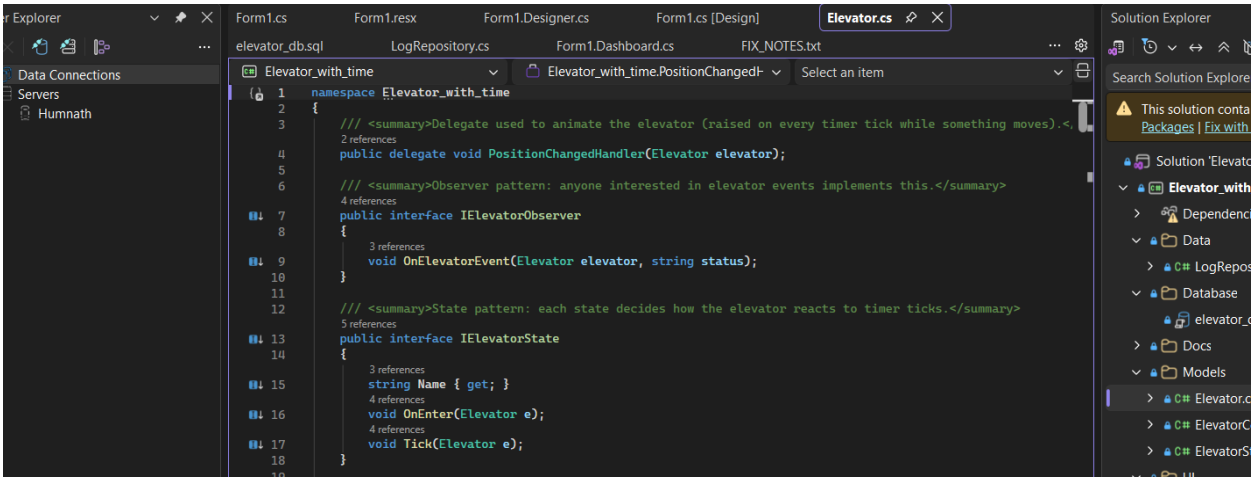


Figure 13 Abstraction

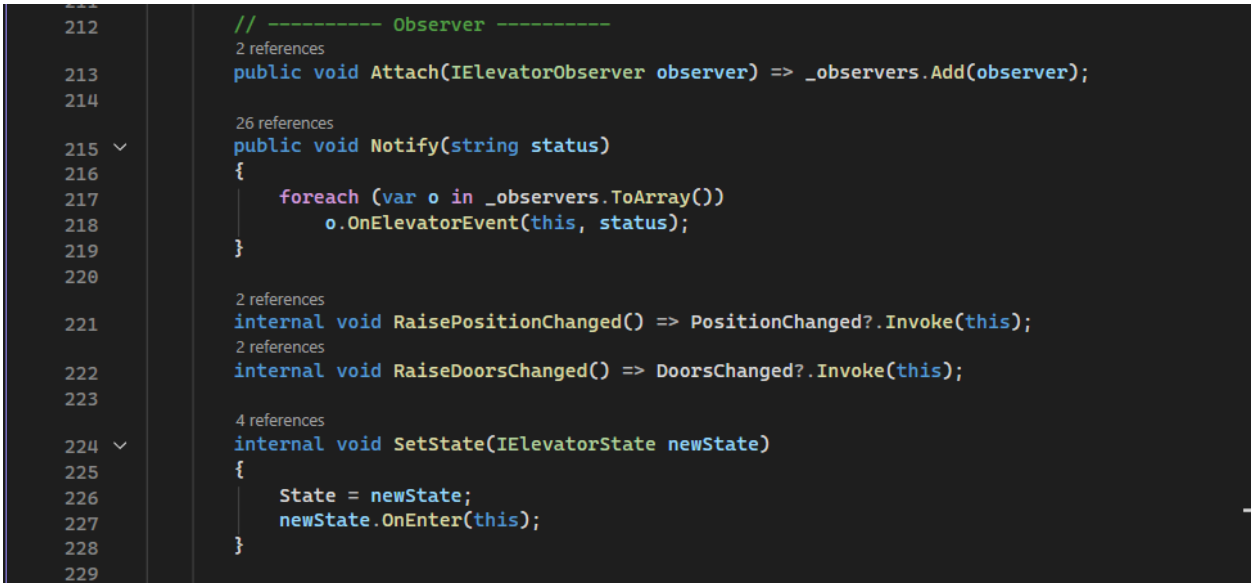


Figure 14 Abstration

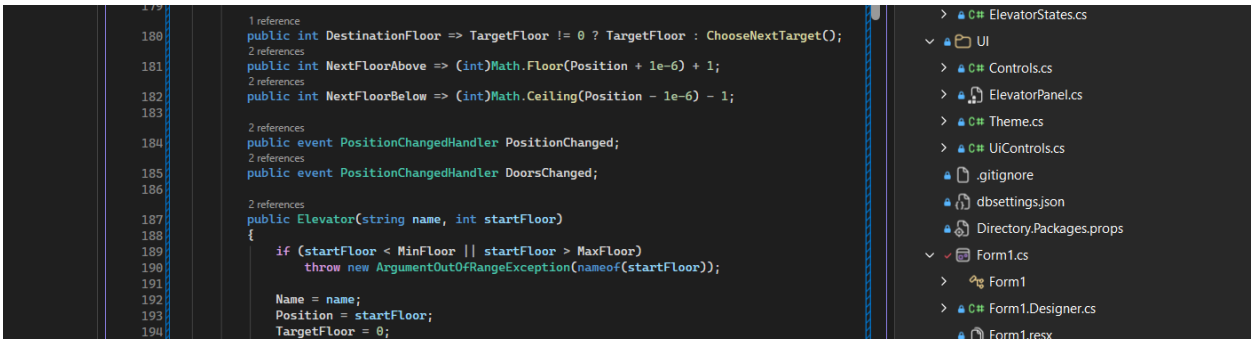


Figure 15 Event-driven programming

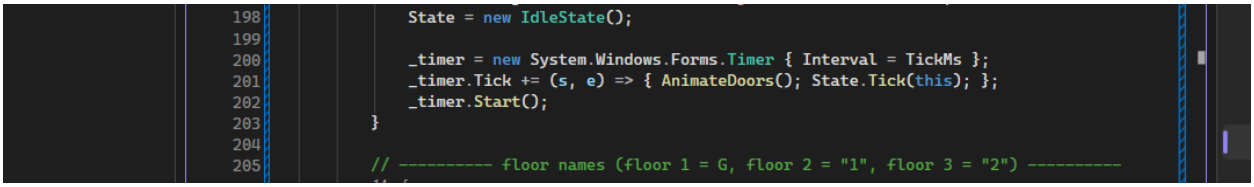


Figure 16 Event-driven programming

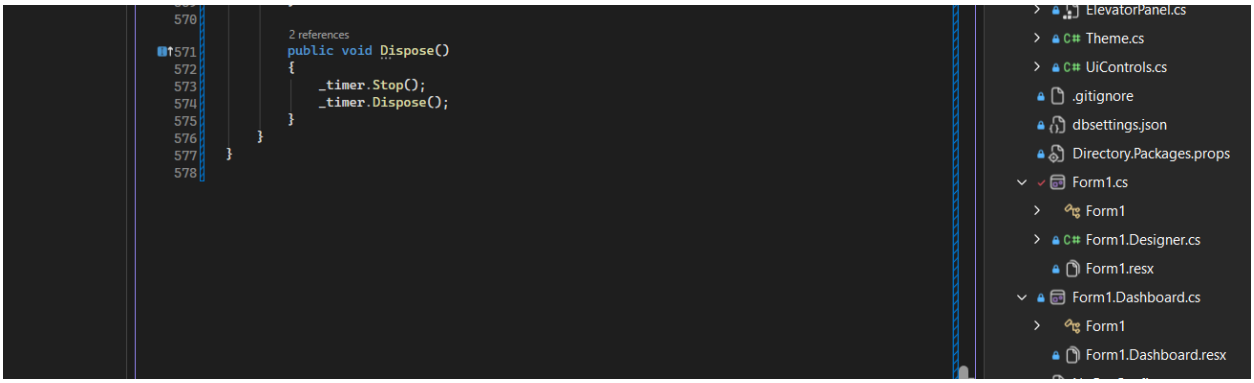


Figure 17 Resource management



Figure 18 Resource management

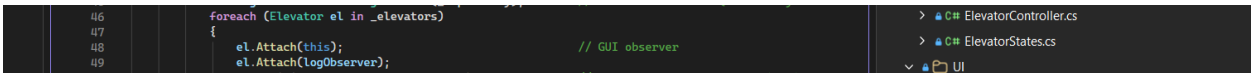


Figure 19 Observer pattern

8.3 Design Patterns Implemented

- **State pattern:** Idle State sends a call to the dispatcher, Moving State relies on the speed profile, checks interlocks and floors, whereas Emergency State waits for RESET [4]. Design Pattern, in which the behavior of the object varies depending on its state (ASME, 2019)

```
4 references
public interface IElevatorObserver
{
    3 references
    void OnElevatorEvent(Elevator elevator, string status);
}

/// <summary>State pattern: each state decides how the elevator reacts to timer ticks.</s
5 references
public interface IElevatorState
{
    3 references
    string Name { get; }
    4 references
    void OnEnter(Elevator e);
    4 references
    void Tick(Elevator e);
```

-

Figure 20 state pattern

```
/// <summary>The five states a door can be in.</summary>
32 references  7 references | 4 references | 5 references | 4 references  9 references
public enum DoorState { Closed, Opening, Open, Closing, Locked }

/// <summary>
/// One physical door (the inner cabin door, or the outer landing door of one floor).
/// Openness: 0 = closed, 1 = fully open. The picture of the door is drawn from these two values only.
/// </summary>
15 references
public class Door
{
    2 references
    public Door(string name, DoorState initial)
    {
        Name = name;
        State = initial;
    }

    1 reference
    public string Name { get; }
    25 references
    public DoorState State { get; internal set; }
    12 references
    public double Openness { get; internal set; }

    3 references
    public string Text => State.ToString().ToUpperInvariant();

    /// <summary>True when the door leaf is completely shut and bolted/closed (not opening, not open, not closing).</summary>
    2 references
    public bool IsSecured => Openness <= 0.0 && (State == DoorState.Closed || State == DoorState.Locked);
}

/// <summary>
/// ONE elevator serving the floors MinFloor..MaxFloor (floor 1 is shown as "G").
///
/// DUAL DOOR SYSTEM
///   * Inner (cabin) door : belongs to the cabin and travels with it.
///   * Outer (landing) doors: one per floor, fixed to the building, they never move with the cabin.
///   The landing door of a floor is coupled to the cabin door only while the cabin is stopped and levelled at
///   exactly that floor. Both doors then open and close together (same speed, same moment).
///
/// SAFETY INTERLOCK
///   * the cabin only moves while the inner door and EVERY landing door are completely closed,
///   * all doors are LOCKED while the cabin travels,
///   * a landing door is only unlocked / opened for the floor the cabin is levelled at and stopped on.
///
/// A single System.Windows.Forms.Timer drives the cabin movement and the door animation.
/// </summary>
93 references
public class Elevator : IDisposable
{
    27 references
    public const int MinFloor = 1;                  // G
    20 references
    public const int MaxFloor = 2;                  // G and 1 only
    3 references
    public const int TickMs = 20;                   // timer interval
    1 reference
    public const double MaxSpeed = 0.022;           // floors per tick at cruising speed (about 1 s per floor)
    1 reference
    public const double RampDistance = 0.30;        // floors used to speed up / slow down
    1 reference
    public const double ApproachDistance = 0.50;    // announce approach when this close to the target floor
    1 reference
    public const double LevelDistance = 0.08;       // announce levelling when this close to the target floor
    1 reference
    public const double MinSpeedFactor = 0.30;      // creeping speed while levelling (share of MaxSpeed)
    1 reference
    public const double BrakeDistance = 0.35;       // a new stop can only be added if it is at least this far ahead
    2 references
```

Figure 21 state pattern

```
/// <summary>Emergency stop: the timer is stopped, nothing moves until RESET.</summary>
internal class EmergencyState : IElevatorState
{
    public string Name => "Emergency";

    public void OnEnter(Elevator e)
    {
        e.Direction = 0;
        e.Notify("EMERGENCY STOP ACTIVATED");
    }

    public void Tick(Elevator e) { }
}
```

Figure 22 state pattern

- **Observer pattern:** Elevator.Notify broadcasts the event string to the subscribed observers; Form1 will then update the log and description, while LogObserver will post an entry in the database [4].

```
7       public interface IElevatorObserver
```

Figure 23 observer pattern

```
public static string FloorTag(int floor) => floor == 1 ? "GROUND FLOOR" : $"FLOOR {floor - 1}";

// ---------- Observer ----------
public void Attach(IElevatorObserver observer) => _observers.Add(observer);

public void Notify(string status)
{
    foreach (var o in _observers.ToArray())
        o.OnElevatorEvent(this, status);
}

internal void RaisePositionChanged() => PositionChanged?.Invoke(this);
```

Figure 24 observer pattern

```
/// <summary>Observer that turns every elevator event into a database log entry.</summary>
public class LogObserver : IElevatorObserver
{
    private readonly LogRepository _repository;

    public LogObserver(LogRepository repository) => _repository = repository;

    public void OnElevatorEvent(Elevator elevator, string status)
    {
        _repository.Enqueue(new LogEntry(elevator.Name, elevator.CurrentFloor, status, DateTime.Now));
    }
}
```

Figure 25 obserber pattern

- **Background processing:** A queue along with BackgroundWorker takes all the database-related stuff out of the main thread.

```
public class LogRepository
{
    private const string TableName = "elevator_log";
    private static readonly DbSettings Settings = DbSettings.Load();

    private readonly ConcurrentQueue<LogEntry> _queue = new();
    private readonly BackgroundWorker _writer = new();
    private readonly object _schemaLock = new();
    private bool _schemaChecked;
    private volatile bool _wroteSomething;

    /// <summary>Raised on the GUI thread when a database operation fails.</summary>
    public event Action<string> ErrorOccurred;

    /// <summary>Raised on the GUI thread after new log entries were stored.</summary>
    public event Action EntriesWritten;

    public static string Description =>
        $"MySQL  {Settings.User}@{Settings.Host}:{Settings.Port}  /  {Settings.Database}";

    public LogRepository()
    {
        _writer.DoWork += Writer_DoWork;
        _writer.RunWorkerCompleted += Writer_Completed;
    }

    // ------------------------------------------------------------ writing (queued)

    public void Enqueue(LogEntry entry)
    {
        _queue.Enqueue(entry);
        StartWriterIfNeeded();
    }

    private void StartWriterIfNeeded()
    {
        if (!_writer.IsBusy && !_queue.IsEmpty)
            _writer.RunWorkerAsync();
    }

    private void Writer_DoWork(object sender, DoWorkEventArgs e)
    {
        while (_queue.TryDequeue(out LogEntry entry))
        {
            try
            {
                Insert(entry);
                _wroteSomething = true;
            }
            catch (Exception ex)
            {
                e.Result = ex.Message;
                while (_queue.TryDequeue(out _)) { }   // database unusable: do not retry every entry
            }
        }
    }
```

Figure 26 backgroundpattern

Data/LogRepository.cs — Background Processing — asynchronous database work

```
134         public void LoadLogsAsync(Action<DataTable> onLoaded, Action<string> onError) =>
135             RunAsync(ReadAll, onLoaded, onError);
136
137         public void ClearAsync(Action onDone, Action<string> onError) =>
138             RunAsync(() => Execute($"TRUNCATE TABLE {TableName}"), _ => onDone(), onError);
139
140         /// <summary>One place that runs work on a BackgroundWorker and reports back on the GUI thread.</summary>
141         private static void RunAsync<T>(Func<T> work, Action<T> onDone, Action<string> onError)
142         {
143             var worker = new BackgroundWorker();
144             worker.DoWork += (s, e) => e.Result = work();
145             worker.RunWorkerCompleted += (s, e) =>
146             {
147                 if (e.Error != null) onError(e.Error.Message);
148                 else onDone((T)e.Result);
149                 worker.Dispose();
150             };
151             worker.RunWorkerAsync();
152         }
```

Figure 27 background pattern

- **Separation of responsibilities:** Simulation, Control, Panels, Database (MySQL, 2026)and Composition of Form1 should never mix together. The method ElevatorController transforms commands such as GoUp and GoDown from Panels into Requests.

Form1.cs — Separation of Responsibilities — composition in Form1

```
24          public Form1()
25          {
26              InitializeComponent();
27              DoubleBuffered = true;
28              _elevators = new[] { _elevatorA, _elevatorB };
29
30              // each elevator gets its OWN controller and its OWN panel: a control can only ever reach its own elevator
31              // (the two panels are created by the designer in Form1.Designer.cs; here they get their elevator)
32              _panelA.Bind(_elevatorA, new ElevatorController(_elevatorA), Ui.Blue, "A");
33              _panelB.Bind(_elevatorB, new ElevatorController(_elevatorB), Ui.Violet, "B");
34              BindDashboard();
35
36              // the log takes about a fifth of the window, so the lifts above it always stay fully visible
37              Resize += (s, e) => _logPanel.Height = Math.Clamp(ClientSize.Height / 5, 150, 260);
38          }
39
40          private void Form1_Load(object sender, EventArgs e)
41          {
42              ApplyTheme();
43
44              _repository = new LogRepository();
45              _repository.ErrorOccurred += OnDatabaseError;
46              _repository.EntriesWritten += () => LoadLog(false);   // keep the grid up to date
47              _lblDbStatus.Text = "Database: " + LogRepository.Description;
48
49              var logObserver = new LogObserver(_repository);        // database observer (rows carry the elevator name)
50              foreach (Elevator el in _elevators)
51              {
52                  el.Attach(this);                                   // GUI observer
53                  el.Attach(logObserver);
54                  el.PositionChanged += OnElevatorAnimated;          // delegates used for the animation
55                  el.DoorsChanged += OnElevatorAnimated;
```

Figure 28 • Separation of responsibilities

Models/ElevatorController.cs — Separation of Responsibilities — controller commands

```
3       /// <summary>Owns the single elevator and translates the control-panel commands into requests.</summary>
4       public class ElevatorController
5       {
6           public Elevator Elevator { get; }
7
8           public ElevatorController(Elevator elevator) => Elevator = elevator;
9
10          /// <summary>UP: request the next floor above the cabin (nothing happens at the top floor).</summary>
11          public void GoUp()
12          {
13              int floor = Elevator.NextFloorAbove;
14              if (floor <= Elevator_with_time.Elevator.MaxFloor) Elevator.RequestFloor(floor);
15          }
16
17          /// <summary>DOWN: request the next floor below the cabin (nothing happens at the ground floor).</summary>
18          public void GoDown()
19          {
20              int floor = Elevator.NextFloorBelow;
21              if (floor >= Elevator_with_time.Elevator.MinFloor) Elevator.RequestFloor(floor);
22          }
```

Figure 29 • Separation of responsibilities

8.4 Database Used

Using MySQL with MySqlConnector v2.4.0. Connection details will be pulled out from dbsettings.json file (host, port, user, password, database) instead of having them hard-coded in the code. At the start of application for the first time, it will create database ‘elevator_db’, table ‘elevator_log’, and view. Useful queries were described in Database/elevator_db.sql.

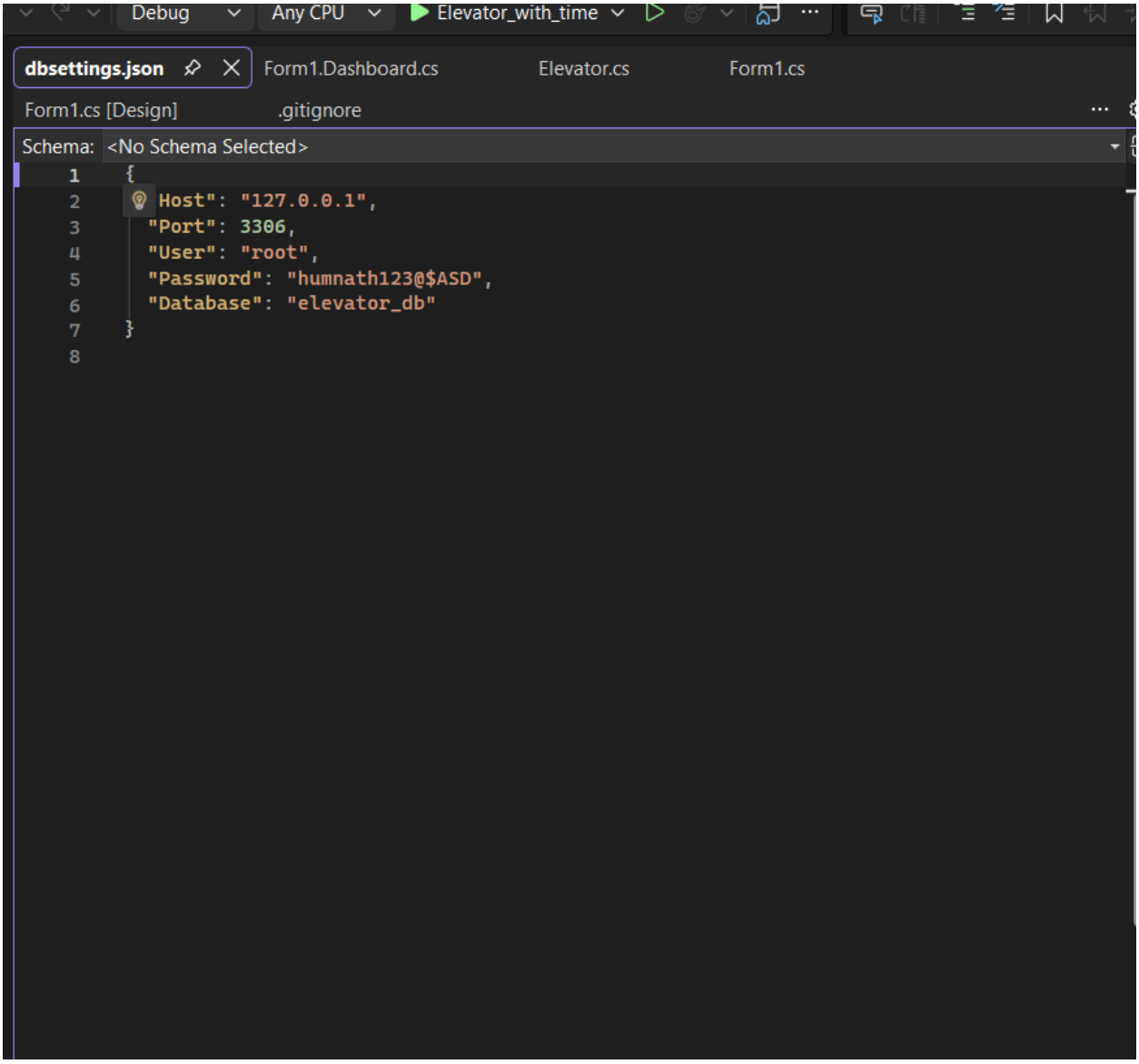


Figure 30 jason file

8.5 Delegation and Timers

There are two cases of Timer component from System.Windows. Forms namespace. TickMs is a 20ms timer for each Elevator which provides animation of cabin and door moving, and clock timer, which is a 250ms timer for Form1 to handle time and alarm blinking. With the help of delegates and events communication is done between layers. PositionChangedHandler (PositionChanged, DoorsChanged) invokes drawing on Form1 at any event of the model.

Form1.cs | 8.5 Delegation and Timers — event subscriptions

```
            var logObserver = new LogObserver(_repository);        // database observer (rows carry the elevator name)
            foreach (Elevator el in _elevators)
            {
                el.Attach(this);                                   // GUI observer
                el.Attach(logObserver);
                el.PositionChanged += OnElevatorAnimated;          // delegates used for the animation
                el.DoorsChanged += OnElevatorAnimated;
            }

            _clockTimer.Tick += ClockTimer_Tick;
            _clockTimer.Start();
            UpdateClock();
```

Form1.cs | 8.5 Delegation and Timers — clock and drawing callbacks

```
        // ------------------------------------------------------------ clock + alarm timeout

        private void ClockTimer_Tick(object sender, EventArgs e)
        {
            UpdateClock();
            _blink = !_blink;

            bool anyAlarm = false;
            foreach (Elevator el in _elevators)
            {
                if (!el.AlarmActive) continue;
                if (DateTime.Now - el.AlarmStarted > AlarmDuration)
                {
                    el.ClearAlarm("ALARM STOPPED AUTOMATICALLY");
                    continue;
                }
                anyAlarm = true;
            }

            if (anyAlarm)
            {
                if ((DateTime.Now - _lastBeep).TotalMilliseconds > 1200)       // repeating alarm tone
                {
                    _lastBeep = DateTime.Now;
                    try { System.Media.SystemSounds.Exclamation.Play(); } catch { /* no sound device: ignore */ }
                }
                RefreshAll();                                                   // flashing alarm indicators
            }
        }

        private void UpdateClock() => _lblClock.Text = DateTime.Now.ToString("ddd, dd MMM yyyy  HH:mm:ss");

        // ------------------------------------------------------------ animation (delegate, every timer tick)

        private void OnElevatorAnimated(Elevator elevator)
        {
            PanelOf(elevator)?.UpdateView(_blink);       // only the elevator that moved
            UpdateOverview();
        }
```

Models/Elevator.cs | 8.5 Delegation and Timers — delegate declaration

```
namespace Elevator_with_time
{
    /// <summary>Delegate used to animate the elevator (raised on every timer tick while something moves).</summary>
    public delegate void PositionChangedHandler(Elevator elevator);

    /// <summary>Observer pattern: anyone interested in elevator events implements this.</summary>
    public interface IElevatorObserver
    {
```

Models/Elevator.cs | 8.5 Delegation and Timers — 20 ms elevator timer

```
59      /// A single System.Windows.Forms.Timer drives the cabin movement and the door animation.
60      /// </summary>
61      public class Elevator : IDisposable
62      {
63          public const int MinFloor = 1;                    // G
64          public const int MaxFloor = 2;                    // G and 1 only
65          public const int TickMs = 20;                     // timer interval
66          public const double MaxSpeed = 0.022;             // floors per tick at cruising speed (about 1 s per floor)
67          public const double RampDistance = 0.30;          // floors used to speed up / slow down
68          public const double ApproachDistance = 0.50;      // announce approach when this close to the target floor
69          public const double LevelDistance = 0.08;         // announce levelling when this close to the target floor
70          public const double MinSpeedFactor = 0.30;        // creeping speed while levelling (share of MaxSpeed)
71          public const double BrakeDistance = 0.35;         // a new stop can only be added if it is at least this far ahead
72          public const double DoorStep = 0.03;              // doors need about 0.7 s to open / close
73          public const int DoorDwellTicks = 4000 / TickMs;  // doors stay open for 4 s
74          public const int ShortDwellTicks = 1500 / TickMs; // ... or only 1.5 s more once another floor was requested
75
76          private readonly List<IElevatorObserver> _observers = new();
77          private string _lastRefusal;                       // last MOVE REFUSED reason (avoids flooding the log)
78          private readonly System.Windows.Forms.Timer _timer;
79          private readonly List<int> _pending = new();       // requested floors that have not been served yet
```

Models/Elevator.cs | 8.5 Delegation and Timers — events and timer Tick

```
180         public int DestinationFloor => TargetFloor != 0 ? TargetFloor : ChooseNextTarget();
181         public int NextFloorAbove => (int)Math.Floor(Position + 1e-6) + 1;
182         public int NextFloorBelow => (int)Math.Ceiling(Position - 1e-6) - 1;
183
184         public event PositionChangedHandler PositionChanged;
185         public event PositionChangedHandler DoorsChanged;
186
187         public Elevator(string name, int startFloor)
188         {
189             if (startFloor < MinFloor || startFloor > MaxFloor)
190                 throw new ArgumentOutOfRangeException(nameof(startFloor));
191
192             Name = name;
193             Position = startFloor;
194             TargetFloor = 0;
195             Inner = new Door("Inner door", DoorState.Closed);
196             for (int f = MinFloor; f <= MaxFloor; f++)
197                 _landing[f] = new Door($"Landing door {FloorName(f)}", f == startFloor ? DoorState.Closed : DoorState.Locked);
198             State = new IdleState();
199
200             _timer = new System.Windows.Forms.Timer { Interval = TickMs };
201             _timer.Tick += (s, e) => { AnimateDoors(); State.Tick(this); };
202             _timer.Start();
203         }
204
205         // ---------- floor names (floor 1 = G, floor 2 = "1", floor 3 = "2") ----------
206         public static string FloorName(int floor) => floor == 1 ? "G" : (floor - 1).ToString();
207         public static string FloorLongName(int floor) => floor == 1 ? "Ground Floor" : floor == 2 ? "First Floor" : $"Floor {floor - 1}";
208
209         /// <summary>Upper-case floor name used in the event log: "GROUND FLOOR", "FLOOR 1", "FLOOR 2".</summary>
210         public static string FloorTag(int floor) => floor == 1 ? "GROUND FLOOR" : $"FLOOR {floor - 1}";
211
212         // ---------- Observer ----------
213         public void Attach(IElevatorObserver observer) => _observers.Add(observer);
214
215         public void Notify(string status)
216         {
217             foreach (var o in _observers.ToArray())
218                 o.OnElevatorEvent(this, status);
219         }
220
221         internal void RaisePositionChanged() => PositionChanged?.Invoke(this);
222         internal void RaiseDoorsChanged() => DoorsChanged?.Invoke(this);
```

Form1.cs | 8.5 Delegation and Timers — 250 ms clock timer

```
15          private bool _dbErrorShown;
16          private bool _dbOffline;
17          private bool _loadingLog;
18          private bool _blink;                    // toggles every 250 ms (flashing alarm)
19          private DateTime _lastBeep = DateTime.MinValue;
20
21          private readonly System.Windows.Forms.Timer _clockTimer = new() { Interval = 250 };
22          private static readonly TimeSpan AlarmDuration = TimeSpan.FromSeconds(6);
```

8.6 BackgroundWorker

LogRepository employs a thread-safe queue and the BackgroundWorker component to handle LogEntry objects asynchronously. Enqueues LogEntry objects and triggers the BackgroundWorker if its state is idle; DoWork takes care of writing them to disk and RunWorkerCompleted catches any errors and publishes the EntriesWritten event.

Data/LogRepository.cs | 8.6 BackgroundWorker — queue, worker and event setup

```
53      public class LogRepository
54      {
55          private const string TableName = "elevator_log";
56          private static readonly DbSettings Settings = DbSettings.Load();
57
58          private readonly ConcurrentQueue<LogEntry> _queue = new();
59          private readonly BackgroundWorker _writer = new();
60          private readonly object _schemaLock = new();
61          private bool _schemaChecked;
62          private volatile bool _wroteSomething;
63
64          /// <summary>Raised on the GUI thread when a database operation fails.</summary>
65          public event Action<string> ErrorOccurred;
66
67          /// <summary>Raised on the GUI thread after new log entries were stored.</summary>
68          public event Action EntriesWritten;
69
70          public static string Description =>
71              $"MySQL  {Settings.User}@{Settings.Host}:{Settings.Port}  /  {Settings.Database}";
72
73          public LogRepository()
74          {
75              _writer.DoWork += Writer_DoWork;
76              _writer.RunWorkerCompleted += Writer_Completed;
77          }
```

Data/LogRepository.cs | 8.6 BackgroundWorker — Enqueue() and start-if-idle

```
81          public void Enqueue(LogEntry entry)
82          {
83              _queue.Enqueue(entry);
84              StartWriterIfNeeded();
85          }
86
87          private void StartWriterIfNeeded()
88          {
89              if (!_writer.IsBusy && !_queue.IsEmpty)
90                  _writer.RunWorkerAsync();
91          }
```

Data/LogRepository.cs | 8.6 BackgroundWorker — DoWork drains queue and writes entries

```
93          private void Writer_DoWork(object sender, DoWorkEventArgs e)
94          {
95              while (_queue.TryDequeue(out LogEntry entry))
96              {
97                  try
98                  {
99                      Insert(entry);
100                     _wroteSomething = true;
101                 }
102                 catch (Exception ex)
103                 {
104                     e.Result = ex.Message;
105                     while (_queue.TryDequeue(out _)) { }   // database unusable: do not retry every entry
106                 }
107             }
108         }
```

Data/LogRepository.cs | 8.6 BackgroundWorker — completion, errors and EntriesWritten

```
110         private void Writer_Completed(object sender, RunWorkerCompletedEventArgs e)
111         {
112             if (e.Error != null) ErrorOccurred?.Invoke(e.Error.Message);
113             else if (e.Result is string message) ErrorOccurred?.Invoke(message);
114
115             if (_wroteSomething)
116             {
117                 _wroteSomething = false;
118                 EntriesWritten?.Invoke();
119             }
120             StartWriterIfNeeded();
121         }
```

Data/LogRepository.cs | 8.6 BackgroundWorker — asynchronous loading and clearing

```
132         // ------------------------------------------------------------ reading / clearing (async)
133
134         public void LoadLogsAsync(Action<DataTable> onLoaded, Action<string> onError) =>
135             RunAsync(ReadAll, onLoaded, onError);
136
137         public void ClearAsync(Action onDone, Action<string> onError) =>
138             RunAsync(() => Execute($"TRUNCATE TABLE {TableName}"), _ => onDone(), onError);
139
140         /// <summary>One place that runs work on a BackgroundWorker and reports back on the GUI thread.</summary>
141         private static void RunAsync<T>(Func<T> work, Action<T> onDone, Action<string> onError)
142         {
143             var worker = new BackgroundWorker();
144             worker.DoWork += (s, e) => e.Result = work();
145             worker.RunWorkerCompleted += (s, e) =>
146             {
147                 if (e.Error != null) onError(e.Error.Message);
148                 else onDone((T)e.Result);
149                 worker.Dispose();
150             };
151             worker.RunWorkerAsync();
152         }
```

8.7 Optimizing the Program

- Batching writes by background worker is done without affecting GUI.
- Database write failures remove items from queue without retries.
- DoubleBuffered and invalidate make flicker-free.
- Using cached themes will enhance rendering performance since only the cabin needs to move.
- 5-second timeout ensures no delay from the server side.

UI/Controls.cs | Double buffering — ElevatorShaft reduces paint flicker

```
391         public ElevatorShaft()
392         {
393             DoubleBuffered = true;
394             ResizeRedraw = true;
395         }
396
397         /// <summary>The elevator that is drawn (position of the cabin, openness of every door, requests ...).</summary>
398         [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
399         public Elevator Model
400         {
401             get => _model;
402             set { _model = value; Invalidate(); }
```

UI/Controls.cs | Repaint optimization — skip Invalidate when hover value is unchanged

```
490         protected override void OnMouseMove(MouseEventArgs e)
491         {
492             base.OnMouseMove(e);
493             int hit = HitCallButton(e.Location);
494             if (hit == _hoverFloor) return;
495             _hoverFloor = hit;
496             Cursor = hit != 0 ? Cursors.Hand : Cursors.Default;
497             Invalidate();
498         }
499
500         protected override void OnMouseLeave(EventArgs e)
501         {
502             base.OnMouseLeave(e);
503             if (_hoverFloor == 0) return;
504             _hoverFloor = 0;
505             Cursor = Cursors.Default;
506             Invalidate();
507         }
```

UI/Controls.cs | Cached shaft layers — static background and foreground bitmaps

```
371     public class ElevatorShaft : Panel
372     {
373         private Elevator _model;
374         private int _hoverFloor;
375         private Bitmap _back;               // static picture behind the cabin
376         private Bitmap _front;              // static picture in front of the cabin (landing door frames), transparent opening
377         private Size _builtFor;
378         private ThemeData _theme = Theme.Current;
```

UI/Controls.cs | Build static layers once, then draw moving cabin separately

```
517         // ------------------------------------------------------------ static layers (built once per size)
518
519         private void BuildLayers()
520         {
521             _back?.Dispose();
522             _front?.Dispose();
523             _back = new Bitmap(Width, Height, PixelFormat.Format32bppPArgb);
524             _front = new Bitmap(Width, Height, PixelFormat.Format32bppPArgb);
525             _builtFor = Size;
526
527             using (Graphics g = Graphics.FromImage(_back)) PaintBack(g);
528             using (Graphics g = Graphics.FromImage(_front)) PaintFront(g);
529         }
```

UI/Controls.cs | Cached theme used while drawing the moving cabin

```
878         }
879
880         /// <summary>The cabin: silver car body, direction display, its own INNER door (bright silver, amber trim).</summary>
881         private void DrawCar(Graphics g, Elevator el, Rectangle car, double position)
882         {
883             ThemeData t = _theme;
884             double open = el != null ? el.Inner.Openness : 0.0;
885             bool emergency = el != null && el.IsEmergency;
886             int direction = el != null ? el.Direction : 0;
887             int y = YFloor(position);
888             var opening = new Rectangle(OpenLeft, y - DoorH, OpenW, DoorH);
889
890             g.SmoothingMode = SmoothingMode.None;
891             Steel.Fill(g, car);                                          // silver car body
892
893             // direction display in the cabin fascia (hidden behind the landing frame while the cabin is levelled)
894             int dw = Math.Min(OpenW, CarW / 3);
895             var disp = new Rectangle(CenterX - dw / 2, car.Top + 3, dw, Math.Max(8, CarTopH - 6));
896             using (var back = new SolidBrush(t.DisplayBack)) g.FillRectangle(back, disp);
897             string arrow = direction > 0 ? "\u2191" : direction < 0 ? "\u2193" : "\u2022";
898             using (var arrowFont = new Font("Segoe UI Symbol", Math.Max(8f, disp.Height * 0.75f), FontStyle.Bold, GraphicsUnit.Pixel))
899                 TextRenderer.DrawText(g, arrow, arrowFont, disp, emergency ? Color.OrangeRed : t.Led,
```

Data/LogRepository.cs | MySQL connection timeout — 5 seconds

```
227             var builder = new MySqlConnectionStringBuilder
228             {
229                 Server = Settings.Host,
230                 Port = (uint)Settings.Port,
231                 UserID = Settings.User,
232                 Password = Settings.Password,      // the builder handles special characters such as @ and $
233                 ConnectionTimeout = 5,
234                 AllowPublicKeyRetrieval = true,    // needed for MySQL 8 on localhost without SSL
235                 CharacterSet = "utf8mb4"
236             };
237
238             var conn = new MySqlConnection(builder.ConnectionString);
```

Data/LogRepository.cs | Database failure — clear remaining queue and report completion

```
81          public void Enqueue(LogEntry entry)
82          {
83              _queue.Enqueue(entry);
84              StartWriterIfNeeded();
85          }
86
87          private void StartWriterIfNeeded()
88          {
89              if (!_writer.IsBusy && !_queue.IsEmpty)
90                  _writer.RunWorkerAsync();
91          }
92
93          private void Writer_DoWork(object sender, DoWorkEventArgs e)
94          {
95              while (_queue.TryDequeue(out LogEntry entry))
96              {
97                  try
98                  {
99                      Insert(entry);
100                     _wroteSomething = true;
101                 }
102                 catch (Exception ex)
103                 {
104                     e.Result = ex.Message;
105                     while (_queue.TryDequeue(out _)) { }   // database unusable: do not retry every entry
106                 }
107             }
108         }
109
110         private void Writer_Completed(object sender, RunWorkerCompletedEventArgs e)
111         {
112             if (e.Error != null) ErrorOccurred?.Invoke(e.Error.Message);
113             else if (e.Result is string message) ErrorOccurred?.Invoke(message);
114
115             if (_wroteSomething)
116             {
117                 _wroteSomething = false;
118                 EntriesWritten?.Invoke();
119             }
120             StartWriterIfNeeded();
```

Data/LogRepository.cs | Queued writes — one BackgroundWorker and worker event setup

```
58          private readonly ConcurrentQueue<LogEntry> _queue = new();
59          private readonly BackgroundWorker _writer = new();
60          private readonly object _schemaLock = new();
61          private bool _schemaChecked;
62          private volatile bool _wroteSomething;
63
64          /// <summary>Raised on the GUI thread when a database operation fails.</summary>
65          public event Action<string> ErrorOccurred;
66
67          /// <summary>Raised on the GUI thread after new log entries were stored.</summary>
68          public event Action EntriesWritten;
69
70          public static string Description =>
71              $"MySQL  {Settings.User}@{Settings.Host}:{Settings.Port}  /  {Settings.Database}";
72
73          public LogRepository()
74          {
75              _writer.DoWork += Writer_DoWork;
76              _writer.RunWorkerCompleted += Writer_Completed;
```

8.8 Exception Handling

- Database error handling includes reporting the error once, as well as indicating an offline state.
- Configurations error handling is done without causing the application to terminate upon startup.
- Panel button failure is detected to prevent application termination.
- Inadequate sound devices do not interfere with other functionalities.
- Log filters are set to be empty in case of invalid configuration.
- Invalid actions such as movement within open doors are disallowed.

```
1 reference
private void Writer_DoWork(object sender, DoWorkEventArgs e)
{
    while (_queue.TryDequeue(out LogEntry entry))
    {
        try
        {
            Insert(entry);
            _wroteSomething = true;
        }
        catch (Exception ex)
        {
            e.Result = ex.Message;
            while (_queue.TryDequeue(out _)) { }   // database unusable: do not retry every entry
        }
    }
}

1 reference
private void Writer_Completed(object sender, RunWorkerCompletedEventArgs e)
{
    if (e.Error != null) ErrorOccurred?.Invoke(e.Error.Message);
    else if (e.Result is string message) ErrorOccurred?.Invoke(message);

    if (_wroteSomething)
    {
        _wroteSomething = false;
        EntriesWritten?.Invoke();
    }
    StartWriterIfNeeded();
}
```

Figure 31 exception handling

```
1 reference
private void Writer_Completed(object sender, RunWorkerCompletedEventArgs e)
{
    if (e.Error != null) ErrorOccurred?.Invoke(e.Error.Message);
    else if (e.Result is string message) ErrorOccurred?.Invoke(message);

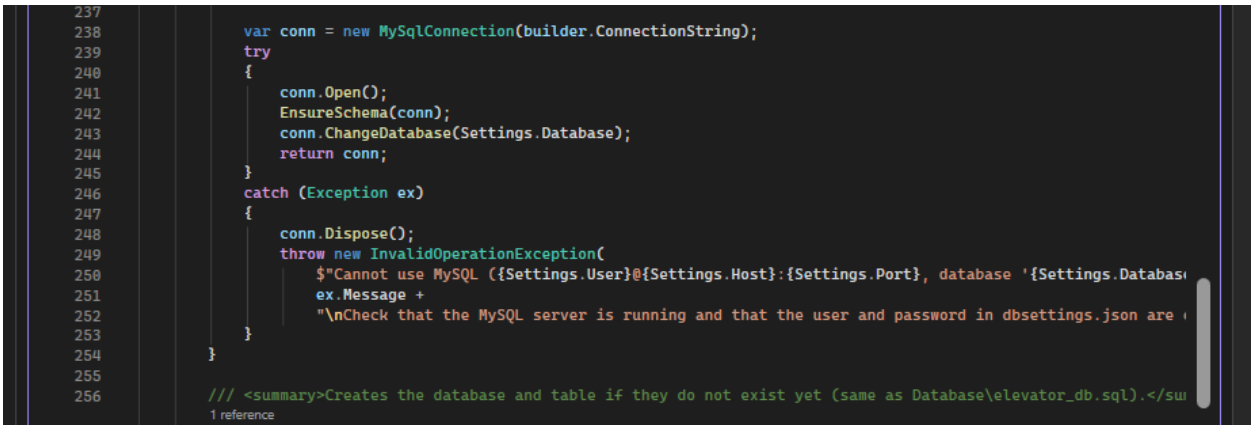
    if (_wroteSomething)
    {
        _wroteSomething = false;
        EntriesWritten?.Invoke();
    }
    StartWriterIfNeeded();
}

/// <summary>Writes anything still queued (called when the form closes).</summary>
1 reference
public void Flush()
{
    while (_queue.TryDequeue(out LogEntry entry))
    {
        try { Insert(entry); } catch { break; }
    }
}

// ------------------------------------------------------------ reading / clearing (async)

1 reference
public void LoadLogsAsync(Action<DataTable> onLoaded, Action<string> onError) =>
```

Figure 32 exception handling



```
237
238             var conn = new MySqlConnection(builder.ConnectionString);
239             try
240             {
241                 conn.Open();
242                 EnsureSchema(conn);
243                 conn.ChangeDatabase(Settings.Database);
244                 return conn;
245             }
246             catch (Exception ex)
247             {
248                 conn.Dispose();
249                 throw new InvalidOperationException(
250                     $"Cannot use MySQL ({Settings.User}@{Settings.Host}:{Settings.Port}, database '{Settings.Databas
251                     ex.Message +
252                     "\nCheck that the MySQL server is running and that the user and password in dbsettings.json are
253             }
254         }
255
256         /// <summary>Creates the database and table if they do not exist yet (same as Database\elevator_db.sql).</su
            1 reference
```

Figure 33 exception handling

9. Prototype

9.1 Main Window

Login now to generate your dashboard design. After logging in, I'll create an industrial, black theme-based elevator dashboard with indicators and cards for navigation and overview.

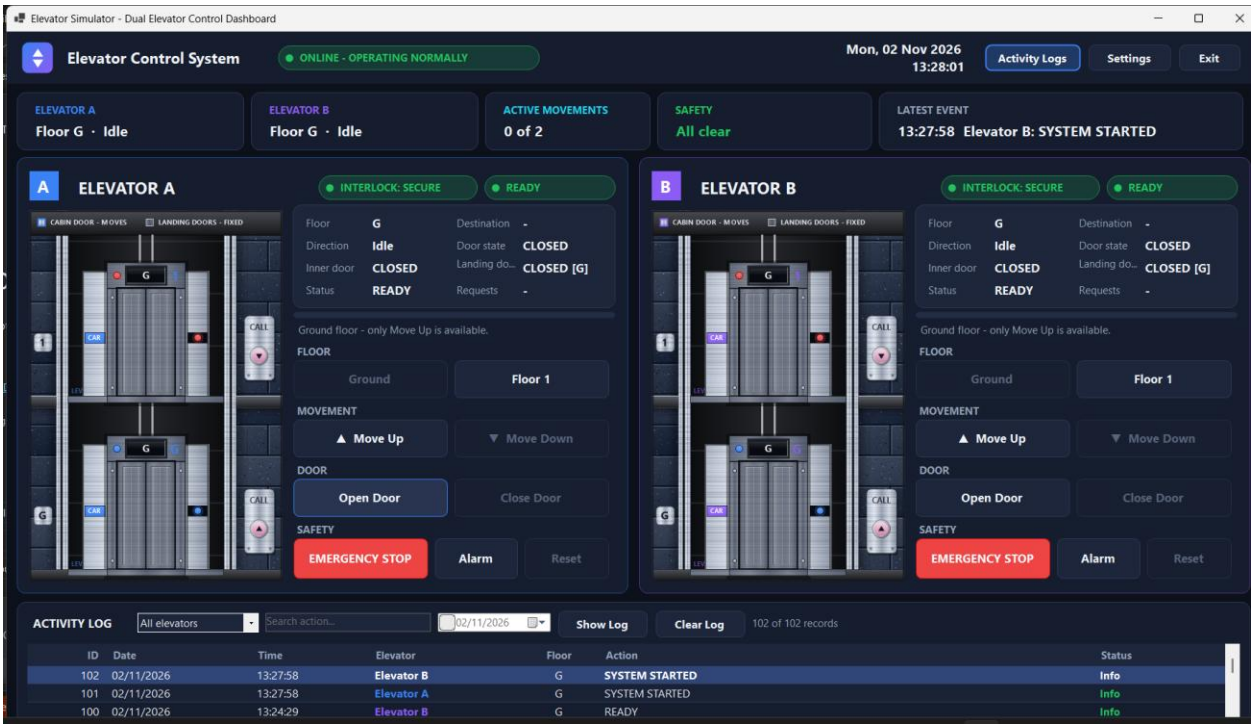


Figure 34 main window

9.2 Lift Control Card

The ElevatorPanel for each elevator includes:

- STATUS: Current Floor, Destination, Direction, Inner/Outer Door, Door Status, Status, Request.
- INDICATOR LAMPS: EMERGENCY (red), ALARM, INTERLOCK (green/amber/red).
- CONTROLS: FLOOR, MOVEMENT, DOOR, SAFETY.
- OPERATION BAR: Shows progression of operation.
- HELP TEXT: Gives reason for control being inhibited.

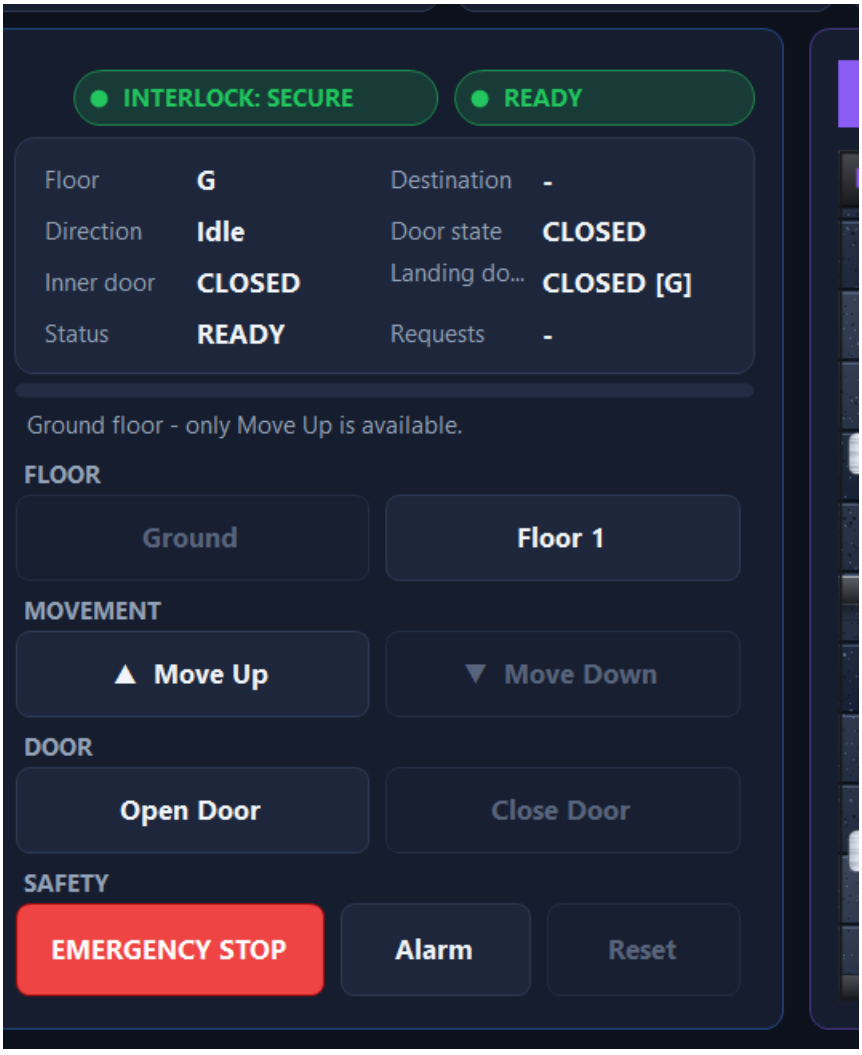


Figure 35 Lift Control Card

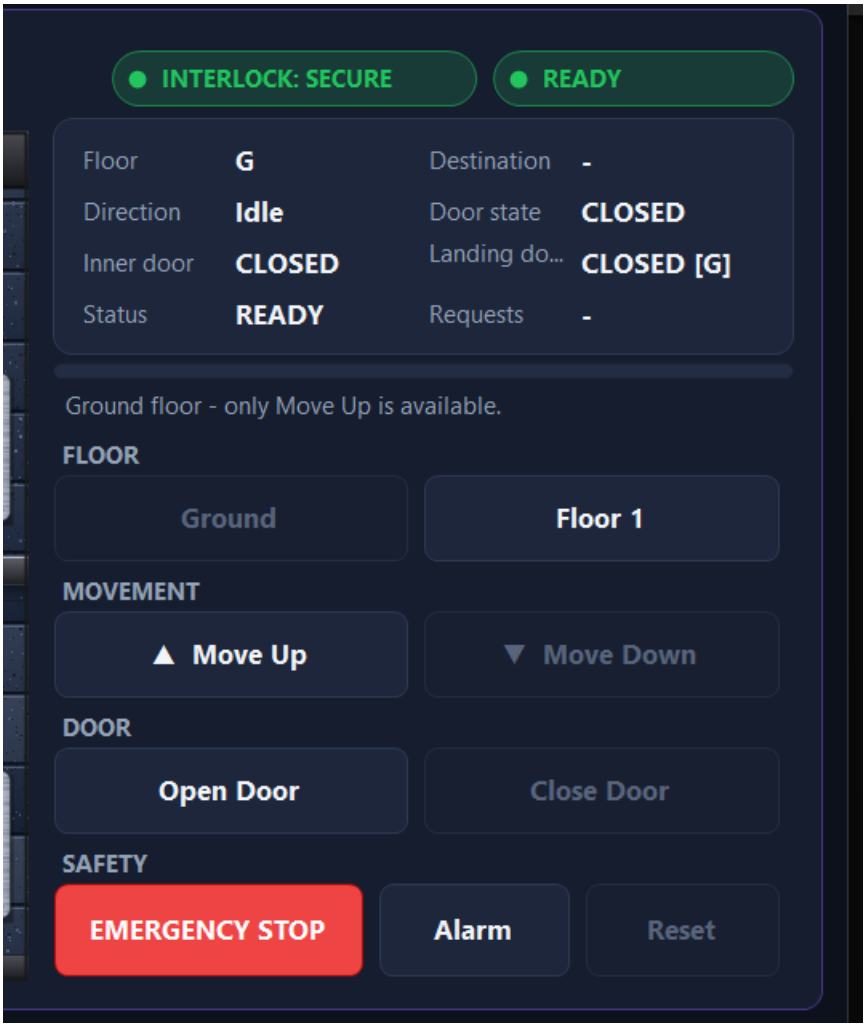


Figure 36 Lift control card

9.3 Shaft, Cabin and Doors

The shaft of the elevator does not move, but its body is silver and has doors on every floor level. It is only the cabin that can move up and down. At each time step of the simulation, both doors open and close together.

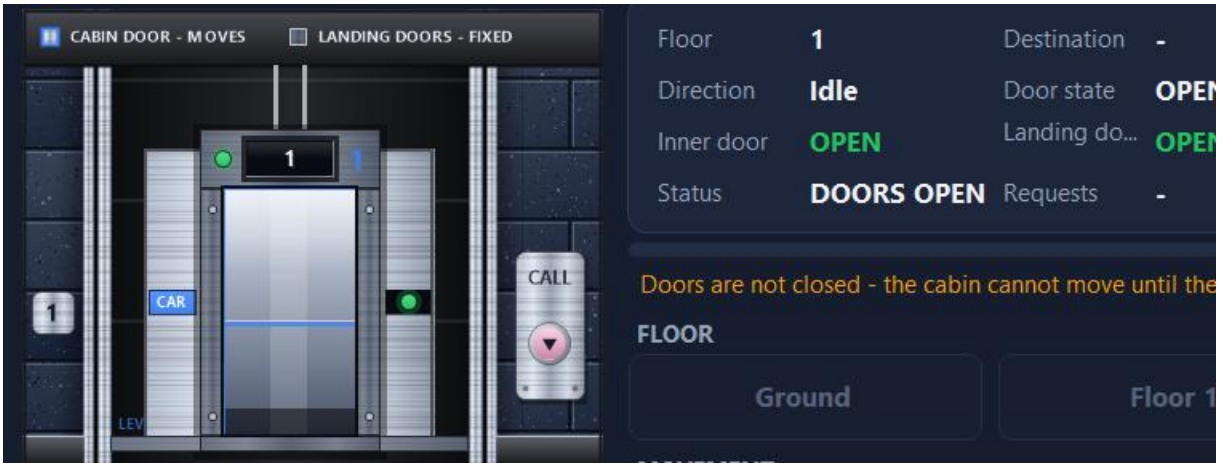


Figure 37 shaft,cabin and doors

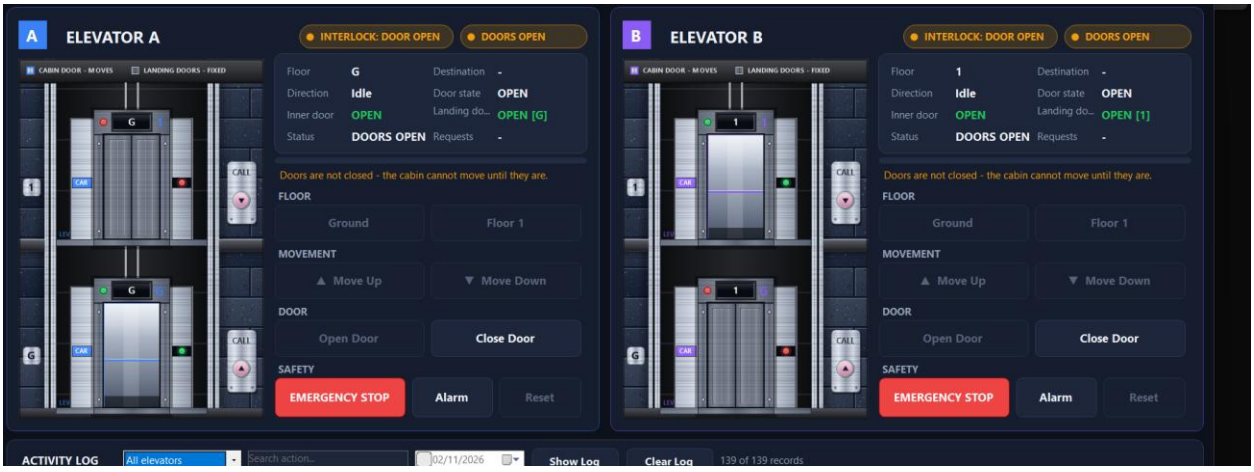
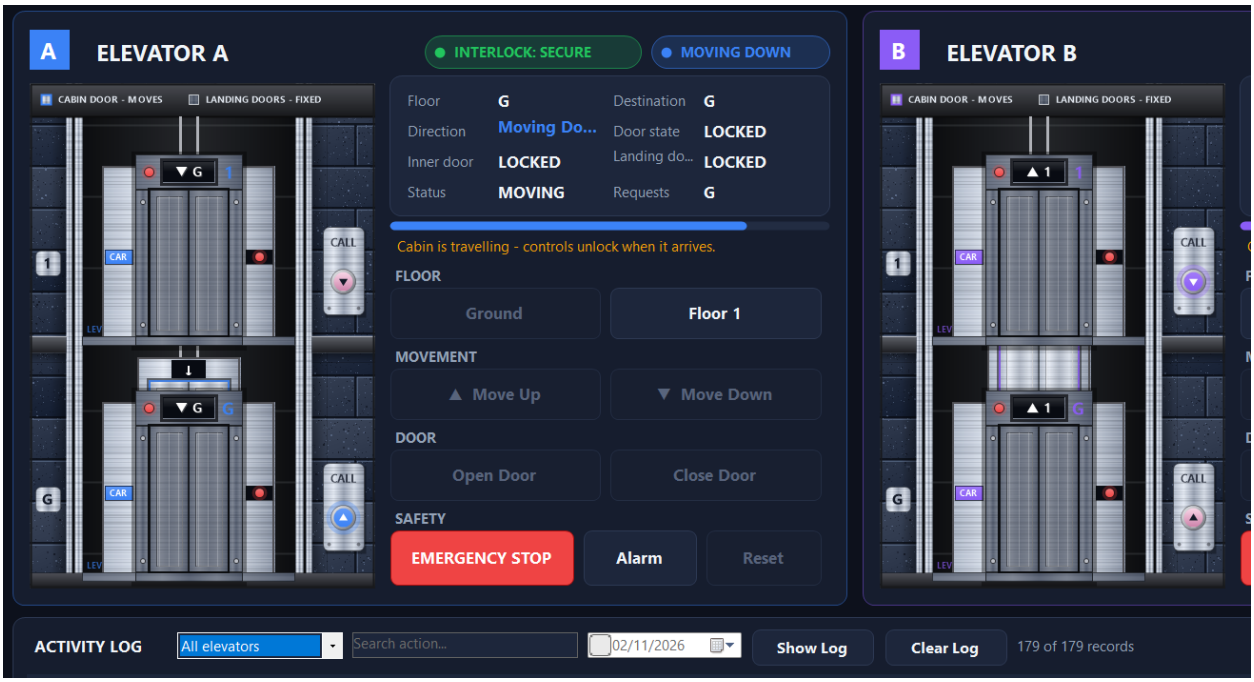


Figure 38 shaft cabin and doors

9.4 Elevator Movement and Door Behaviour

- Movement: Speed up, maintain speed, and slow down (maximum rate 0.022 floors per tick, 0.30-floor ramp, 30% creeping speed during level changes). Stop at the right floor.
- Arrival procedure: ARRIVED AT FLOOR -> Doors unlocked -> Opening -> Open -> 4-second pause (1.5 seconds if other floor selected) -> Closing -> Closed -> Ready.
- Departure: Document ALL DOORS LOCKED prior to movement.
- Door statuses: Closed, Opening, Open, Closing, Locked.



9.5 Safety Behaviour

- Interlocks: All doors must be closed; faults cause emergency stop.
- Landing doors: Doors should unlock only when stopped on that floor.
- Emergency Stop: Halts operation and cancels all calls until reset.
- Alarm: Flashes, rings, and logs; stays on for 6 seconds or until reset.
- Control: Rejects all invalid floor and movement requests.

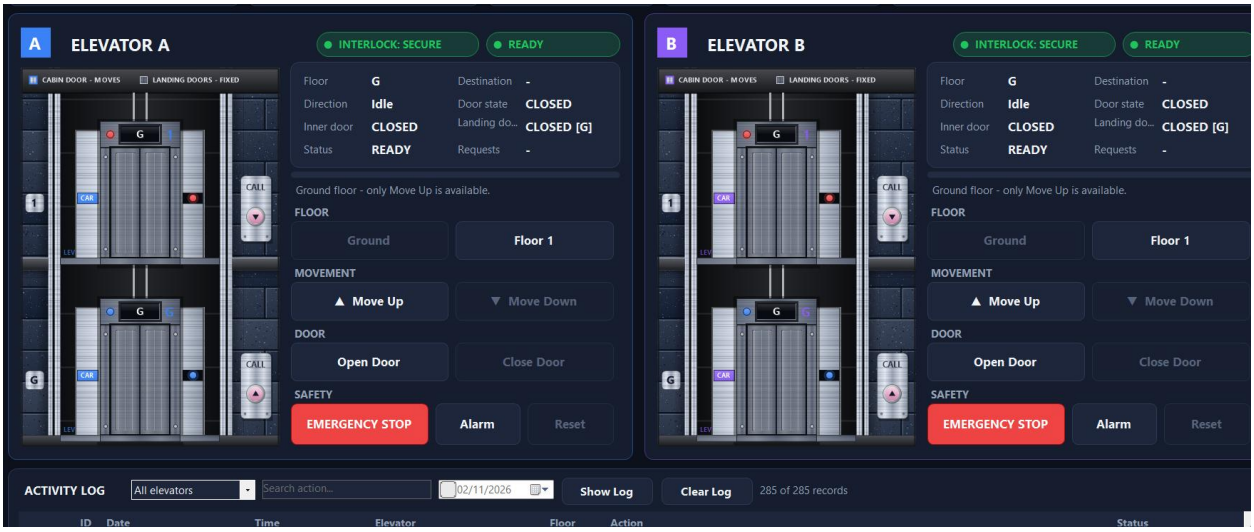


Figure 39 interlocks

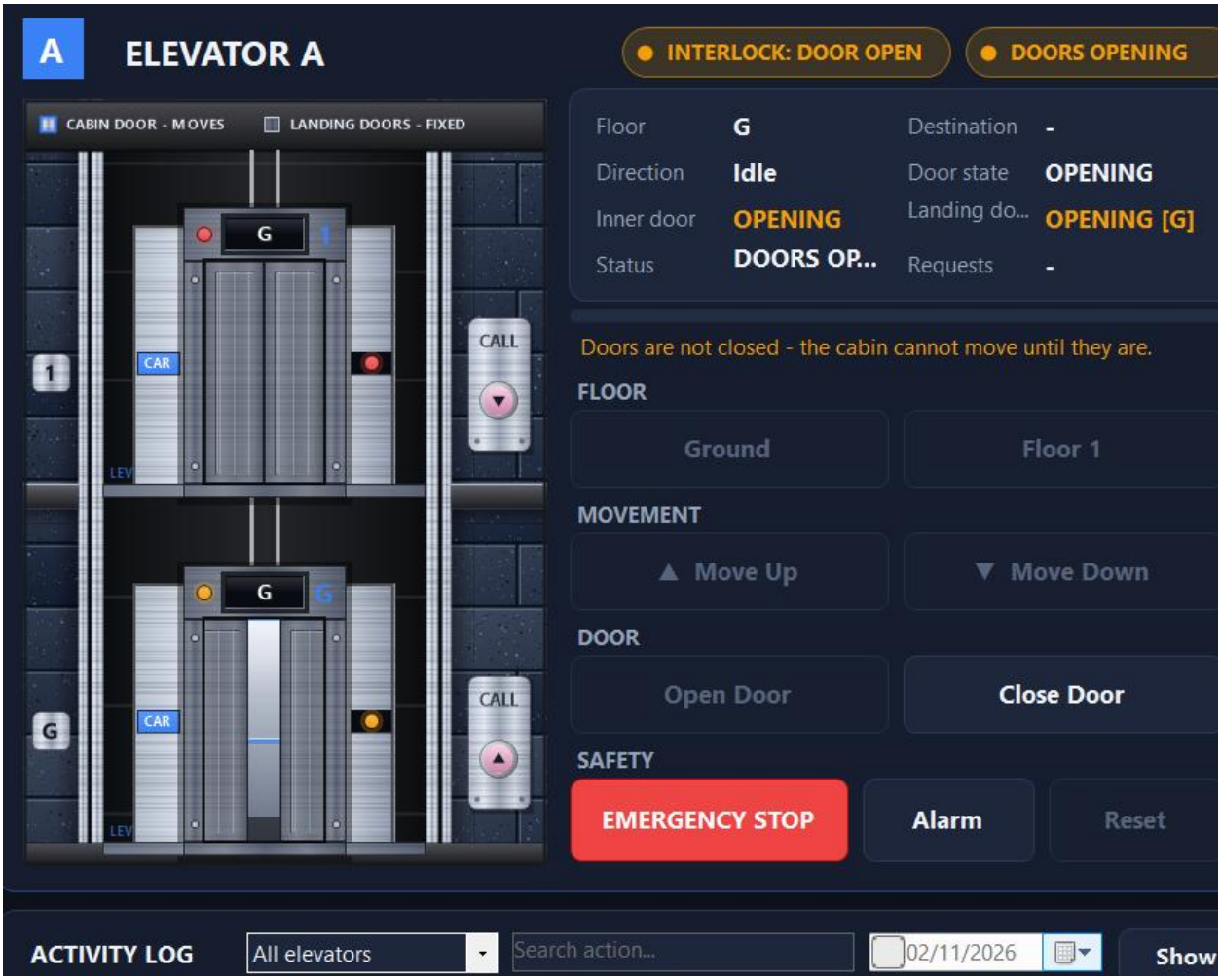


Figure 40 closing the door



Figure 41 elevator going up

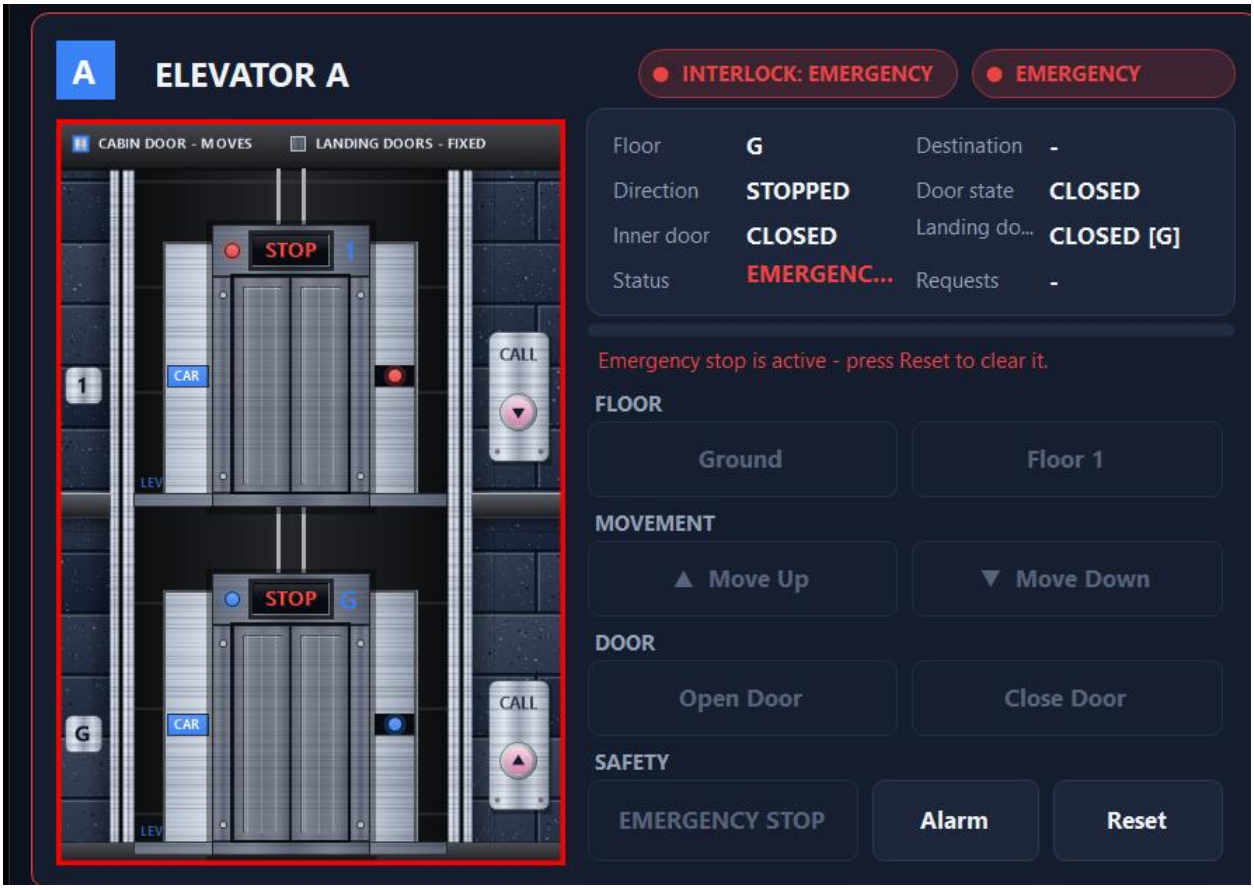


Figure 42 emergency stop

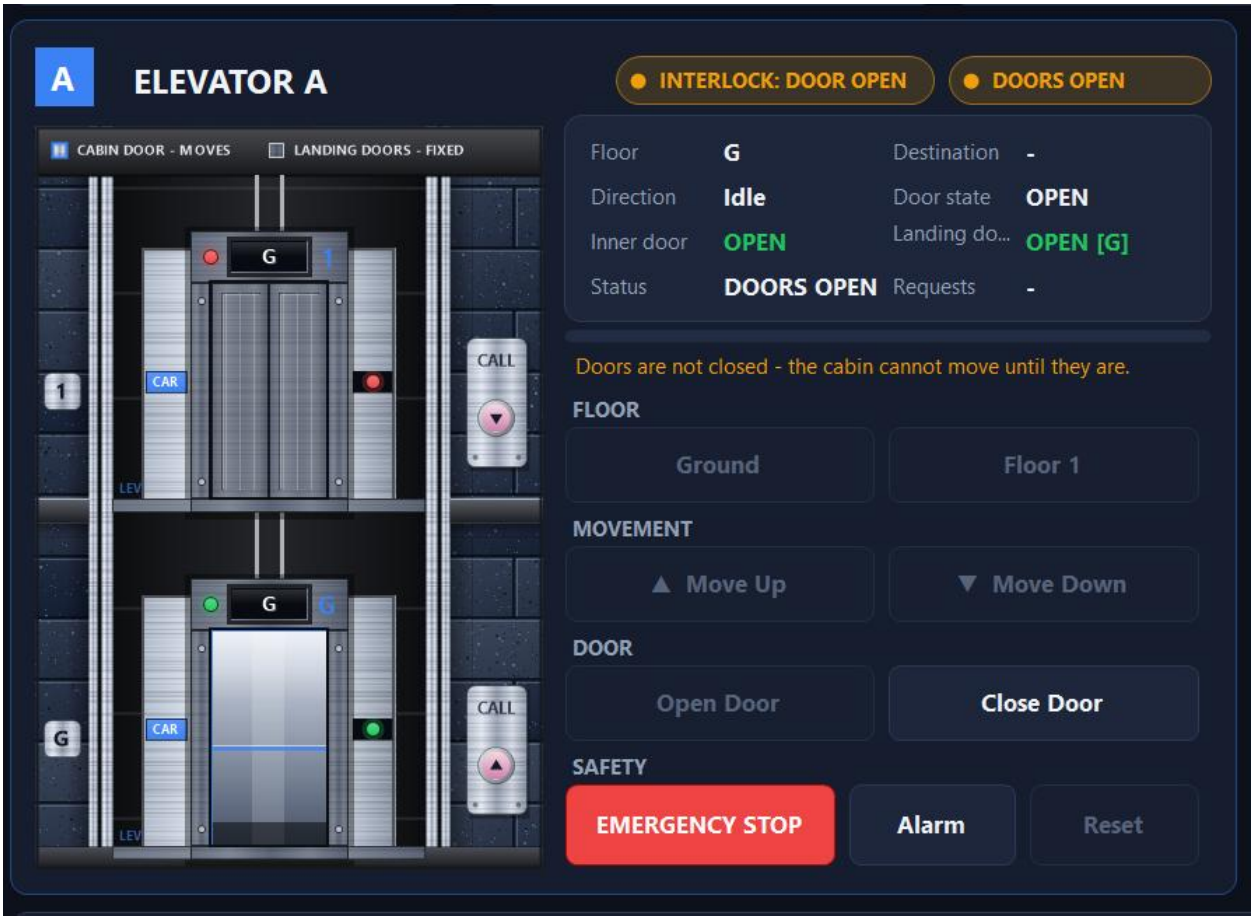


Figure 43 opening of door

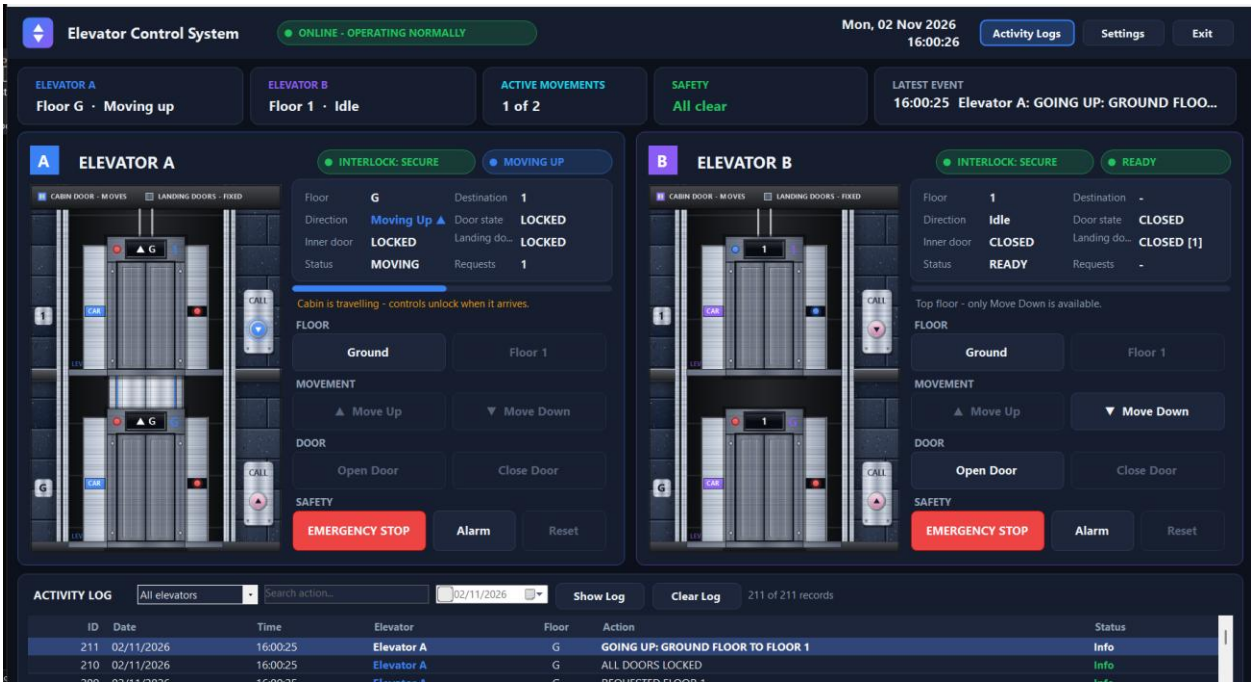
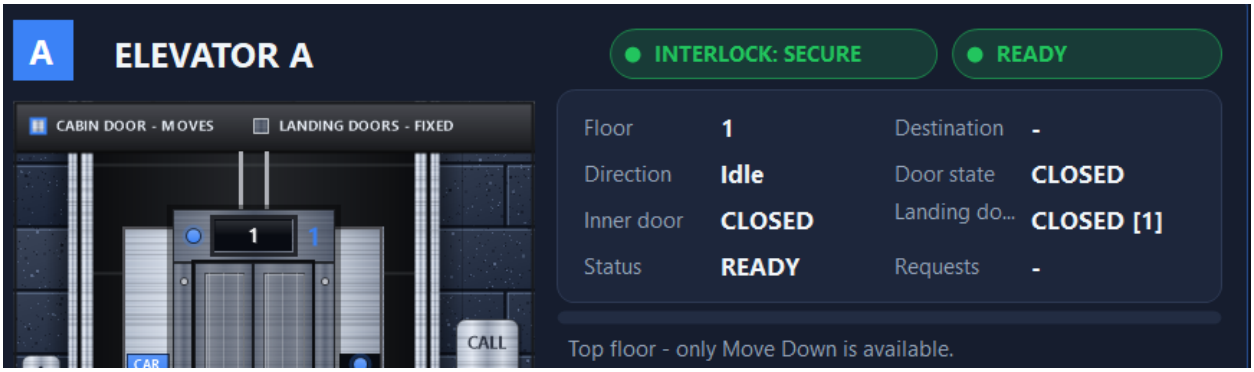
- 

Figure 44 going down

- 

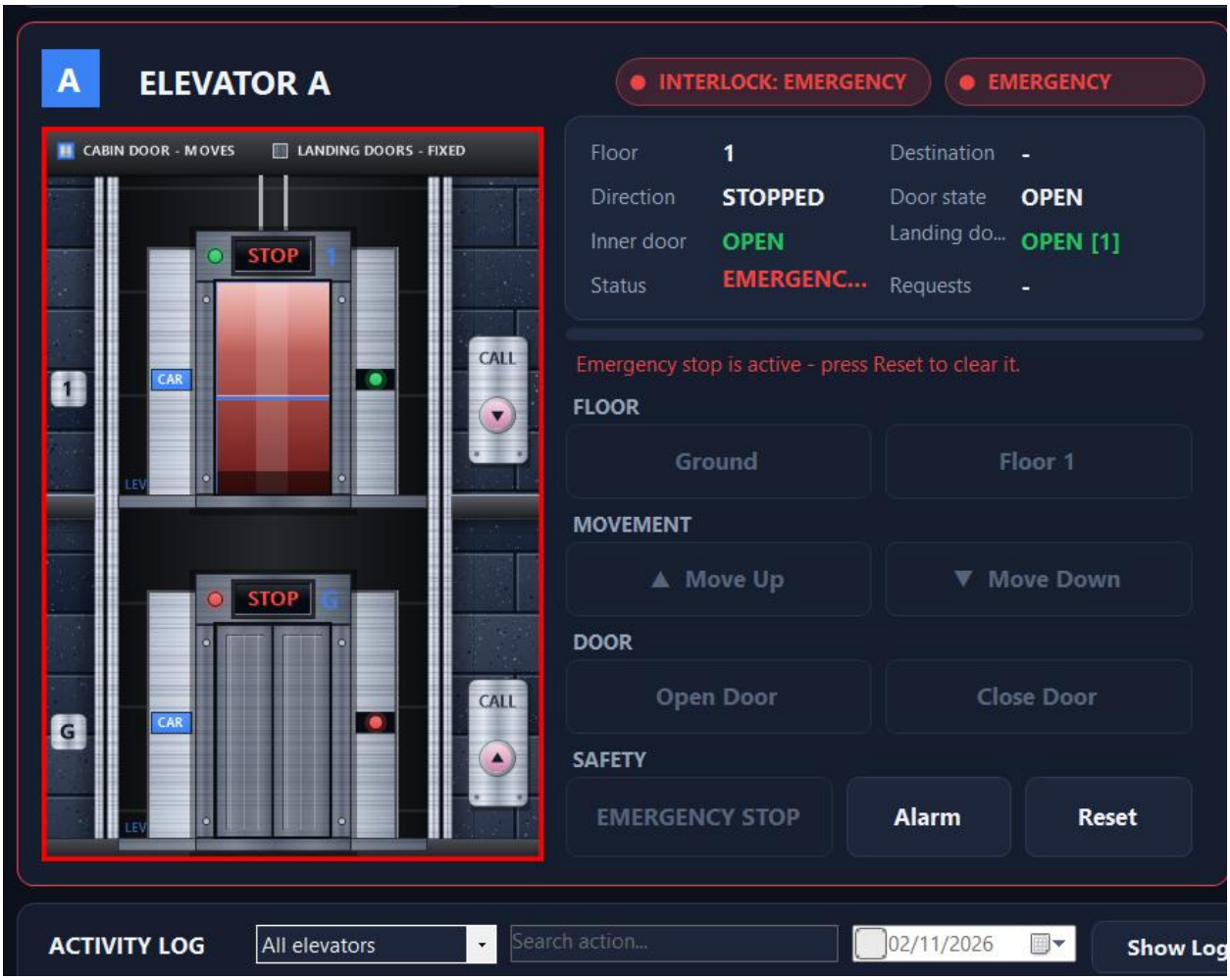


Figure 45 emergency stop

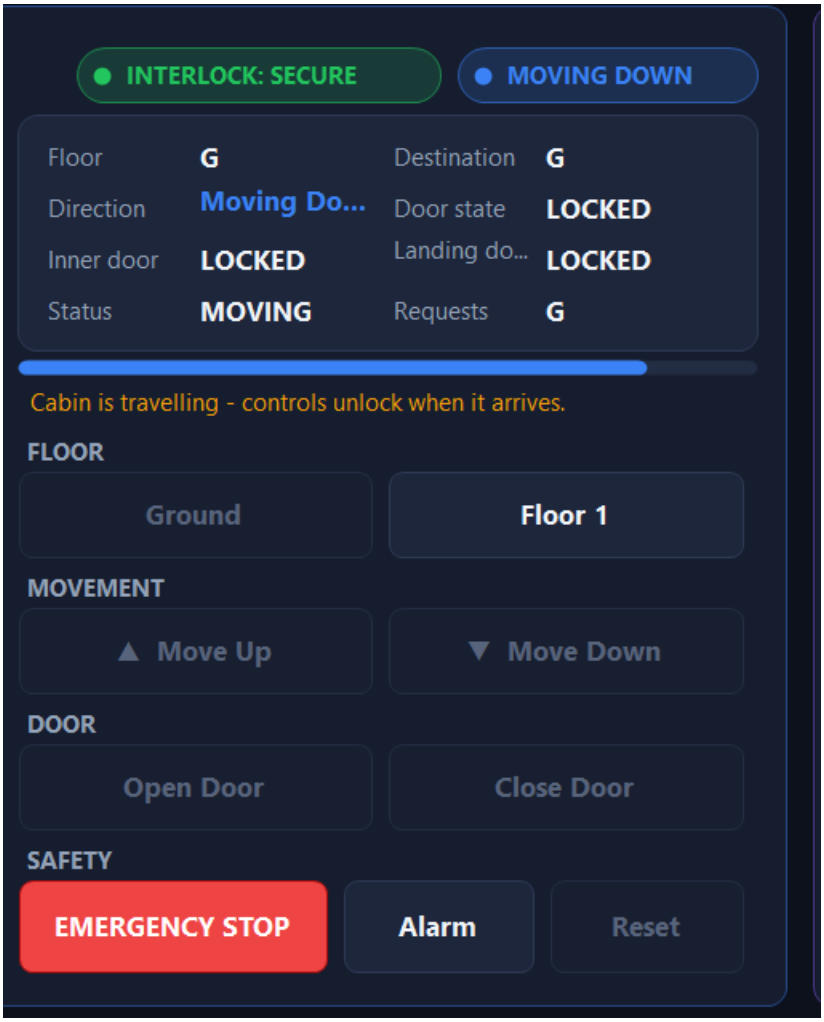


Figure 46 canin trvaeeling

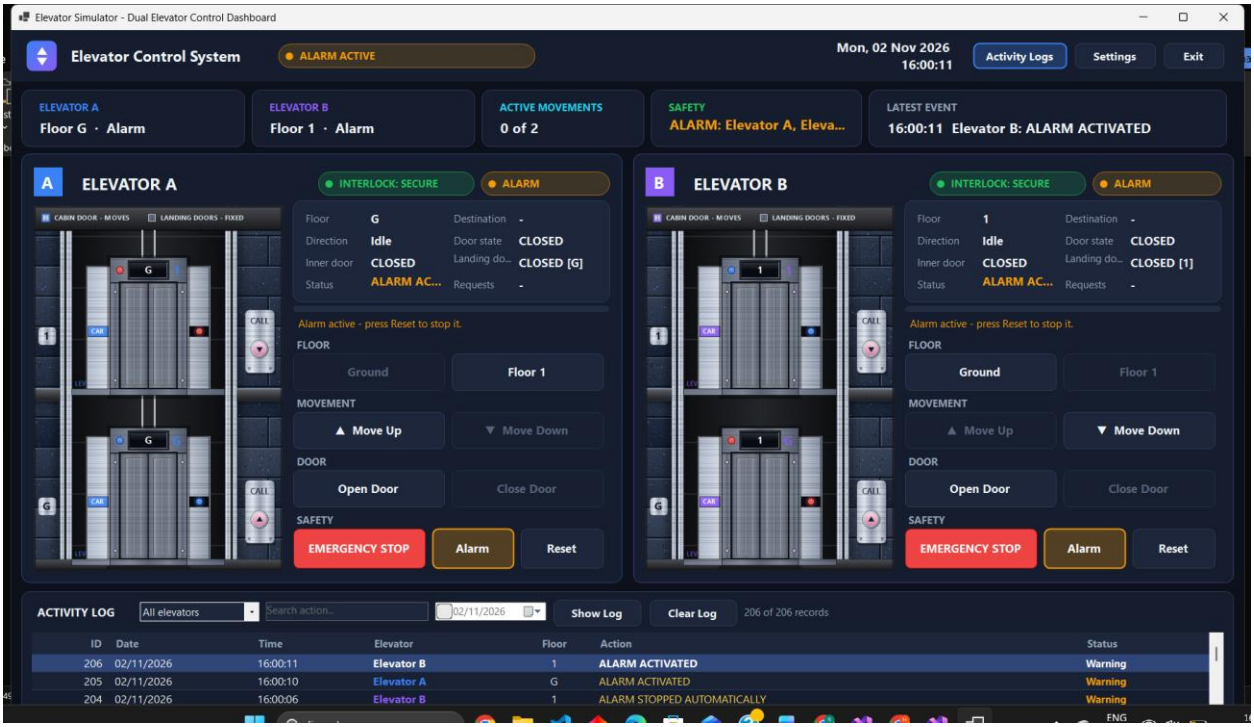
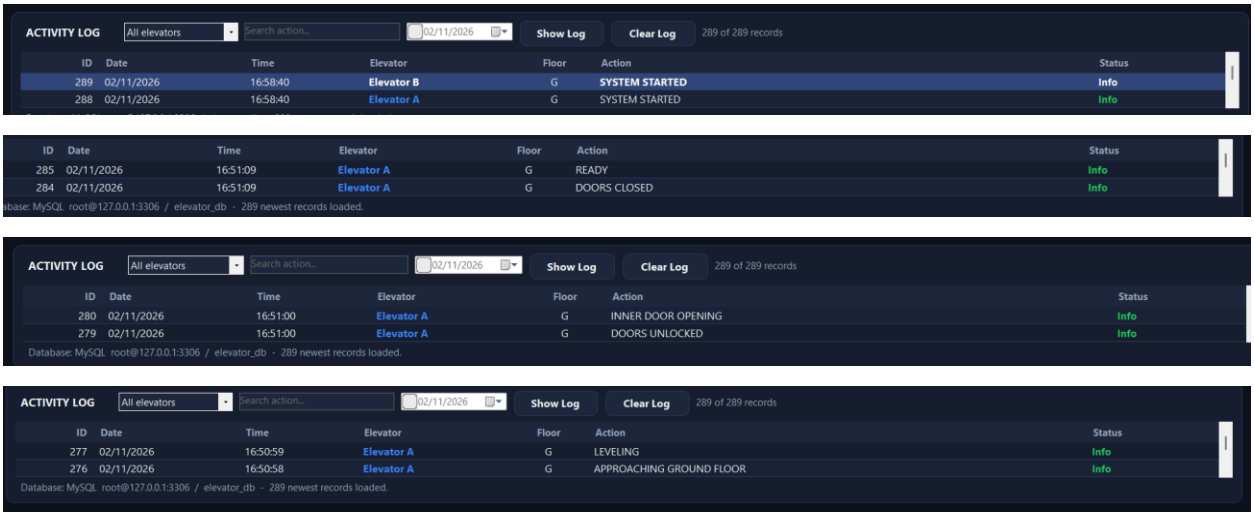


Figure 47 alarm activated

9.6 Event Log

- Log Grid: ID, Date, Time, Elevator, Floor, Action, Status.
- Filters: Elevator, text, date.
- Highlighting: Critical, Warning, and Latest rows.
- Empty State: Shown when there are no results.
- Events: Startup, Requests, Movement, Door Events, Arrival, Alarm, Emergency Stop, Reset, Faults.



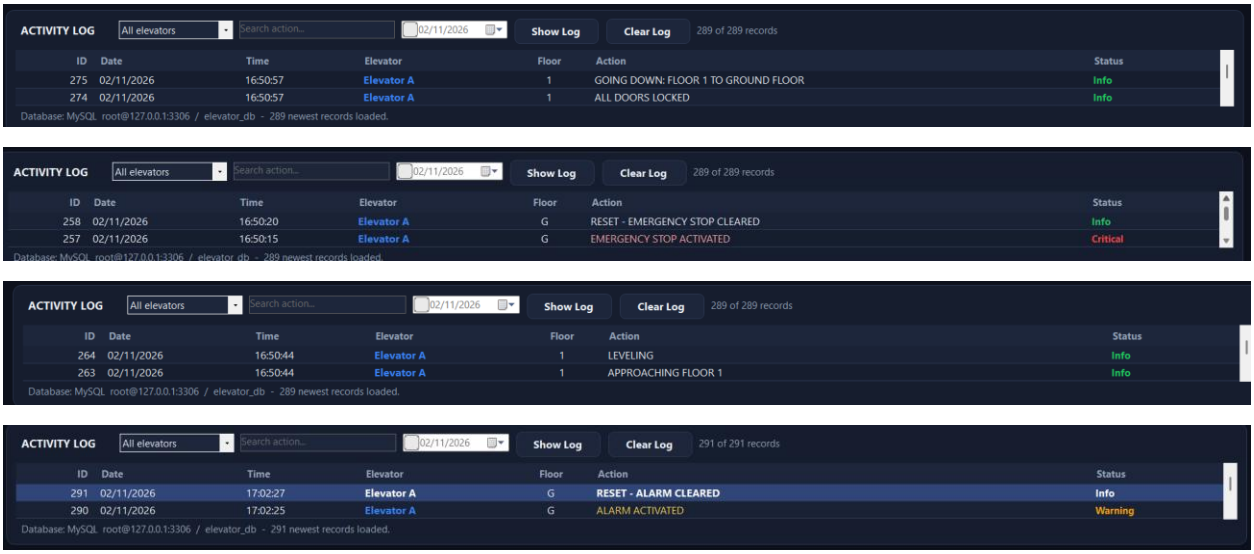


Figure 48 event log

10. Testing and Evaluation

Testing combines black box tests (what the user sees, without looking at code) and white box tests (paths through the code) [6]. Expected results come from the design; record the actual result and Pass or Fail in the last column when each test is run.

10.1 Black Box Testing

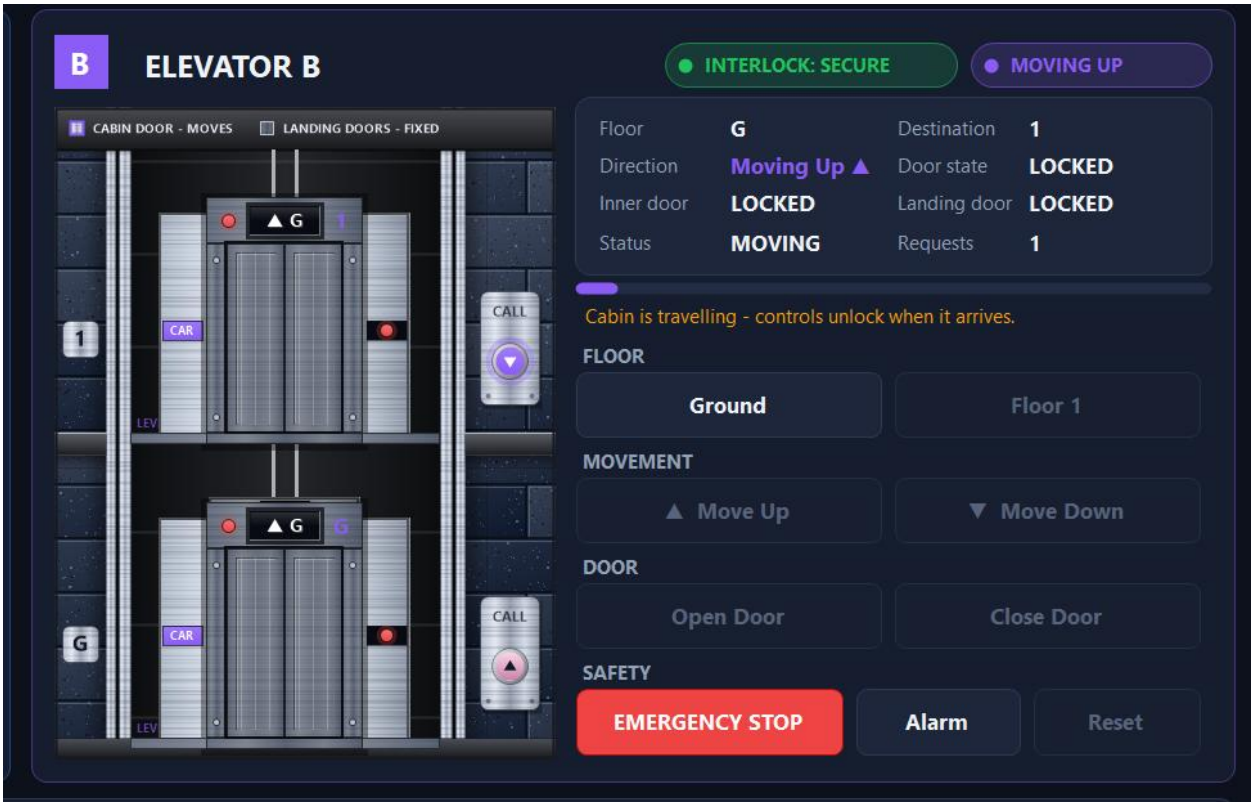
Techniques used: equivalence partitioning (valid and invalid inputs), boundary value analysis (floors G and 1) and state-based testing (door and emergency states).

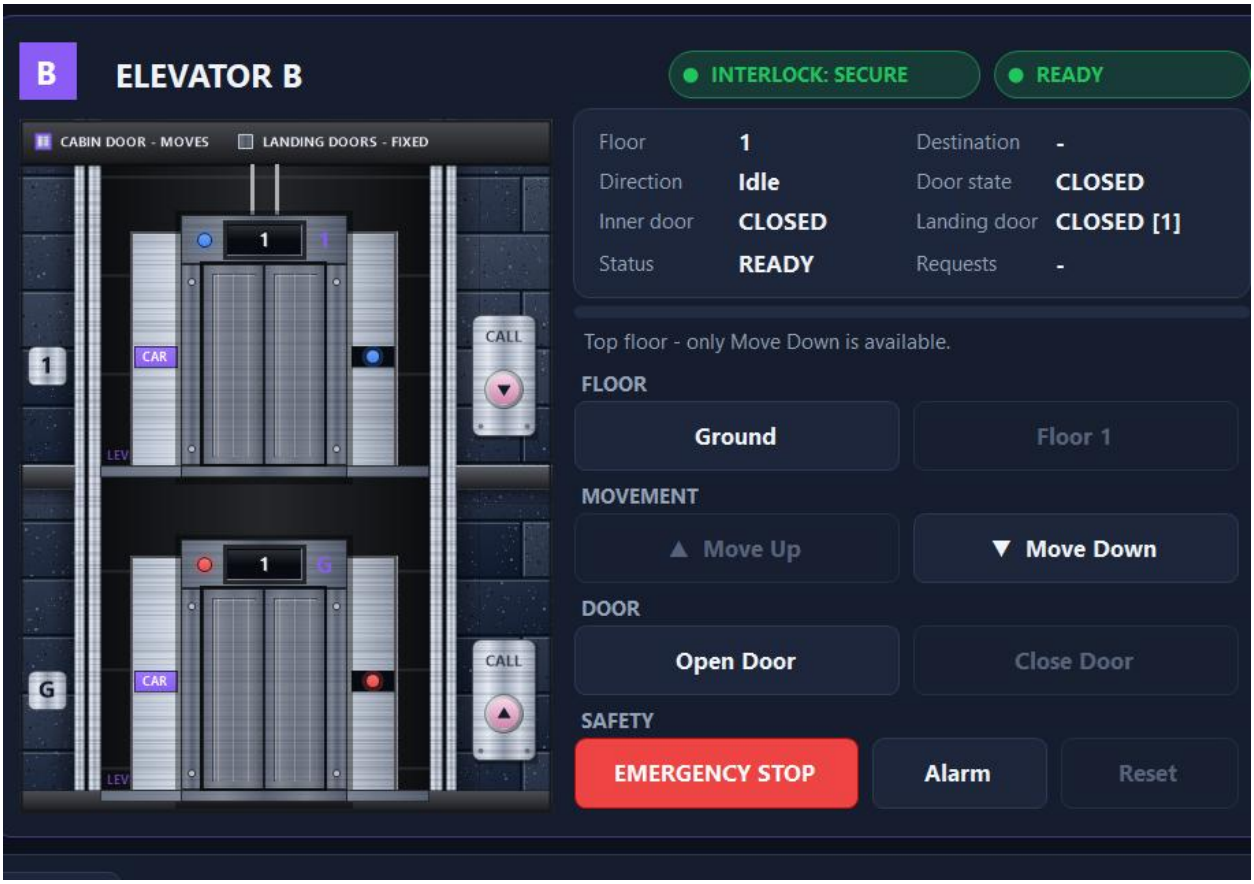
Table 8 Black box test cases

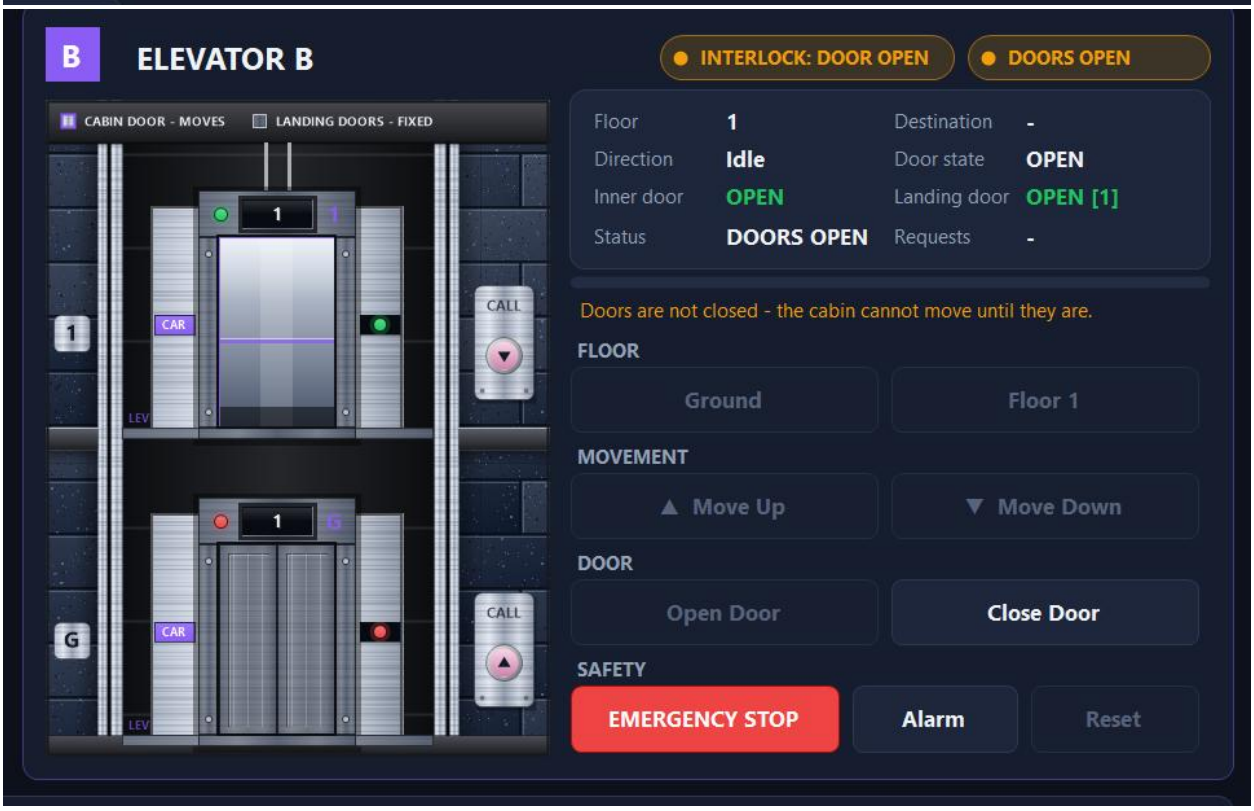
ID	Test case	Input / action	Expected result	Result
BB1	Request upper floor	Elevator at G, press floor 1	REQUESTED FLOOR logged; doors lock; cabin goes up; ARRIVED AT FLOOR 1; doors open together	To run
BB2	Request current floor	Press the floor where the cabin stands	Doors open, no movement	To run
BB3	DOWN at ground (boundary)	Press DOWN at G	Button disabled; nothing happens	To run
BB4	UP at top (boundary)	Press UP at floor 1	Button disabled; nothing happens	To run
BB5	Door auto-close	Wait after DOORS OPEN	Doors close after 4 s (1.5 s if another floor requested)	To run
BB6	Manual open	Press OPEN DOOR at a floor	Inner and that floor's outer door open	To run
BB7	Manual close	Press CLOSE DOOR with doors open	Doors close, status READY	To run
BB8	Movement with open door	Press floor while doors are open	Floor buttons disabled, hint shown	To run
BB9	Emergency stop while moving	Press EMERGENCY STOP	Cabin and doors freeze, requests dropped, only ALARM and RESET enabled	To run
BB10	Reset	Press RESET after emergency	System returns to READY, controls enabled	To run
BB11	Alarm	Press ALARM	Flashing lamp, tone, log entry; stops after 6 s or on RESET	To run

ID	Test case	Input / action	Expected result	Result
BB12	Two elevators	Request floors on both elevators at once	Both move independently with their own logs	To run
BB13	Log display	Click Show Log / Activity Logs	Grid shows ID, Date, Time, Elevator, Floor, Action	To run
BB14	Log filter	Filter by elevator and text ARRIVED	Only matching rows remain	To run
BB15	Clear log	Click Clear Log, confirm	Table emptied; empty-state message	To run
BB16	Database offline	Stop MySQL, start the program	Status shows database offline, one error message, simulation still works	To run
BB17	Window resize	Shrink below about 1260 px	Scroll bars appear, nothing clipped	To run

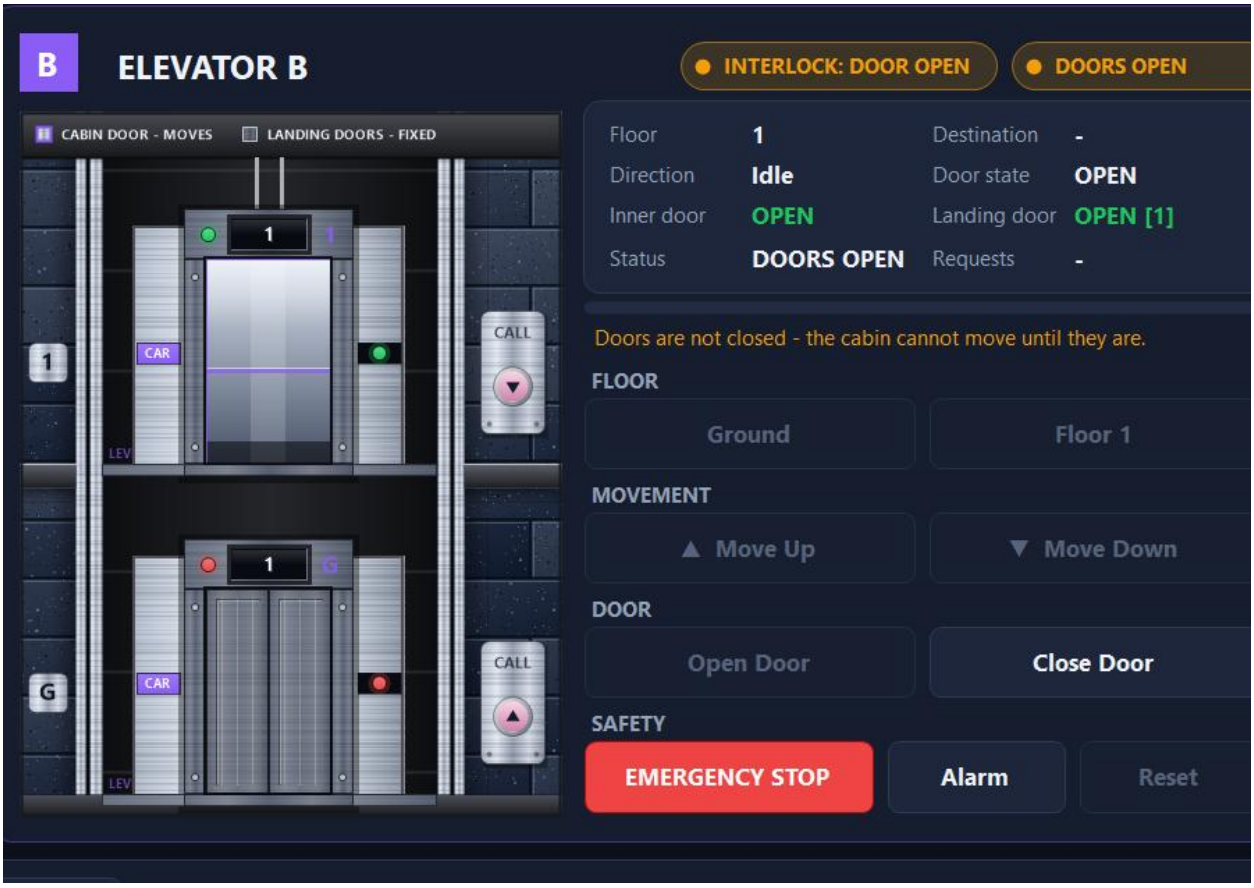
BB1



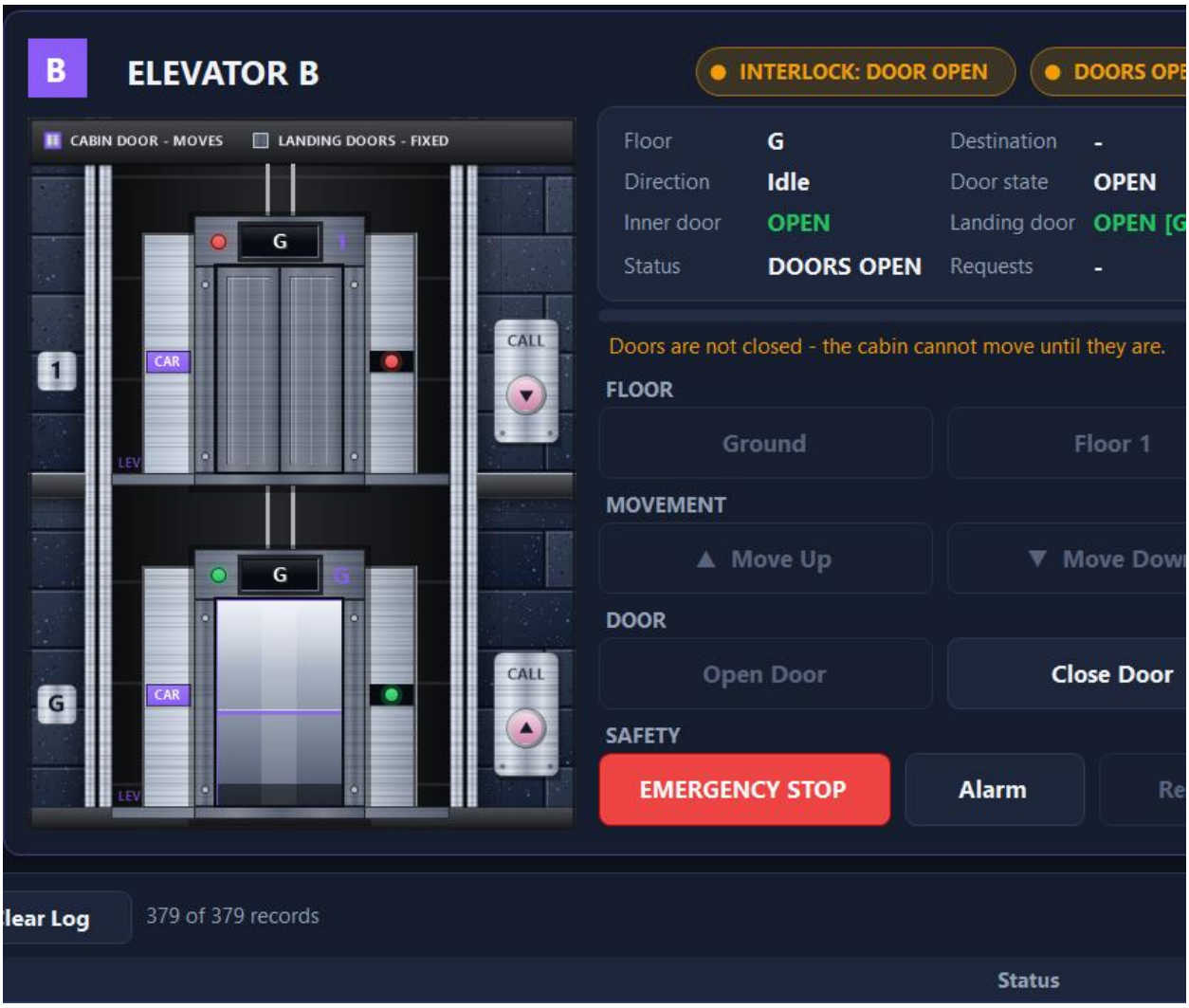

B
ELEVATOR B
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

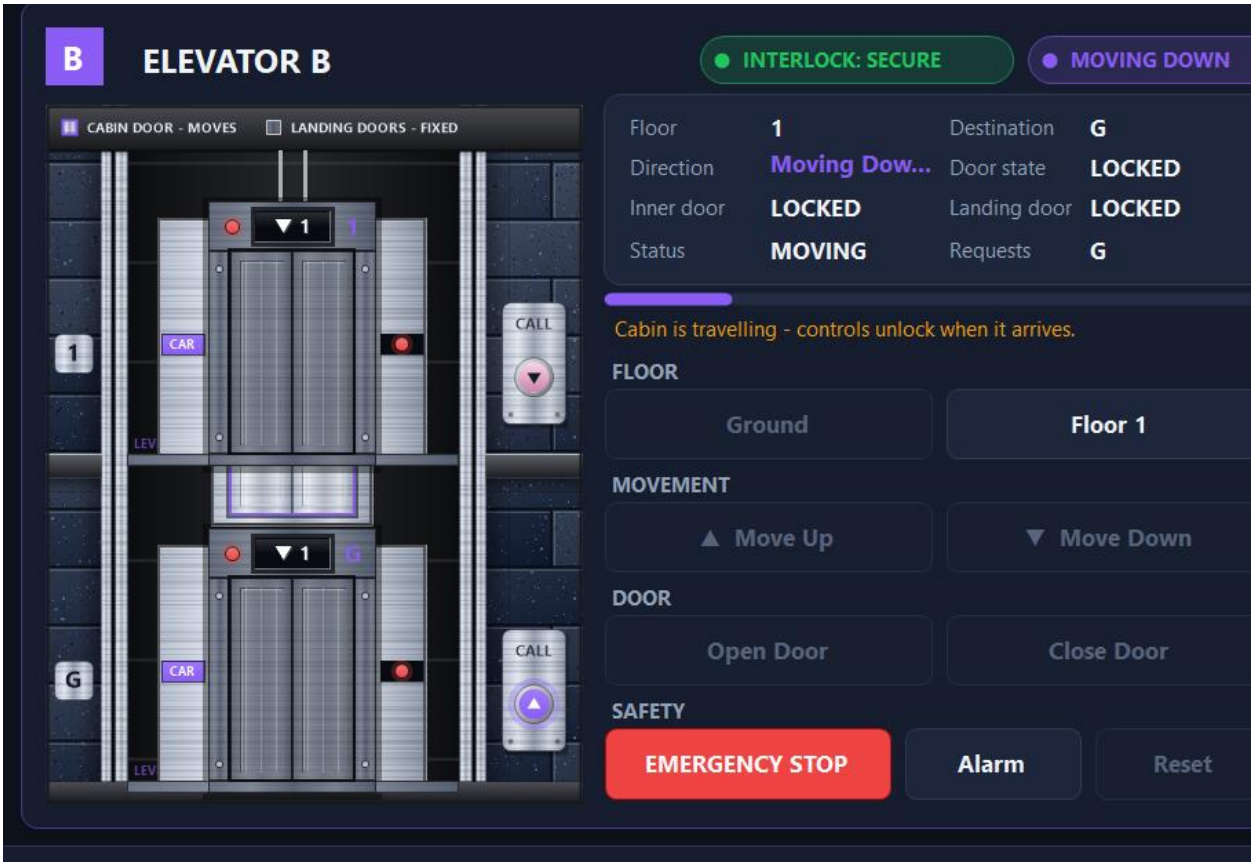

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing door OPEN [1]
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

BB2

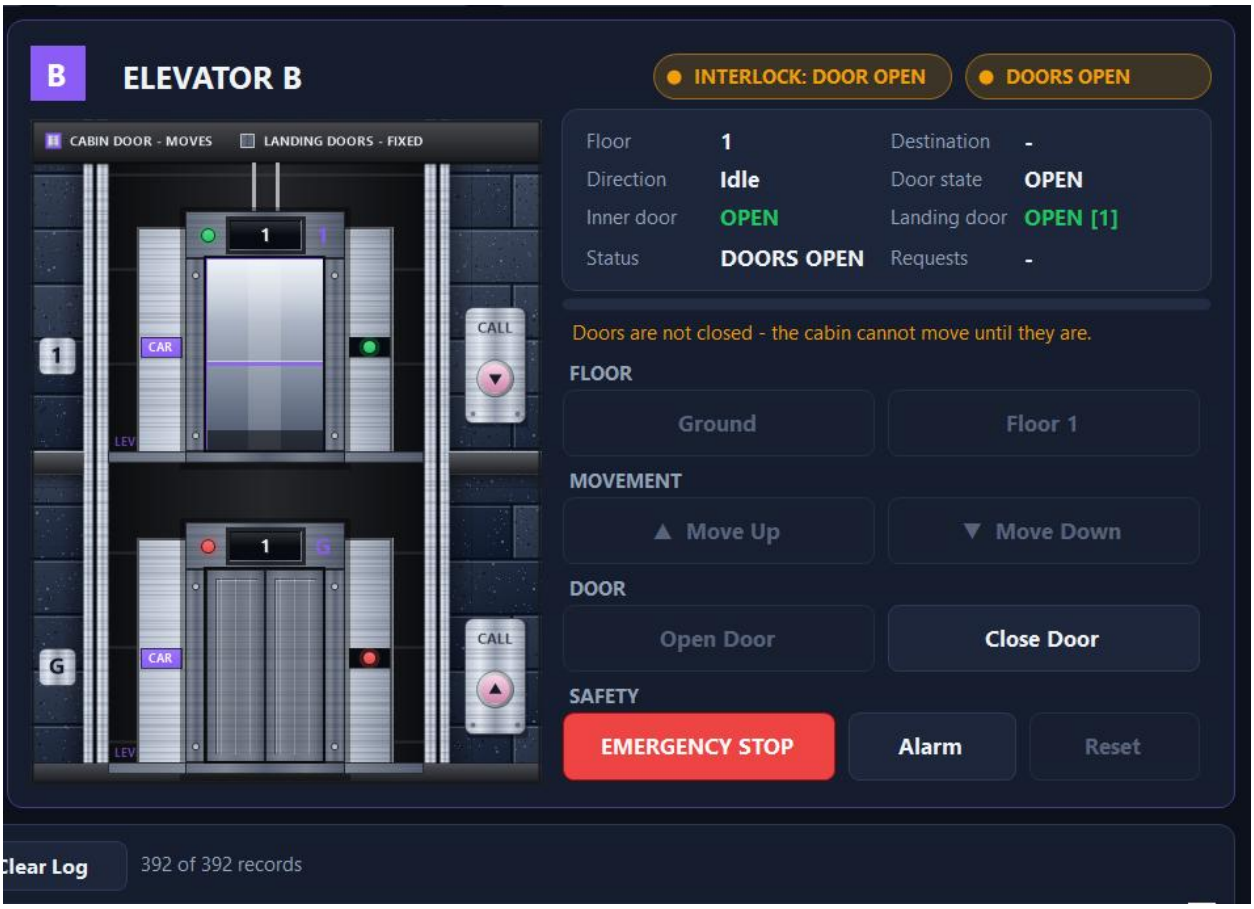

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Direction Idle
Inner door OPEN
Status DOORS OPEN
Destination -
Door state OPEN
Landing door OPEN [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
1
G

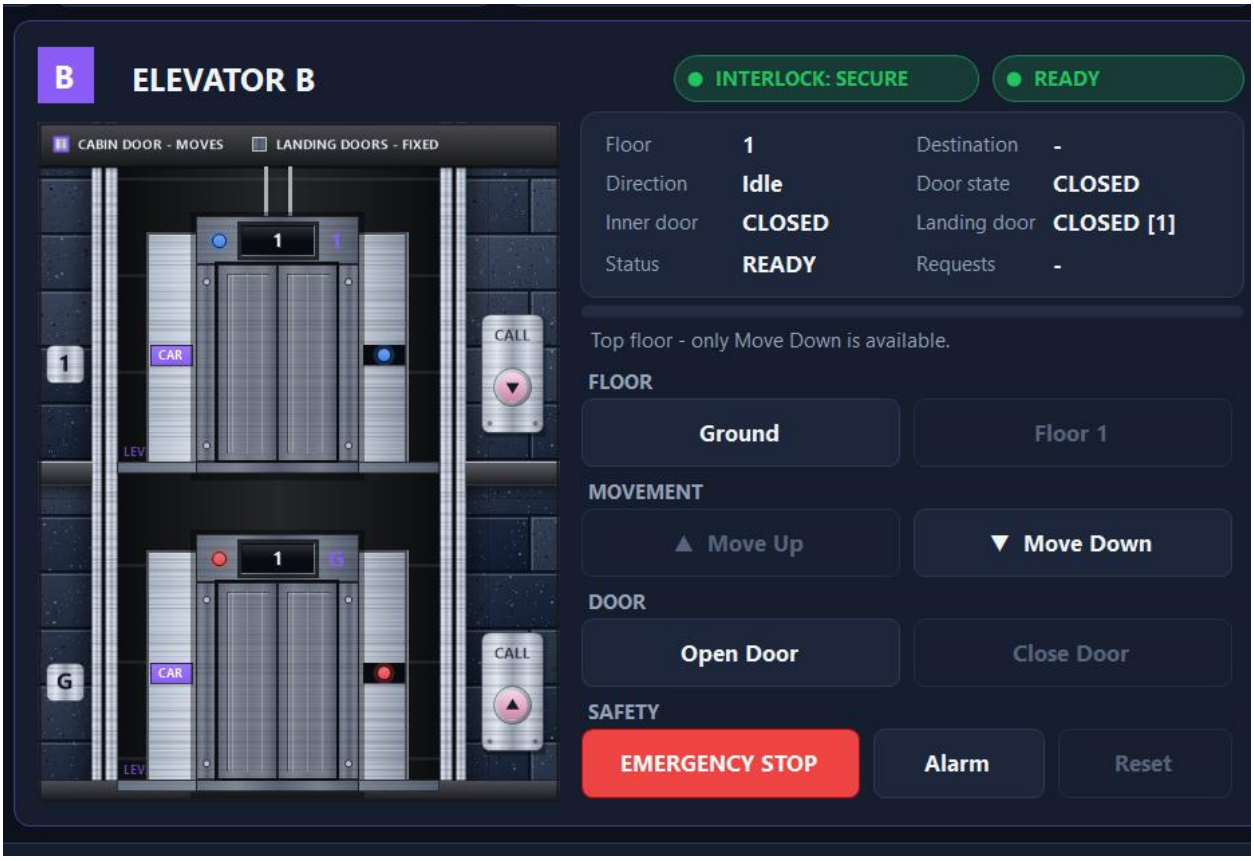
BB3


B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPE
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor G
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing door OPEN [G
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
CALL
CAR
1
G
379 of 379 records
Status

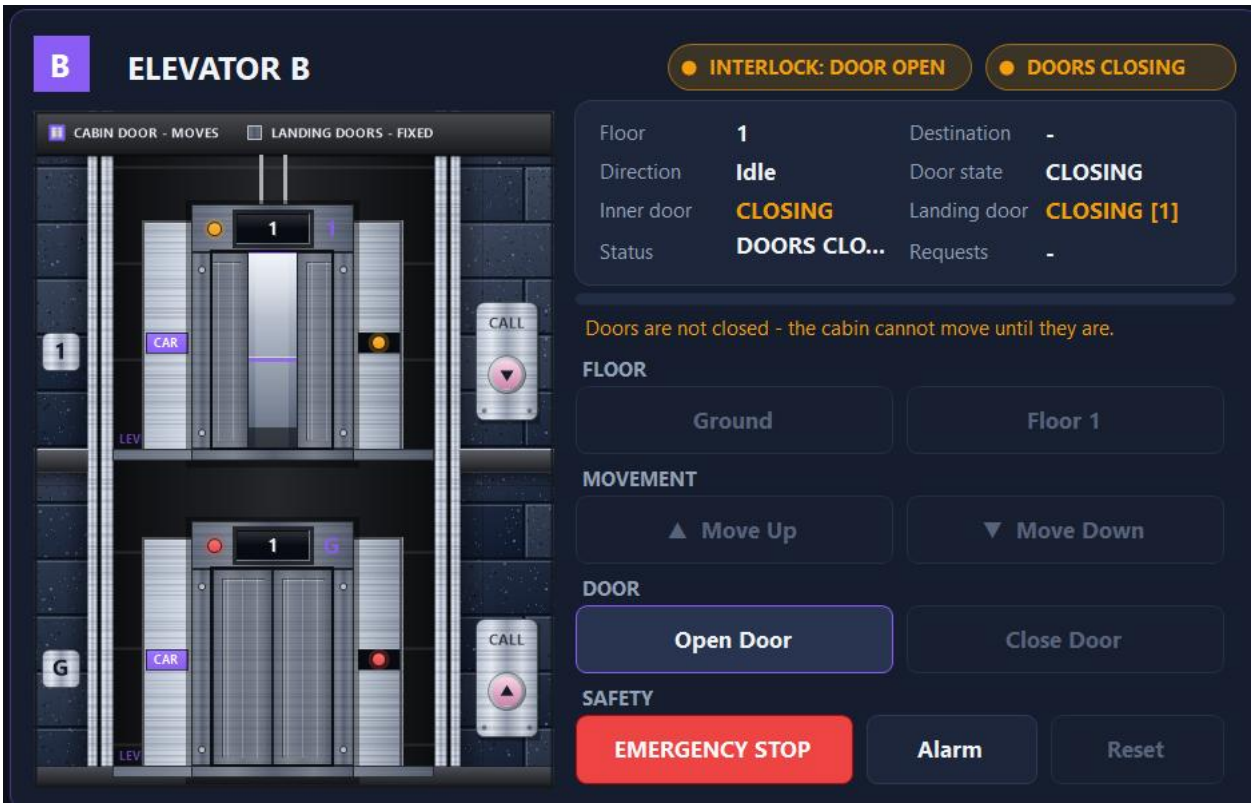

B
ELEVATOR B
INTERLOCK: SECURE
MOVING DOWN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Direction Moving Dow...
Inner door LOCKED
Status MOVING
Destination G
Door state LOCKED
Landing door LOCKED
Requests G
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR

BB4

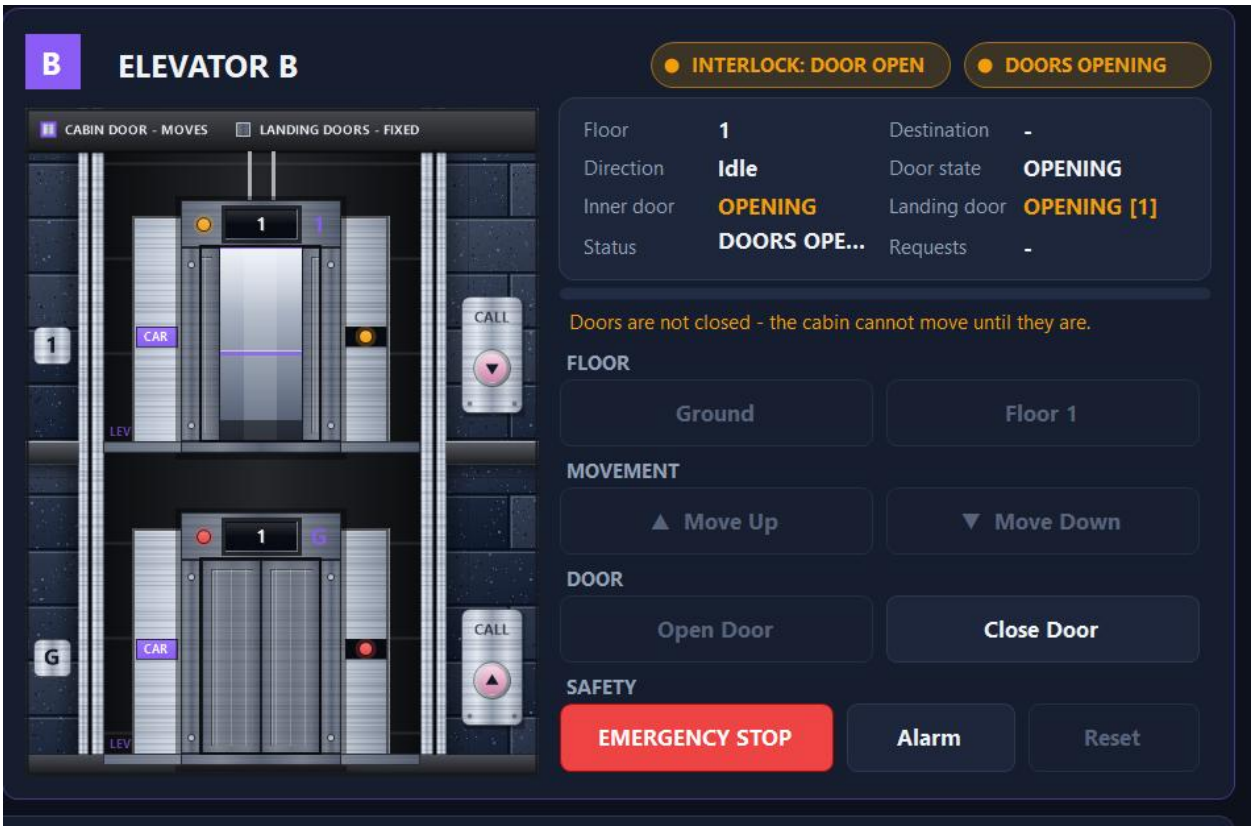

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing door OPEN [1]
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
Clear Log
392 of 392 records


B
ELEVATOR B
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
1
G

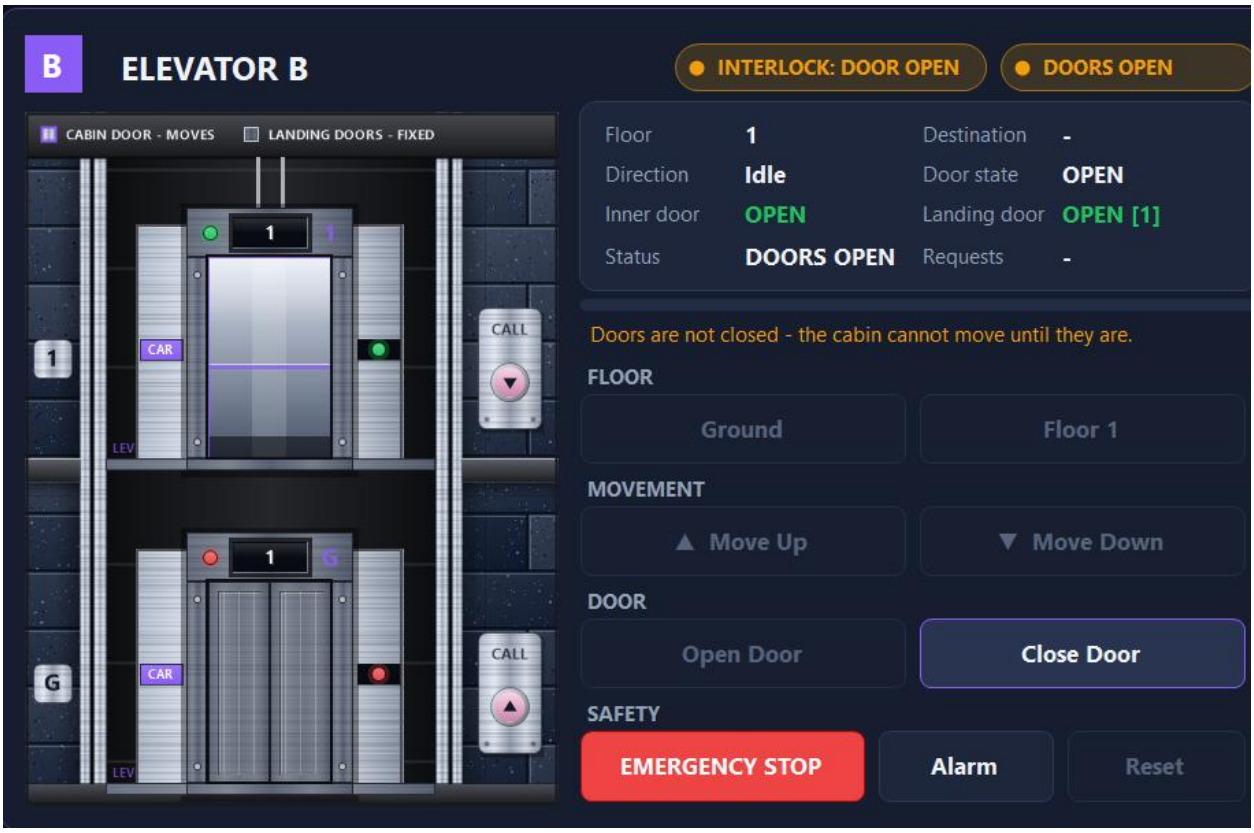
BB5

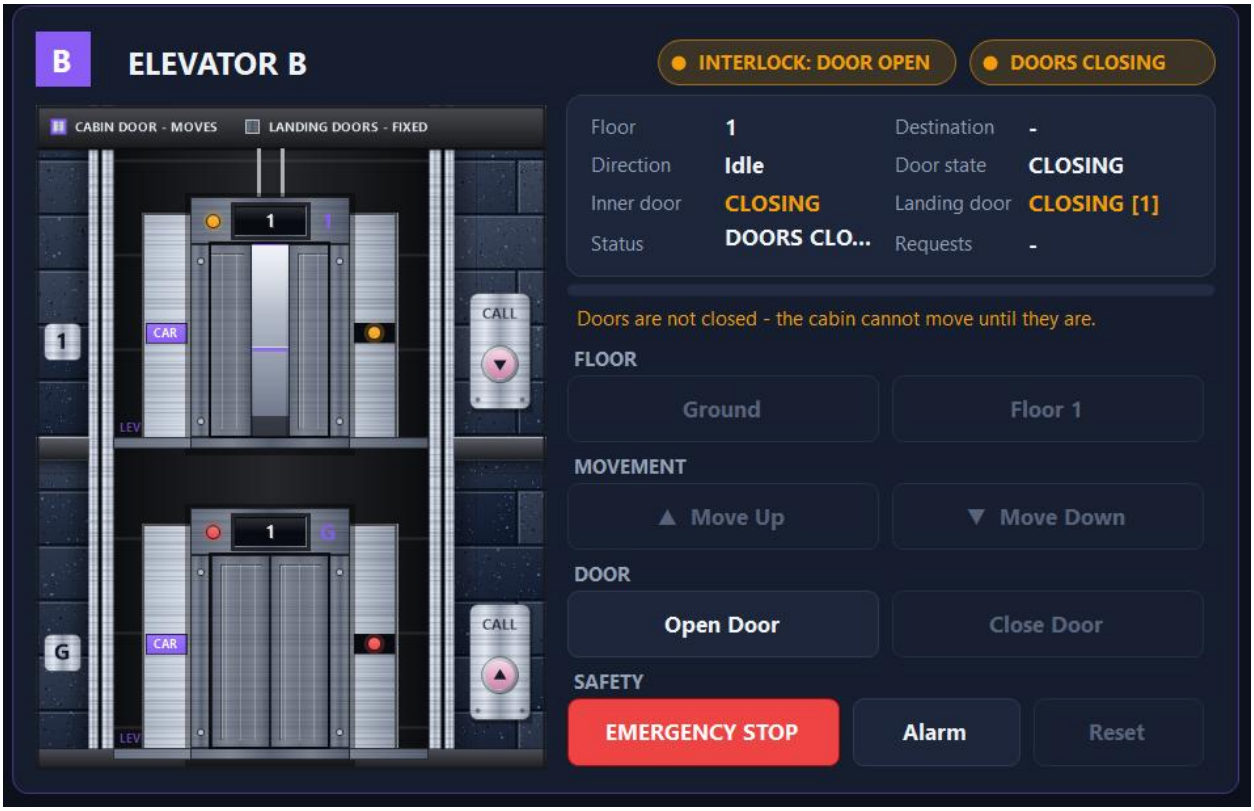

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS CLOSING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSING
Inner door CLOSING
Landing door CLOSING [1]
Status DOORS CLO...
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
LEV

BB6

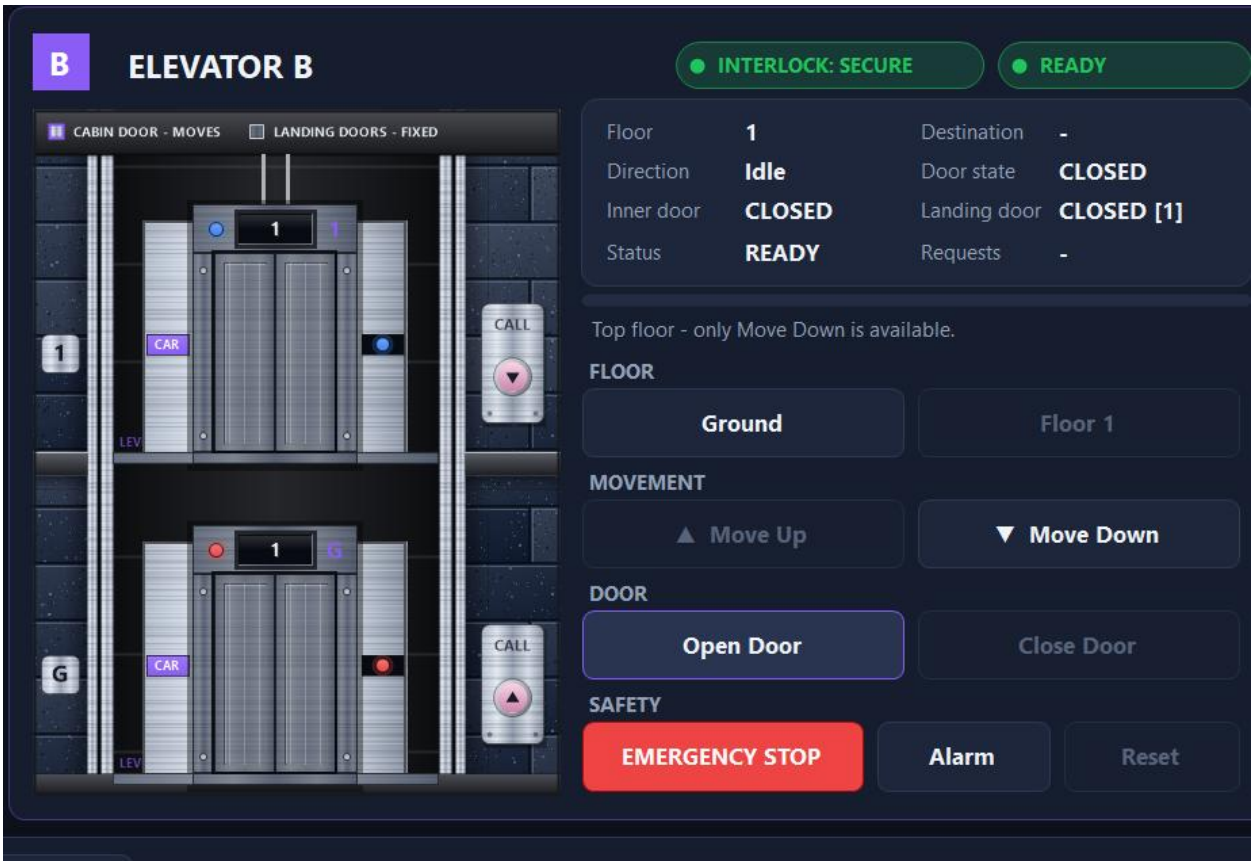

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPENING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state OPENING
Inner door OPENING
Landing door OPENING [1]
Status DOORS OPE...
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
1
G

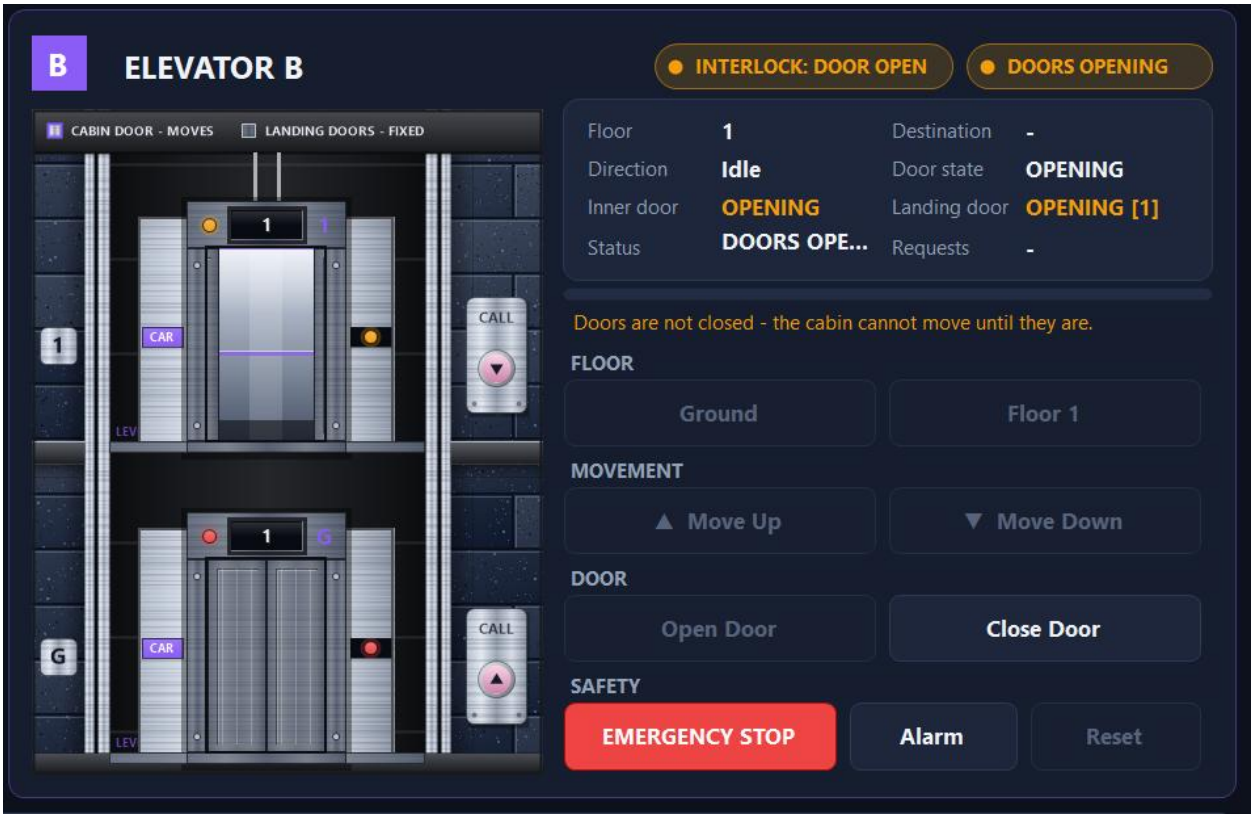
BB7


B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing door OPEN [1]
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
1
G

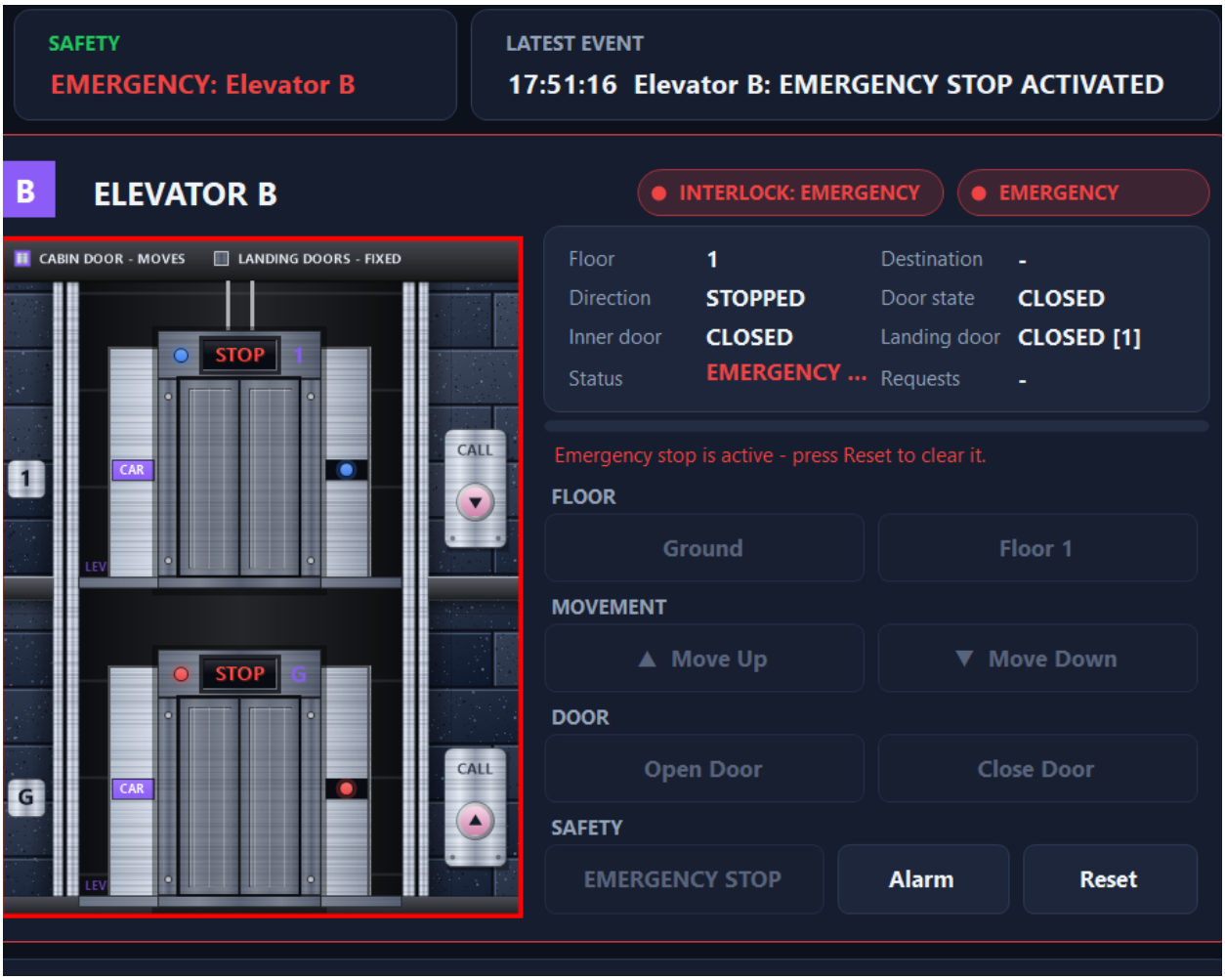

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS CLOSING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Direction Idle
Inner door CLOSING
Status DOORS CLO...
Destination -
Door state CLOSING
Landing door CLOSING [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

BB8

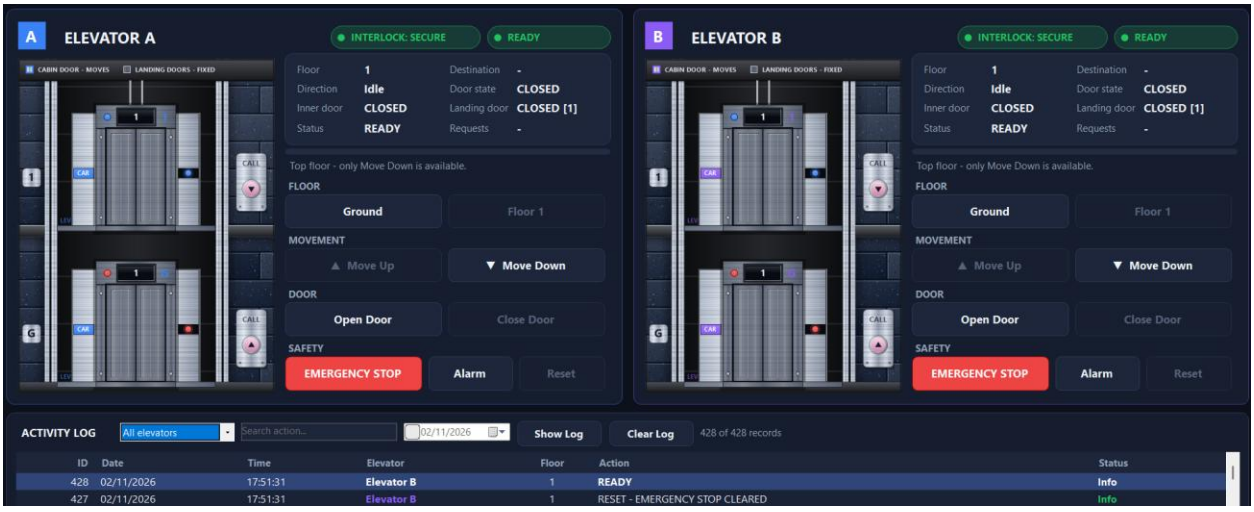

B
ELEVATOR B
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
1
G


B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPENING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Direction Idle
Inner door OPENING
Status DOORS OPE...
Destination -
Door state OPENING
Landing door OPENING [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
1
G

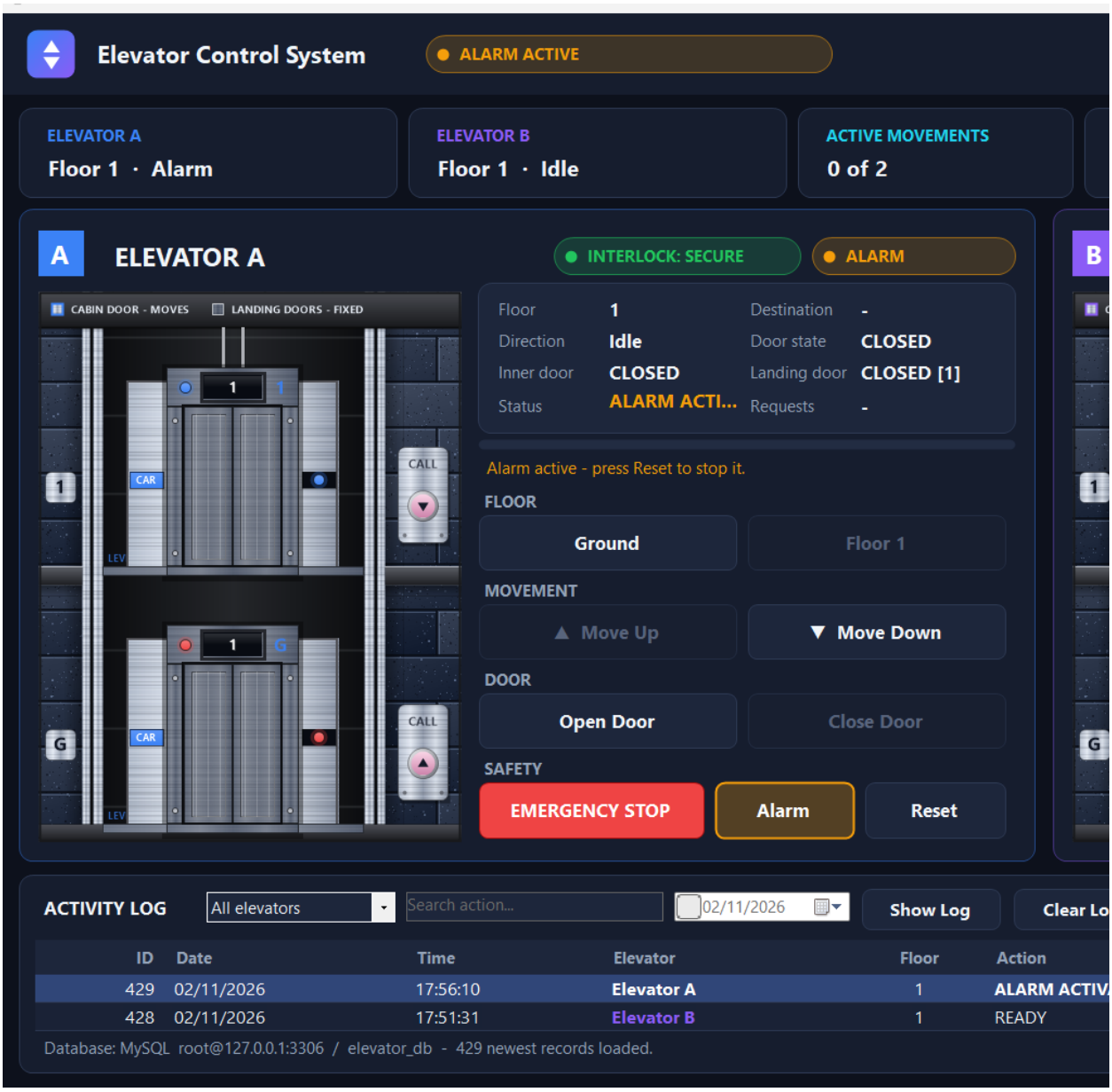
BB9


SAFETY
EMERGENCY: Elevator B
LATEST EVENT
17:51:16 Elevator B: EMERGENCY STOP ACTIVATED
B
ELEVATOR B
INTERLOCK: EMERGENCY
EMERGENCY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
STOP
CAR
CALL
STOP
CAR
CALL
Floor 1
Direction STOPPED
Inner door CLOSED
Status EMERGENCY ...
Destination -
Door state CLOSED
Landing door CLOSED [1]
Requests -
Emergency stop is active - press Reset to clear it.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

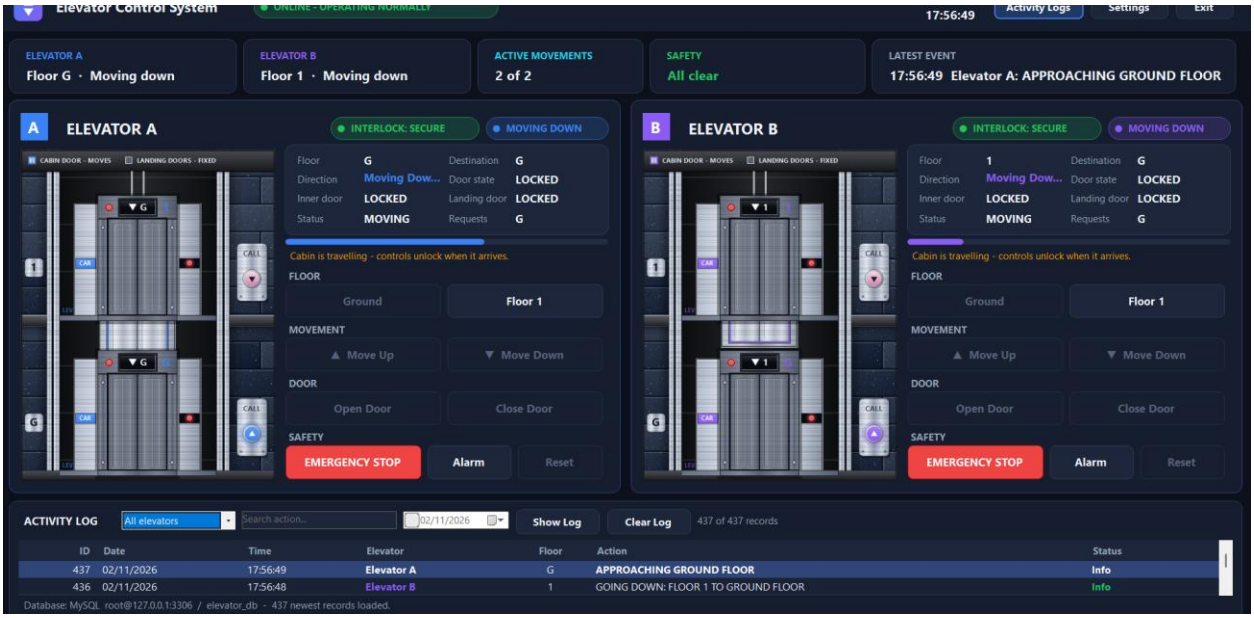
BB10


A
ELEVATOR A
INTERLOCK: SECURE
READY
B
ELEVATOR B
INTERLOCK: SECURE
READY
ACTIVITY LOG
Show Log
Clear Log
ID Date Time Elevator Floor Action Status
428 02/11/2026 17:51:31 Elevator B 1 READY Info
427 02/11/2026 17:51:31 Elevator B 1 RESET - EMERGENCY STOP CLEARED Info

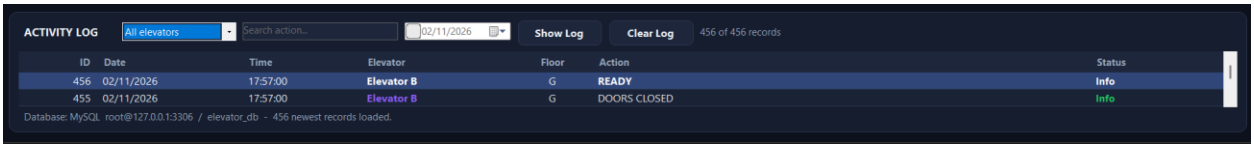
BB11


Elevator Control System
ALARM ACTIVE
ELEVATOR A
Floor 1 · Alarm
ELEVATOR B
Floor 1 · Idle
ACTIVE MOVEMENTS
0 of 2
A
ELEVATOR A
INTERLOCK: SECURE
ALARM
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status ALARM ACTI...
Requests -
Alarm active - press Reset to stop it.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Lo
ID Date Time Elevator Floor Action
429 02/11/2026 17:56:10 Elevator A 1 ALARM ACTIV
428 02/11/2026 17:51:31 Elevator B 1 READY
Database: MySQL root@127.0.0.1:3306 / elevator_db - 429 newest records loaded.

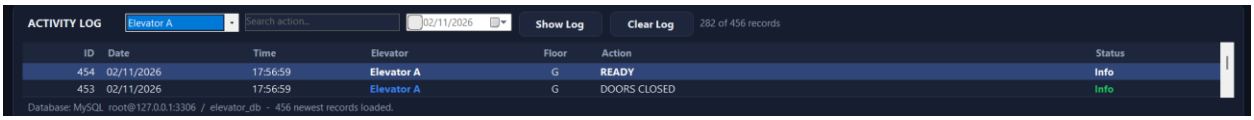
BB12



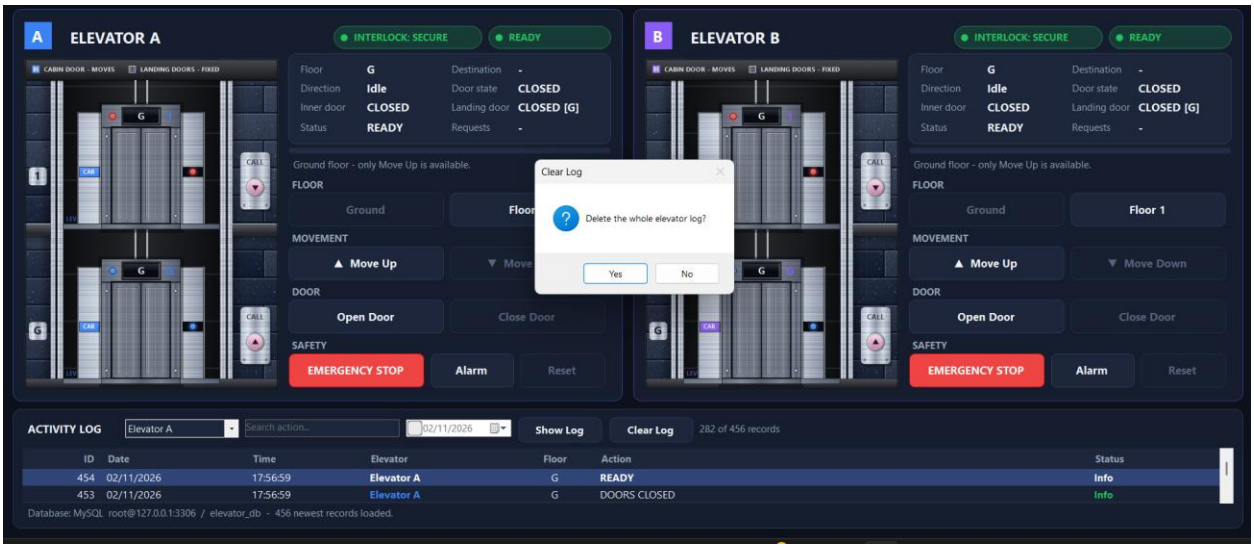
BB13

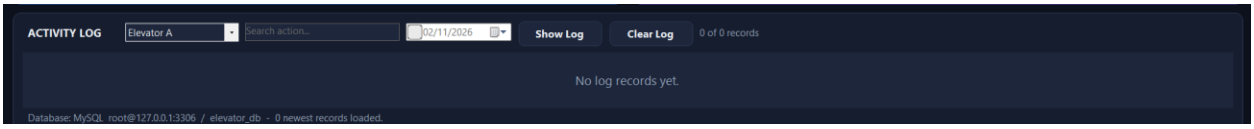


BB14

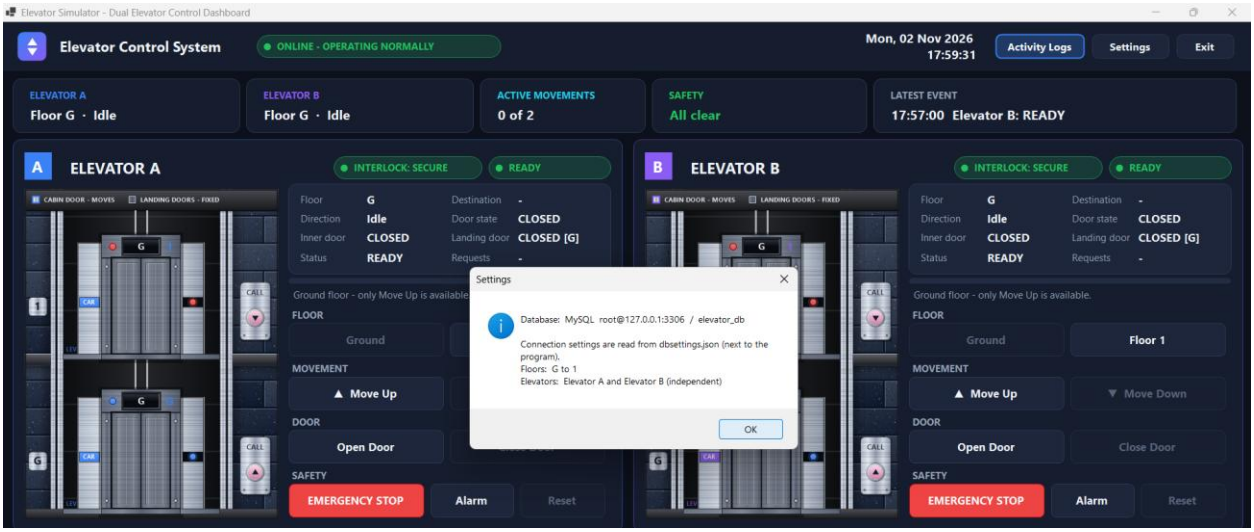


BB15

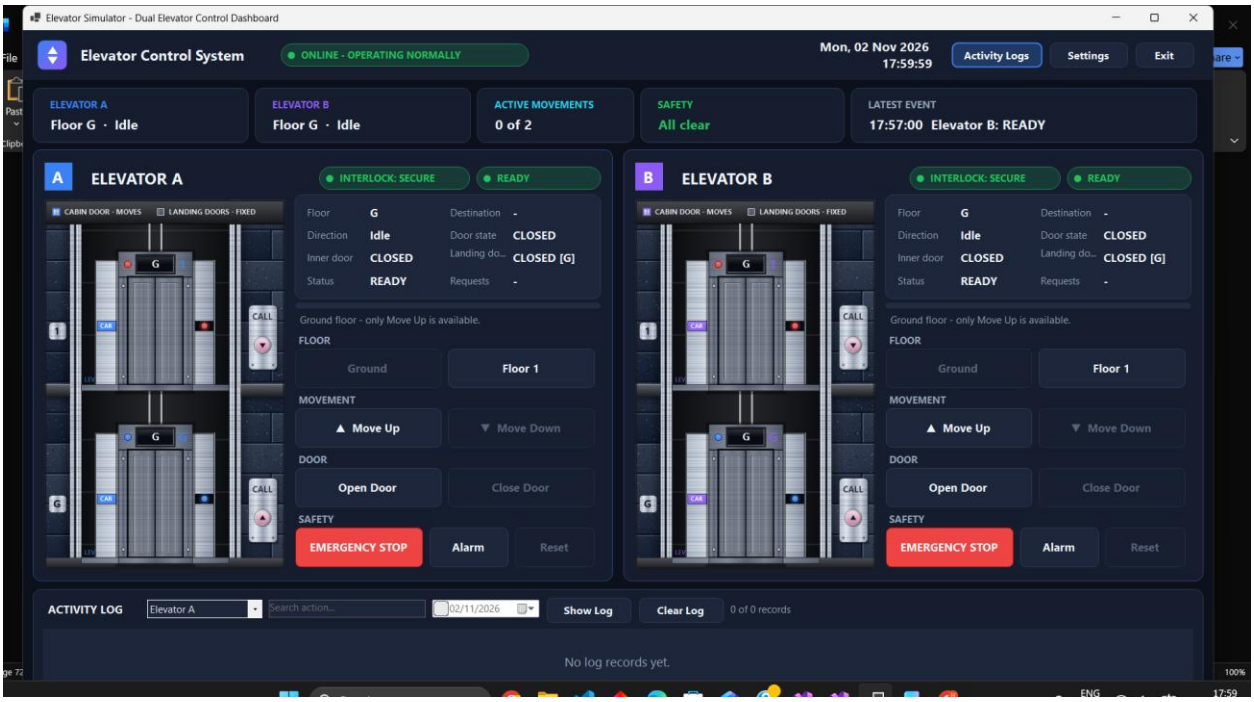

ELEVATOR A
INTERLOCK: SECURE
READY
ELEVATOR B
Clear Log
Delete the whole elevator log?
Yes
No
Move Up
Move Down
Open Door
Close Door
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
Show Log
Clear Log
282 of 456 records
454 02/11/2026 17:56:59 Elevator A G READY Info
453 02/11/2026 17:56:59 Elevator A G DOORS CLOSED Info


ACTIVITY LOG
Show Log
Clear Log
0 of 0 records
No log records yet.

BB16


Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
17:59:31
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Idle
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
17:57:00 Elevator B: READY
Settings
Database: MySQL root@127.0.0.1:3306 / elevator_db
Connection settings are read from dbsettings.json (next to the program).
Floors: G to 1
Elevators: Elevator A and Elevator B (independent)
OK

BB17



10.2 White Box Testing

Techniques used: statement coverage, branch (decision) coverage and path testing on the key methods. The flow diagram below shows the decisions inside MovingState.Tick.

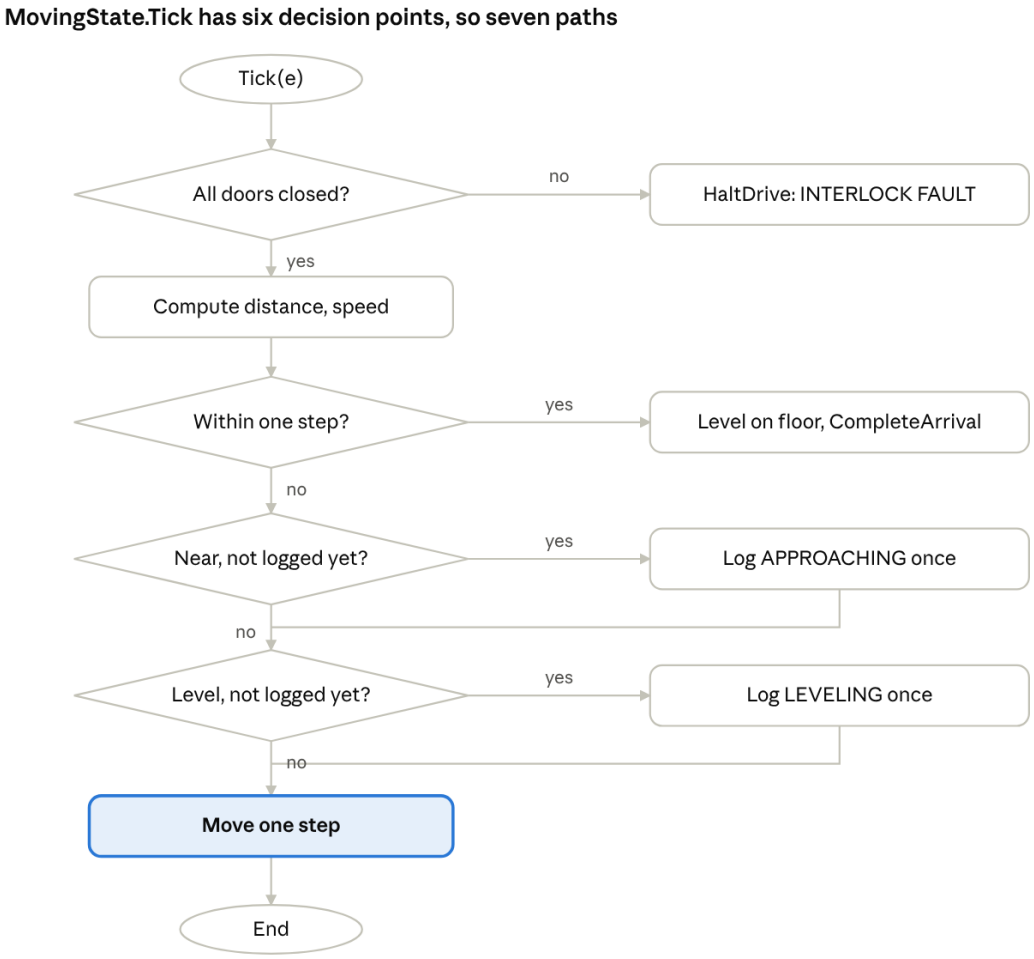


Figure 49 White box flow diagram: MovingState.Tick

Table 9 White box test cases

ID	Method	Branch / path covered	Input condition	Expected result	Result
WB1	MovingState.Tick	Interlock true branch	Doors not closed while moving	HaltDrive logs INTERLOCK FAULT, emergency state	To run
WB2	MovingState.Tick	Interlock false branch	All doors closed	Movement continues	To run
WB3	MovingState.Tick	dist <= speed (arrival)	Cabin within one step of target	Position set to target, CompleteArrival called	To run

ID	Method	Branch / path covered	Input condition	Expected result	Result
WB4	MovingState.Tick	Approach announcement	dist <= 0.50 and not yet logged	APPROACHING logged once only	To run
WB5	MovingState.Tick	Levelling announcement	dist <= 0.08 and not yet logged	LEVELING logged once only	To run
WB6	MovingState.Tick	Speed profile	Near start and near target	Speed ramps between 30% and 100% of MaxSpeed	To run
WB7	MovingState.OnEnter	Direction up and down	Target above, then below	Logs GOING UP, then GOING DOWN	To run
WB8	Elevator.CanMove	Each operand false	Emergency, moving, door open	Returns false; all true returns true	To run
WB9	Door.IsSecured	State and openness combinations	Closed or Locked with Openness 0, others	True only when Openness <= 0 and state is Closed or Locked	To run
WB10	ElevatorController.GoUp / GoDown	Boundary branches	At top, at bottom	No request created at the limit	To run
WB11	IdleState.Tick	Dispatcher call	Requests pending, doors closed	IdleTick serves the next request	To run
WB12	EmergencyState.Tick	Empty body	Tick during emergency	No change in position or doors	To run
WB13	LogRepository.Writer_DoWork	Insert success and exception branch	Database up, then down	Rows written; on failure queue cleared and error reported	To run
WB14	LogRepository.Flush	Loop and break on error	Queue with entries, database down	Stops at first failure without crashing	To run

Statement and branch coverage is achieved when each WB case above has run at least once; cyclomatic complexity of MovingState.Tick is 7 (six decision points: four if statements and two && operators, plus one), so seven independent paths need testing.

WB1

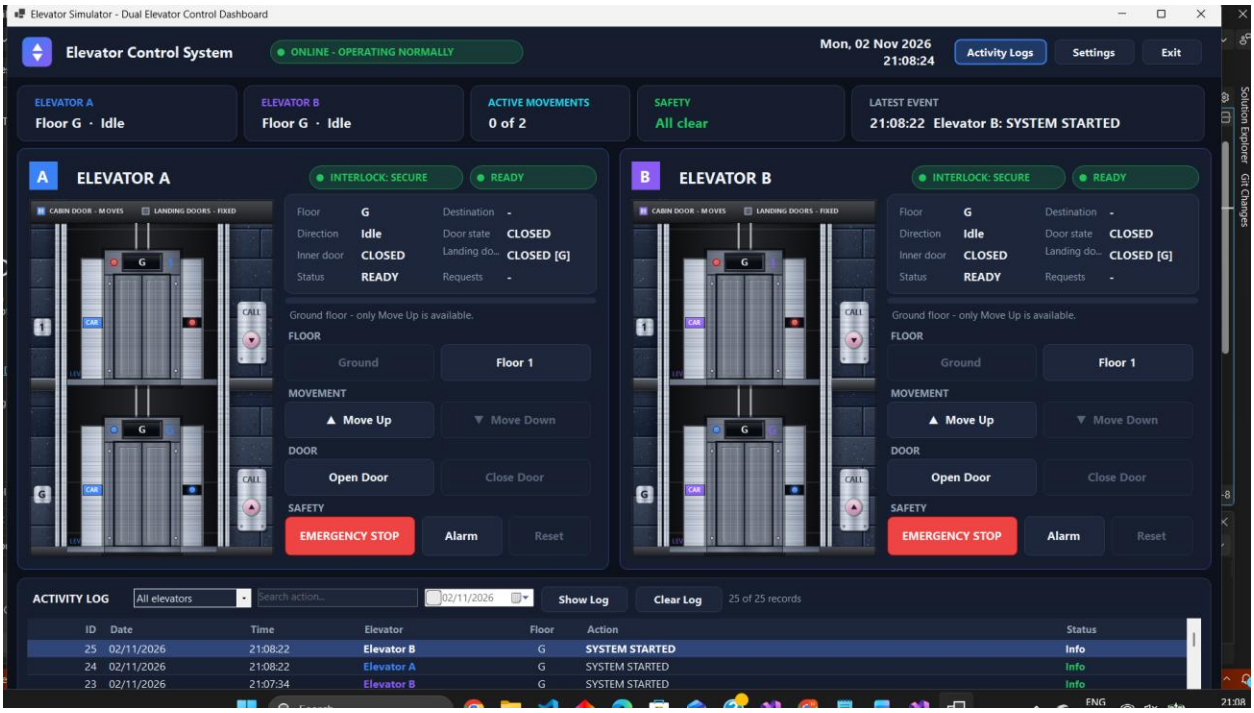
```
39          {
40              // safety interlock: the cabin must never travel with a door that is not closed
41              if (!e.AllDoorsClosed)
42              {
43                  e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
44                  return;
45              }
46
```

Elevator_with_time Elevator_with_time.Program

```
1   namespace Elevator_with_time
2   {
        0 references
3       internal static class Program
4       {
5           [STAThread]
            0 references
6           static void Main()
7           {
8               ApplicationConfiguration.Initialize();   ≤ 1ms elapsed
9               Application.Run(new Form1());
10          }
11      }
12  }
13
```

Elevator_with_time Elevator_with_time.Program Main()

```
1   namespace Elevator_with_time
2   {
        0 references
3       internal static class Program
4       {
5           [STAThread]
            0 references
6           static void Main()
7           {
8               ApplicationConfiguration.Initialize();
9               Application.Run(new Form1());   ≤ 7ms elapsed
10          }
11      }
12  }
13
```

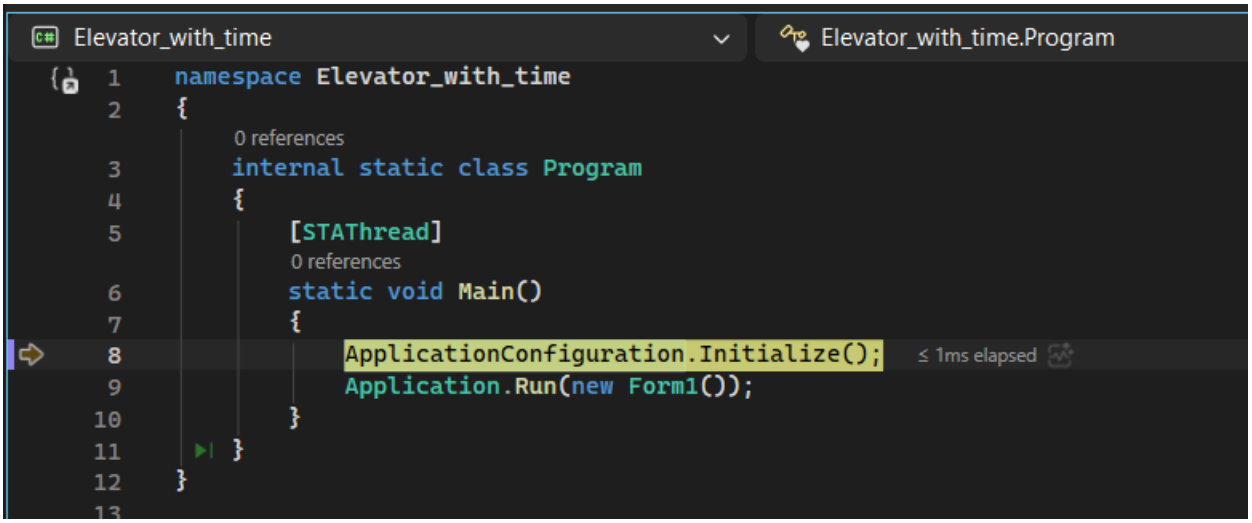


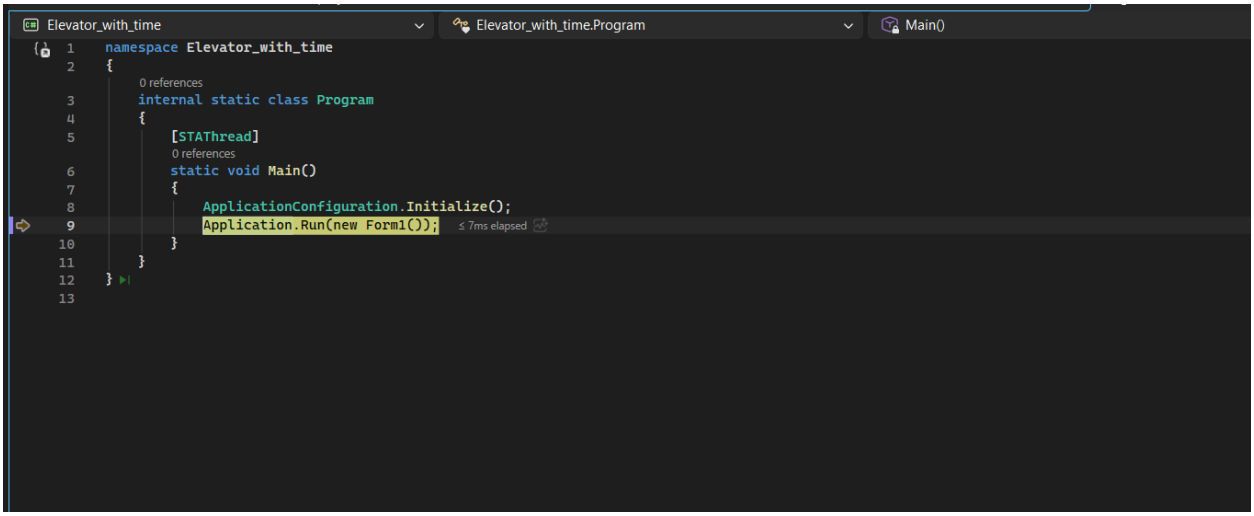

```
41      public void Tick(Elevator e)    e: {Elevator_with_time.Elevator}
42      {
43          // safety interlock: the cabin must never travel with a door that is not closed
44          if (!e.AllDoorsClosed)
45          {
46              e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
47              return;
48          }
49
50          double diff = e.TargetFloor - e.Position;
51          double dist = Math.Abs(diff);
52
```

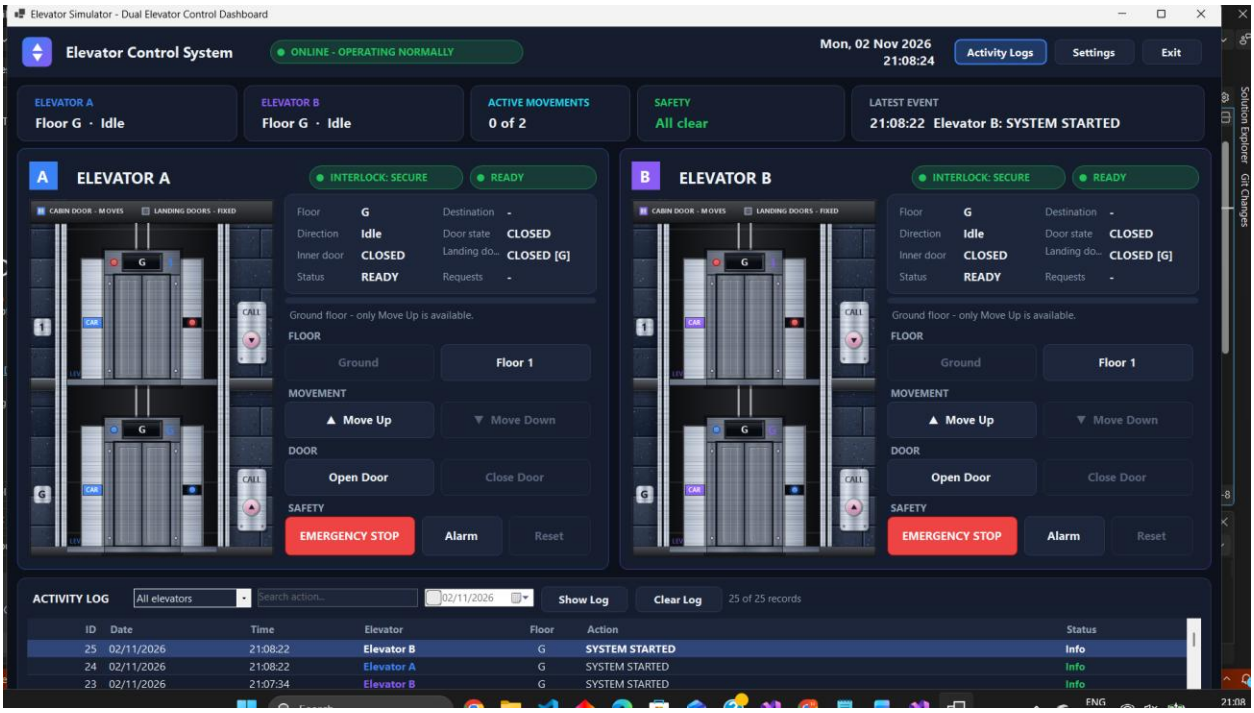
WB2

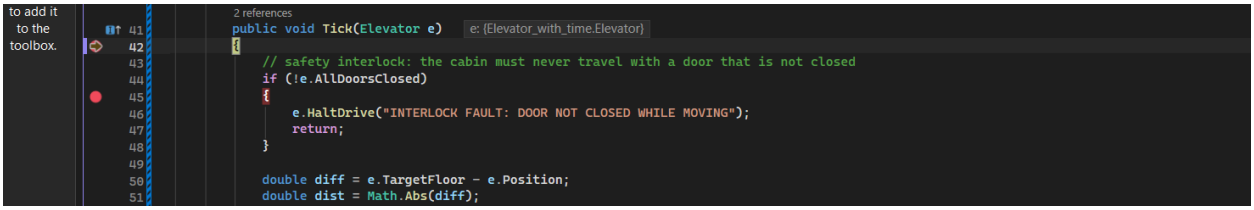
```
39      {
40          // safety interlock: the cabin must never travel with a door that is not closed
41          if (!e.AllDoorsClosed)
42          {
43              e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
44              return;
45          }
46
47          double diff = e.TargetFloor - e.Position;
48          double dist = Math.Abs(diff);
```

```
6       static void Main()
7       {
8           ApplicationConfiguration.Initialize();
9           Application.Run(new Form1());
10      }
11  }
12  }
```

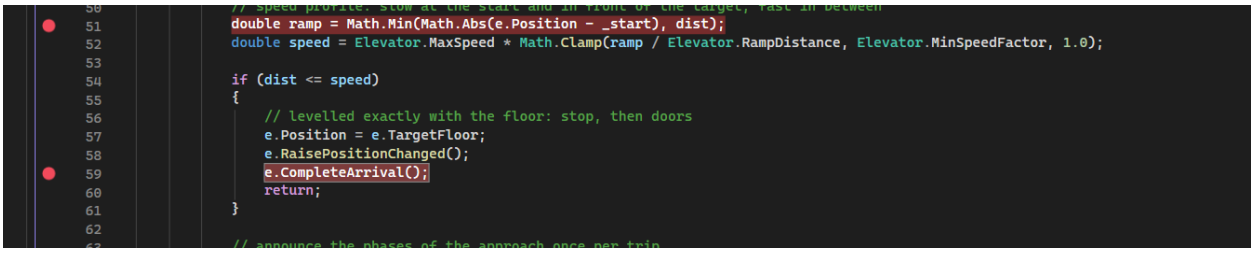

Elevator_with_time
Elevator_with_time.Program
namespace Elevator_with_time
{
0 references
internal static class Program
{
[STAThread]
0 references
static void Main()
{
ApplicationConfiguration.Initialize();
≤ 1ms elapsed
Application.Run(new Form1());
}
}
}

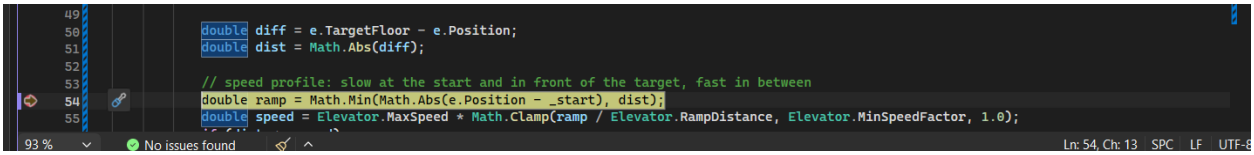

Elevator_with_time
Elevator_with_time.Program
Main()
namespace Elevator_with_time
{
0 references
internal static class Program
{
[STAThread]
0 references
static void Main()
{
ApplicationConfiguration.Initialize();
Application.Run(new Form1());
≤ 7ms elapsed
}
}
}


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
21:08:24
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Idle
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
21:08:22 Elevator B: SYSTEM STARTED
ELEVATOR A
INTERLOCK: SECURE
READY
ELEVATOR B
INTERLOCK: SECURE
READY
Floor G
Direction Idle
Inner door CLOSED
Status READY
Destination -
Door state CLOSED
Landing do... CLOSED [G]
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
25 of 25 records
ID Date Time Elevator Floor Action Status
25 02/11/2026 21:08:22 Elevator B G SYSTEM STARTED Info
24 02/11/2026 21:08:22 Elevator A G SYSTEM STARTED Info
23 02/11/2026 21:07:34 Elevator B G SYSTEM STARTED Info


to add it to the toolbox.
2 references
public void Tick(Elevator e)
e: {Elevator_with_time.Elevator}
{
// safety interlock: the cabin must never travel with a door that is not closed
if (!e.AllDoorsClosed)
{
e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
return;
}
double diff = e.TargetFloor - e.Position;
double dist = Math.Abs(diff);

WB3


double ramp = Math.Min(Math.Abs(e.Position - _start), dist);
double speed = Elevator.MaxSpeed * Math.Clamp(ramp / Elevator.RampDistance, Elevator.MinSpeedFactor, 1.0);
if (dist <= speed)
{
// levelled exactly with the floor: stop, then doors
e.Position = e.TargetFloor;
e.RaisePositionChanged();
e.CompleteArrival();
return;
}


double diff = e.TargetFloor - e.Position;
double dist = Math.Abs(diff);
// speed profile: slow at the start and in front of the target, fast in between
double ramp = Math.Min(Math.Abs(e.Position - _start), dist);
double speed = Elevator.MaxSpeed * Math.Clamp(ramp / Elevator.RampDistance, Elevator.MinSpeedFactor, 1.0);
93 %
No issues found
Ln: 54, Ch: 13
SPC
LF
UTF-8

WB4

```
64          if (!e.ApproachLogged && dist <= Elevator.ApproachDistance)
65          {
66              e.ApproachLogged = true;
67              e.Notify($"APPROACHING {Elevator.FloorTag(e.TargetFloor)}");
68          }
```

```
64
65      // announce the phases of the approach once per trip
66      if (!e.ApproachLogged && dist <= Elevator.ApproachDistance)   → false
67      {
93 %    No issues found                                   Ln: 66, Ch: 13   OVR   SPC   LF
Error List
Entire Solution   0 Errors   0 of 1 Warning   0 of 36 Messages   Build + IntelliSense   Search Error List
Searcl   Code   Description   Project   File   Li...   Suppression State
Call Stack   Breakpoints   Exception Settings   Command Window   Immediate Window   Output   Error List
0 / 0   2   main   Down
Search   ENG US
```

WB5

```
68          }
69          if (!e.LevelLogged && dist <= Elevator.LevelDistance)
70          {
71              e.LevelLogged = true;
72              e.Notify("LEVELING");
73          }
74
75          double before = e.Position;
76          e.Position += Math.Sign(diff) * speed;
77          e.AnnouncePassing(before, e.Position);
```

6

```
49
50          double diff = e.TargetFloor - e.Position;
51          double dist = Math.Abs(diff);
52
53          // speed profile: slow at the start and in front of the target, fast in between
54          double ramp = Math.Min(Math.Abs(e.Position - _start), dist);
55          double speed = Elevator.MaxSpeed * Math.Clamp(ramp / Elevator.RampDistance, Elevator.MinSpeedFactor, 1.0);
56          if (dist <= speed)
57          {
58              // levelled exactly with the floor: stop, then doors
59              e.Position = e.TargetFloor;
```

```
Drag an item onto this text to add it to the toolbox.
48          }
49
50          double diff = e.TargetFloor - e.Position;
51          double dist = Math.Abs(diff);
52
53          // speed profile: slow at the start and in front of the target, fast in between
54          double ramp = Math.Min(Math.Abs(e.Position - _start), dist);
55          double speed = Elevator.MaxSpeed * Math.Clamp(ramp / Elevator.RampDistance, Elevator.MinSpeedFactor, 1.0);
56          if (dist <= speed)
57          {
58              // levelled exactly with the floor: stop, then doors
59              TargetFloor;
namespace Elevator_with_time
```

WB7

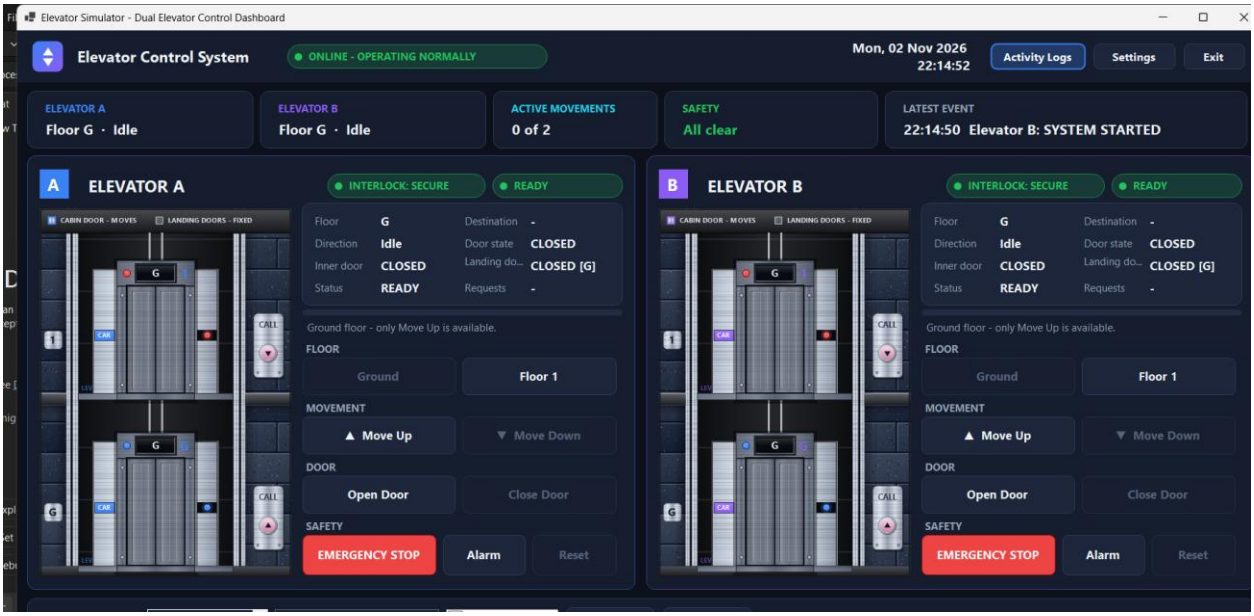
```
2 references
public void OnEnter(Elevator e)
{
    _start = e.Position;
    e.Direction = Math.Sign(e.TargetFloor - e.Position);
    e.LastDirection = e.Direction;
    e.StatusText = "MOVING";

    string fro   string Elevator.StatusText { get; internal set; }
    string to    Describe
    e.Notify(e.Direction > 0 ? $"GOING UP: {from} TO {to}" : $"GOING DOWN: {from} TO {to}");
}

2 references
```

```
{
    _start = e.Position;
    e.Direction = Math.Sign(e.TargetFloor - e.Position);
    e.LastDirection = e.Direction;
    e.StatusText = "MOVING";

    string from = Elevator.FloorTag(e.CurrentFloor);
    string to = Elevator.FloorTag(e.TargetFloor);
    e.Notify(e.Direction > 0 ? $"GOING UP: {from} TO {to}" : $"GOING DOWN: {from} TO {to}");
}
```




```
private double _start;

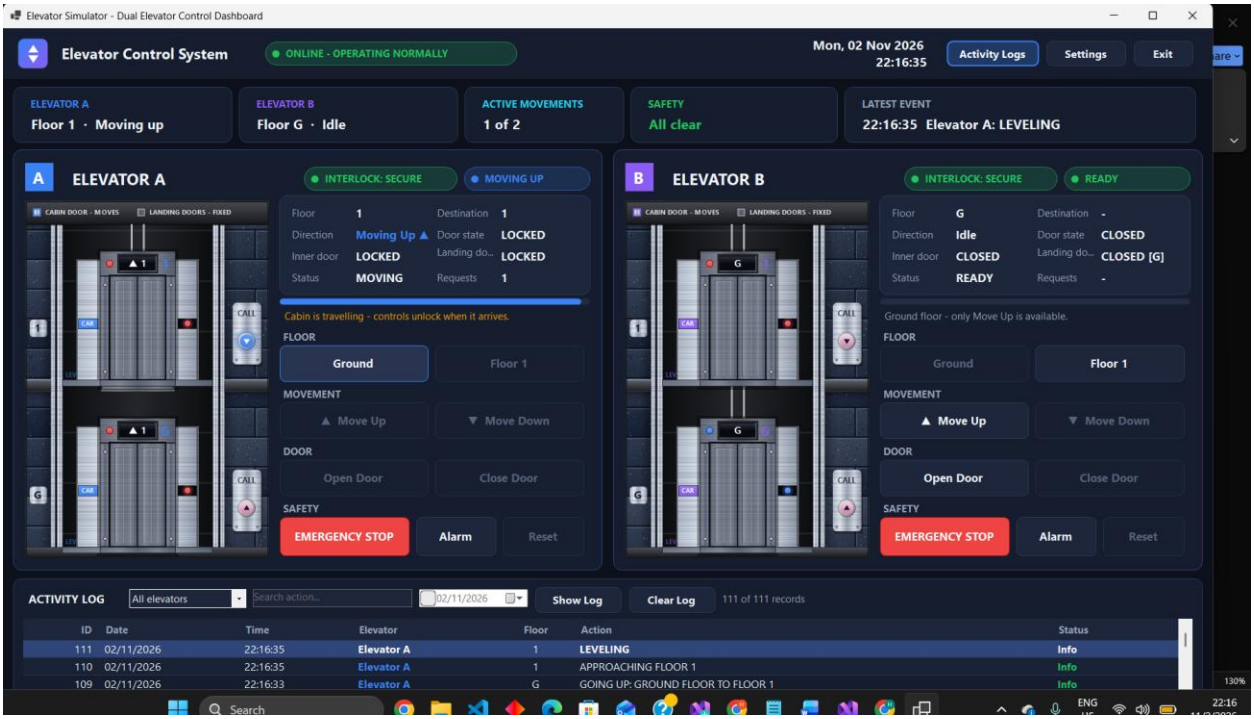
1 reference
public string Name => "Moving";

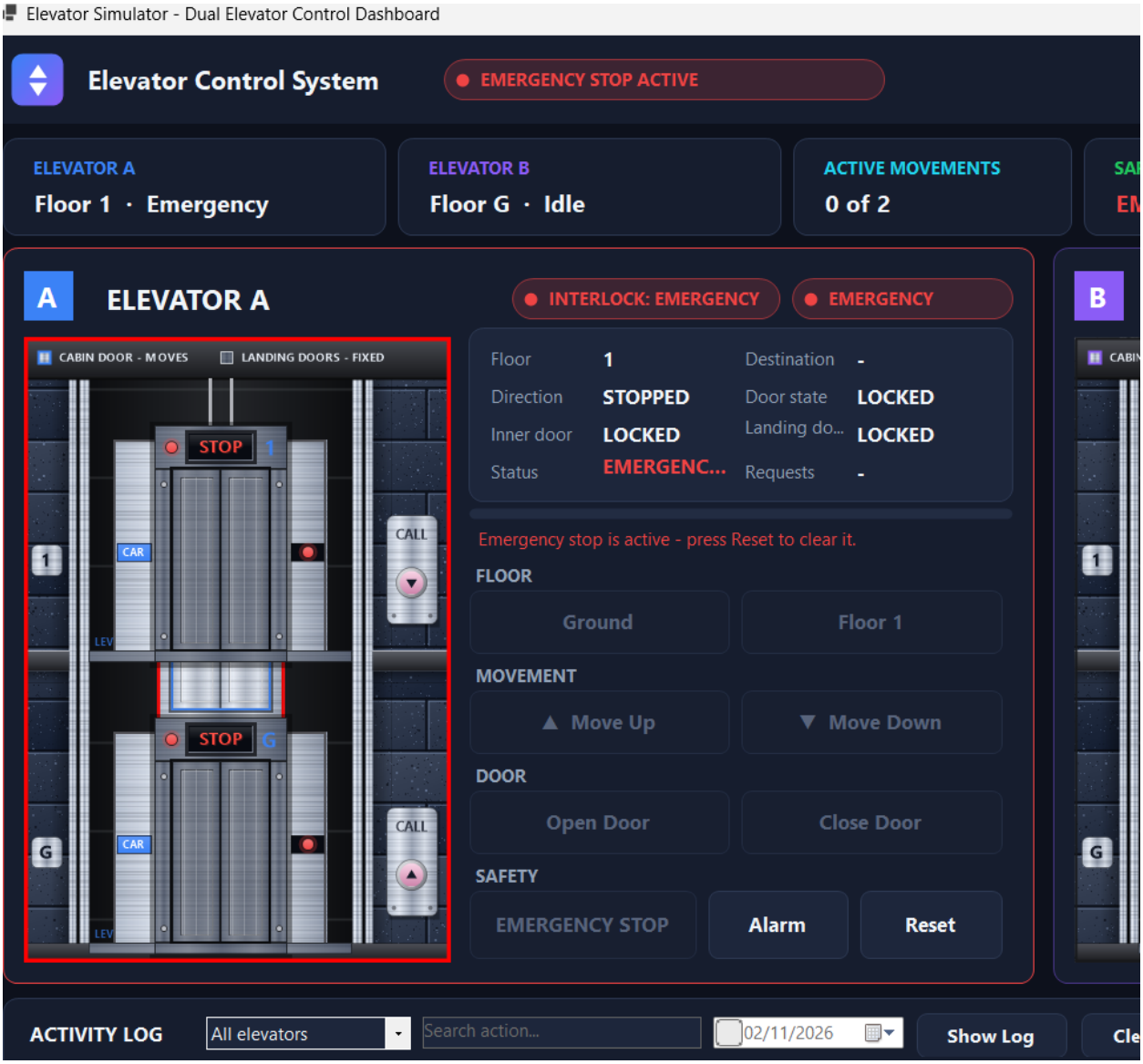
2 references
public void OnEnter(Elevator e)   e: {Elevator_with_time.Elevator}
{
    _start = e.Position;
    e.Direction = Math.Sign(e.TargetFloor - e.Position);
    e.LastDirection = e.Direction;
    e.StatusText = "MOVING";

    string from = Elevator.FloorTag(e.CurrentFloor);
    string to = Elevator.FloorTag(e.TargetFloor);
```

WB8

```
public bool CanMove => !IsEmergency && !IsMoving && AllDoorsClosed && !Overload;
```


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
22:16:35
Activity Logs
Settings
Exit
ELEVATOR A
Floor 1 · Moving up
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
1 of 2
SAFETY
All clear
LATEST EVENT
22:16:35 Elevator A: LEVELING
A
ELEVATOR A
INTERLOCK: SECURE
MOVING UP
Floor 1
Destination 1
Direction Moving Up
Door state LOCKED
Inner door LOCKED
Landing do... LOCKED
Status MOVING
Requests 1
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ELEVATOR B
INTERLOCK: SECURE
READY
Floor G
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [G]
Status READY
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
111 of 111 records
ID Date Time Elevator Floor Action Status
111 02/11/2026 22:16:35 Elevator A 1 LEVELING Info
110 02/11/2026 22:16:35 Elevator A 1 APPROACHING FLOOR 1 Info
109 02/11/2026 22:16:33 Elevator A G GOING UP: GROUND FLOOR TO FLOOR 1 Info
Search
ENG
22:16



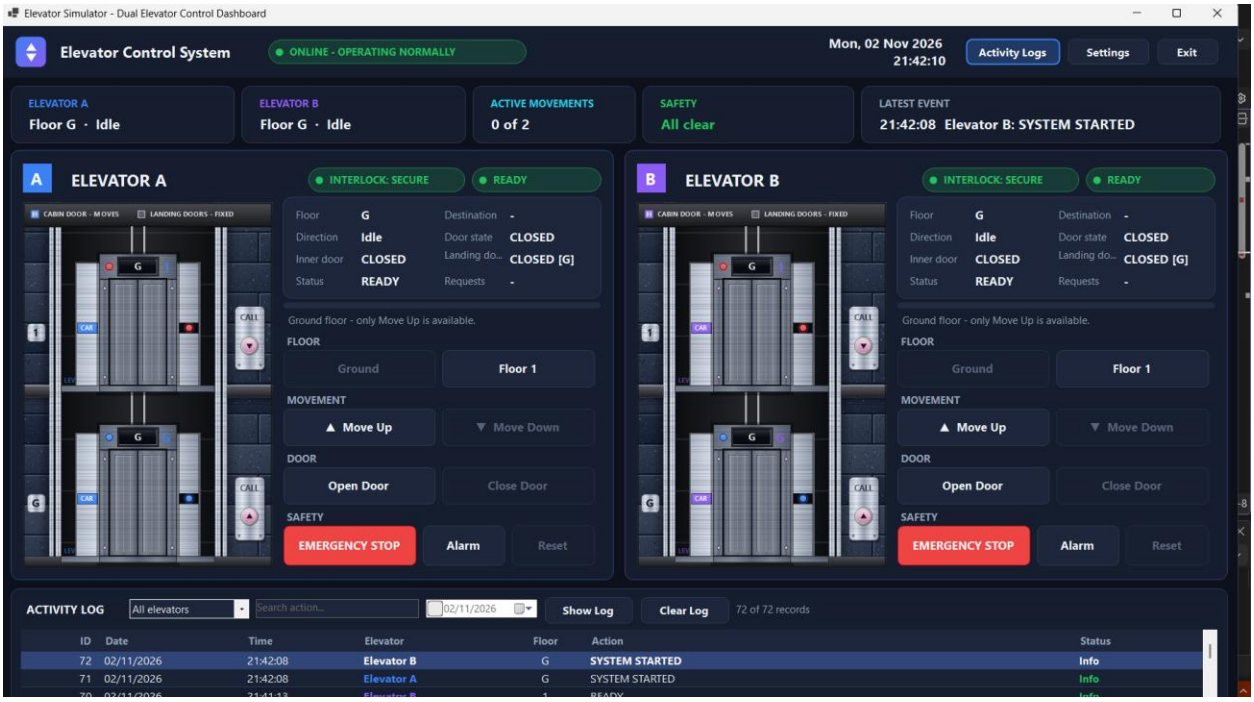
WB9

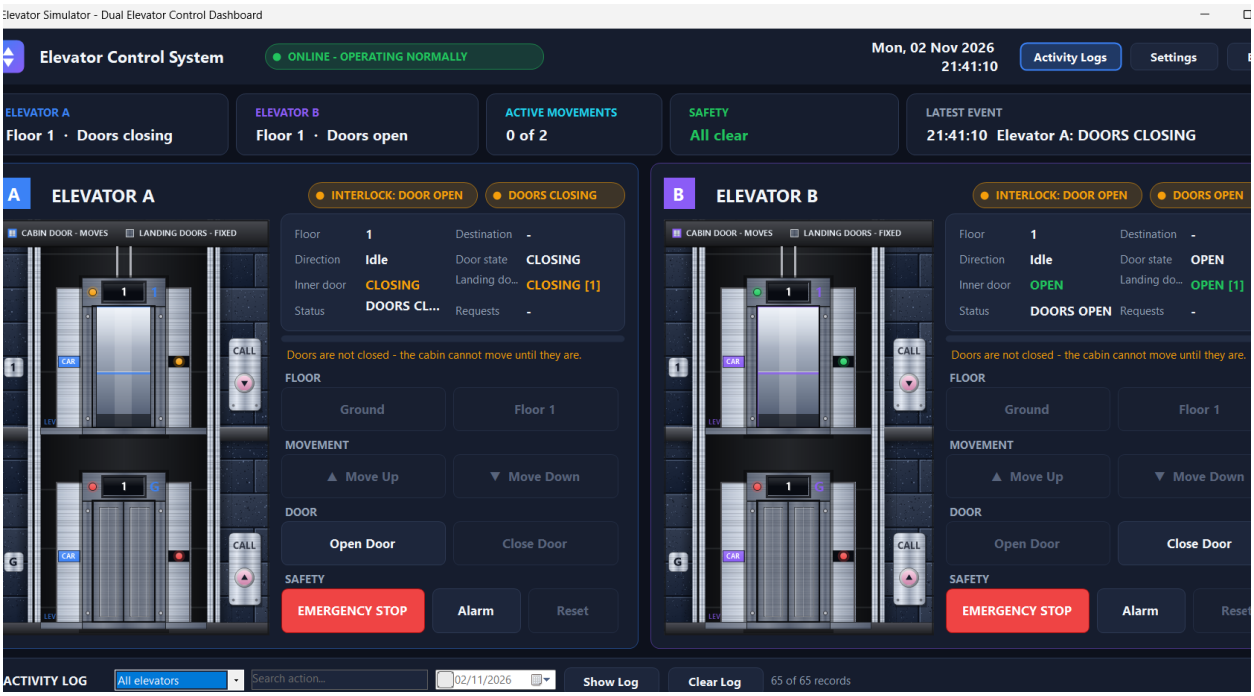
```
40
41          /// <summary>True when the door leaf is completely shut and bolted/closed (not opening, not open, not closing).</summary>
            2 references
42          public bool IsSecured => Openness <= 0.0 && (State == DoorState.Closed || State == DoorState.Locked);
43      }
44
45      /// <summary>
46      /// ONE elevator serving the floors MinFloor..MaxFloor (floor 1 is shown as "G").
```

```
36          public DoorState State { get; internal set; }
            12 references
37          public double Openness { get; internal set; }
38
            3 references
39          public string Text => State.ToString().ToUpperInvariant();
40
41          /// <summary>True when the door leaf is completely shut and bolted/closed (not opening, not open, not closing).</summary>
            2 references
42          public bool IsSecured => Openness <= 0.0 && (State == DoorState.Closed || State == DoorState.Locked);
43      }
44
45      /// <summary>
```

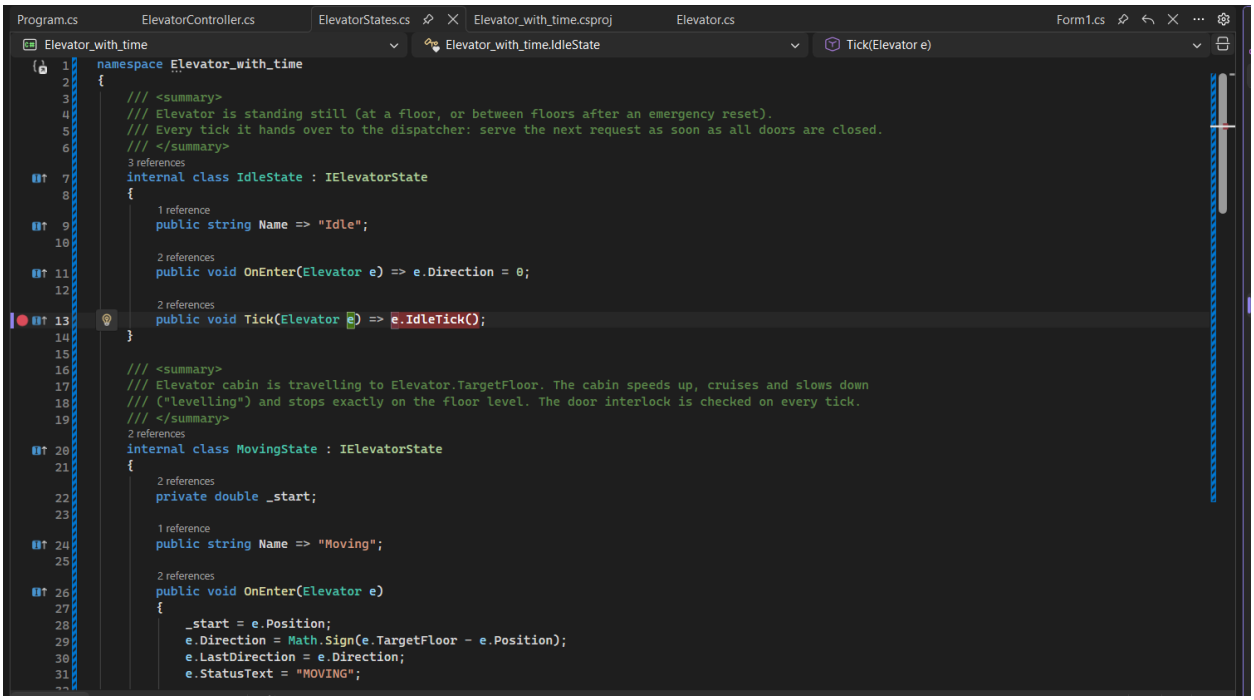
WB10

```
12          {
13              int floor = Elevator.NextFloorAbove;
14              if (floor <= Elevator_with_time.Elevator.MaxFloor) Elevator.RequestFloor(floor);
15          }
16
17          /// <summary>DOWN: request the next floor below the cabin (nothing happens at the ground floor).</summary>
            namespace Elevator_with_time
                public class ElevatorController
18          {
19              int floor = Elevator.NextFloorBelow;
20              if (floor >= Elevator_with_time.Elevator.MinFloor) Elevator.RequestFloor(floor);
21          }
22      }
23  }
```




Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
21:41:10
Activity Logs
Settings
ELEVATOR A
Floor 1 · Doors closing
ELEVATOR B
Floor 1 · Doors open
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
21:41:10 Elevator A: DOORS CLOSING
A
ELEVATOR A
INTERLOCK: DOOR OPEN
DOORS CLOSING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSING
Inner door CLOSING
Landing do... CLOSING [1]
Status DOORS CL...
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
Floor 1
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing do... OPEN [1]
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
65 of 65 records

WB11


Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Form1.cs
Elevator_with_time
Elevator_with_time.IdleState
Tick(Elevator e)
namespace Elevator_with_time
{
/// <summary>
/// Elevator is standing still (at a floor, or between floors after an emergency reset).
/// Every tick it hands over to the dispatcher: serve the next request as soon as all doors are closed.
/// </summary>
3 references
internal class IdleState : IElevatorState
{
1 reference
public string Name => "Idle";
2 references
public void OnEnter(Elevator e) => e.Direction = 0;
2 references
public void Tick(Elevator e) => e.IdleTick();
}
/// <summary>
/// Elevator cabin is travelling to Elevator.TargetFloor. The cabin speeds up, cruises and slows down
/// ("levelling") and stops exactly on the floor level. The door interlock is checked on every tick.
/// </summary>
2 references
internal class MovingState : IElevatorState
{
2 references
private double _start;
1 reference
public string Name => "Moving";
2 references
public void OnEnter(Elevator e)
{
_start = e.Position;
e.Direction = Math.Sign(e.TargetFloor - e.Position);
e.LastDirection = e.Direction;
e.StatusText = "MOVING";

```
302
303         /// <summary>Called on every tick while the cabin stands still (IdleState).</summary>
            1 reference
304         internal void IdleTick()
305         {
306             // a cabin that is levelled at a floor (e.g. after a RESET) releases the doors of THAT floor only
307             if (IsAtFloor) UnlockDoorZone(CurrentFloor);
308
309             if (_doorFloor != 0) return;              // a door cycle is running: wait until it is finished
310             if (!AllDoorsClosed) return;              // interlock: never start while any door is not closed
311
312             if (_pending.Count > 0)
313             {
314                 StartNextMove();
315                 return;
316             }
317
```

93 % | No issues found | Ln: 305, Ch: 9 | SPC | CRLF

```
12
            2 references
13          public void Tick(Elevator e) => e.IdleTick();   e: {Elevator_with_time.Elevator}
```

WB12

```
91              e.Direction = 0;
92              e.Notify("EMERGENCY STOP ACTIVATED");
93          }
94
            2 references
95          public void Tick(Elevator e) { }
96      }
97  }
98
```

Process: [28684] Elevator_with_time.exe | Lifecycle Events | Thread: [10776] Main Thread | Stack Frame: Elevator_with_time.EmergencyState.On

Chat — New Thread

Debugger Agent

I can help you diagnose and fix bugs, analyze exceptions, understand call stacks, and suggest breakpoints.

See Debugger Agent concepts and example prompts

I might make mistakes, so check for accuracy.

Explain the code based on debugger state

Set breakpoints to debug <describe acti...

Debug <paste a GitHub/Azure DevOps b...

Ask Copilot

Debugger | Interactive

Search — General

There are no usable controls in this group. Drag an item onto this text to add it to the toolbox.

Program.cs | ElevatorController.cs | ElevatorStates.cs | Elevator_with_time.csproj | Elevator.cs | Form1.cs

Elevator_with_time | Elevator_with_time.EmergencyState | OnEnter(Elevator e)

```
83
84          /// <summary>Emergency stop: the timer is stopped, nothing moves until RESET.</summary>
            2 references
85          internal class EmergencyState : IElevatorState
86          {
                1 reference
87              public string Name => "Emergency";
88
                2 references
89              public void OnEnter(Elevator e)
90              {
91                  e.Direction = 0;
92                  e.Notify("EMERGENCY STOP ACTIVATED");
93              }
94
                2 references
95              public void Tick(Elevator e) { }
96          }
97      }
98
```

93 % | No issues found | Ln: 93, Ch: 9

Error List — Entire Solution | 0 Errors | 0 of 1 Warning | 0 of 16 Messages | Build + IntelliSense | Search Error List

Code | Description | Project | File

Call Stack | Breakpoints | Exception Settings | Command Window | Immediate Window | Output | Error List

WB13

```
1 reference
private void Writer_DoWork(object sender, DoWorkEventArgs e)
{
    while (_queue.TryDequeue(out LogEntry entry))
    {
        try
        {
            Insert(entry);
            _wroteSomething = true;
        }
        catch (Exception ex)
        {
            e.Result = ex.Message;
            while (_queue.TryDequeue(out _)) { }   // database unusable: do not retry every entry
        }
    }
}

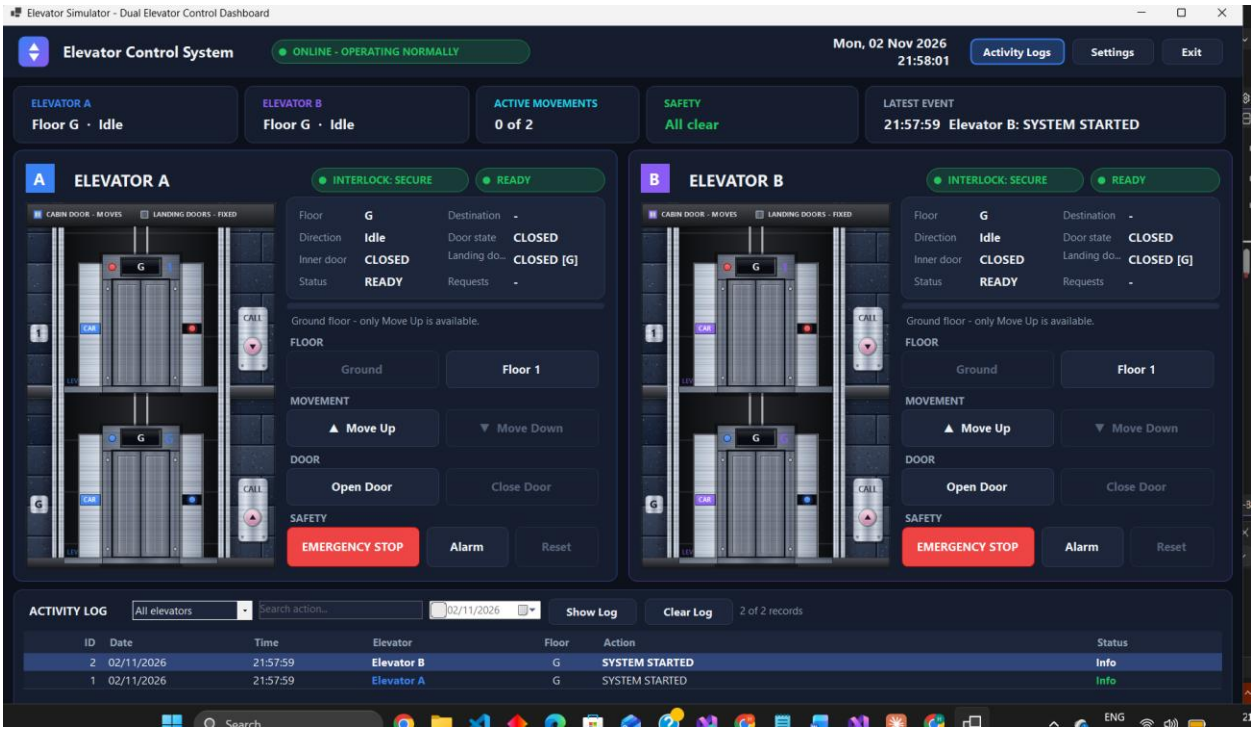
1 reference
```

```
1 reference
private void Writer_DoWork(object sender, DoWorkEventArgs e)   sender: {System.ComponentModel.BackgroundWorker}   e: {Sy
{   ≤ 41,401ms elapsed
    while (_queue.TryDequeue(out LogEntry entry))
    {
        try
        {
            Insert(entry);
93 %   No issues found
```

WB14

```
public void Flush()
{
    while (_queue.TryDequeue(out LogEntry entry))
    {
        try { Insert(entry); } catch { break; }
    }
}

// ------------------------------------------------------------ reading / clearing (async)
```



11. Critical Thinking

11.1 Advantages

- Safe to run with interlocks, emergency buttons, and closed doors.
- Clean design utilizing State and Observer design patterns.
- Good user interface with background thread database processing.
- Two separate elevators with resizable dashboard and filterable logs.
- Automatically configured database through dbsettings.json.

11.2 Limitations

- Restricted to two floors with two independent elevators.
- Database password stored as clear text.
- Single log table using text status field.
- Windows-based WinForms app with no unit testing.
- Any failure to database may delete log queue and one button can delete all logs.

11.3 Future Improvements

- Extend floors along with priority queues using nearest elevator assignment algorithm.
- Implement credential encryption and role-based authentication in Clear Log.
- Create event types and severity columns in the database.
- Perform xUnit tests on the state class and the repository.
- Connect the IoT hardware to a web/mobile app for monitoring purposes.

12. Conclusion

In conclusion, the project has been successful in developing a program to simulate elevators including the dispatcher and speed scheduler. Synchronization, interlocking, and safety features have been included along with resizable GUI and Asynchronous MySQL logging. The design pattern that has been used includes State and Observer designs. Both black box and white box testing has been used.

13. References

References

ASME, 2019. *ASME A17.1-2019/CSA B44-19: Safety Code for Elevators and Escalators,* s.l.: ASME.

ASME, ASME A17.1-2019/CSA B44-19: Safety Code for Elevators and Escalators. ASME A17.1-2019/CSA B44-19: Safety Code for Elevators and Escalators. In: New York: American Society of Mechanical Engineers.

Gamma, E., Helm, R., Johnson, R. & Vlissides, J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. In: Reading, Massachusetts: Addison-Wesley.

MySQL, 2026. *MySQL 8.4 Reference Manual.* [Online]
Available at: https://dev.mysql.com/doc/refman/8.4/en/
[Accessed 3 November 2026].

MySQL, O. C. /., 2026. *MySQL 8.4 Reference Manual,* s.l.: Oracle Corporation.

14. Marking Matrix with Self-Assessment

Fill in the marks after checking your assignment brief. The evidence column points to the section of this report and the source file.

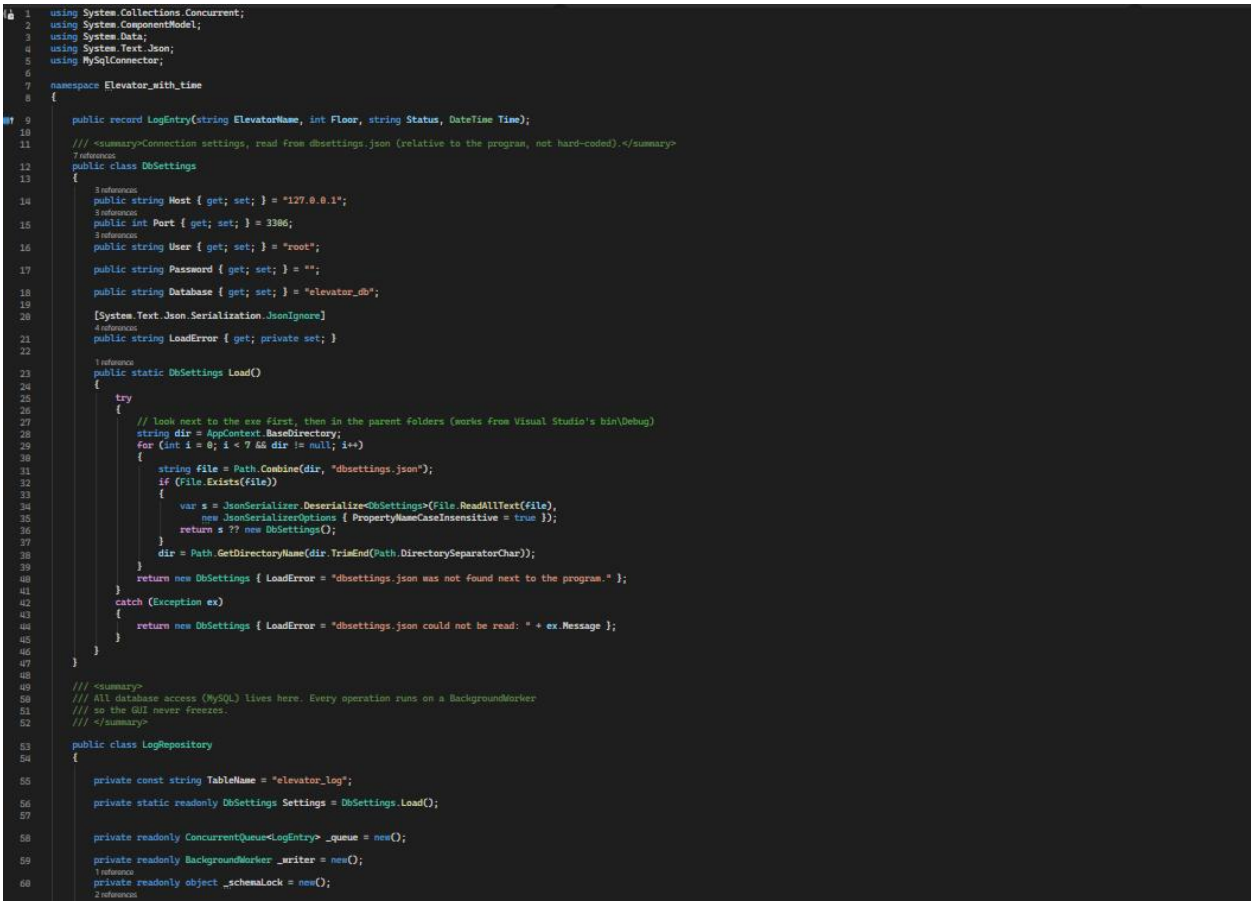
Table 10 Marking matrix with self-assessment

Criterion	Evidence	Max	Self mark
GUI	Section 9; Form1.cs, UI/ElevatorPanel.cs, UI/Controls.cs		
Database connection	Section 8.4; Data/LogRepository.cs, dbsettings.json		
Delegation and timers	Section 8.5; Elevator.cs, Form1.cs		
Optimizing in program	Section 8.7		
Exception handling	Section 8.8		
Object-oriented design and patterns	Sections 7.2 and 8.2 to 8.3		
Testing (black box and white box)	Section 10		
Documentation and diagrams	Sections 7 and 15		

15. Appendices

Appendix A: GitHub Link to Complete Source Code

Source code Screenshot



```
using System.Collections.Concurrent;
using System.ComponentModel;
using System.Data;
using System.Text.Json;
using MySqlConnector;

namespace Elevator_with_time
{

    public record LogEntry(string ElevatorName, int Floor, string Status, DateTime Time);

    /// <summary>Connection settings, read from dbsettings.json (relative to the program, not hard-coded).</summary>
    7 references
    public class DbSettings
    {
        3 references
        public string Host { get; set; } = "127.0.0.1";
        3 references
        public int Port { get; set; } = 3306;
        3 references
        public string User { get; set; } = "root";

        public string Password { get; set; } = "";

        public string Database { get; set; } = "elevator_db";

        [System.Text.Json.Serialization.JsonIgnore]
        4 references
        public string LoadError { get; private set; }

        1 reference
        public static DbSettings Load()
        {
            try
            {
                // look next to the exe first, then in the parent folders (works from Visual Studio's bin\Debug)
                string dir = AppContext.BaseDirectory;
                for (int i = 0; i < 7 && dir != null; i++)
                {
                    string file = Path.Combine(dir, "dbsettings.json");
                    if (File.Exists(file))
                    {
                        var s = JsonSerializer.Deserialize<DbSettings>(File.ReadAllText(file),
                            new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
                        return s ?? new DbSettings();
                    }
                    dir = Path.GetDirectoryName(dir.TrimEnd(Path.DirectorySeparatorChar));
                }
                return new DbSettings { LoadError = "dbsettings.json was not found next to the program." };
            }
            catch (Exception ex)
            {
                return new DbSettings { LoadError = "dbsettings.json could not be read: " + ex.Message };
            }
        }
    }

    /// <summary>
    /// All database access (MySQL) lives here. Every operation runs on a BackgroundWorker
    /// so the GUI never freezes.
    /// </summary>

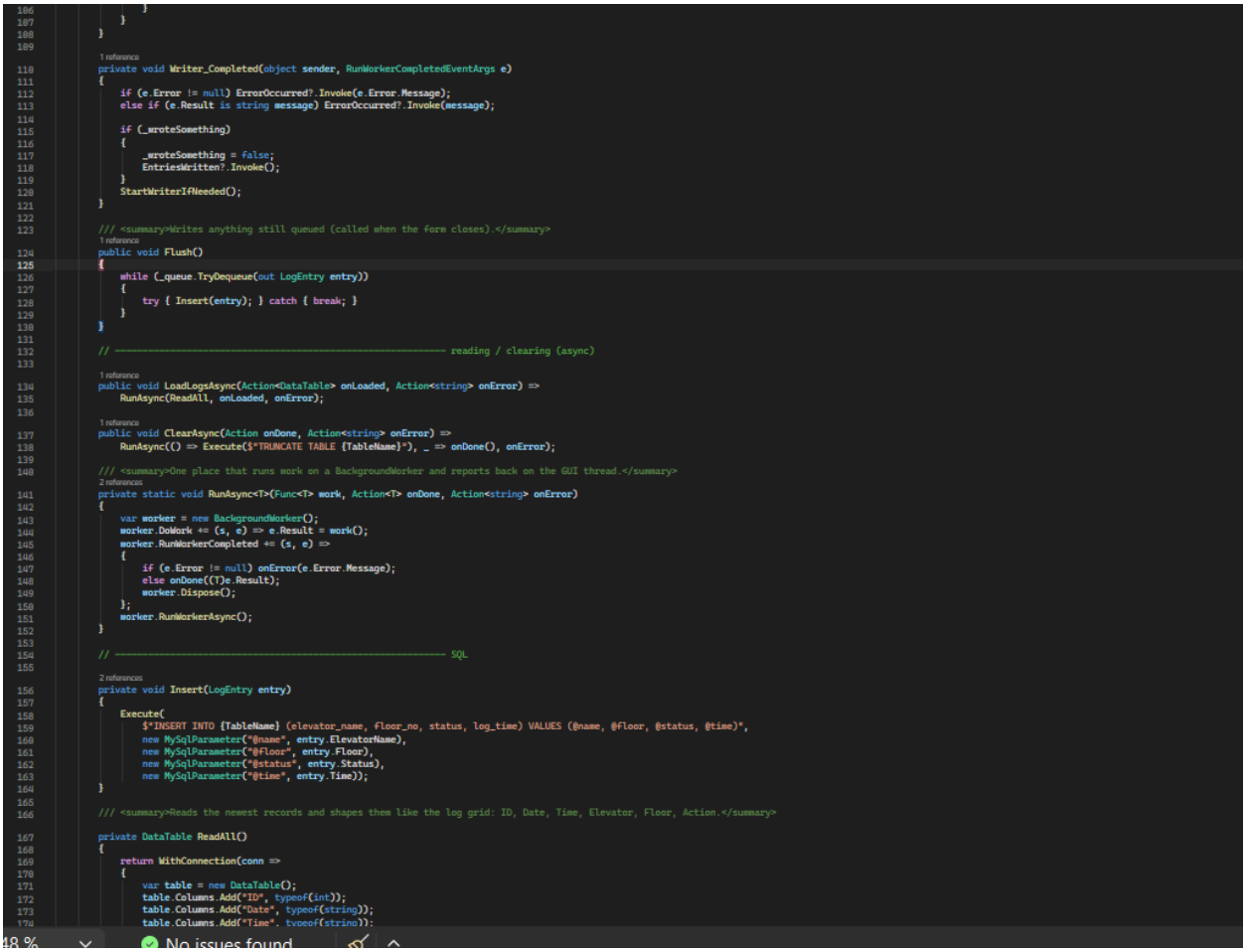
    public class LogRepository
    {

        private const string TableName = "elevator_log";

        private static readonly DbSettings Settings = DbSettings.Load();


        private readonly ConcurrentQueue<LogEntry> _queue = new();

        private readonly BackgroundWorker _writer = new();
        1 reference
        private readonly object _schemaLock = new();
        2 references
```



```
            }
        }
    }

    1 reference
    private void Writer_Completed(object sender, RunWorkerCompletedEventArgs e)
    {
        if (e.Error != null) ErrorOccurred?.Invoke(e.Error.Message);
        else if (e.Result is string message) ErrorOccurred?.Invoke(message);

        if (_wroteSomething)
        {
            _wroteSomething = false;
            EntriesWritten?.Invoke();
        }
        StartWriterIfNeeded();
    }

    /// <summary>Writes anything still queued (called when the form closes).</summary>
    1 reference
    public void Flush()
    {
        while (_queue.TryDequeue(out LogEntry entry))
        {
            try { Insert(entry); } catch { break; }
        }
    }

    // ---------------------------------------------------------- reading / clearing (async)

    1 reference
    public void LoadLogsAsync(Action<DataTable> onLoaded, Action<string> onError) =>
        RunAsync(ReadAll, onLoaded, onError);

    1 reference
    public void ClearAsync(Action onDone, Action<string> onError) =>
        RunAsync(() => Execute($"TRUNCATE TABLE {TableName}"), _ => onDone(), onError);

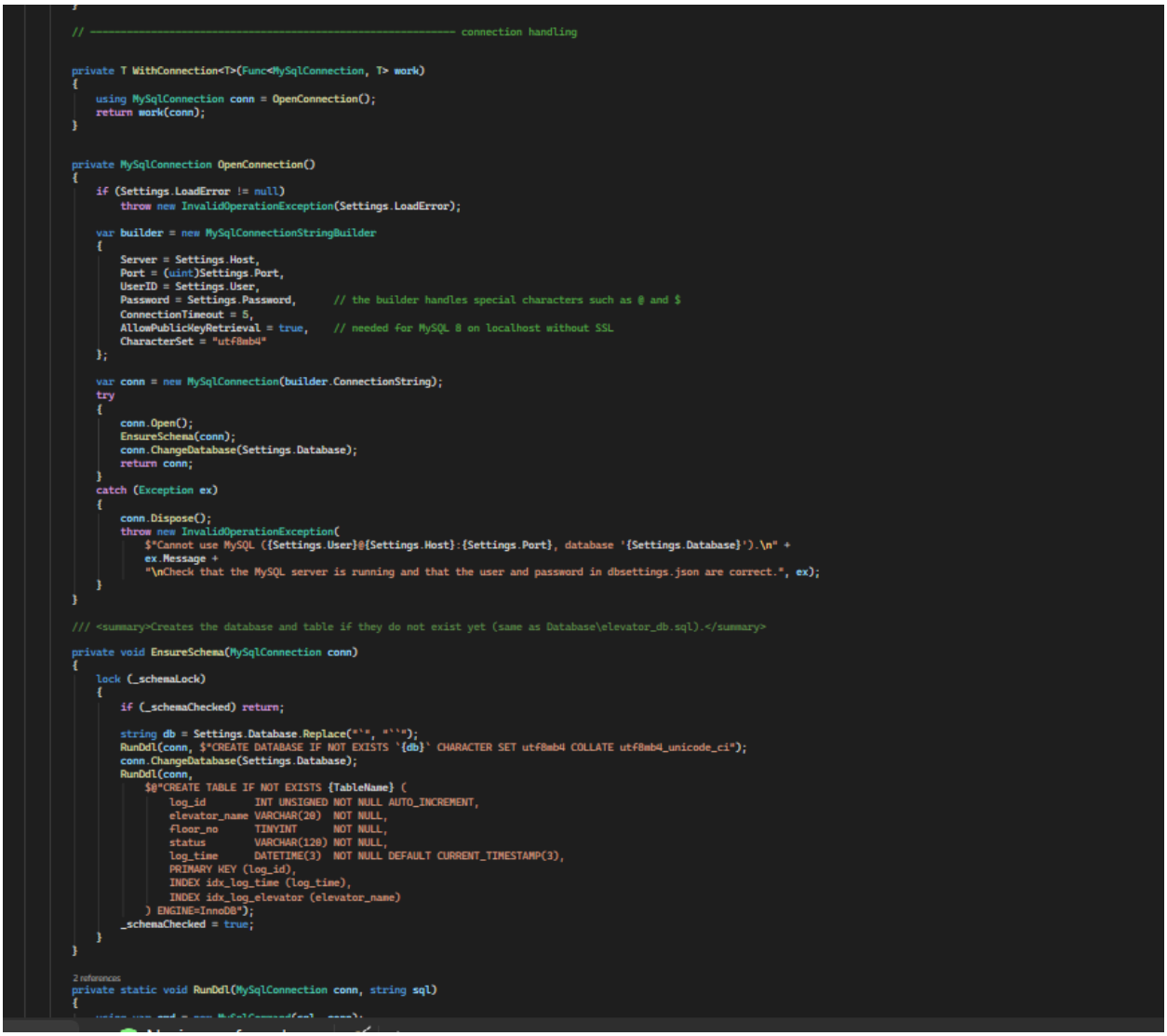
    /// <summary>One place that runs work on a BackgroundWorker and reports back on the GUI thread.</summary>
    2 references
    private static void RunAsync<T>(Func<T> work, Action<T> onDone, Action<string> onError)
    {
        var worker = new BackgroundWorker();
        worker.DoWork += (s, e) => e.Result = work();
        worker.RunWorkerCompleted += (s, e) =>
        {
            if (e.Error != null) onError(e.Error.Message);
            else onDone((T)e.Result);
            worker.Dispose();
        };
        worker.RunWorkerAsync();
    }

    // ---------------------------------------------------------- SQL

    2 references
    private void Insert(LogEntry entry)
    {
        Execute(
            $"INSERT INTO {TableName} (elevator_name, floor_no, status, log_time) VALUES (@name, @floor, @status, @time)",
            new MySqlParameter("@name", entry.ElevatorName),
            new MySqlParameter("@floor", entry.Floor),
            new MySqlParameter("@status", entry.Status),
            new MySqlParameter("@time", entry.Time));
    }

    /// <summary>Reads the newest records and shapes them like the log grid: ID, Date, Time, Elevator, Floor, Action.</summary>

    private DataTable ReadAll()
    {
        return WithConnection(conn =>
        {
            var table = new DataTable();
            table.Columns.Add("ID", typeof(int));
            table.Columns.Add("Date", typeof(string));
            table.Columns.Add("Time", typeof(string));
48 %                    No issues found
```



```
    // ------------------------------------------------------------ connection handling

    private T WithConnection<T>(Func<MySqlConnection, T> work)
    {
        using MySqlConnection conn = OpenConnection();
        return work(conn);
    }

    private MySqlConnection OpenConnection()
    {
        if (Settings.LoadError != null)
            throw new InvalidOperationException(Settings.LoadError);

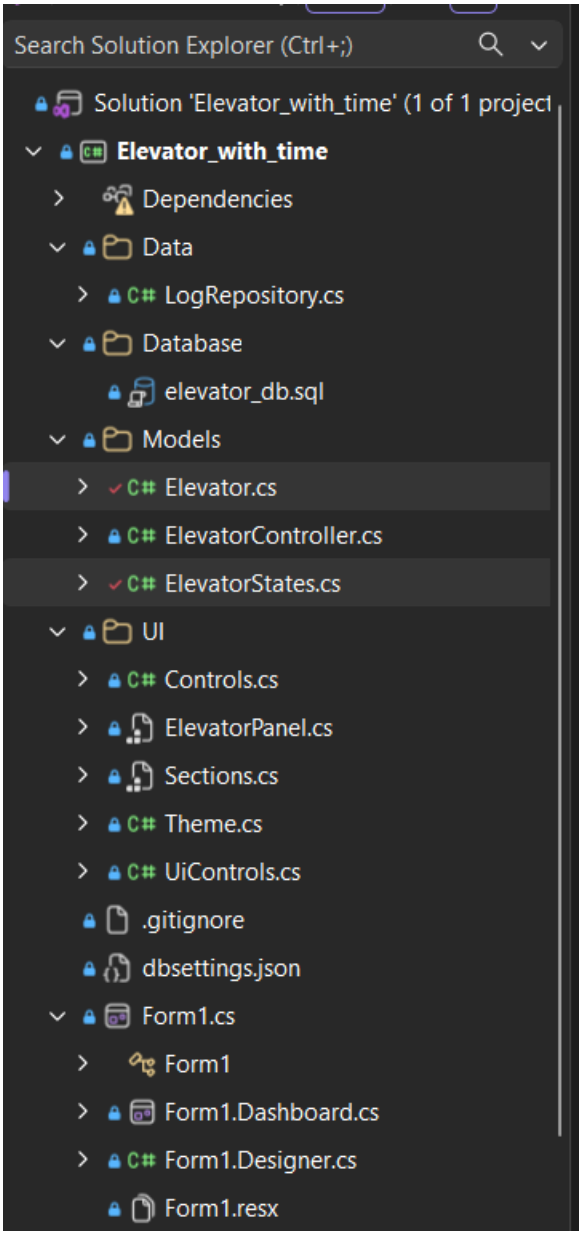
        var builder = new MySqlConnectionStringBuilder
        {
            Server = Settings.Host,
            Port = (uint)Settings.Port,
            UserID = Settings.User,
            Password = Settings.Password,      // the builder handles special characters such as @ and $
            ConnectionTimeout = 5,
            AllowPublicKeyRetrieval = true,    // needed for MySQL 8 on localhost without SSL
            CharacterSet = "utf8mb4"
        };

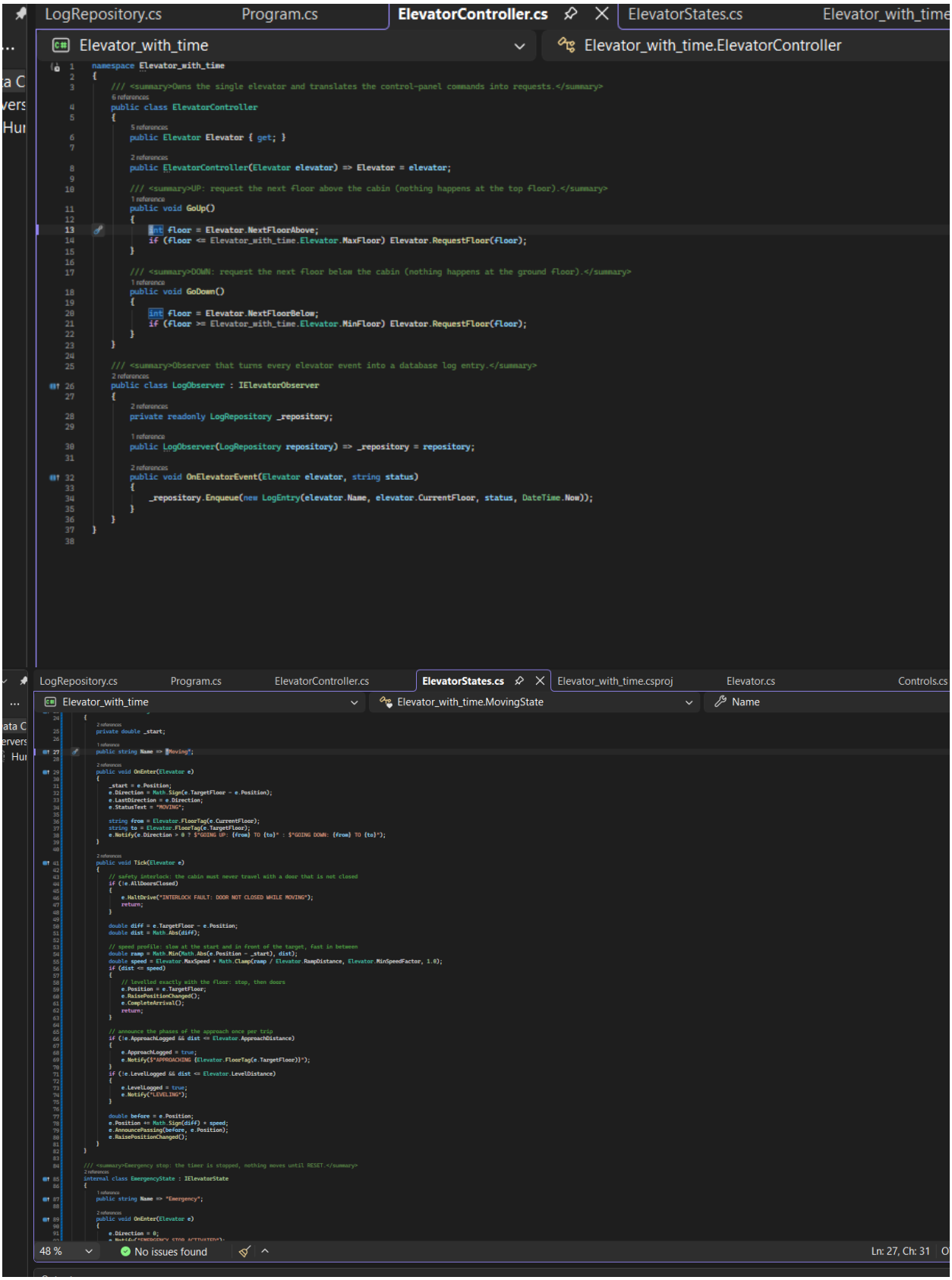
        var conn = new MySqlConnection(builder.ConnectionString);
        try
        {
            conn.Open();
            EnsureSchema(conn);
            conn.ChangeDatabase(Settings.Database);
            return conn;
        }
        catch (Exception ex)
        {
            conn.Dispose();
            throw new InvalidOperationException(
                $"Cannot use MySQL ({Settings.User}@{Settings.Host}:{Settings.Port}, database '{Settings.Database}').\n" +
                ex.Message +
                "\nCheck that the MySQL server is running and that the user and password in dbsettings.json are correct.", ex);
        }
    }

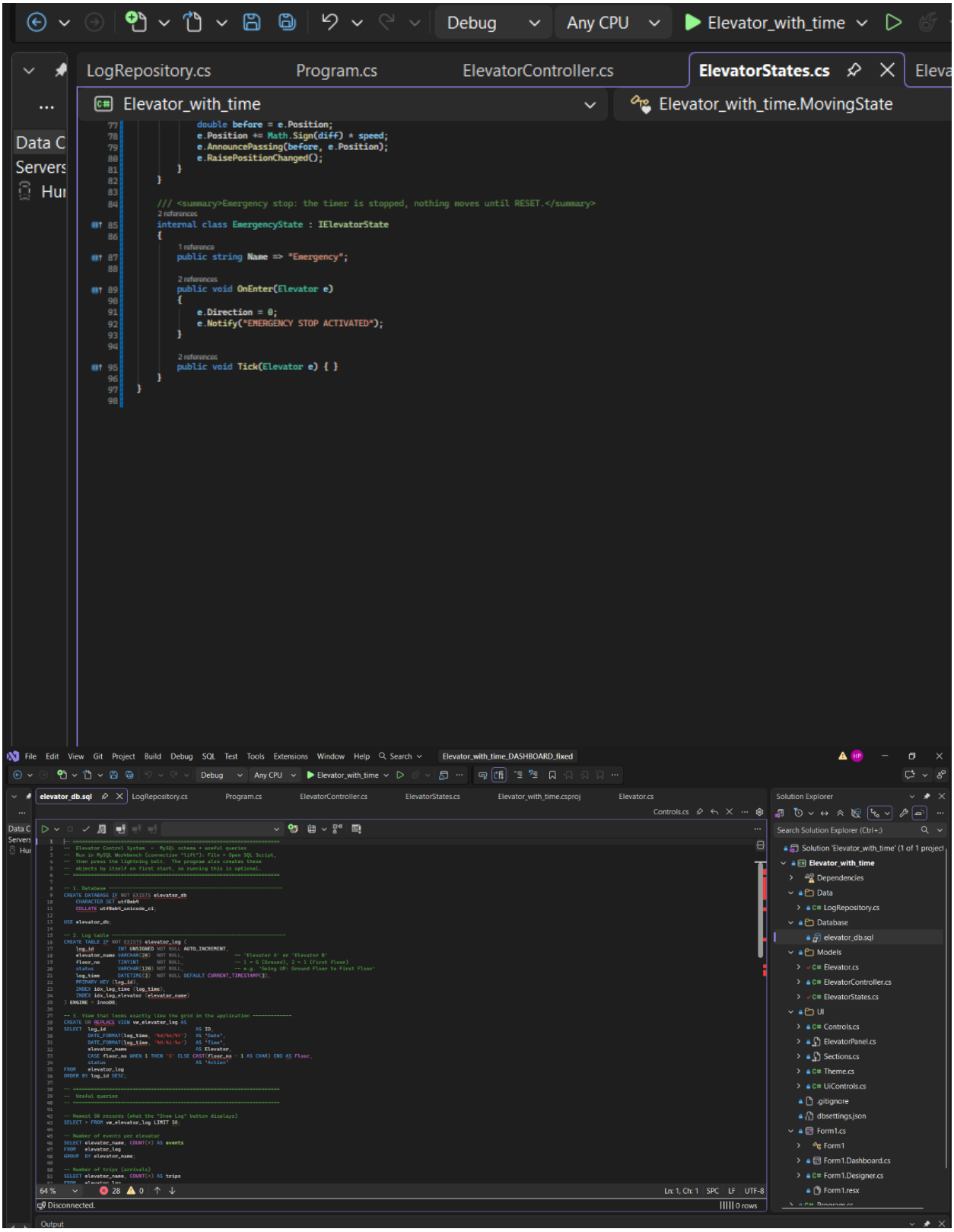
    /// <summary>Creates the database and table if they do not exist yet (same as Database\elevator_db.sql).</summary>
    private void EnsureSchema(MySqlConnection conn)
    {
        lock (_schemaLock)
        {
            if (_schemaChecked) return;

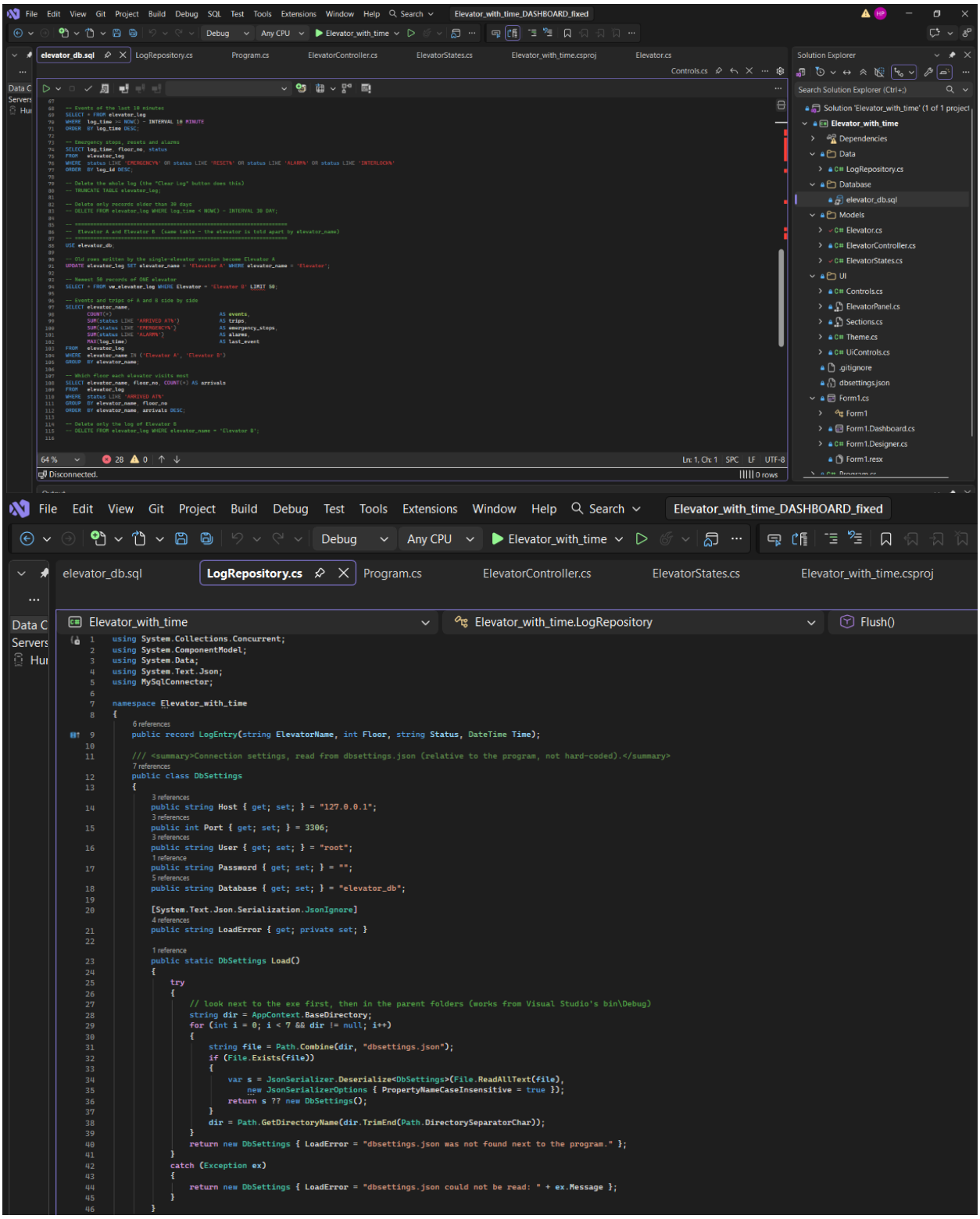
            string db = Settings.Database.Replace("`", "``");
            RunDdl(conn, $"CREATE DATABASE IF NOT EXISTS `{db}` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci");
            conn.ChangeDatabase(Settings.Database);
            RunDdl(conn,
                $@"CREATE TABLE IF NOT EXISTS {TableName} (
                    log_id       INT UNSIGNED NOT NULL AUTO_INCREMENT,
                    elevator_name VARCHAR(20)  NOT NULL,
                    floor_no     TINYINT      NOT NULL,
                    status       VARCHAR(120) NOT NULL,
                    log_time     DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
                    PRIMARY KEY (log_id),
                    INDEX idx_log_time (log_time),
                    INDEX idx_log_elevator (elevator_name)
                ) ENGINE=InnoDB");
            _schemaChecked = true;
        }
    }

    2 references
    private static void RunDdl(MySqlConnection conn, string sql)
    {
```


Search Solution Explorer (Ctrl+;)
Solution 'Elevator_with_time' (1 of 1 project
Elevator_with_time
Dependencies
Data
LogRepository.cs
Database
elevator_db.sql
Models
Elevator.cs
ElevatorController.cs
ElevatorStates.cs
UI
Controls.cs
ElevatorPanel.cs
Sections.cs
Theme.cs
UiControls.cs
.gitignore
dbsettings.json
Form1.cs
Form1
Form1.Dashboard.cs
Form1.Designer.cs
Form1.resx


LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time
Elevator_with_time.ElevatorController
namespace Elevator_with_time
{
/// <summary>Owns the single elevator and translates the control-panel commands into requests.</summary>
6 references
public class ElevatorController
{
5 references
public Elevator Elevator { get; }
2 references
public ElevatorController(Elevator elevator) => Elevator = elevator;
/// <summary>UP: request the next floor above the cabin (nothing happens at the top floor).</summary>
1 reference
public void GoUp()
{
int floor = Elevator.NextFloorAbove;
if (floor <= Elevator_with_time.Elevator.MaxFloor) Elevator.RequestFloor(floor);
}
/// <summary>DOWN: request the next floor below the cabin (nothing happens at the ground floor).</summary>
1 reference
public void GoDown()
{
int floor = Elevator.NextFloorBelow;
if (floor >= Elevator_with_time.Elevator.MinFloor) Elevator.RequestFloor(floor);
}
}
/// <summary>Observer that turns every elevator event into a database log entry.</summary>
2 references
public class LogObserver : IElevatorObserver
{
2 references
private readonly LogRepository _repository;
1 reference
public LogObserver(LogRepository repository) => _repository = repository;
2 references
public void OnElevatorEvent(Elevator elevator, string status)
{
_repository.Enqueue(new LogEntry(elevator.Name, elevator.CurrentFloor, status, DateTime.Now));
}
}
}
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Controls.cs
Elevator_with_time
Elevator_with_time.MovingState
Name
{
2 references
private double _start;
1 reference
public string Name => "Moving";
2 references
public void OnEnter(Elevator e)
{
_start = e.Position;
e.Direction = Math.Sign(e.TargetFloor - e.Position);
e.LastDirection = e.Direction;
e.StatusText = "MOVING";
string from = Elevator.FloorTag(e.CurrentFloor);
string to = Elevator.FloorTag(e.TargetFloor);
e.Notify(e.Direction > 0 ? $"GOING UP: {from} TO {to}" : $"GOING DOWN: {from} TO {to}");
}
2 references
public void Tick(Elevator e)
{
// safety interlock: the cabin must never travel with a door that is not closed
if (!e.AllDoorsClosed)
{
e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
return;
}
double diff = e.TargetFloor - e.Position;
double dist = Math.Abs(diff);
// speed profile: slow at the start and in front of the target, fast in between
double ramp = Math.Min(Math.Abs(e.Position - _start), dist);
double speed = Elevator.MaxSpeed * Math.Clamp(ramp / Elevator.RampDistance, Elevator.MinSpeedFactor, 1.0);
if (dist <= speed)
{
// levelled exactly with the floor: stop, then doors
e.Position = e.TargetFloor;
e.RaisePositionChanged();
e.CompleteArrival();
return;
}
// announce the phases of the approach once per trip
if (!e.ApproachLogged && dist <= Elevator.ApproachDistance)
{
e.ApproachLogged = true;
e.Notify($"APPROACHING {Elevator.FloorTag(e.TargetFloor)}");
}
if (!e.LevelLogged && dist <= Elevator.LevelDistance)
{
e.LevelLogged = true;
e.Notify("LEVELING");
}
double before = e.Position;
e.Position += Math.Sign(diff) * speed;
e.AnnouncePassing(before, e.Position);
e.RaisePositionChanged();
}
}
/// <summary>Emergency stop: the timer is stopped, nothing moves until RESET.</summary>
2 references
internal class EmergencyState : IElevatorState
{
1 reference
public string Name => "Emergency";
2 references
public void OnEnter(Elevator e)
{
e.Direction = 0;
48 %
No issues found
Ln: 27, Ch: 31


Debug
Any CPU
Elevator_with_time
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time
Elevator_with_time.MovingState
double before = e.Position;
e.Position += Math.Sign(diff) * speed;
e.AnnouncePassing(before, e.Position);
e.RaisePositionChanged();
/// <summary>Emergency stop: the timer is stopped, nothing moves until RESET.</summary>
internal class EmergencyState : IElevatorState
public string Name => "Emergency";
public void OnEnter(Elevator e)
e.Direction = 0;
e.Notify("EMERGENCY STOP ACTIVATED");
public void Tick(Elevator e) { }
Elevator_with_time_DASHBOARD_fixed
elevator_db.sql
Solution Explorer
Solution 'Elevator_with_time' (1 of 1 project)
Elevator_with_time
Dependencies
Data
LogRepository.cs
Database
elevator_db.sql
Models
Elevator.cs
ElevatorController.cs
ElevatorStates.cs
UI
Controls.cs
ElevatorPanel.cs
Sections.cs
Theme.cs
UiControls.cs
.gitignore
dbsettings.json
Form1.cs
Form1
Form1.Dashboard.cs
Form1.Designer.cs
Form1.resx
Disconnected.
Output



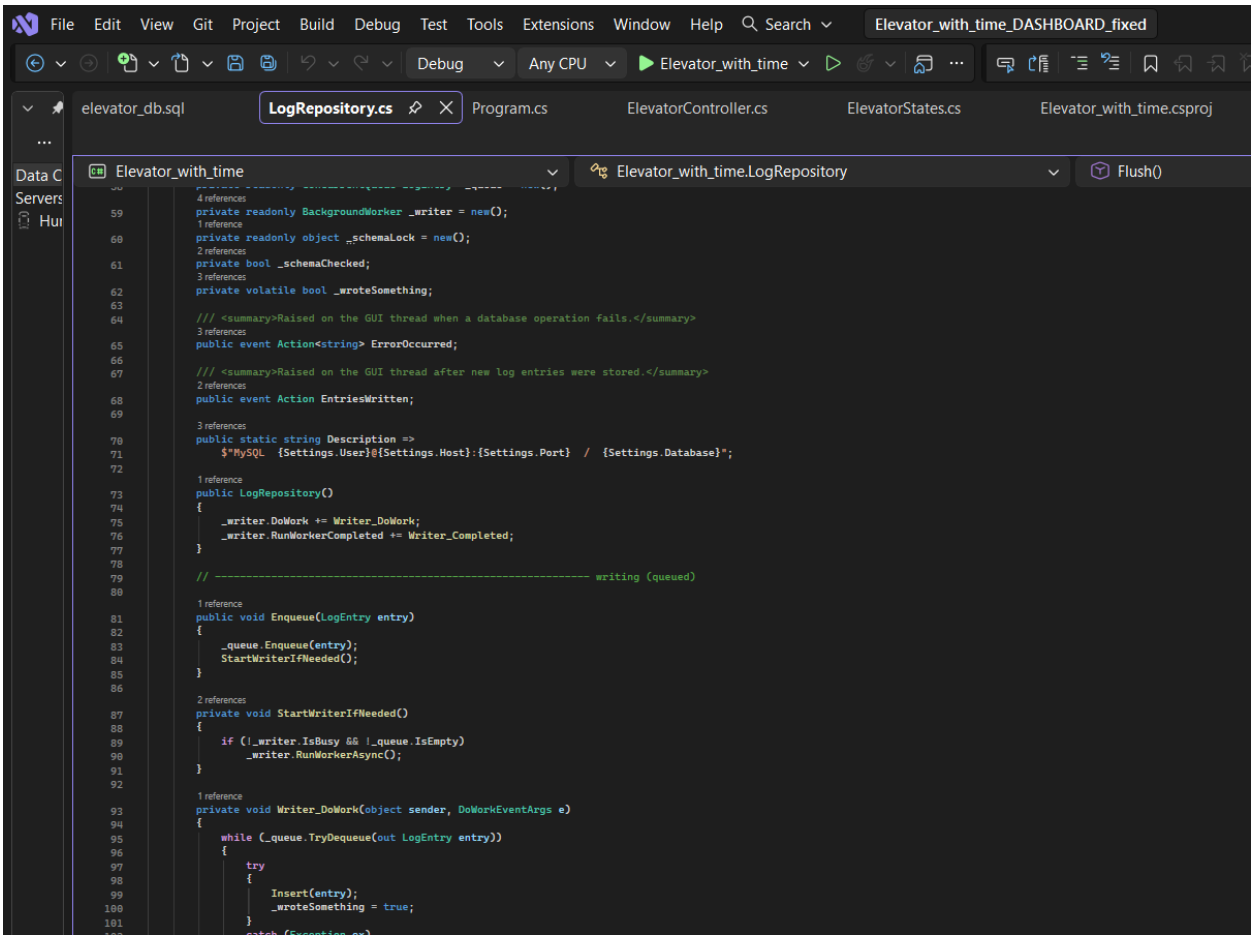
```
using System.Collections.Concurrent;
using System.ComponentModel;
using System.Data;
using System.Text.Json;
using MySqlConnector;

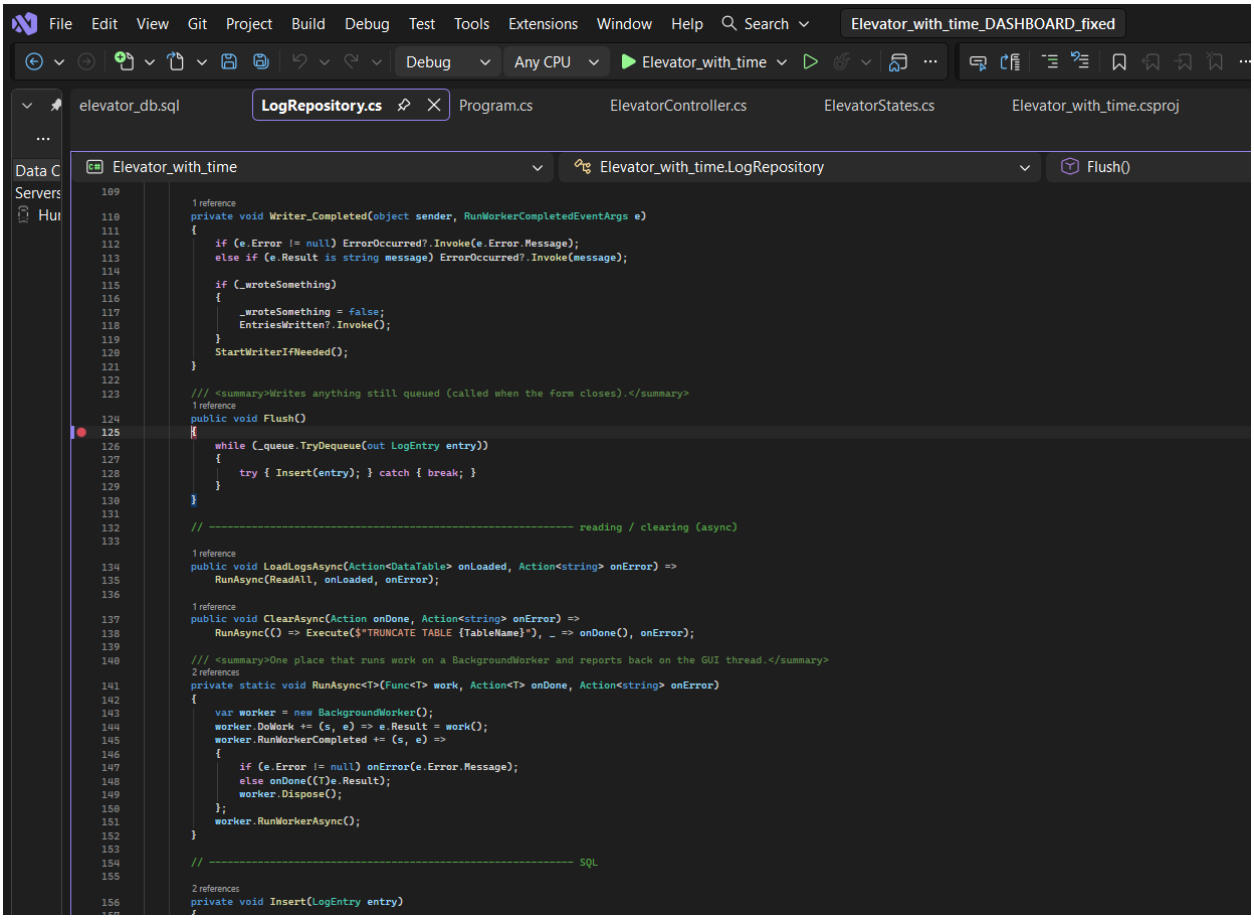
namespace Elevator_with_time
{
    6 references
    public record LogEntry(string ElevatorName, int Floor, string Status, DateTime Time);

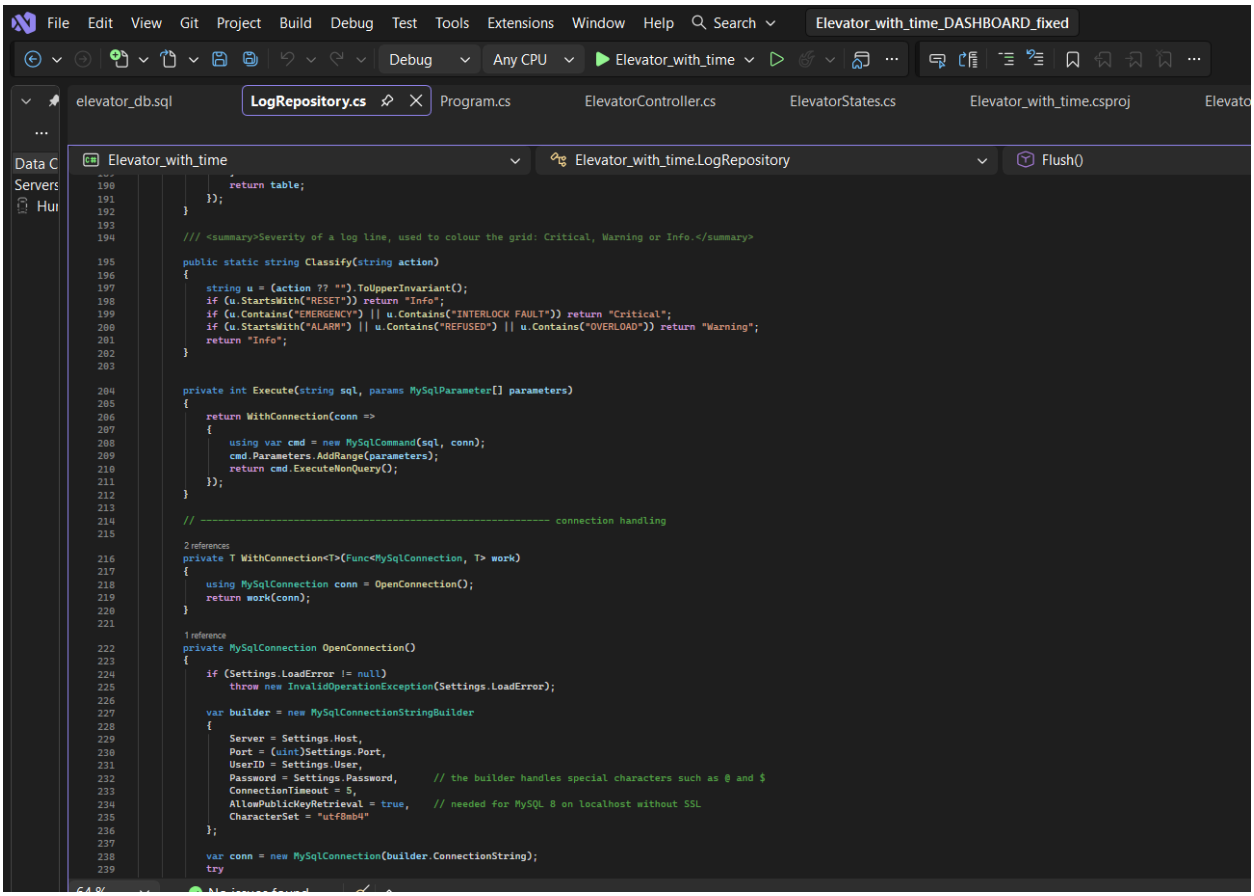
    /// <summary>Connection settings, read from dbsettings.json (relative to the program, not hard-coded).</summary>
    7 references
    public class DbSettings
    {
        3 references
        public string Host { get; set; } = "127.0.0.1";
        3 references
        public int Port { get; set; } = 3306;
        3 references
        public string User { get; set; } = "root";
        1 reference
        public string Password { get; set; } = "";
        5 references
        public string Database { get; set; } = "elevator_db";

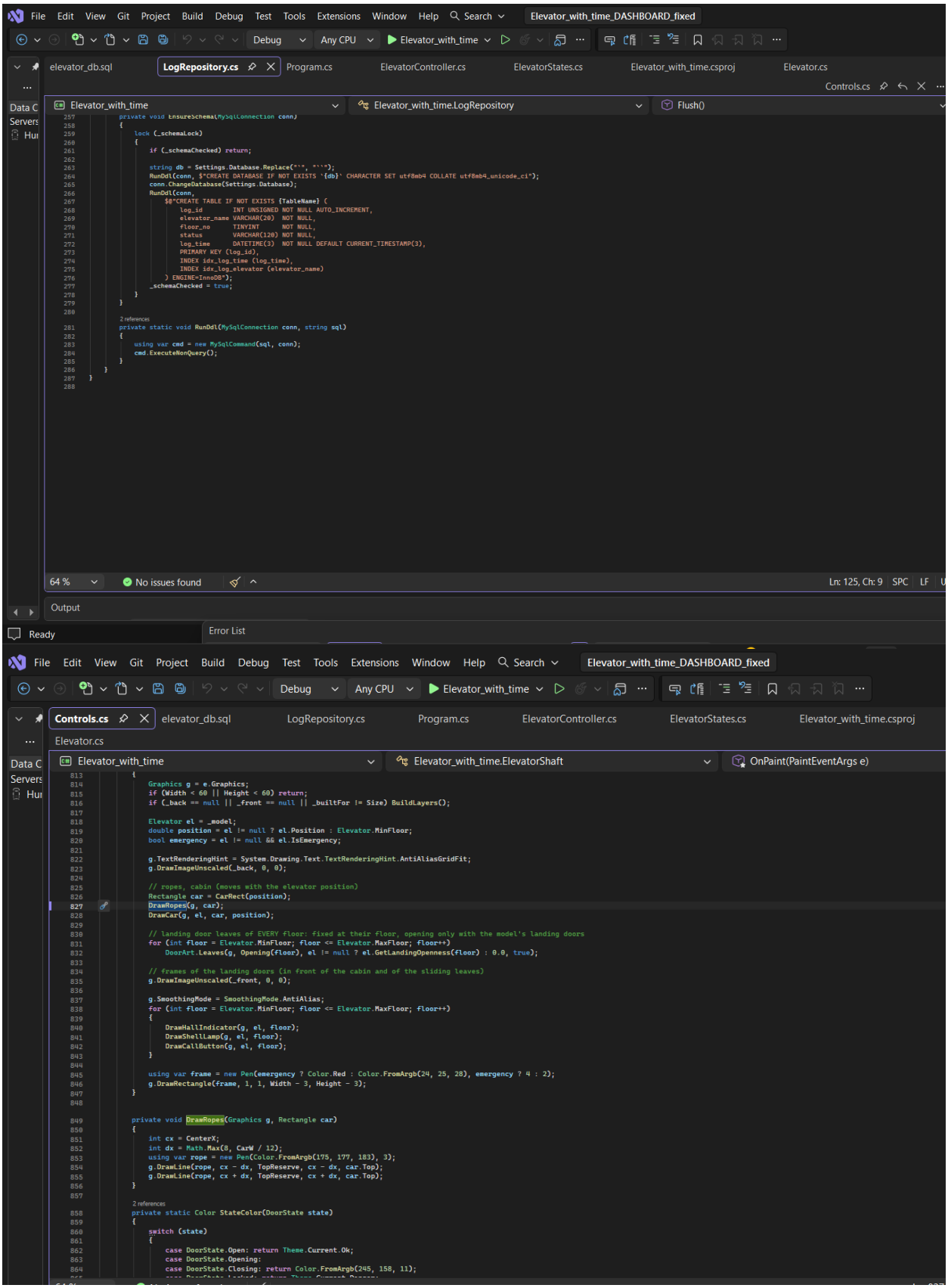
        [System.Text.Json.Serialization.JsonIgnore]
        4 references
        public string LoadError { get; private set; }

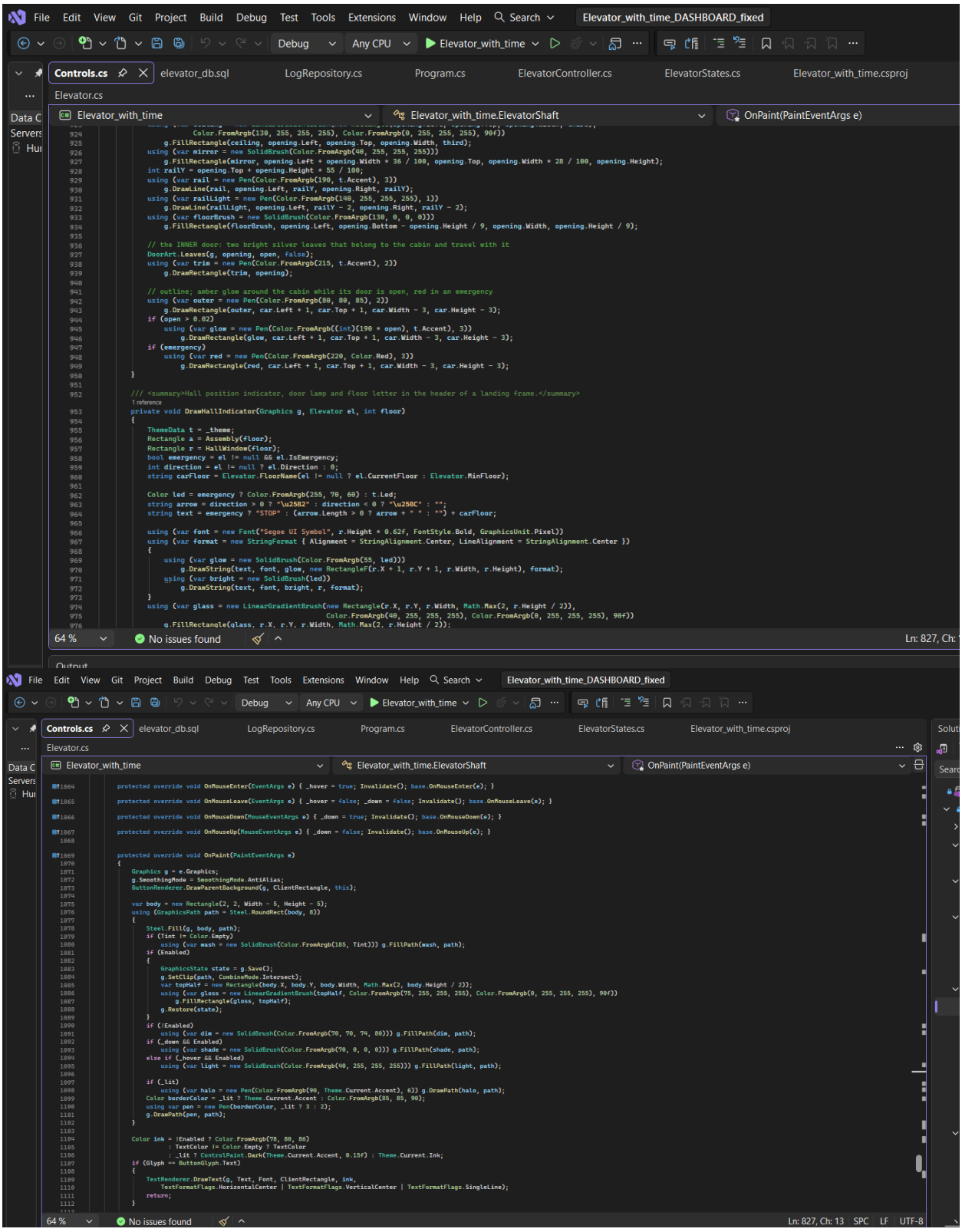
        1 reference
        public static DbSettings Load()
        {
            try
            {
                // look next to the exe first, then in the parent folders (works from Visual Studio's bin\Debug)
                string dir = AppContext.BaseDirectory;
                for (int i = 0; i < 7 && dir != null; i++)
                {
                    string file = Path.Combine(dir, "dbsettings.json");
                    if (File.Exists(file))
                    {
                        var s = JsonSerializer.Deserialize<DbSettings>(File.ReadAllText(file),
                            new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
                        return s ?? new DbSettings();
                    }
                    dir = Path.GetDirectoryName(dir.TrimEnd(Path.DirectorySeparatorChar));
                }
                return new DbSettings { LoadError = "dbsettings.json was not found next to the program." };
            }
            catch (Exception ex)
            {
                return new DbSettings { LoadError = "dbsettings.json could not be read: " + ex.Message };
            }
        }
```

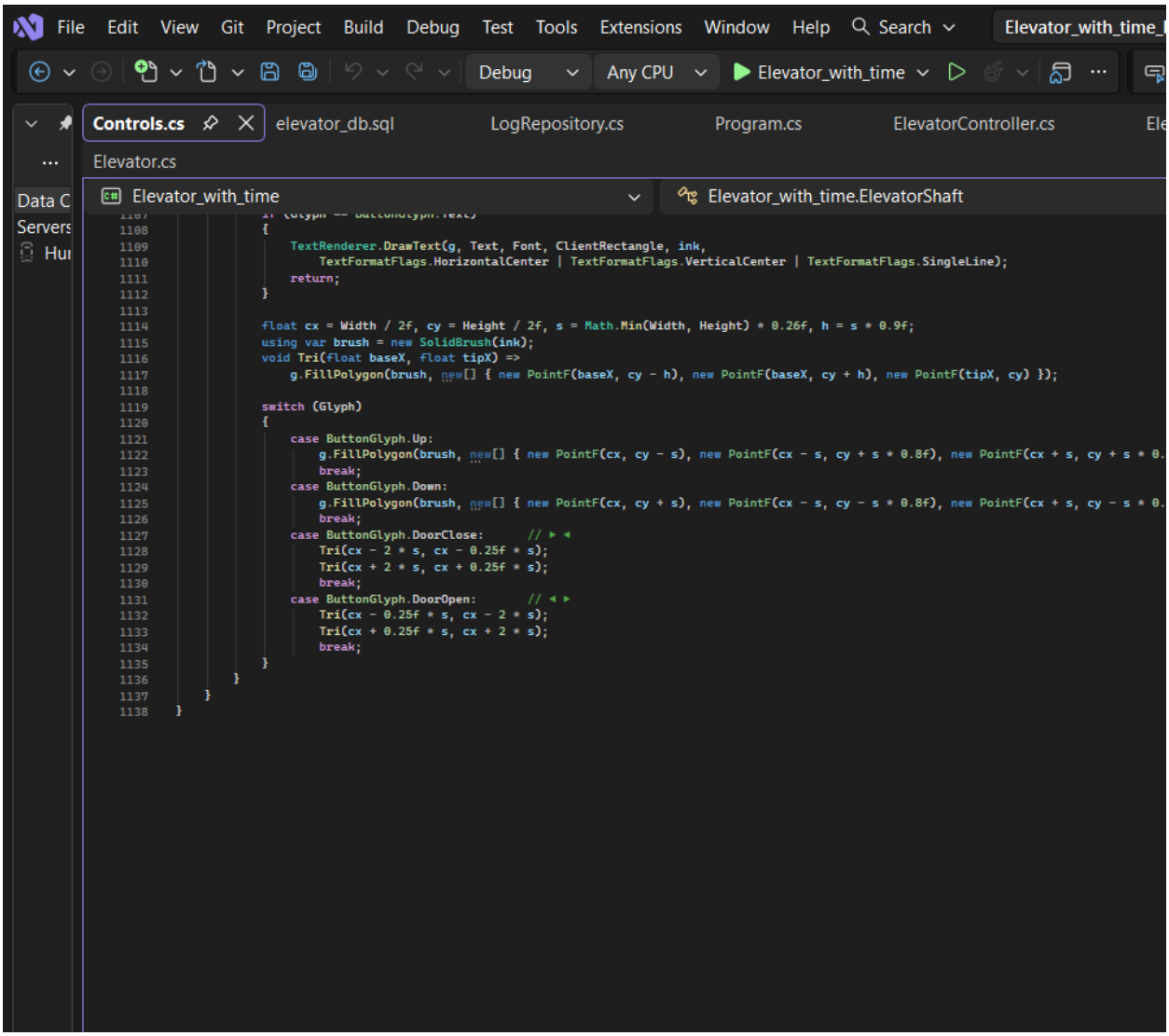

File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator_with_time
Elevator_with_time.LogRepository
Flush()
4 references
private readonly BackgroundWorker _writer = new();
1 reference
private readonly object _schemaLock = new();
2 references
private bool _schemaChecked;
3 references
private volatile bool _wroteSomething;
/// <summary>Raised on the GUI thread when a database operation fails.</summary>
3 references
public event Action<string> ErrorOccurred;
/// <summary>Raised on the GUI thread after new log entries were stored.</summary>
2 references
public event Action EntriesWritten;
3 references
public static string Description =>
$"MySQL {Settings.User}@{Settings.Host}:{Settings.Port} / {Settings.Database}";
1 reference
public LogRepository()
{
_writer.DoWork += Writer_DoWork;
_writer.RunWorkerCompleted += Writer_Completed;
}
// ---------- writing (queued)
1 reference
public void Enqueue(LogEntry entry)
{
_queue.Enqueue(entry);
StartWriterIfNeeded();
}
2 references
private void StartWriterIfNeeded()
{
if (!_writer.IsBusy && !_queue.IsEmpty)
_writer.RunWorkerAsync();
}
1 reference
private void Writer_DoWork(object sender, DoWorkEventArgs e)
{
while (_queue.TryDequeue(out LogEntry entry))
{
try
{
Insert(entry);
_wroteSomething = true;
}

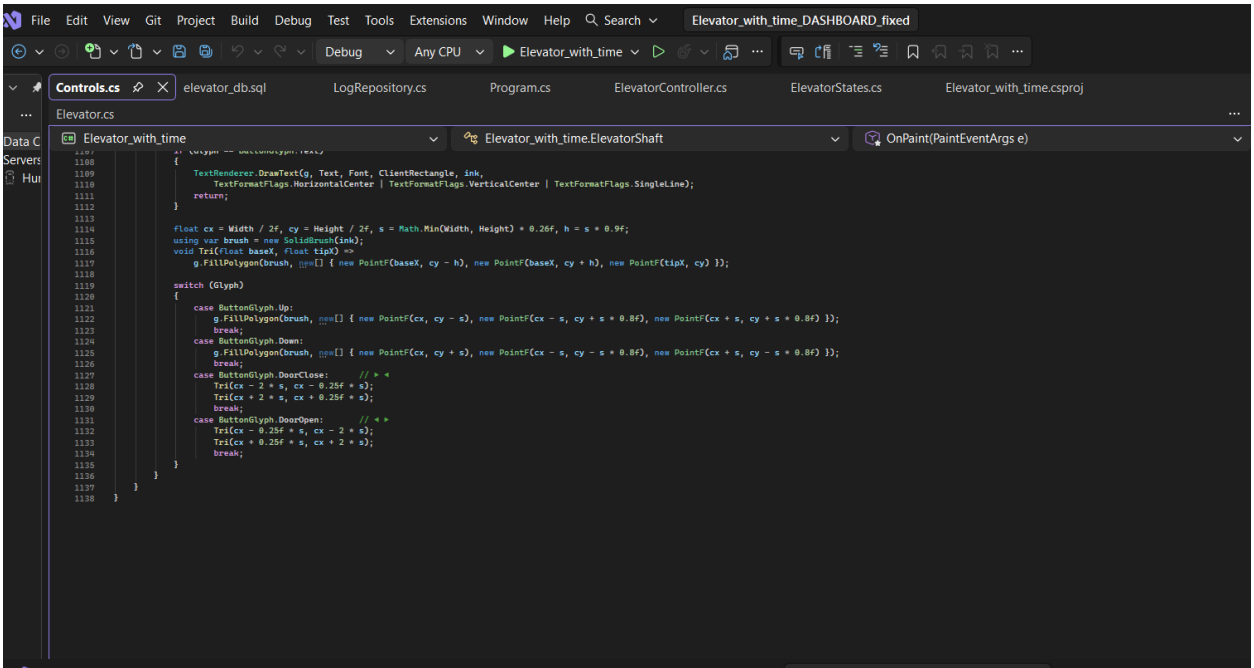

File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator_with_time
Elevator_with_time.LogRepository
Flush()
1 reference
private void Writer_Completed(object sender, RunWorkerCompletedEventArgs e)
{
if (e.Error != null) ErrorOccurred?.Invoke(e.Error.Message);
else if (e.Result is string message) ErrorOccurred?.Invoke(message);
if (_wroteSomething)
{
_wroteSomething = false;
EntriesWritten?.Invoke();
}
StartWriterIfNeeded();
}
/// <summary>Writes anything still queued (called when the form closes).</summary>
1 reference
public void Flush()
{
while (_queue.TryDequeue(out LogEntry entry))
{
try { Insert(entry); } catch { break; }
}
}
// ---------- reading / clearing (async)
1 reference
public void LoadLogsAsync(Action<DataTable> onLoaded, Action<string> onError) =>
RunAsync(ReadAll, onLoaded, onError);
1 reference
public void ClearAsync(Action onDone, Action<string> onError) =>
RunAsync(() => Execute($"TRUNCATE TABLE {TableName}"), _ => onDone(), onError);
/// <summary>One place that runs work on a BackgroundWorker and reports back on the GUI thread.</summary>
2 references
private static void RunAsync<T>(Func<T> work, Action<T> onDone, Action<string> onError)
{
var worker = new BackgroundWorker();
worker.DoWork += (s, e) => e.Result = work();
worker.RunWorkerCompleted += (s, e) =>
{
if (e.Error != null) onError(e.Error.Message);
else onDone((T)e.Result);
worker.Dispose();
};
worker.RunWorkerAsync();
}
// ---------- SQL
2 references
private void Insert(LogEntry entry)

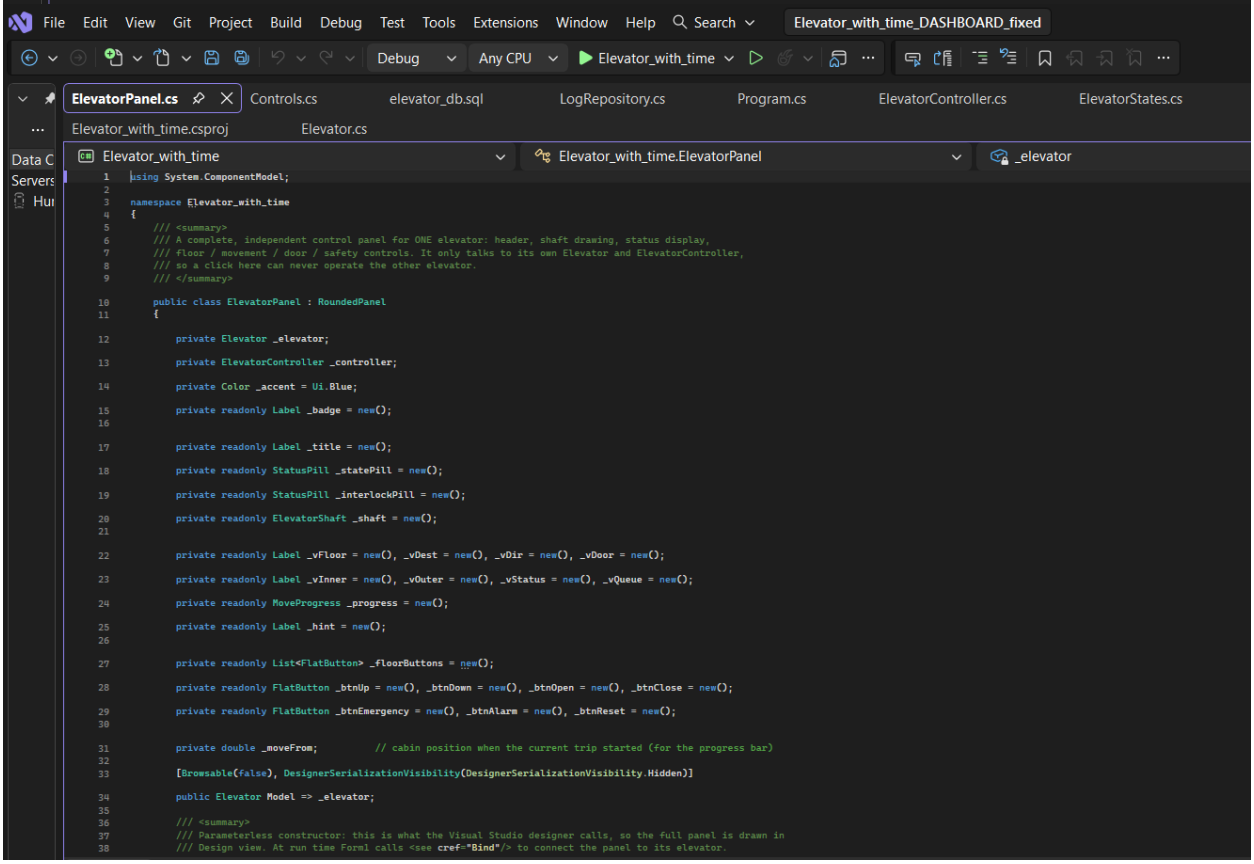

File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator_with_time
Elevator_with_time.LogRepository
Flush()
return table;
});
}
/// <summary>Severity of a log line, used to colour the grid: Critical, Warning or Info.</summary>
public static string Classify(string action)
{
string u = (action ?? "").ToUpperInvariant();
if (u.StartsWith("RESET")) return "Info";
if (u.Contains("EMERGENCY") || u.Contains("INTERLOCK FAULT")) return "Critical";
if (u.StartsWith("ALARM") || u.Contains("REFUSED") || u.Contains("OVERLOAD")) return "Warning";
return "Info";
}
private int Execute(string sql, params MySqlParameter[] parameters)
{
return WithConnection(conn =>
{
using var cmd = new MySqlCommand(sql, conn);
cmd.Parameters.AddRange(parameters);
return cmd.ExecuteNonQuery();
});
}
// ---------- connection handling
2 references
private T WithConnection<T>(Func<MySqlConnection, T> work)
{
using MySqlConnection conn = OpenConnection();
return work(conn);
}
1 reference
private MySqlConnection OpenConnection()
{
if (Settings.LoadError != null)
throw new InvalidOperationException(Settings.LoadError);
var builder = new MySqlConnectionStringBuilder
{
Server = Settings.Host,
Port = (uint)Settings.Port,
UserID = Settings.User,
Password = Settings.Password,
// the builder handles special characters such as @ and $
ConnectionTimeout = 5,
AllowPublicKeyRetrieval = true,
// needed for MySQL 8 on localhost without SSL
CharacterSet = "utf8mb4"
};
var conn = new MySqlConnection(builder.ConnectionString);
try

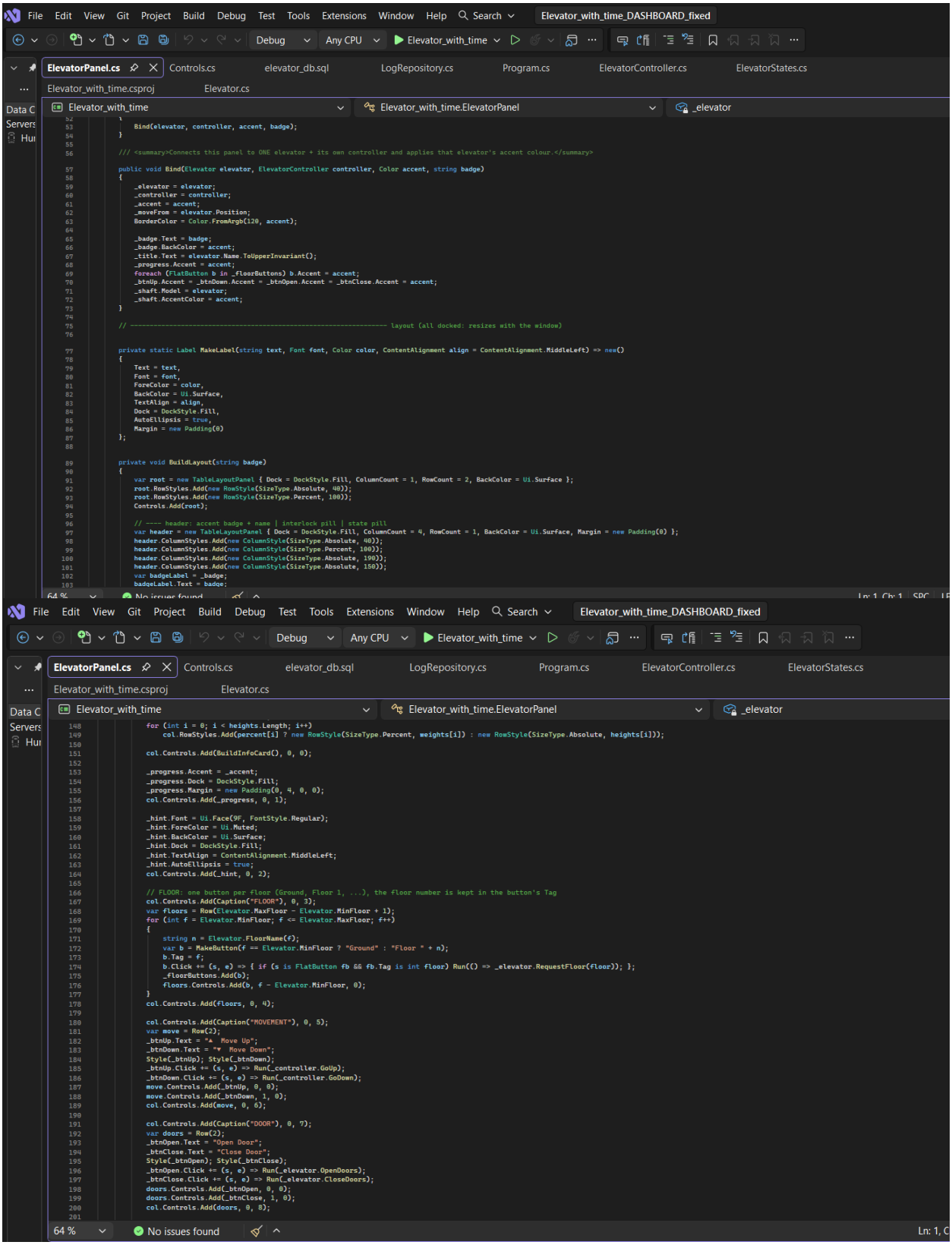



File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.ElevatorShaft
OnPaint(PaintEventArgs e)
No issues found
Ln: 827, Ch: 13
SPC
LF
UTF-8


File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Debug
Any CPU
Elevator_with_time
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
Elevator.cs
Elevator_with_time
Elevator_with_time.ElevatorShaft
{
TextRenderer.DrawText(g, Text, Font, ClientRectangle, ink,
TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter | TextFormatFlags.SingleLine);
return;
}
float cx = Width / 2f, cy = Height / 2f, s = Math.Min(Width, Height) * 0.26f, h = s * 0.9f;
using var brush = new SolidBrush(ink);
void Tri(float baseX, float tipX) =>
g.FillPolygon(brush, new[] { new PointF(baseX, cy - h), new PointF(baseX, cy + h), new PointF(tipX, cy) });
switch (Glyph)
{
case ButtonGlyph.Up:
g.FillPolygon(brush, new[] { new PointF(cx, cy - s), new PointF(cx - s, cy + s * 0.8f), new PointF(cx + s, cy + s * 0.
break;
case ButtonGlyph.Down:
g.FillPolygon(brush, new[] { new PointF(cx, cy + s), new PointF(cx - s, cy - s * 0.8f), new PointF(cx + s, cy - s * 0.
break;
case ButtonGlyph.DoorClose: // ▸ ◂
Tri(cx - 2 * s, cx - 0.25f * s);
Tri(cx + 2 * s, cx + 0.25f * s);
break;
case ButtonGlyph.DoorOpen: // ◂ ▸
Tri(cx - 0.25f * s, cx - 2 * s);
Tri(cx + 0.25f * s, cx + 2 * s);
break;
}
}
}
}


File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.ElevatorShaft
OnPaint(PaintEventArgs e)
TextRenderer.DrawText(g, Text, Font, ClientRectangle, ink,
TextFormatFlags.HorizontalCenter | TextFormatFlags.VerticalCenter | TextFormatFlags.SingleLine);
return;
float cx = Width / 2f, cy = Height / 2f, s = Math.Min(Width, Height) * 0.26f, h = s * 0.9f;
using var brush = new SolidBrush(ink);
void Tri(float baseX, float tipX) =>
g.FillPolygon(brush, new[] { new PointF(baseX, cy - h), new PointF(baseX, cy + h), new PointF(tipX, cy) });
switch (Glyph)
case ButtonGlyph.Up:
g.FillPolygon(brush, new[] { new PointF(cx, cy - s), new PointF(cx - s, cy + s * 0.8f), new PointF(cx + s, cy + s * 0.8f) });
break;
case ButtonGlyph.Down:
g.FillPolygon(brush, new[] { new PointF(cx, cy + s), new PointF(cx - s, cy - s * 0.8f), new PointF(cx + s, cy - s * 0.8f) });
break;
case ButtonGlyph.DoorClose:
Tri(cx - 2 * s, cx - 0.25f * s);
Tri(cx + 2 * s, cx + 0.25f * s);
break;
case ButtonGlyph.DoorOpen:
Tri(cx - 0.25f * s, cx - 2 * s);
Tri(cx + 0.25f * s, cx + 2 * s);
break;


File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
ElevatorPanel.cs
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.ElevatorPanel
_elevator
using System.ComponentModel;
namespace Elevator_with_time
{
/// <summary>
/// A complete, independent control panel for ONE elevator: header, shaft drawing, status display,
/// floor / movement / door / safety controls. It only talks to its own Elevator and ElevatorController,
/// so a click here can never operate the other elevator.
/// </summary>
public class ElevatorPanel : RoundedPanel
{
private Elevator _elevator;
private ElevatorController _controller;
private Color _accent = Ui.Blue;
private readonly Label _badge = new();
private readonly Label _title = new();
private readonly StatusPill _statePill = new();
private readonly StatusPill _interlockPill = new();
private readonly ElevatorShaft _shaft = new();
private readonly Label _vFloor = new(), _vDest = new(), _vDir = new(), _vDoor = new();
private readonly Label _vInner = new(), _vOuter = new(), _vStatus = new(), _vQueue = new();
private readonly MoveProgress _progress = new();
private readonly Label _hint = new();
private readonly List<FlatButton> _floorButtons = new();
private readonly FlatButton _btnUp = new(), _btnDown = new(), _btnOpen = new(), _btnClose = new();
private readonly FlatButton _btnEmergency = new(), _btnAlarm = new(), _btnReset = new();
private double _moveFrom; // cabin position when the current trip started (for the progress bar)
[Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
public Elevator Model => _elevator;
/// <summary>
/// Parameterless constructor: this is what the Visual Studio designer calls, so the full panel is drawn in
/// Design view. At run time Form1 calls <see cref="Bind"/> to connect the panel to its elevator.



```
        Bind(elevator, controller, accent, badge);
    }

    /// <summary>Connects this panel to ONE elevator + its own controller and applies that elevator's accent colour.</summary>

    public void Bind(Elevator elevator, ElevatorController controller, Color accent, string badge)
    {
        _elevator = elevator;
        _controller = controller;
        _accent = accent;
        _moveFrom = elevator.Position;
        BorderColor = Color.FromArgb(120, accent);

        _badge.Text = badge;
        _badge.BackColor = accent;
        _title.Text = elevator.Name.ToUpperInvariant();
        _progress.Accent = accent;
        foreach (FlatButton b in _floorButtons) b.Accent = accent;
        _btnUp.Accent = _btnDown.Accent = _btnOpen.Accent = _btnClose.Accent = accent;
        _shaft.Model = elevator;
        _shaft.AccentColor = accent;
    }

    // ------------------------------------------------------------ layout (all docked: resizes with the window)


    private static Label MakeLabel(string text, Font font, Color color, ContentAlignment align = ContentAlignment.MiddleLeft) => new()
    {
        Text = text,
        Font = font,
        ForeColor = color,
        BackColor = Ui.Surface,
        TextAlign = align,
        Dock = DockStyle.Fill,
        AutoEllipsis = true,
        Margin = new Padding(0)
    };


    private void BuildLayout(string badge)
    {
        var root = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 1, RowCount = 2, BackColor = Ui.Surface };
        root.RowStyles.Add(new RowStyle(SizeType.Absolute, 40));
        root.RowStyles.Add(new RowStyle(SizeType.Percent, 100));
        Controls.Add(root);

        // ---- header: accent badge + name | interlock pill | state pill
        var header = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 4, RowCount = 1, BackColor = Ui.Surface, Margin = new Padding(0) };
        header.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 40));
        header.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100));
        header.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 190));
        header.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 150));
        var badgeLabel = _badge;
        badgeLabel.Text = badge;
```

```
            for (int i = 0; i < heights.Length; i++)
                col.RowStyles.Add(percent[i] ? new RowStyle(SizeType.Percent, weights[i]) : new RowStyle(SizeType.Absolute, heights[i]));

            col.Controls.Add(BuildInfoCard(), 0, 0);

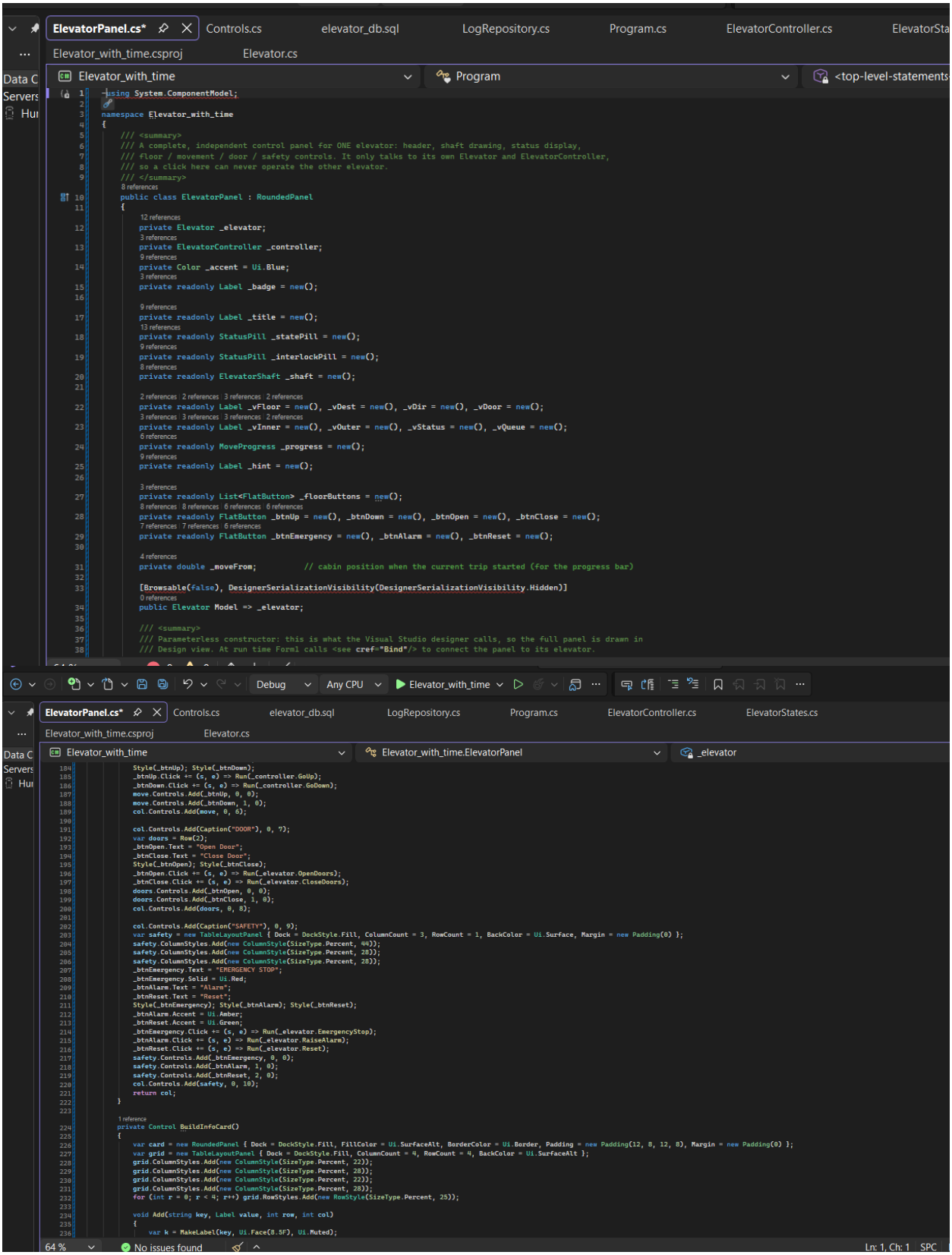
            _progress.Accent = _accent;
            _progress.Dock = DockStyle.Fill;
            _progress.Margin = new Padding(0, 4, 0, 0);
            col.Controls.Add(_progress, 0, 1);

            _hint.Font = Ui.Face(9F, FontStyle.Regular);
            _hint.ForeColor = Ui.Muted;
            _hint.BackColor = Ui.Surface;
            _hint.Dock = DockStyle.Fill;
            _hint.TextAlign = ContentAlignment.MiddleLeft;
            _hint.AutoEllipsis = true;
            col.Controls.Add(_hint, 0, 2);

            // FLOOR: one button per floor (Ground, Floor 1, ...), the floor number is kept in the button's Tag
            col.Controls.Add(Caption("FLOOR"), 0, 3);
            var floors = Row(Elevator.MaxFloor - Elevator.MinFloor + 1);
            for (int f = Elevator.MinFloor; f <= Elevator.MaxFloor; f++)
            {
                string n = Elevator.FloorName(f);
                var b = MakeButton(f == Elevator.MinFloor ? "Ground" : "Floor " + n);
                b.Tag = f;
                b.Click += (s, e) => { if (s is FlatButton fb && fb.Tag is int floor) Run(() => _elevator.RequestFloor(floor)); };
                _floorButtons.Add(b);
                floors.Controls.Add(b, f - Elevator.MinFloor, 0);
            }
            col.Controls.Add(floors, 0, 4);

            col.Controls.Add(Caption("MOVEMENT"), 0, 5);
            var move = Row(2);
            _btnUp.Text = "▲  Move Up";
            _btnDown.Text = "▼  Move Down";
            Style(_btnUp); Style(_btnDown);
            _btnUp.Click += (s, e) => Run(_controller.GoUp);
            _btnDown.Click += (s, e) => Run(_controller.GoDown);
            move.Controls.Add(_btnUp, 0, 0);
            move.Controls.Add(_btnDown, 1, 0);
            col.Controls.Add(move, 0, 6);

            col.Controls.Add(Caption("DOOR"), 0, 7);
            var doors = Row(2);
            _btnOpen.Text = "Open Door";
            _btnClose.Text = "Close Door";
            Style(_btnOpen); Style(_btnClose);
            _btnOpen.Click += (s, e) => Run(_elevator.OpenDoors);
            _btnClose.Click += (s, e) => Run(_elevator.CloseDoors);
            doors.Controls.Add(_btnOpen, 0, 0);
            doors.Controls.Add(_btnClose, 1, 0);
            col.Controls.Add(doors, 0, 8);
```



```
using System.ComponentModel;

namespace Elevator_with_time
{
    /// <summary>
    /// A complete, independent control panel for ONE elevator: header, shaft drawing, status display,
    /// floor / movement / door / safety controls. It only talks to its own Elevator and ElevatorController,
    /// so a click here can never operate the other elevator.
    /// </summary>
    8 references
    public class ElevatorPanel : RoundedPanel
    {
        12 references
        private Elevator _elevator;
        3 references
        private ElevatorController _controller;
        9 references
        private Color _accent = Ui.Blue;
        3 references
        private readonly Label _badge = new();

        9 references
        private readonly Label _title = new();
        13 references
        private readonly StatusPill _statePill = new();
        9 references
        private readonly StatusPill _interlockPill = new();
        8 references
        private readonly ElevatorShaft _shaft = new();

        2 references | 2 references | 3 references | 2 references
        private readonly Label _vFloor = new(), _vDest = new(), _vDir = new(), _vDoor = new();
        3 references | 3 references | 3 references | 2 references
        private readonly Label _vInner = new(), _vOuter = new(), _vStatus = new(), _vQueue = new();
        6 references
        private readonly MoveProgress _progress = new();
        9 references
        private readonly Label _hint = new();

        3 references
        private readonly List<FlatButton> _floorButtons = new();
        8 references | 8 references | 6 references | 6 references
        private readonly FlatButton _btnUp = new(), _btnDown = new(), _btnOpen = new(), _btnClose = new();
        7 references | 7 references | 6 references
        private readonly FlatButton _btnEmergency = new(), _btnAlarm = new(), _btnReset = new();

        4 references
        private double _moveFrom;          // cabin position when the current trip started (for the progress bar)

        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        0 references
        public Elevator Model => _elevator;

        /// <summary>
        /// Parameterless constructor: this is what the Visual Studio designer calls, so the full panel is drawn in
        /// Design view. At run time Form1 calls <see cref="Bind"/> to connect the panel to its elevator.
```

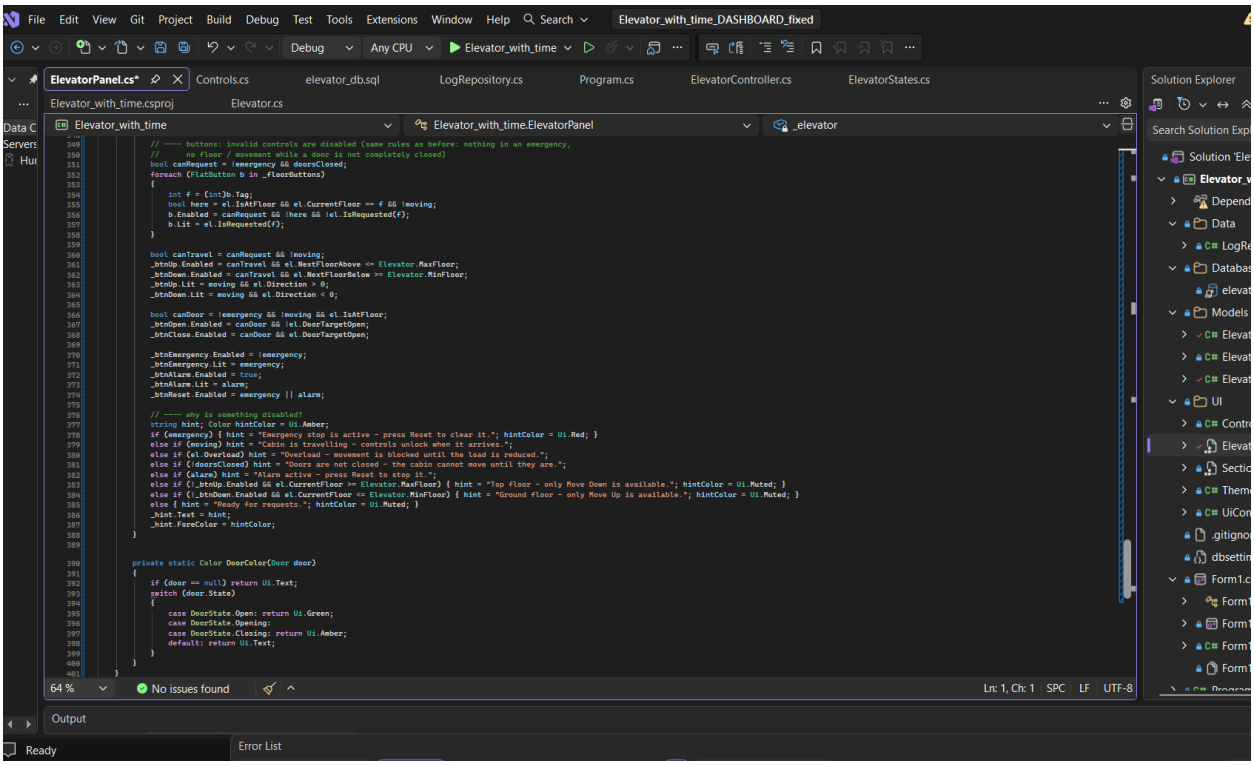
```
            Style(_btnUp); Style(_btnDown);
            _btnUp.Click += (s, e) => Run(_controller.GoUp);
            _btnDown.Click += (s, e) => Run(_controller.GoDown);
            move.Controls.Add(_btnUp, 0, 0);
            move.Controls.Add(_btnDown, 1, 0);
            col.Controls.Add(move, 0, 6);

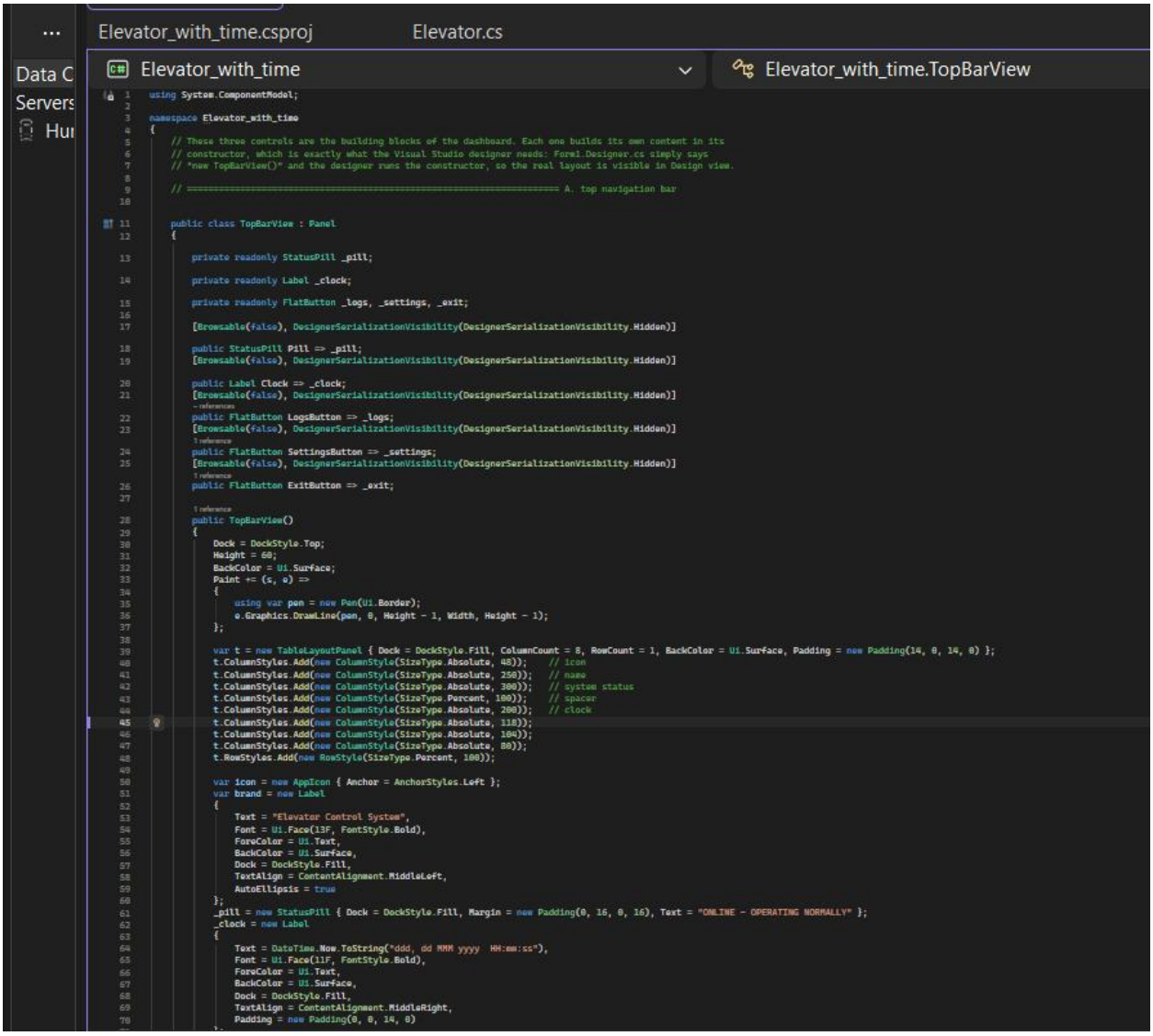
            col.Controls.Add(Caption("DOOR"), 0, 7);
            var doors = Row(2);
            _btnOpen.Text = "Open Door";
            _btnClose.Text = "Close Door";
            Style(_btnOpen); Style(_btnClose);
            _btnOpen.Click += (s, e) => Run(_elevator.OpenDoors);
            _btnClose.Click += (s, e) => Run(_elevator.CloseDoors);
            doors.Controls.Add(_btnOpen, 0, 0);
            doors.Controls.Add(_btnClose, 1, 0);
            col.Controls.Add(doors, 0, 8);

            col.Controls.Add(Caption("SAFETY"), 0, 9);
            var safety = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 3, RowCount = 1, BackColor = Ui.Surface, Margin = new Padding(0) };
            safety.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 44));
            safety.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 28));
            safety.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 28));
            _btnEmergency.Text = "EMERGENCY STOP";
            _btnEmergency.Solid = Ui.Red;
            _btnAlarm.Text = "Alarm";
            _btnReset.Text = "Reset";
            Style(_btnEmergency); Style(_btnAlarm); Style(_btnReset);
            _btnAlarm.Accent = Ui.Amber;
            _btnReset.Accent = Ui.Green;
            _btnEmergency.Click += (s, e) => Run(_elevator.EmergencyStop);
            _btnAlarm.Click += (s, e) => Run(_elevator.RaiseAlarm);
            _btnReset.Click += (s, e) => Run(_elevator.Reset);
            safety.Controls.Add(_btnEmergency, 0, 0);
            safety.Controls.Add(_btnAlarm, 1, 0);
            safety.Controls.Add(_btnReset, 2, 0);
            col.Controls.Add(safety, 0, 10);
            return col;
        }

        1 reference
        private Control BuildInfoCard()
        {
            var card = new RoundedPanel { Dock = DockStyle.Fill, FillColor = Ui.SurfaceAlt, BorderColor = Ui.Border, Padding = new Padding(12, 8, 12, 8), Margin = new Padding(0) };
            var grid = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 4, RowCount = 4, BackColor = Ui.SurfaceAlt };
            grid.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 22));
            grid.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 28));
            grid.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 22));
            grid.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 28));
            for (int r = 0; r < 4; r++) grid.RowStyles.Add(new RowStyle(SizeType.Percent, 25));

            void Add(string key, Label value, int row, int col)
            {
                var k = MakeLabel(key, Ui.Face(8.5F), Ui.Muted);
```


File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.ElevatorPanel
_elevator
// ---- buttons: invalid controls are disabled (same rules as before: nothing in an emergency,
// no floor / movement while a door is not completely closed)
bool canRequest = !emergency && doorsClosed;
foreach (FlatButton b in _floorButtons)
{
int f = (int)b.Tag;
bool here = el.IsAtFloor && el.CurrentFloor == f && !moving;
b.Enabled = canRequest && !here && !el.IsRequested(f);
b.Lit = el.IsRequested(f);
}
bool canTravel = canRequest && !moving;
_btnUp.Enabled = canTravel && el.NextFloorAbove <= Elevator.MaxFloor;
_btnDown.Enabled = canTravel && el.NextFloorBelow >= Elevator.MinFloor;
_btnUp.Lit = moving && el.Direction > 0;
_btnDown.Lit = moving && el.Direction < 0;
bool canDoor = !emergency && !moving && el.IsAtFloor;
_btnOpen.Enabled = canDoor && !el.DoorTargetOpen;
_btnClose.Enabled = canDoor && el.DoorTargetOpen;
_btnEmergency.Enabled = !emergency;
_btnEmergency.Lit = emergency;
_btnAlarm.Enabled = true;
_btnAlarm.Lit = alarm;
_btnReset.Enabled = emergency || alarm;
// ---- why is something disabled?
string hint; Color hintColor = Ui.Amber;
if (emergency) { hint = "Emergency stop is active - press Reset to clear it."; hintColor = Ui.Red; }
else if (moving) hint = "Cabin is travelling - controls unlock when it arrives.";
else if (el.Overload) hint = "Overload - movement is blocked until the load is reduced.";
else if (!doorsClosed) hint = "Doors are not closed - the cabin cannot move until they are.";
else if (alarm) hint = "Alarm active - press Reset to stop it.";
else if (!_btnUp.Enabled && el.CurrentFloor >= Elevator.MaxFloor) { hint = "Top floor - only Move Down is available."; hintColor = Ui.Muted; }
else if (!_btnDown.Enabled && el.CurrentFloor <= Elevator.MinFloor) { hint = "Ground floor - only Move Up is available."; hintColor = Ui.Muted; }
else { hint = "Ready for requests."; hintColor = Ui.Muted; }
_hint.Text = hint;
_hint.ForeColor = hintColor;
}
private static Color DoorColor(Door door)
{
if (door == null) return Ui.Text;
switch (door.State)
{
case DoorState.Open: return Ui.Green;
case DoorState.Opening:
case DoorState.Closing: return Ui.Amber;
default: return Ui.Text;
}
}
}
64 %
No issues found
Ln: 1, Ch: 1
SPC
LF
UTF-8
Output
Ready
Error List
Solution Explorer
Search Solution Explorer
Solution 'Elevator_with_time'
Elevator_with_time
Dependencies
Data
LogRepository.cs
Database
elevator_db.sql
Models
Elevator.cs
ElevatorController.cs
ElevatorStates.cs
UI
Controls.cs
ElevatorPanel.cs
SectionPanel.cs
ThemeManager.cs
UiControls.cs
.gitignore
dbsettings.ini
Form1.cs
Form1.Designer.cs
Form1.resx
Form2.cs
Form2.Designer.cs
Form2.resx
Program.cs



```
Elevator_with_time.csproj        Elevator.cs
Elevator_with_time               Elevator_with_time.TopBarView
```

```
using System.ComponentModel;

namespace Elevator_with_time
{
    // These three controls are the building blocks of the dashboard. Each one builds its own content in its
    // constructor, which is exactly what the Visual Studio designer needs: Form1.Designer.cs simply says
    // "new TopBarView()" and the designer runs the constructor, so the real layout is visible in Design view.

    // ============================================================ A. top navigation bar


    public class TopBarView : Panel
    {

        private readonly StatusPill _pill;

        private readonly Label _clock;

        private readonly FlatButton _logs, _settings, _exit;

        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

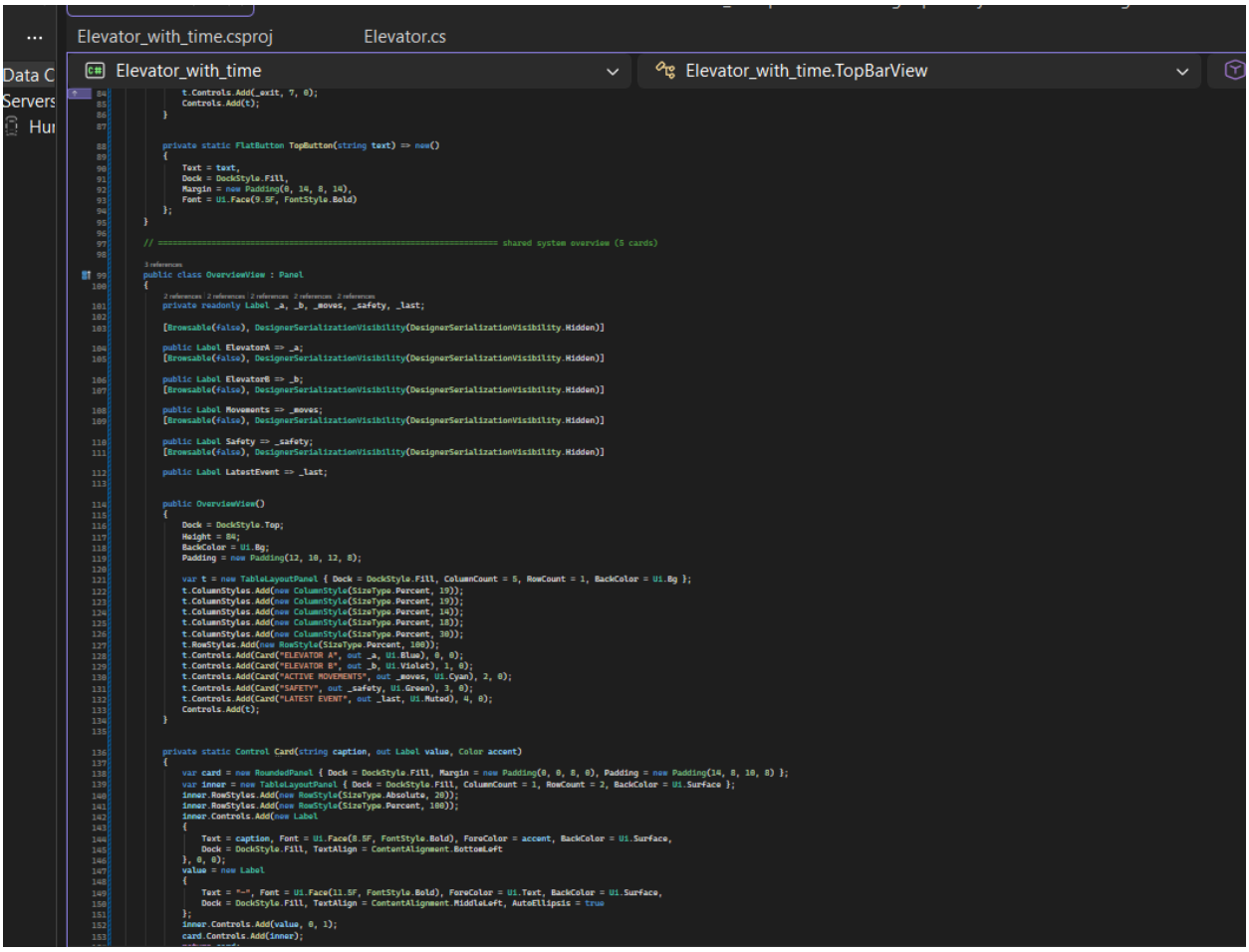
        public StatusPill Pill => _pill;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

        public Label Clock => _clock;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        0 references
        public FlatButton LogsButton => _logs;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        1 reference
        public FlatButton SettingsButton => _settings;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        1 reference
        public FlatButton ExitButton => _exit;

        1 reference
        public TopBarView()
        {
            Dock = DockStyle.Top;
            Height = 60;
            BackColor = Ui.Surface;
            Paint += (s, e) =>
            {
                using var pen = new Pen(Ui.Border);
                e.Graphics.DrawLine(pen, 0, Height - 1, Width, Height - 1);
            };

            var t = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 8, RowCount = 1, BackColor = Ui.Surface, Padding = new Padding(14, 0, 14, 0) };
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 48));   // icon
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 250));  // name
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 300));  // system status
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100));   // spacer
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 200));  // clock
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 118));
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 104));
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 88));
            t.RowStyles.Add(new RowStyle(SizeType.Percent, 100));

            var icon = new AppIcon { Anchor = AnchorStyles.Left };
            var brand = new Label
            {
                Text = "Elevator Control System",
                Font = Ui.Face(13F, FontStyle.Bold),
                ForeColor = Ui.Text,
                BackColor = Ui.Surface,
                Dock = DockStyle.Fill,
                TextAlign = ContentAlignment.MiddleLeft,
                AutoEllipsis = true
            };
            _pill = new StatusPill { Dock = DockStyle.Fill, Margin = new Padding(0, 16, 0, 16), Text = "ONLINE - OPERATING NORMALLY" };
            _clock = new Label
            {
                Text = DateTime.Now.ToString("ddd, dd MMM yyyy  HH:mm:ss"),
                Font = Ui.Face(11F, FontStyle.Bold),
                ForeColor = Ui.Text,
                BackColor = Ui.Surface,
                Dock = DockStyle.Fill,
                TextAlign = ContentAlignment.MiddleRight,
                Padding = new Padding(0, 0, 14, 0)
            };
```




```
            t.Controls.Add(_exit, 7, 0);
            Controls.Add(t);
        }


        private static FlatButton TopButton(string text) => new()
        {
            Text = text,
            Dock = DockStyle.Fill,
            Margin = new Padding(0, 14, 8, 14),
            Font = Ui.Face(9.5F, FontStyle.Bold)
        };
    }

    // ============================================================ shared system overview (5 cards)


    3 references
    public class OverviewView : Panel
    {
        2 references | 2 references | 2 references | 2 references | 2 references
        private readonly Label _a, _b, _moves, _safety, _last;

        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

        public Label ElevatorA => _a;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

        public Label ElevatorB => _b;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

        public Label Movements => _moves;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

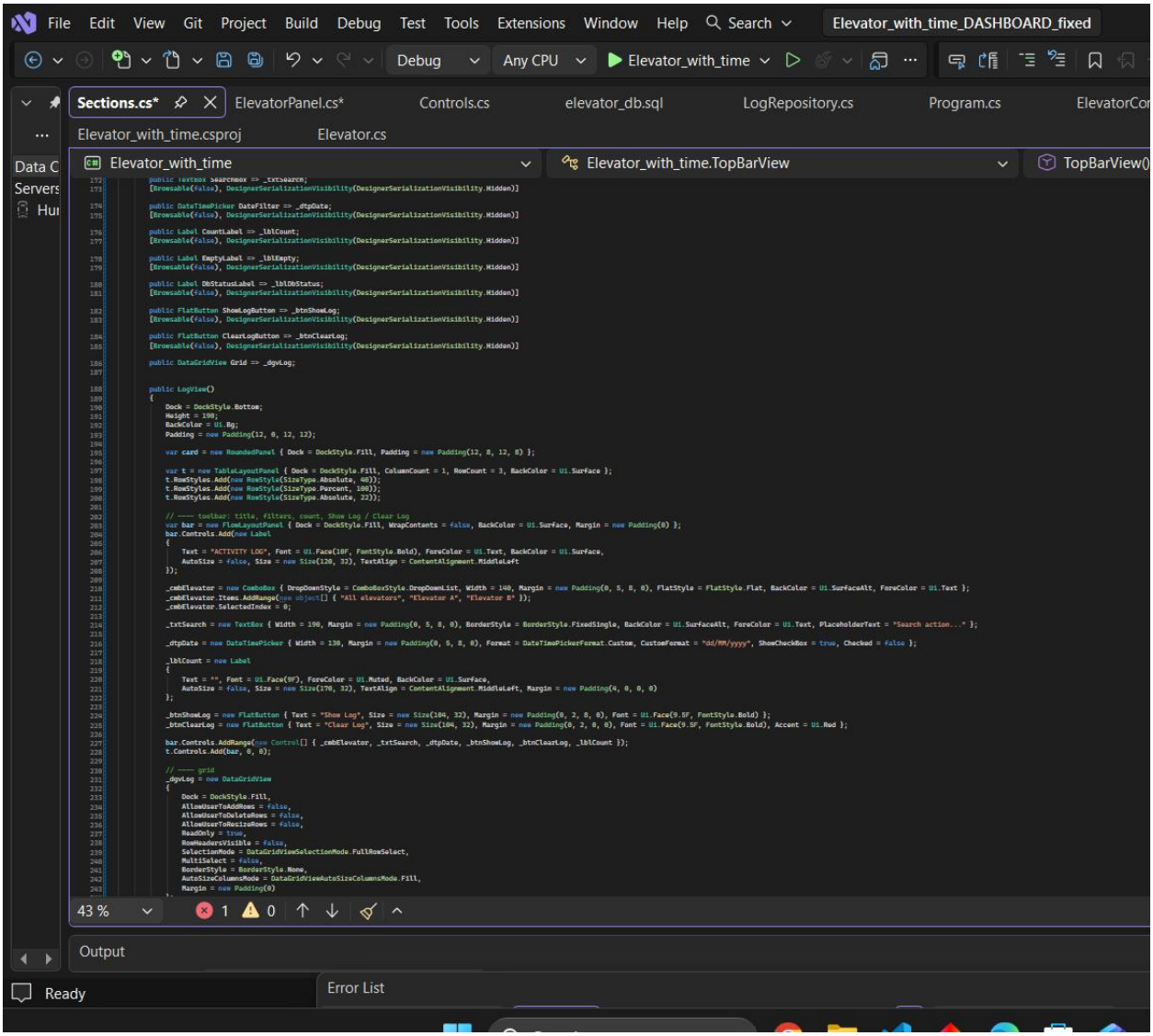
        public Label Safety => _safety;
        [Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]

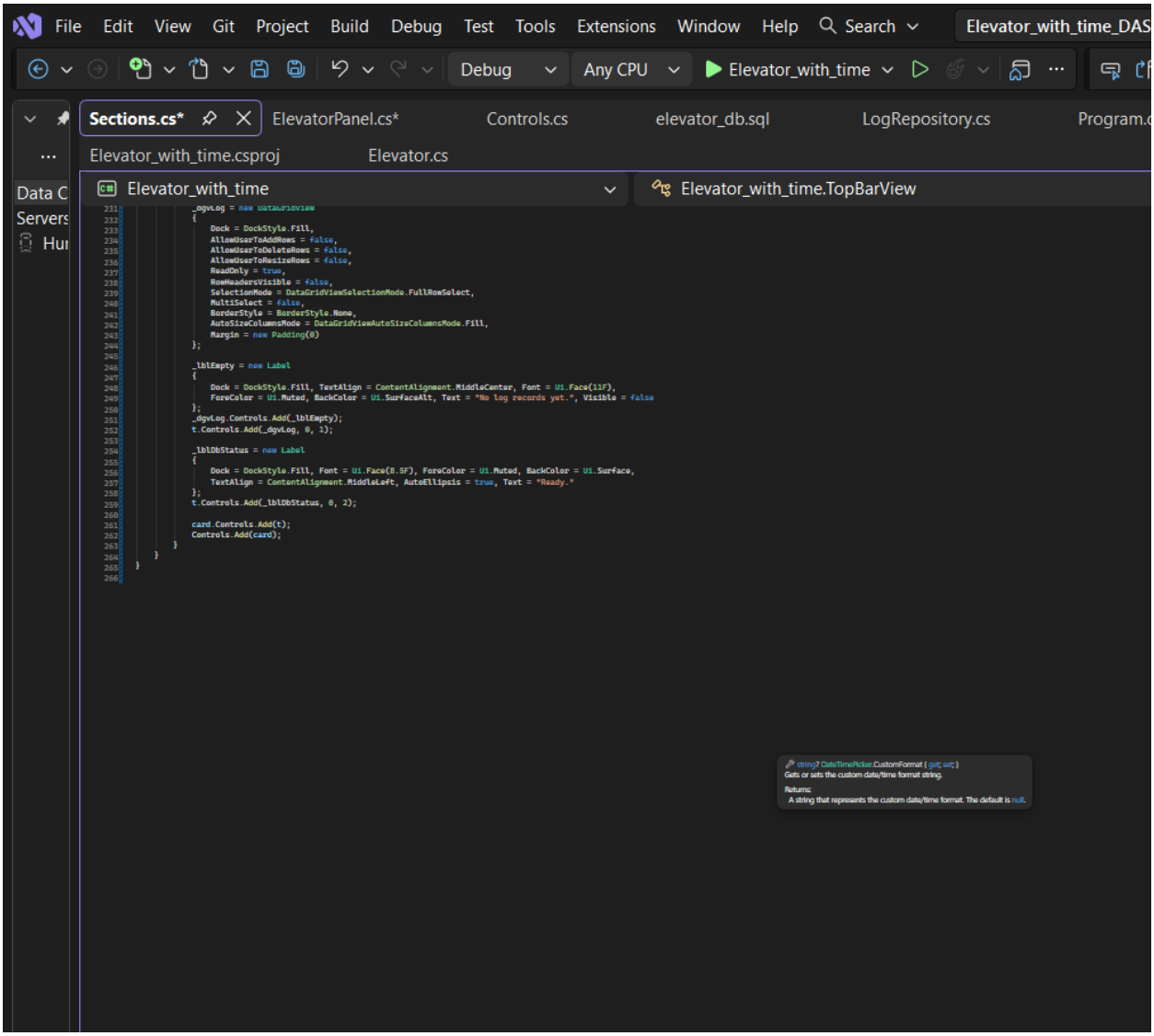
        public Label LatestEvent => _last;

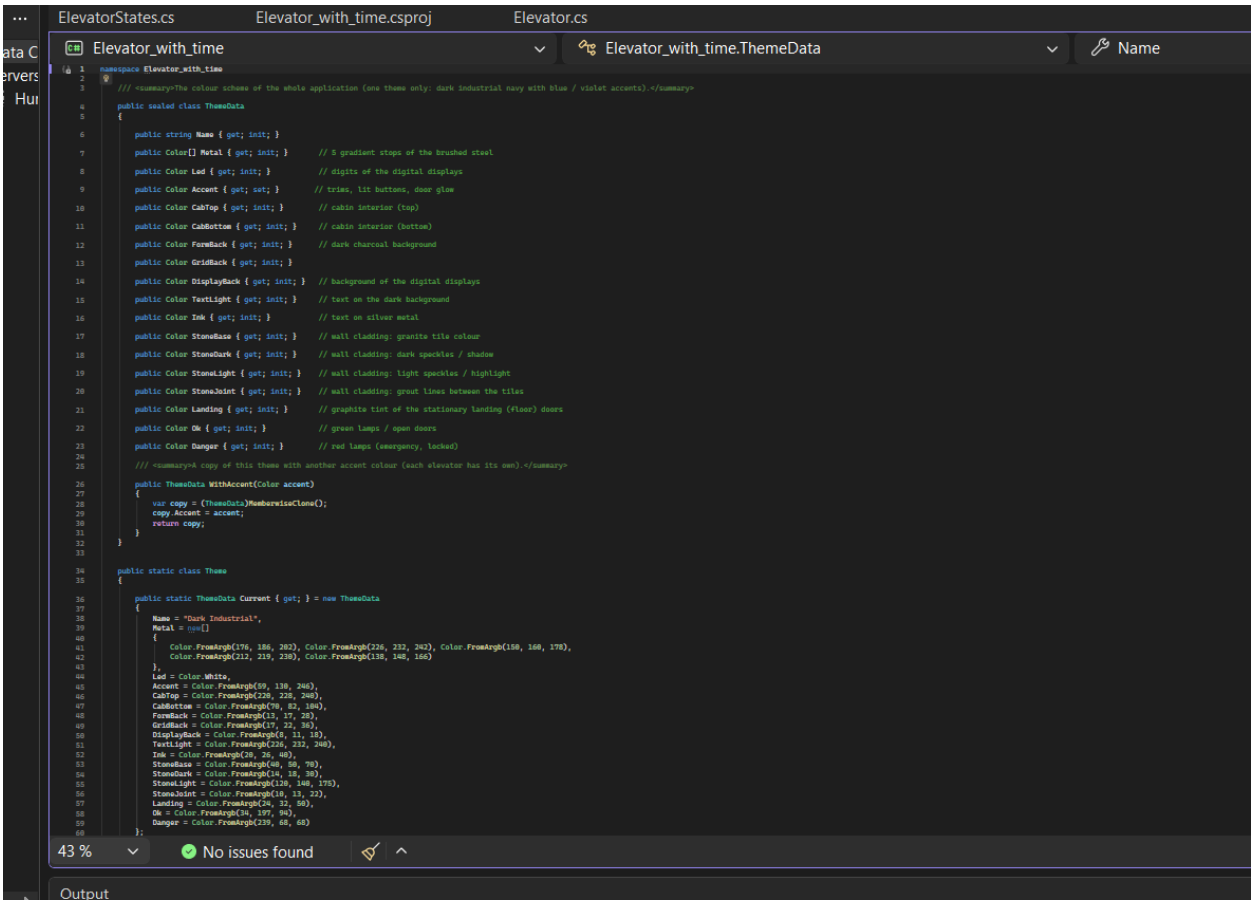

        public OverviewView()
        {
            Dock = DockStyle.Top;
            Height = 84;
            BackColor = Ui.Bg;
            Padding = new Padding(12, 10, 12, 8);

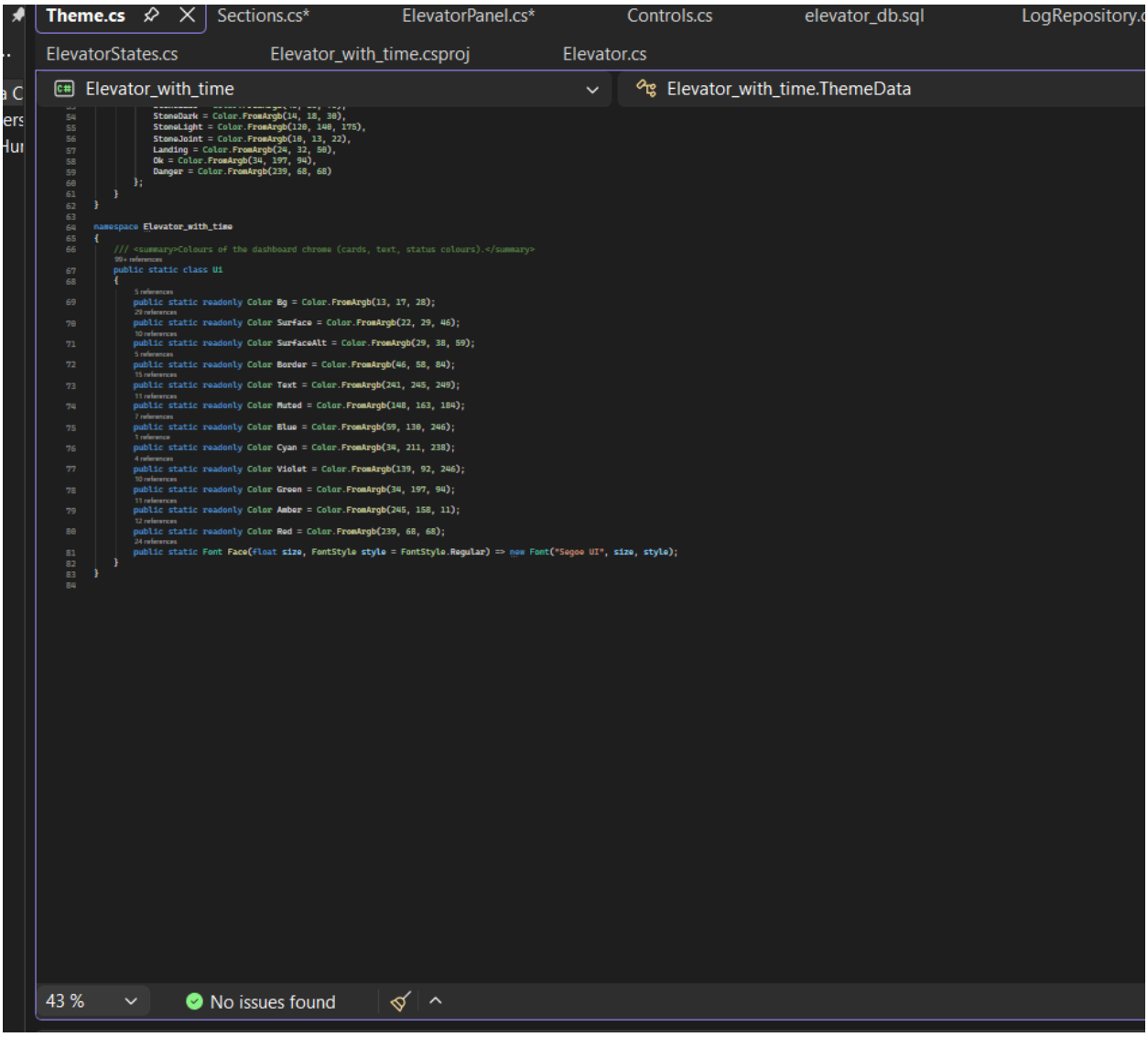
            var t = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 5, RowCount = 1, BackColor = Ui.Bg };
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 19));
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 19));
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 14));
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 18));
            t.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 30));
            t.RowStyles.Add(new RowStyle(SizeType.Percent, 100));
            t.Controls.Add(Card("ELEVATOR A", out _a, Ui.Blue), 0, 0);
            t.Controls.Add(Card("ELEVATOR B", out _b, Ui.Violet), 1, 0);
            t.Controls.Add(Card("ACTIVE MOVEMENTS", out _moves, Ui.Cyan), 2, 0);
            t.Controls.Add(Card("SAFETY", out _safety, Ui.Green), 3, 0);
            t.Controls.Add(Card("LATEST EVENT", out _last, Ui.Muted), 4, 0);
            Controls.Add(t);
        }

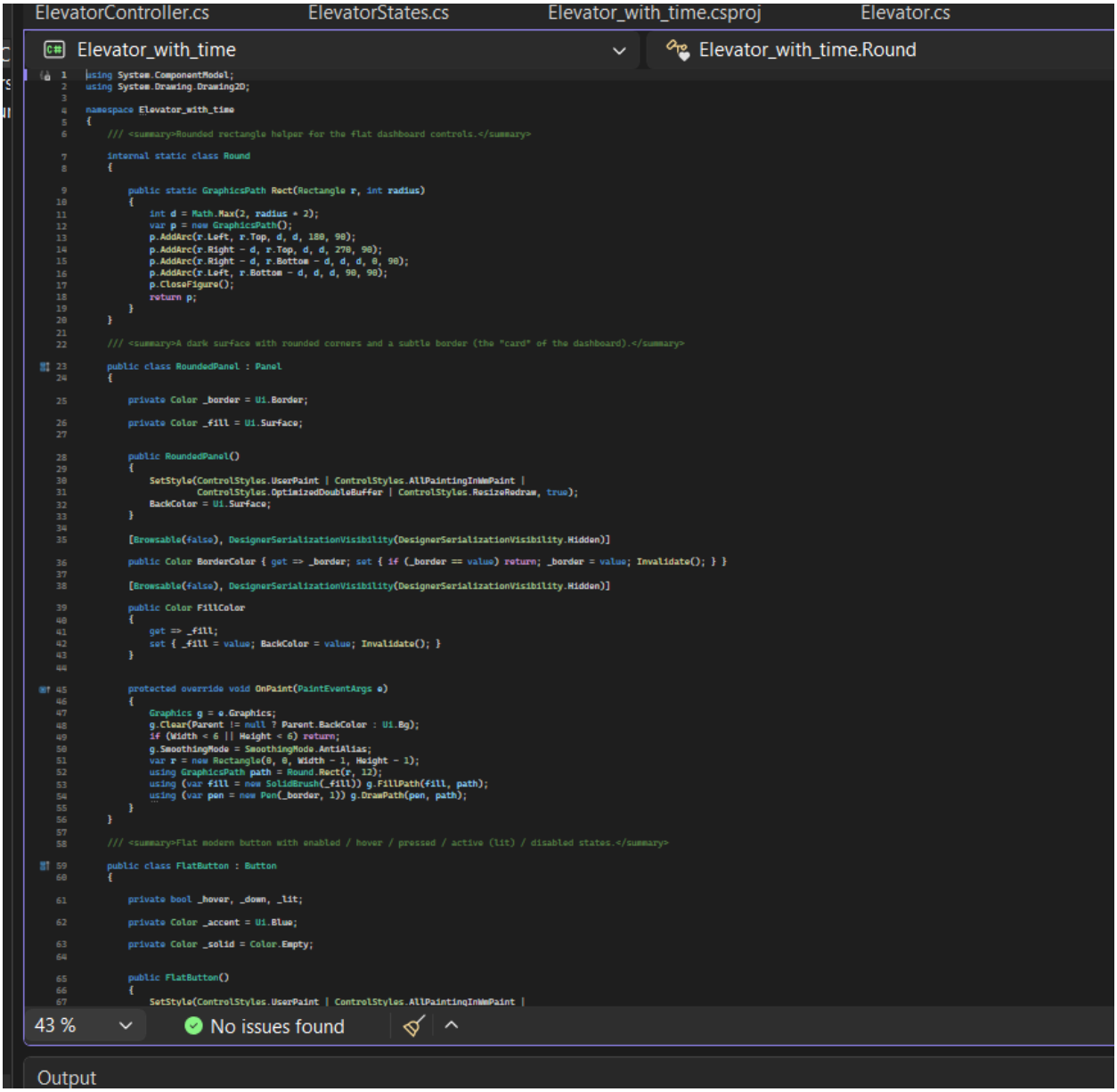

        private static Control Card(string caption, out Label value, Color accent)
        {
            var card = new RoundedPanel { Dock = DockStyle.Fill, Margin = new Padding(0, 0, 8, 0), Padding = new Padding(14, 8, 10, 8) };
            var inner = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 1, RowCount = 2, BackColor = Ui.Surface };
            inner.RowStyles.Add(new RowStyle(SizeType.Absolute, 20));
            inner.RowStyles.Add(new RowStyle(SizeType.Percent, 100));
            inner.Controls.Add(new Label
            {
                Text = caption, Font = Ui.Face(8.5F, FontStyle.Bold), ForeColor = accent, BackColor = Ui.Surface,
                Dock = DockStyle.Fill, TextAlign = ContentAlignment.BottomLeft
            }, 0, 0);
            value = new Label
            {
                Text = "-", Font = Ui.Face(11.5F, FontStyle.Bold), ForeColor = Ui.Text, BackColor = Ui.Surface,
                Dock = DockStyle.Fill, TextAlign = ContentAlignment.MiddleLeft, AutoEllipsis = true
            };
            inner.Controls.Add(value, 0, 1);
            card.Controls.Add(inner);
```

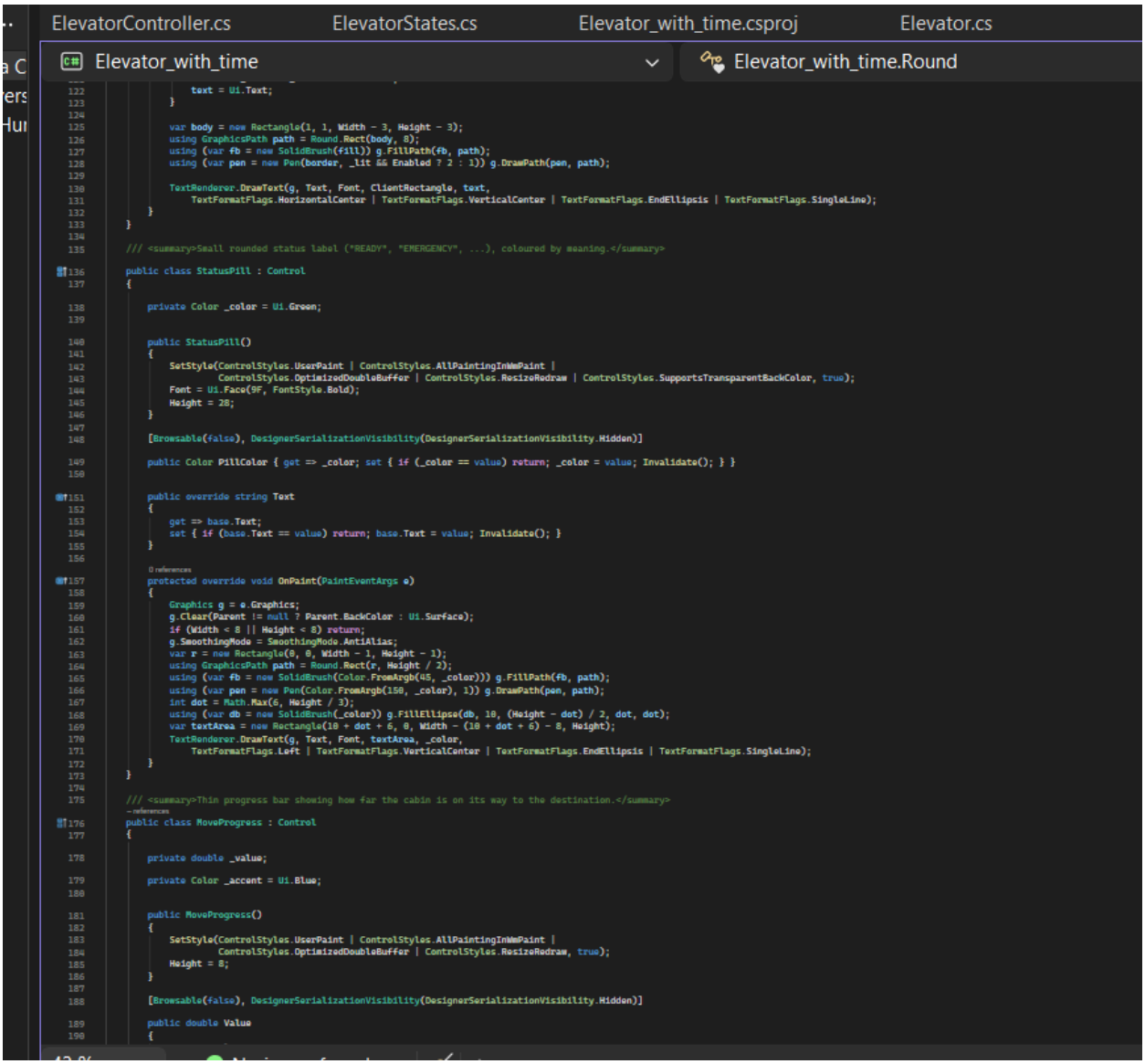

File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DASHBOARD_fixed
Debug
Any CPU
Elevator_with_time
Sections.cs*
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.TopBarView
TopBarView()
43 %
Output
Ready
Error List

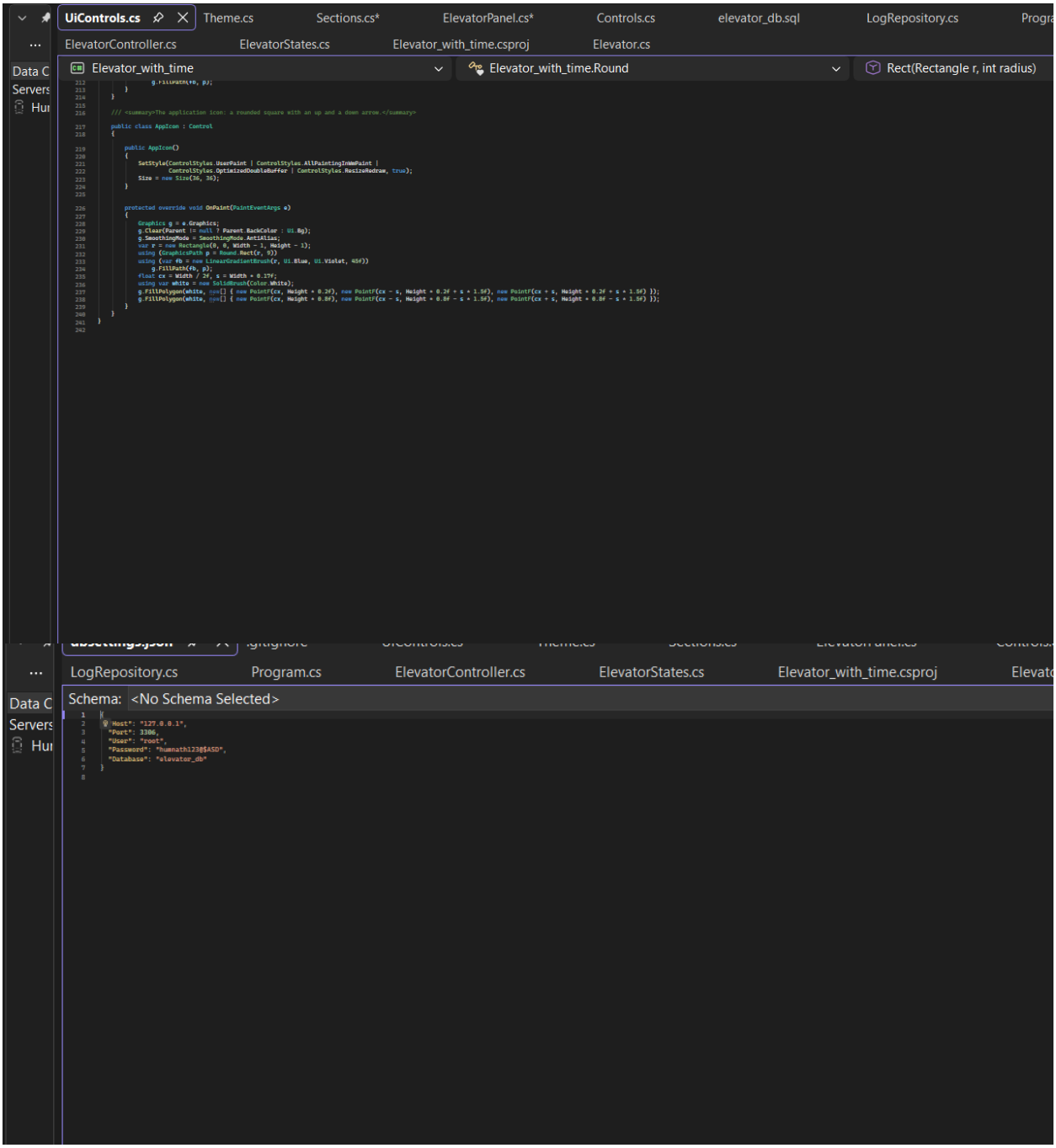

File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Elevator_with_time_DAS
Debug
Any CPU
Elevator_with_time
Sections.cs*
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
Program.c
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.TopBarView
_dgvLog = new DataGridView
{
Dock = DockStyle.Fill,
AllowUserToAddRows = false,
AllowUserToDeleteRows = false,
AllowUserToResizeRows = false,
ReadOnly = true,
RowHeadersVisible = false,
SelectionMode = DataGridViewSelectionMode.FullRowSelect,
MultiSelect = false,
BorderStyle = BorderStyle.None,
AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill,
Margin = new Padding(0)
};
_lblEmpty = new Label
{
Dock = DockStyle.Fill, TextAlign = ContentAlignment.MiddleCenter, Font = Ui.Face(11F),
ForeColor = Ui.Muted, BackColor = Ui.SurfaceAlt, Text = "No log records yet.", Visible = false
};
_dgvLog.Controls.Add(_lblEmpty);
t.Controls.Add(_dgvLog, 0, 1);
_lblDbStatus = new Label
{
Dock = DockStyle.Fill, Font = Ui.Face(8.5F), ForeColor = Ui.Muted, BackColor = Ui.Surface,
TextAlign = ContentAlignment.MiddleLeft, AutoEllipsis = true, Text = "Ready."
};
t.Controls.Add(_lblDbStatus, 0, 2);
card.Controls.Add(t);
Controls.Add(card);
}
}
}
string? DateTimePicker.CustomFormat { get; set; }
Gets or sets the custom date/time format string.
Returns:
A string that represents the custom date/time format. The default is null.

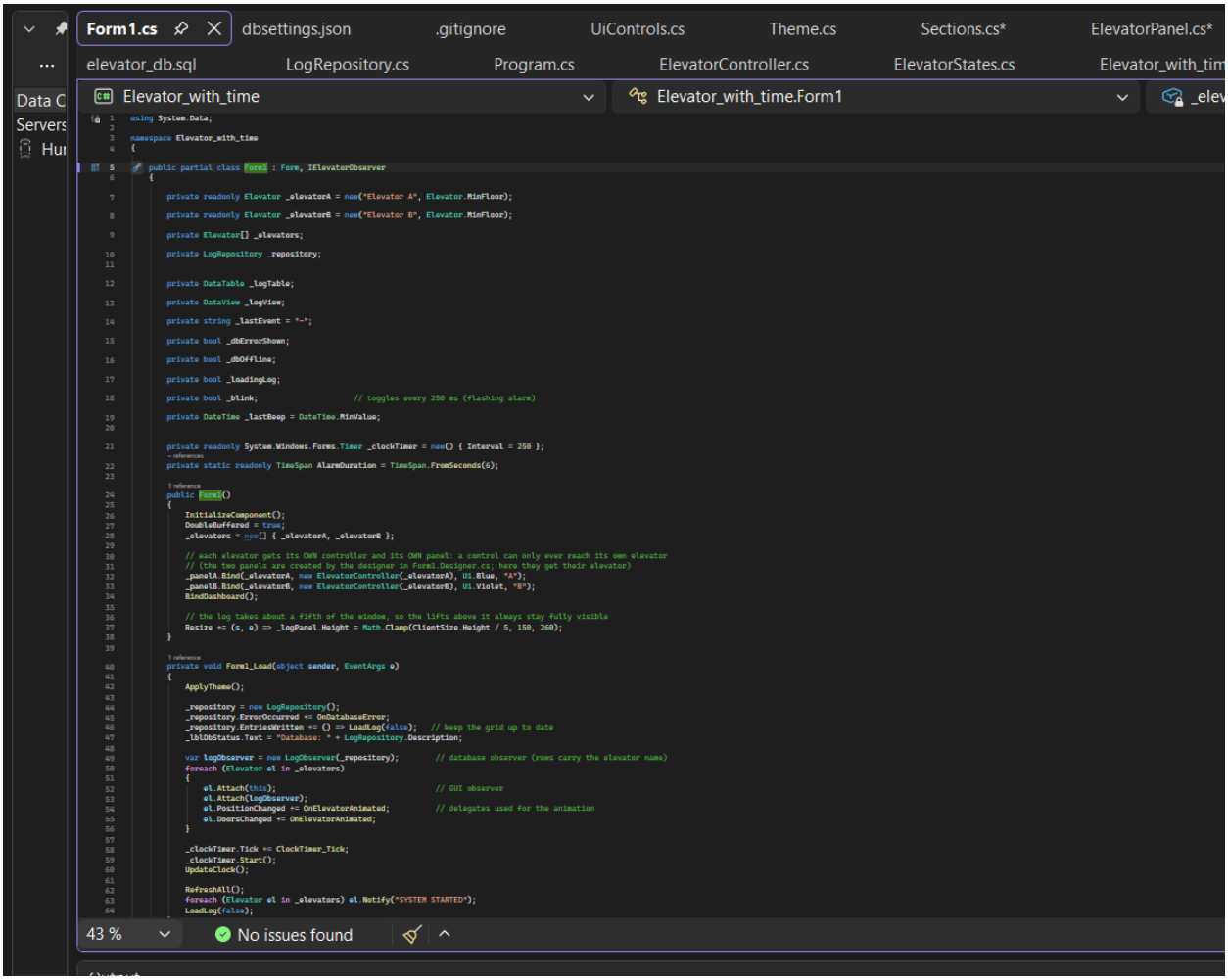

ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.ThemeData
Name
43 %
No issues found
Output

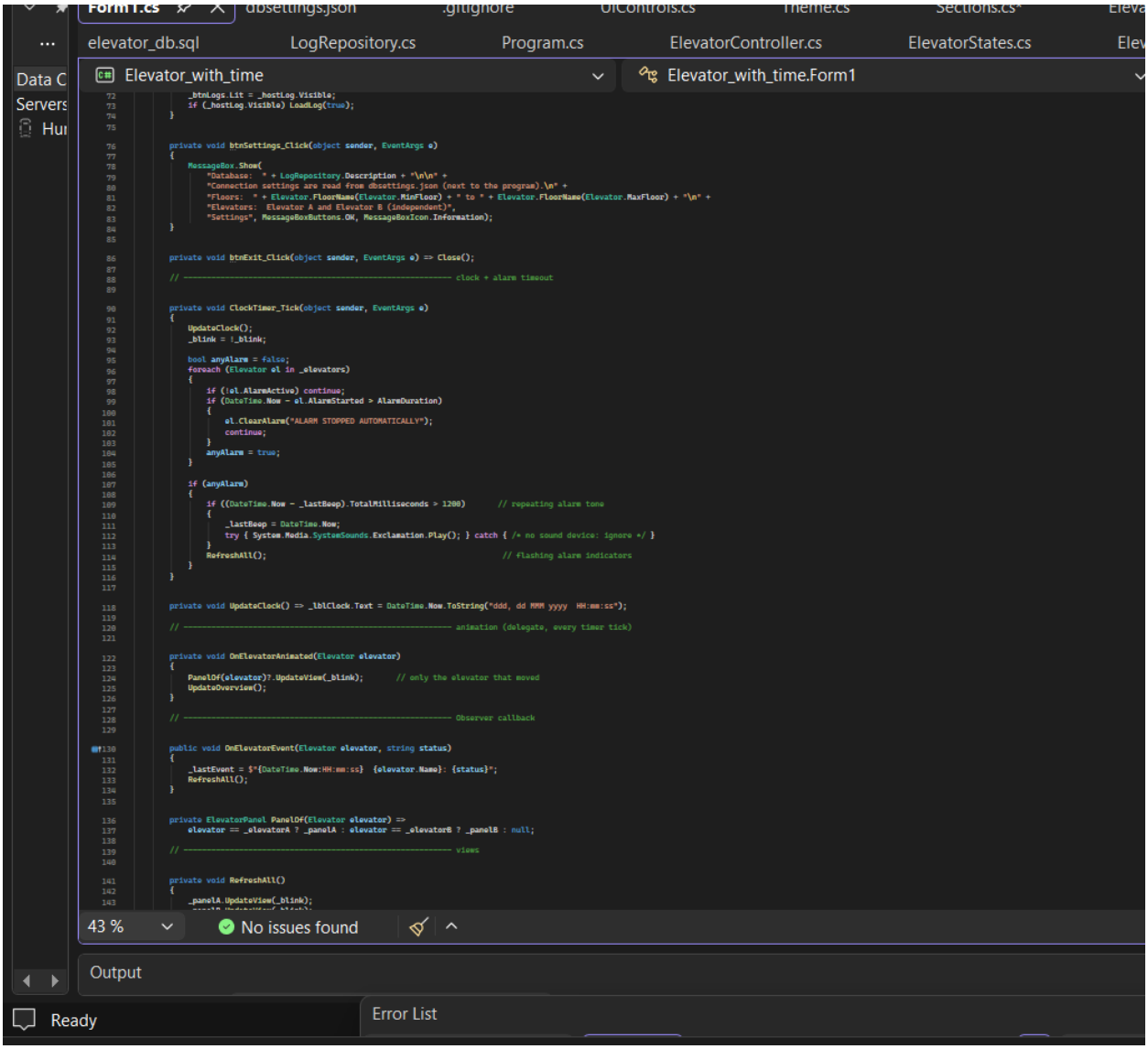

Theme.cs
Sections.cs*
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.c
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.ThemeData
StoneDark = Color.FromArgb(14, 18, 30),
StoneLight = Color.FromArgb(120, 140, 175),
StoneJoint = Color.FromArgb(18, 13, 22),
Landing = Color.FromArgb(24, 32, 50),
Ok = Color.FromArgb(34, 197, 94),
Danger = Color.FromArgb(239, 68, 68)
};
}
}
namespace Elevator_with_time
{
/// <summary>Colours of the dashboard chrome (cards, text, status colours).</summary>
99+ references
public static class Ui
{
5 references
public static readonly Color Bg = Color.FromArgb(13, 17, 28);
29 references
public static readonly Color Surface = Color.FromArgb(22, 29, 46);
10 references
public static readonly Color SurfaceAlt = Color.FromArgb(29, 38, 59);
5 references
public static readonly Color Border = Color.FromArgb(46, 58, 84);
15 references
public static readonly Color Text = Color.FromArgb(241, 245, 249);
11 references
public static readonly Color Muted = Color.FromArgb(148, 163, 184);
7 references
public static readonly Color Blue = Color.FromArgb(59, 130, 246);
1 reference
public static readonly Color Cyan = Color.FromArgb(34, 211, 238);
4 references
public static readonly Color Violet = Color.FromArgb(139, 92, 246);
10 references
public static readonly Color Green = Color.FromArgb(34, 197, 94);
11 references
public static readonly Color Amber = Color.FromArgb(245, 158, 11);
12 references
public static readonly Color Red = Color.FromArgb(239, 68, 68);
24 references
public static Font Face(float size, FontStyle style = FontStyle.Regular) => new Font("Segoe UI", size, style);
}
}
43 %
No issues found

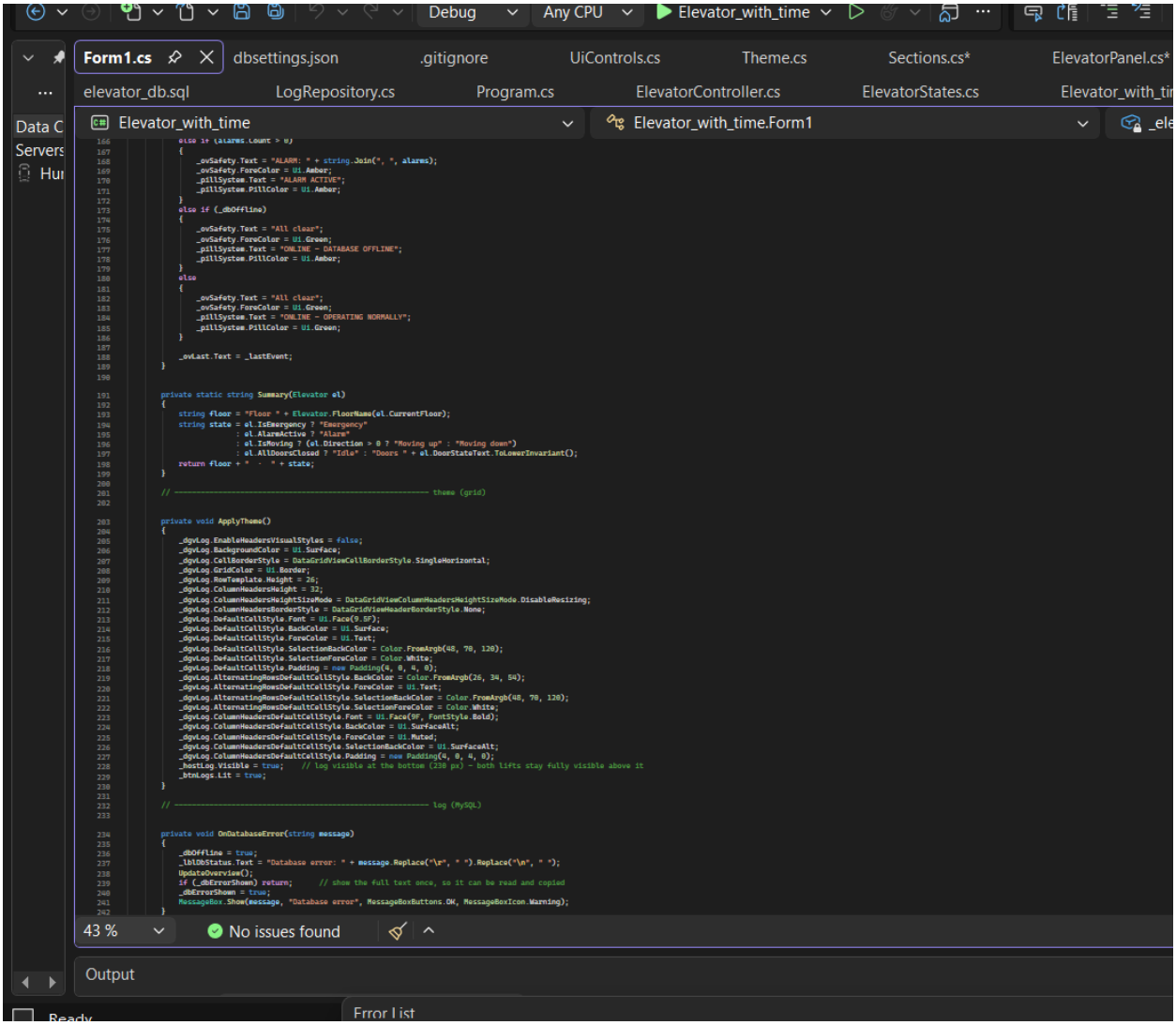

ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.Round
using System.ComponentModel;
using System.Drawing.Drawing2D;
namespace Elevator_with_time
{
/// <summary>Rounded rectangle helper for the flat dashboard controls.</summary>
internal static class Round
{
public static GraphicsPath Rect(Rectangle r, int radius)
{
int d = Math.Max(2, radius * 2);
var p = new GraphicsPath();
p.AddArc(r.Left, r.Top, d, d, 180, 90);
p.AddArc(r.Right - d, r.Top, d, d, 270, 90);
p.AddArc(r.Right - d, r.Bottom - d, d, d, 0, 90);
p.AddArc(r.Left, r.Bottom - d, d, d, 90, 90);
p.CloseFigure();
return p;
}
}
/// <summary>A dark surface with rounded corners and a subtle border (the "card" of the dashboard).</summary>
public class RoundedPanel : Panel
{
private Color _border = Ui.Border;
private Color _fill = Ui.Surface;
public RoundedPanel()
{
SetStyle(ControlStyles.UserPaint | ControlStyles.AllPaintingInWmPaint |
ControlStyles.OptimizedDoubleBuffer | ControlStyles.ResizeRedraw, true);
BackColor = Ui.Surface;
}
[Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
public Color BorderColor { get => _border; set { if (_border == value) return; _border = value; Invalidate(); } }
[Browsable(false), DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
public Color FillColor
{
get => _fill;
set { _fill = value; BackColor = value; Invalidate(); }
}
protected override void OnPaint(PaintEventArgs e)
{
Graphics g = e.Graphics;
g.Clear(Parent != null ? Parent.BackColor : Ui.Bg);
if (Width < 6 || Height < 6) return;
g.SmoothingMode = SmoothingMode.AntiAlias;
var r = new Rectangle(0, 0, Width - 1, Height - 1);
using GraphicsPath path = Round.Rect(r, 12);
using (var fill = new SolidBrush(_fill)) g.FillPath(fill, path);
using (var pen = new Pen(_border, 1)) g.DrawPath(pen, path);
}
}
/// <summary>Flat modern button with enabled / hover / pressed / active (lit) / disabled states.</summary>
public class FlatButton : Button
{
private bool _hover, _down, _lit;
private Color _accent = Ui.Blue;
private Color _solid = Color.Empty;
public FlatButton()
{
SetStyle(ControlStyles.UserPaint | ControlStyles.AllPaintingInWmPaint |
43 %
No issues found
Output

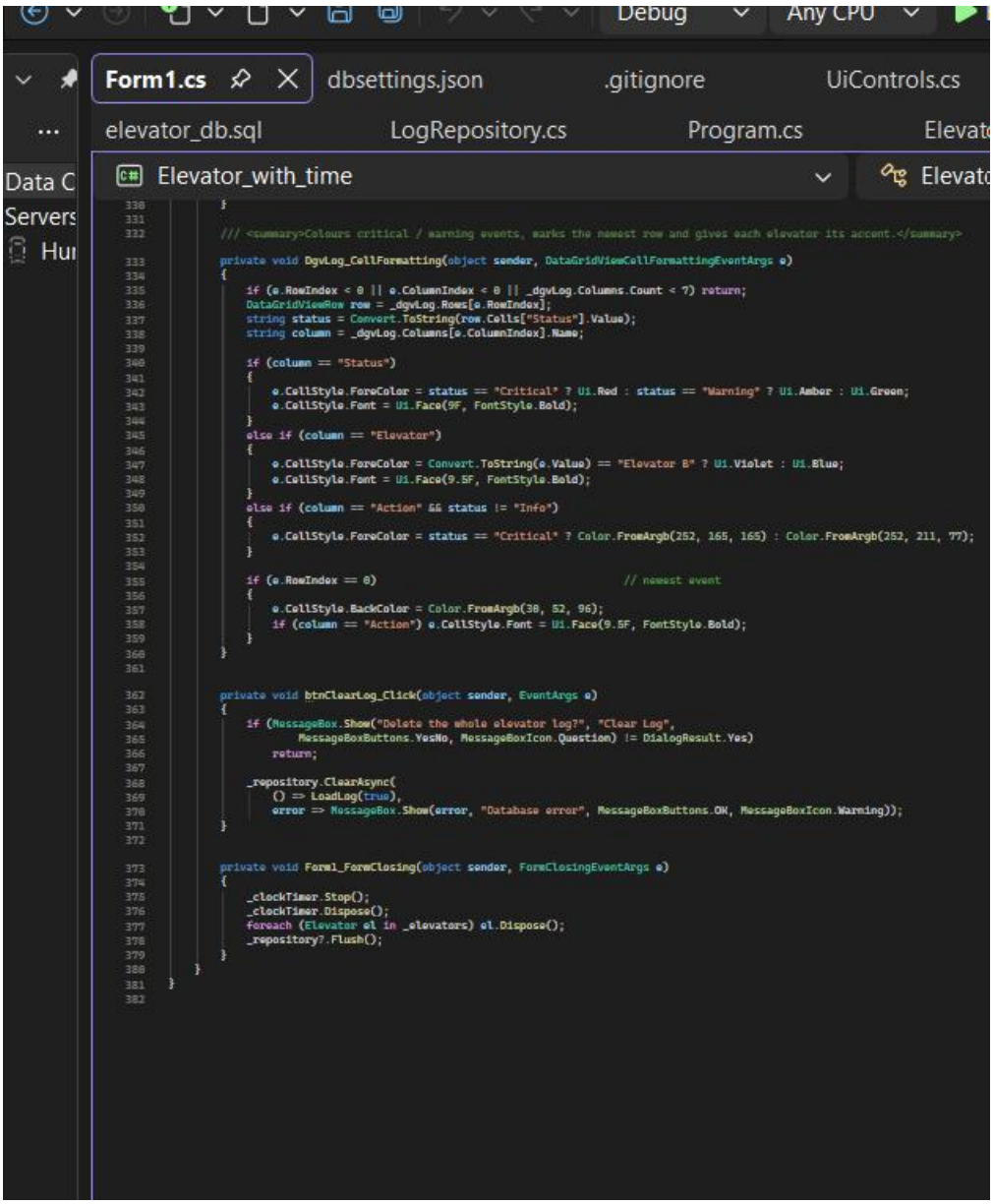

ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.Round


UiControls.cs
Theme.cs
Sections.cs*
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.Round
Rect(Rectangle r, int radius)
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time.csproj
Schema: <No Schema Selected>


Form1.cs
dbsettings.json
.gitignore
UiControls.cs
Theme.cs
Sections.cs*
ElevatorPanel.cs*
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time
Elevator_with_time
Elevator_with_time.Form1
using System.Data;
namespace Elevator_with_time
{
public partial class Form1 : Form, IElevatorObserver
{
private readonly Elevator _elevatorA = new("Elevator A", Elevator.MinFloor);
private readonly Elevator _elevatorB = new("Elevator B", Elevator.MinFloor);
private Elevator[] _elevators;
private LogRepository _repository;
private DataTable _logTable;
private DataView _logView;
private string _lastEvent = "~";
private bool _dbErrorShown;
private bool _dbOffline;
private bool _loadingLog;
private bool _blink; // toggles every 250 ms (flashing alarm)
private DateTime _lastBeep = DateTime.MinValue;
private readonly System.Windows.Forms.Timer _clockTimer = new() { Interval = 250 };
private static readonly TimeSpan AlarmDuration = TimeSpan.FromSeconds(6);
public Form1()
{
InitializeComponent();
DoubleBuffered = true;
_elevators = new[] { _elevatorA, _elevatorB };
// each elevator gets its OWN controller and its OWN panel: a control can only ever reach its own elevator
// (the two panels are created by the designer in Form1.Designer.cs; here they get their elevator)
_panelA.Bind(_elevatorA, new ElevatorController(_elevatorA), UiBlue, "A");
_panelB.Bind(_elevatorB, new ElevatorController(_elevatorB), UiViolet, "B");
BindDashboard();
// the log takes about a fifth of the window, so the lifts above it always stay fully visible
Resize += (s, e) => _logPanel.Height = Math.Clamp(ClientSize.Height / 5, 150, 260);
}
private void Form1_Load(object sender, EventArgs e)
{
ApplyTheme();
_repository = new LogRepository();
_repository.ErrorOccurred += OnDatabaseError;
_repository.EntriesWritten += () => LoadLog(false); // keep the grid up to date
_lblDbStatus.Text = "Database: " + LogRepository.Description;
var logObserver = new LogObserver(_repository); // database observer (rows carry the elevator name)
foreach (Elevator el in _elevators)
{
el.Attach(this); // GUI observer
el.Attach(logObserver);
el.PositionChanged += OnElevatorAnimated; // delegates used for the animation
el.DoorsChanged += OnElevatorAnimated;
}
_clockTimer.Tick += ClockTimer_Tick;
_clockTimer.Start();
UpdateClock();
RefreshAll();
foreach (Elevator el in _elevators) el.Notify("SYSTEM STARTED");
LoadLog(false);
43 %
No issues found


Form1.cs
dbsettings.json
.gitignore
uiControls.cs
Theme.cs
Sections.cs*
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time
Elevator_with_time.Form1
43 %
No issues found
Output
Ready
Error List


Debug
Any CPU
Elevator_with_time
Form1.cs
dbsettings.json
.gitignore
UiControls.cs
Theme.cs
Sections.cs*
ElevatorPanel.cs*
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time
Elevator_with_time.Form1
43 %
No issues found
Output
Error List
Ready




```
        }

        /// <summary>Colours critical / warning events, marks the newest row and gives each elevator its accent.</summary>
        private void DgvLog_CellFormatting(object sender, DataGridViewCellFormattingEventArgs e)
        {
            if (e.RowIndex < 0 || e.ColumnIndex < 0 || _dgvLog.Columns.Count < 7) return;
            DataGridViewRow row = _dgvLog.Rows[e.RowIndex];
            string status = Convert.ToString(row.Cells["Status"].Value);
            string column = _dgvLog.Columns[e.ColumnIndex].Name;

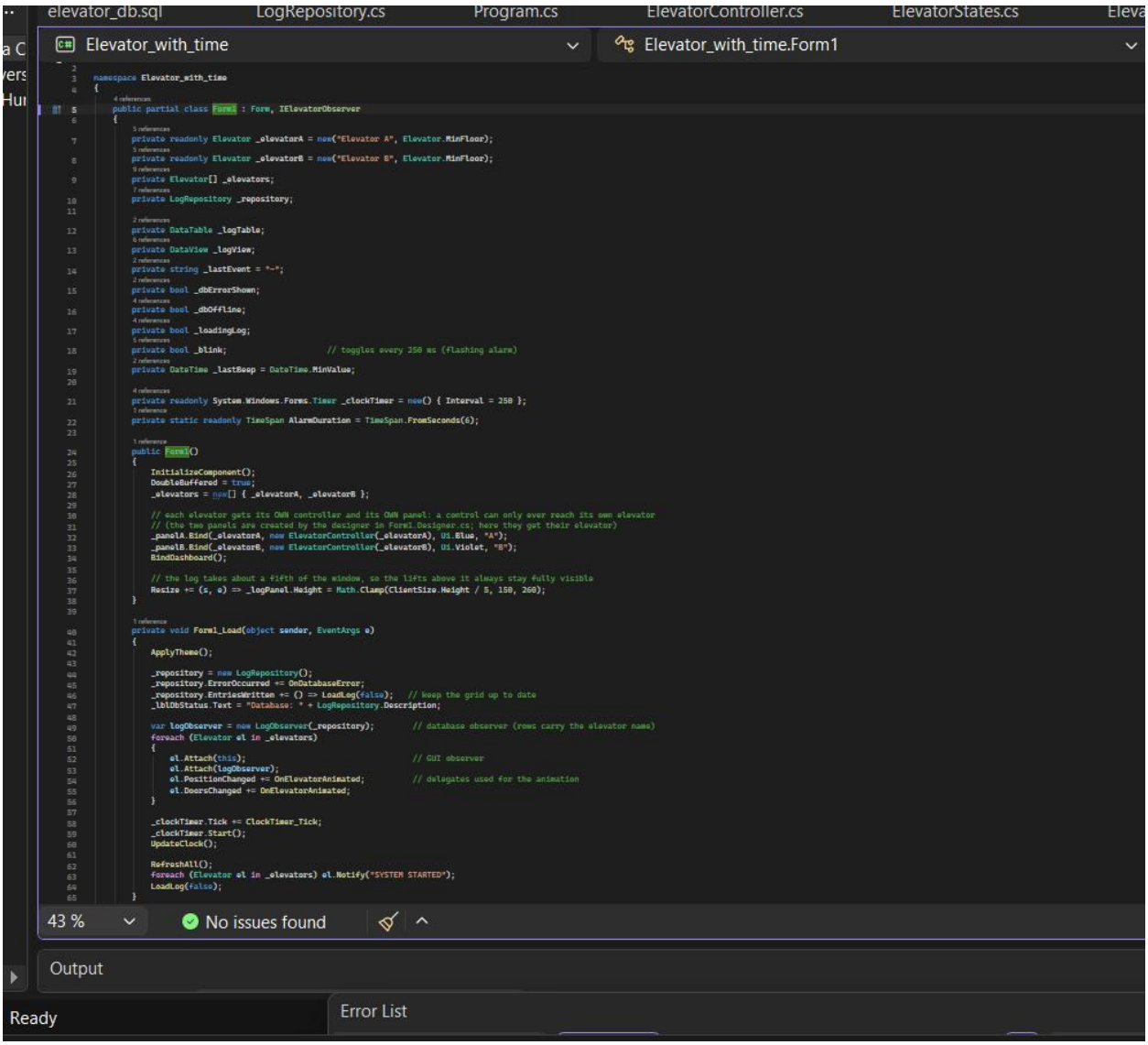
            if (column == "Status")
            {
                e.CellStyle.ForeColor = status == "Critical" ? Ui.Red : status == "Warning" ? Ui.Amber : Ui.Green;
                e.CellStyle.Font = Ui.Face(9F, FontStyle.Bold);
            }
            else if (column == "Elevator")
            {
                e.CellStyle.ForeColor = Convert.ToString(e.Value) == "Elevator B" ? Ui.Violet : Ui.Blue;
                e.CellStyle.Font = Ui.Face(9.5F, FontStyle.Bold);
            }
            else if (column == "Action" && status != "Info")
            {
                e.CellStyle.ForeColor = status == "Critical" ? Color.FromArgb(252, 165, 165) : Color.FromArgb(252, 211, 77);
            }

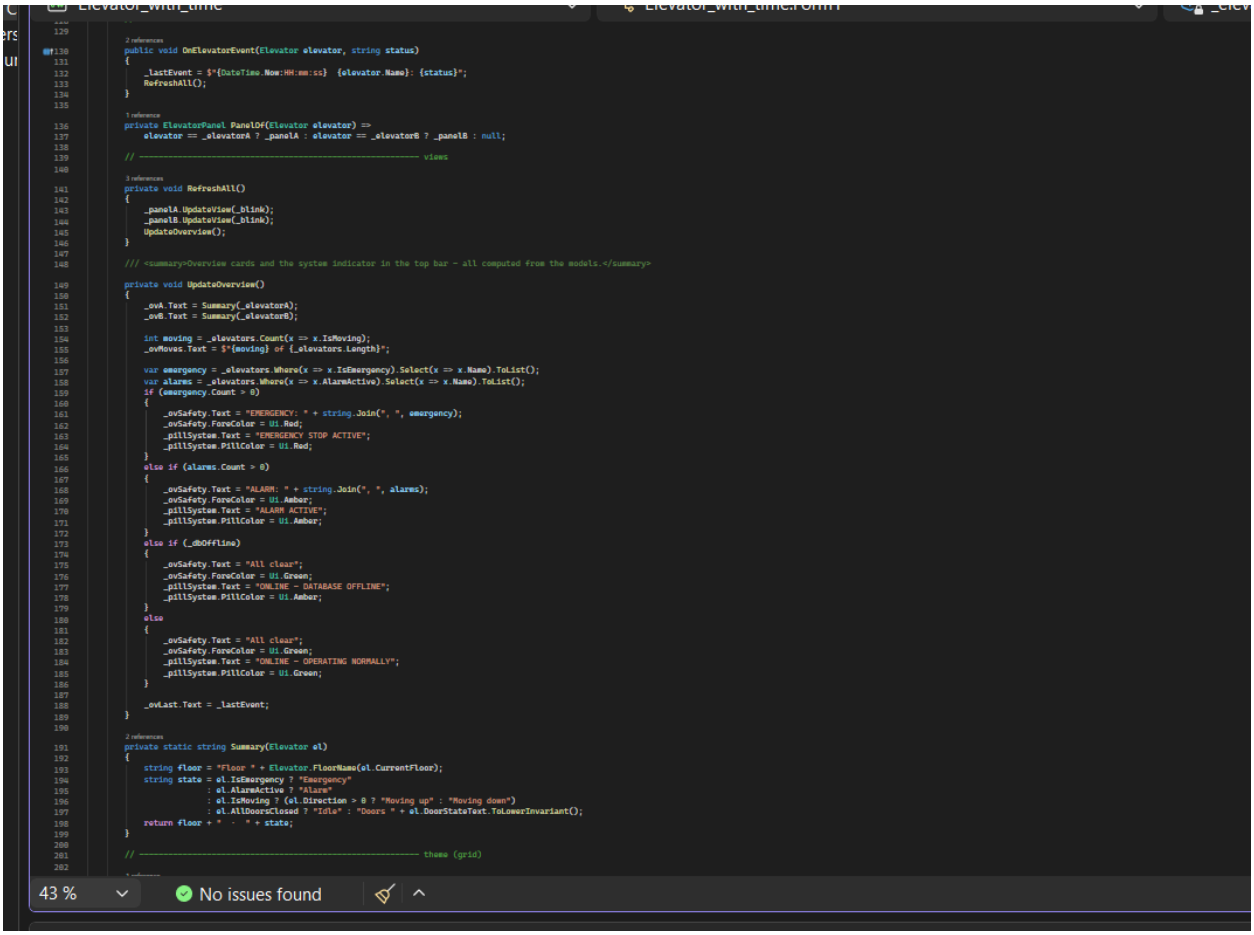
            if (e.RowIndex == 0)                                    // newest event
            {
                e.CellStyle.BackColor = Color.FromArgb(38, 52, 96);
                if (column == "Action") e.CellStyle.Font = Ui.Face(9.5F, FontStyle.Bold);
            }
        }


        private void btnClearLog_Click(object sender, EventArgs e)
        {
            if (MessageBox.Show("Delete the whole elevator log?", "Clear Log",
                    MessageBoxButtons.YesNo, MessageBoxIcon.Question) != DialogResult.Yes)
                return;

            _repository.ClearAsync(
                () => LoadLog(true),
                error => MessageBox.Show(error, "Database error", MessageBoxButtons.OK, MessageBoxIcon.Warning));
        }


        private void Form1_FormClosing(object sender, FormClosingEventArgs e)
        {
            _clockTimer.Stop();
            _clockTimer.Dispose();
            foreach (Elevator el in _elevators) el.Dispose();
            _repository?.Flush();
        }
    }
}
```


elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController.cs
ElevatorStates.cs
Elevator_with_time
Elevator_with_time.Form1
43 %
No issues found
Output
Ready
Error List



```
2 references
public void OnElevatorEvent(Elevator elevator, string status)
{
    _lastEvent = $"{DateTime.Now:HH:mm:ss}  {elevator.Name}: {status}";
    RefreshAll();
}

1 reference
private ElevatorPanel PanelOf(Elevator elevator) =>
    elevator == _elevatorA ? _panelA : elevator == _elevatorB ? _panelB : null;

// ---------------------------------------------------------- views

3 references
private void RefreshAll()
{
    _panelA.UpdateView(_blink);
    _panelB.UpdateView(_blink);
    UpdateOverview();
}

/// <summary>Overview cards and the system indicator in the top bar - all computed from the models.</summary>

private void UpdateOverview()
{
    _ovA.Text = Summary(_elevatorA);
    _ovB.Text = Summary(_elevatorB);

    int moving = _elevators.Count(x => x.IsMoving);
    _ovMoves.Text = $"{moving} of {_elevators.Length}";

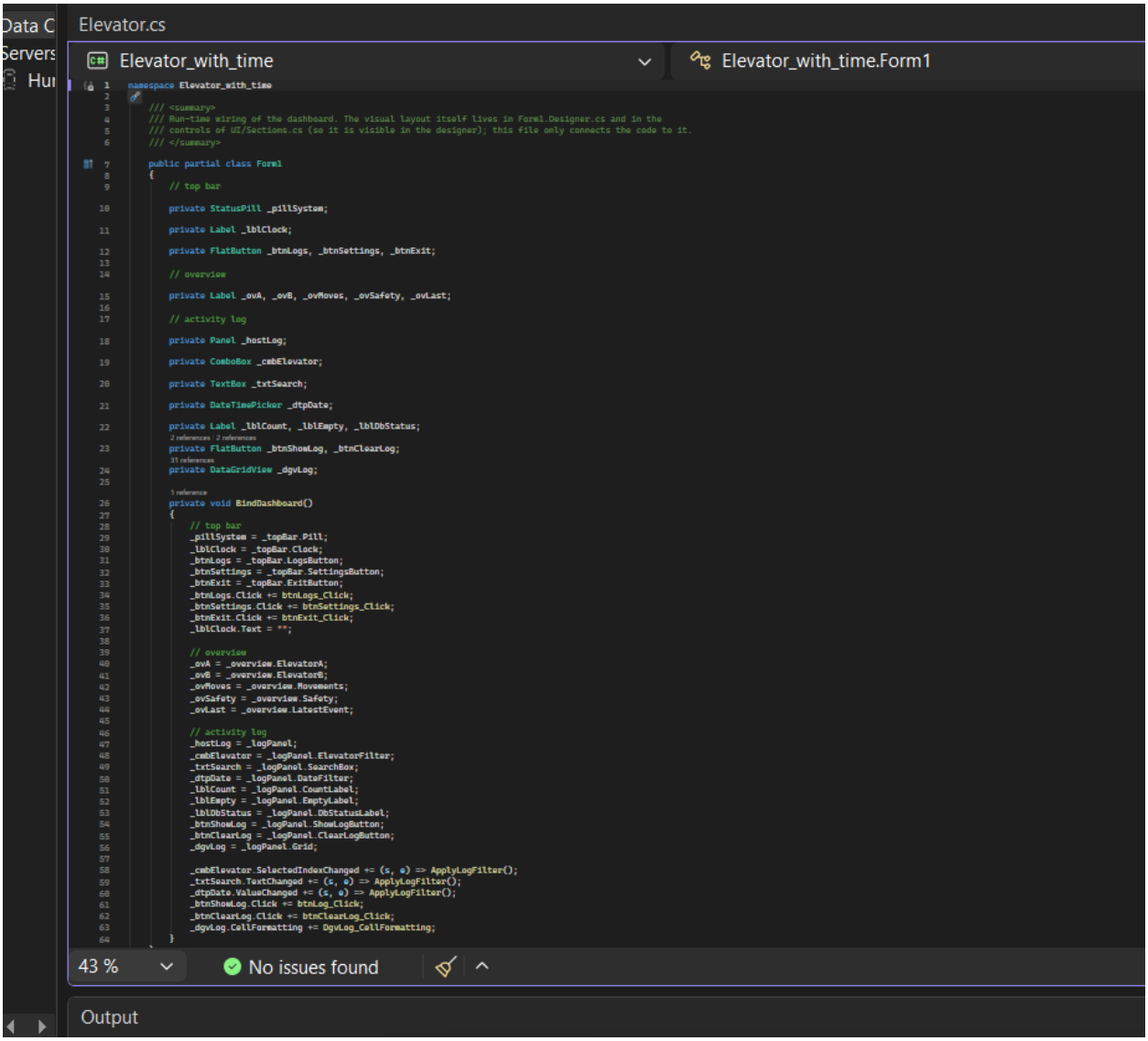
    var emergency = _elevators.Where(x => x.IsEmergency).Select(x => x.Name).ToList();
    var alarms = _elevators.Where(x => x.AlarmActive).Select(x => x.Name).ToList();
    if (emergency.Count > 0)
    {
        _ovSafety.Text = "EMERGENCY: " + string.Join(", ", emergency);
        _ovSafety.ForeColor = Ui.Red;
        _pillSystem.Text = "EMERGENCY STOP ACTIVE";
        _pillSystem.PillColor = Ui.Red;
    }
    else if (alarms.Count > 0)
    {
        _ovSafety.Text = "ALARM: " + string.Join(", ", alarms);
        _ovSafety.ForeColor = Ui.Amber;
        _pillSystem.Text = "ALARM ACTIVE";
        _pillSystem.PillColor = Ui.Amber;
    }
    else if (_dbOffline)
    {
        _ovSafety.Text = "All clear";
        _ovSafety.ForeColor = Ui.Green;
        _pillSystem.Text = "ONLINE - DATABASE OFFLINE";
        _pillSystem.PillColor = Ui.Amber;
    }
    else
    {
        _ovSafety.Text = "All clear";
        _ovSafety.ForeColor = Ui.Green;
        _pillSystem.Text = "ONLINE - OPERATING NORMALLY";
        _pillSystem.PillColor = Ui.Green;
    }

    _ovLast.Text = _lastEvent;
}

2 references
private static string Summary(Elevator el)
{
    string floor = "Floor " + Elevator.FloorName(el.CurrentFloor);
    string state = el.IsEmergency ? "Emergency"
                 : el.AlarmActive ? "Alarm"
                 : el.IsMoving ? (el.Direction > 0 ? "Moving up" : "Moving down")
                 : el.AllDoorsClosed ? "Idle" : "Doors " + el.DoorStateText.ToLowerInvariant();
    return floor + "  -  " + state;
}

// ---------------------------------------------------------- theme (grid)
```

43 % No issues found




```
namespace Elevator_with_time
{
    /// <summary>
    /// Run-time wiring of the dashboard. The visual layout itself lives in Form1.Designer.cs and in the
    /// controls of UI/Sections.cs (so it is visible in the designer); this file only connects the code to it.
    /// </summary>

    public partial class Form1
    {
        // top bar

        private StatusPill _pillSystem;

        private Label _lblClock;

        private FlatButton _btnLogs, _btnSettings, _btnExit;

        // overview

        private Label _ovA, _ovB, _ovMoves, _ovSafety, _ovLast;

        // activity log

        private Panel _hostLog;

        private ComboBox _cmbElevator;

        private TextBox _txtSearch;

        private DateTimePicker _dtpDate;

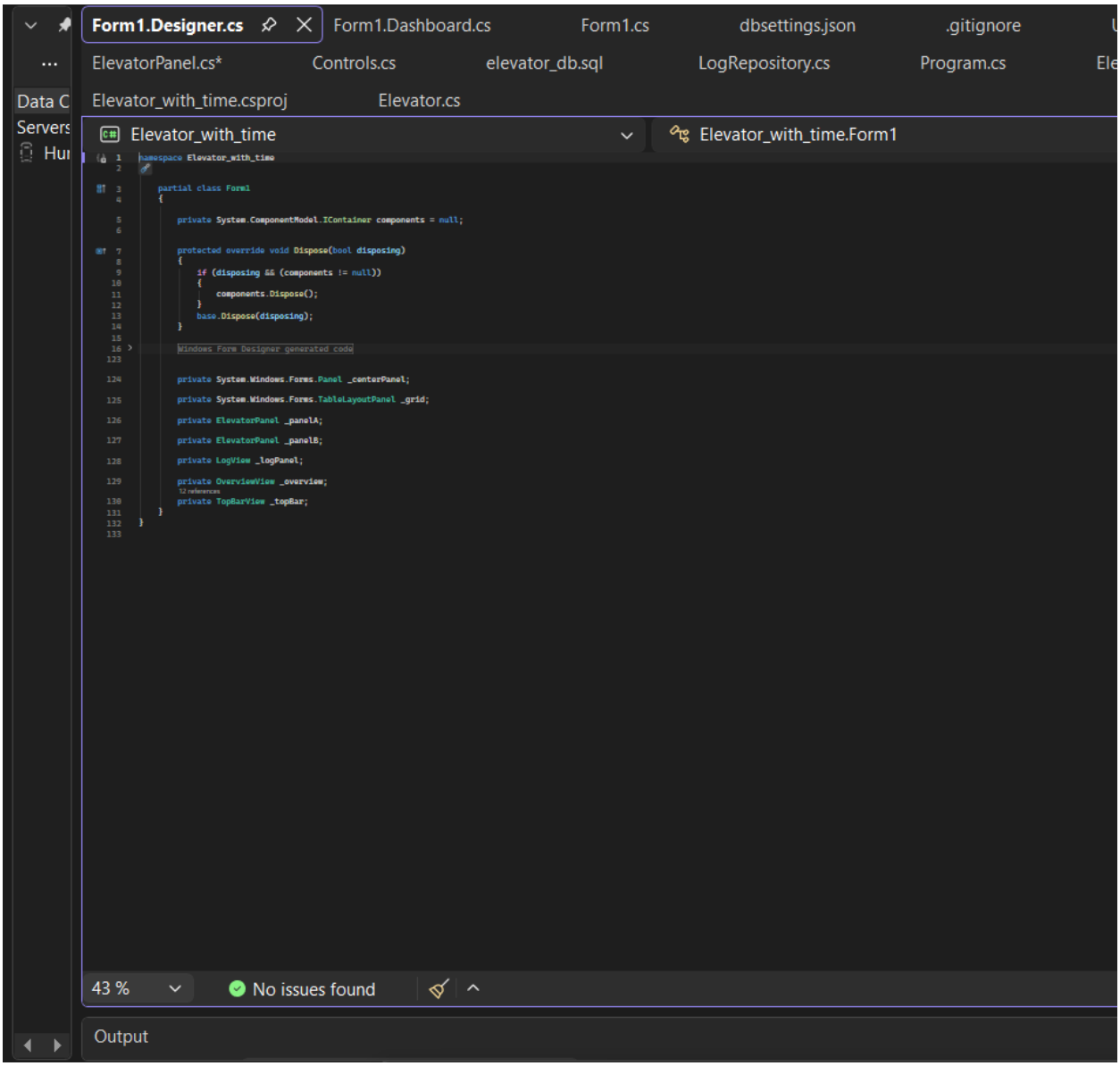
        private Label _lblCount, _lblEmpty, _lblDbStatus;
        2 references | 2 references
        private FlatButton _btnShowLog, _btnClearLog;
        31 references
        private DataGridView _dgvLog;

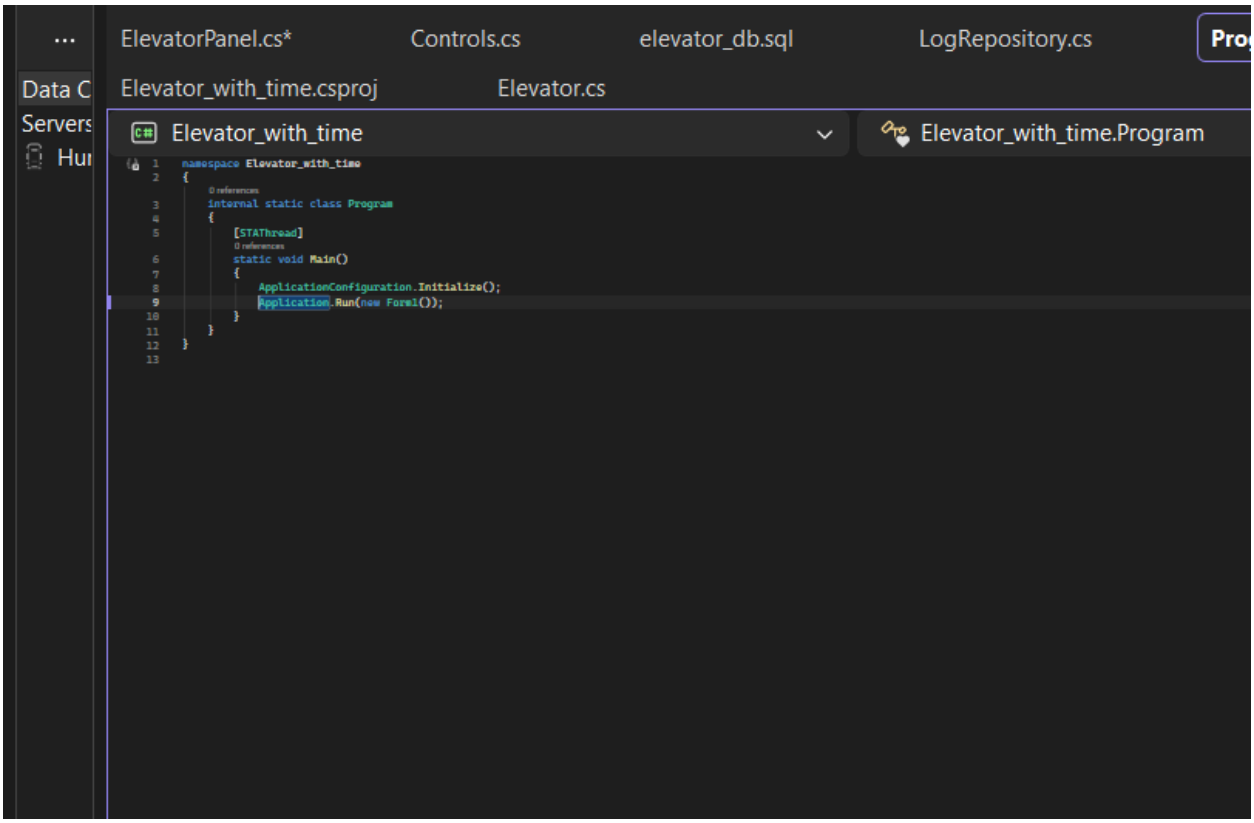
        1 reference
        private void BindDashboard()
        {
            // top bar
            _pillSystem = _topBar.Pill;
            _lblClock = _topBar.Clock;
            _btnLogs = _topBar.LogsButton;
            _btnSettings = _topBar.SettingsButton;
            _btnExit = _topBar.ExitButton;
            _btnLogs.Click += btnLogs_Click;
            _btnSettings.Click += btnSettings_Click;
            _btnExit.Click += btnExit_Click;
            _lblClock.Text = "";

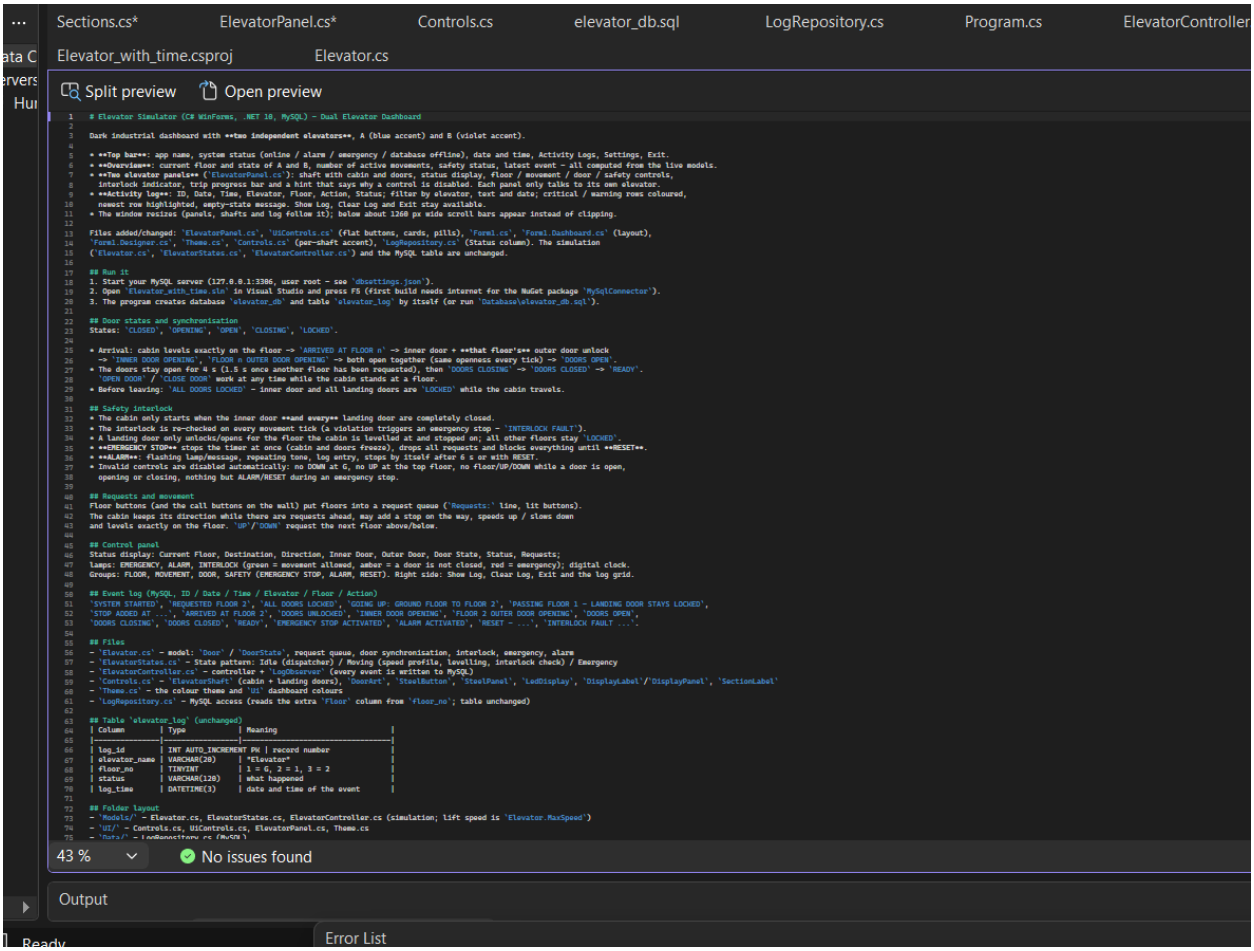
            // overview
            _ovA = _overview.ElevatorA;
            _ovB = _overview.ElevatorB;
            _ovMoves = _overview.Movements;
            _ovSafety = _overview.Safety;
            _ovLast = _overview.LatestEvent;

            // activity log
            _hostLog = _logPanel;
            _cmbElevator = _logPanel.ElevatorFilter;
            _txtSearch = _logPanel.SearchBox;
            _dtpDate = _logPanel.DateFilter;
            _lblCount = _logPanel.CountLabel;
            _lblEmpty = _logPanel.EmptyLabel;
            _lblDbStatus = _logPanel.DbStatusLabel;
            _btnShowLog = _logPanel.ShowLogButton;
            _btnClearLog = _logPanel.ClearLogButton;
            _dgvLog = _logPanel.Grid;

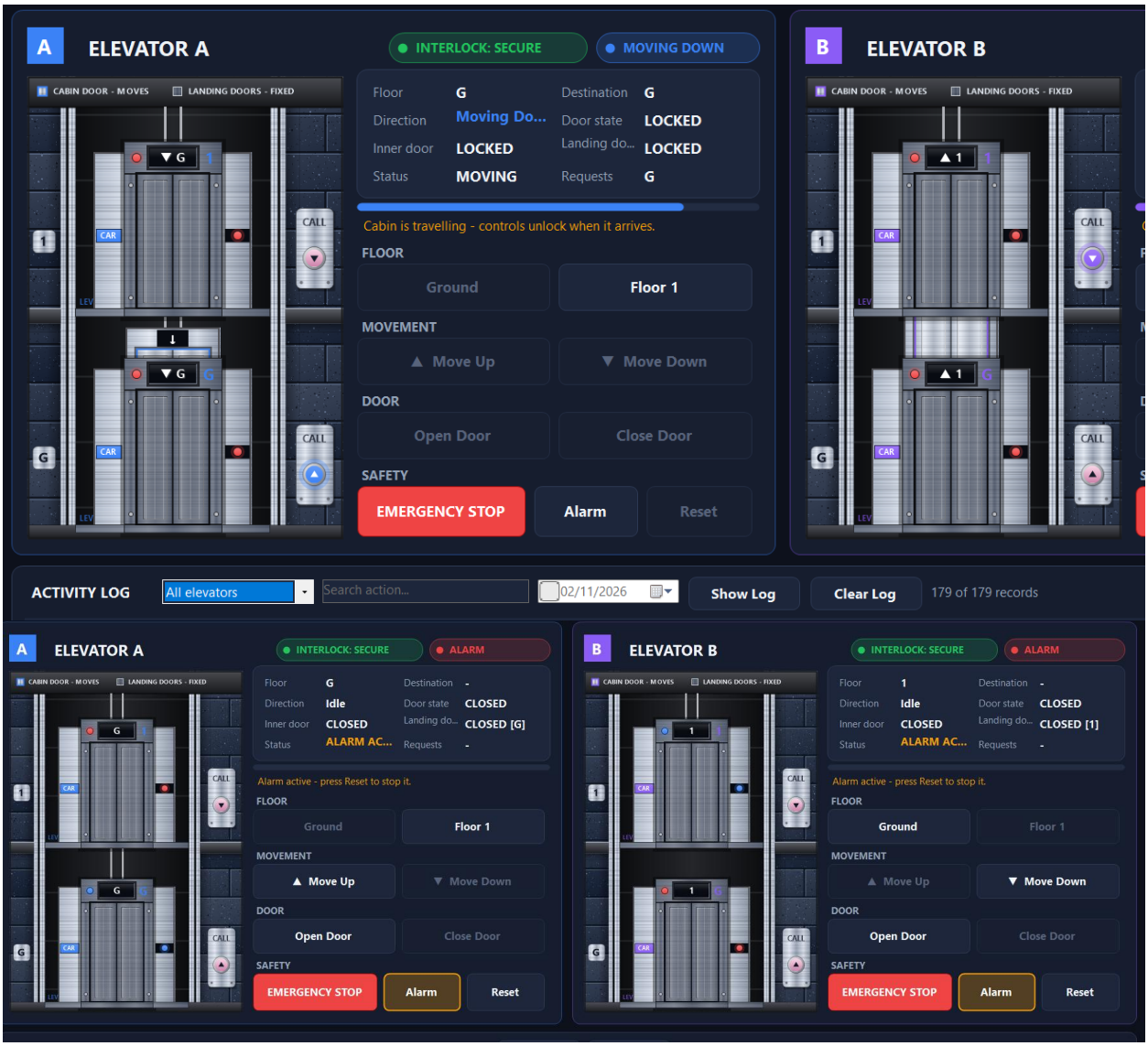
            _cmbElevator.SelectedIndexChanged += (s, e) => ApplyLogFilter();
            _txtSearch.TextChanged += (s, e) => ApplyLogFilter();
            _dtpDate.ValueChanged += (s, e) => ApplyLogFilter();
            _btnShowLog.Click += btnLog_Click;
            _btnClearLog.Click += btnClearLog_Click;
            _dgvLog.CellFormatting += DgvLog_CellFormatting;
        }
```

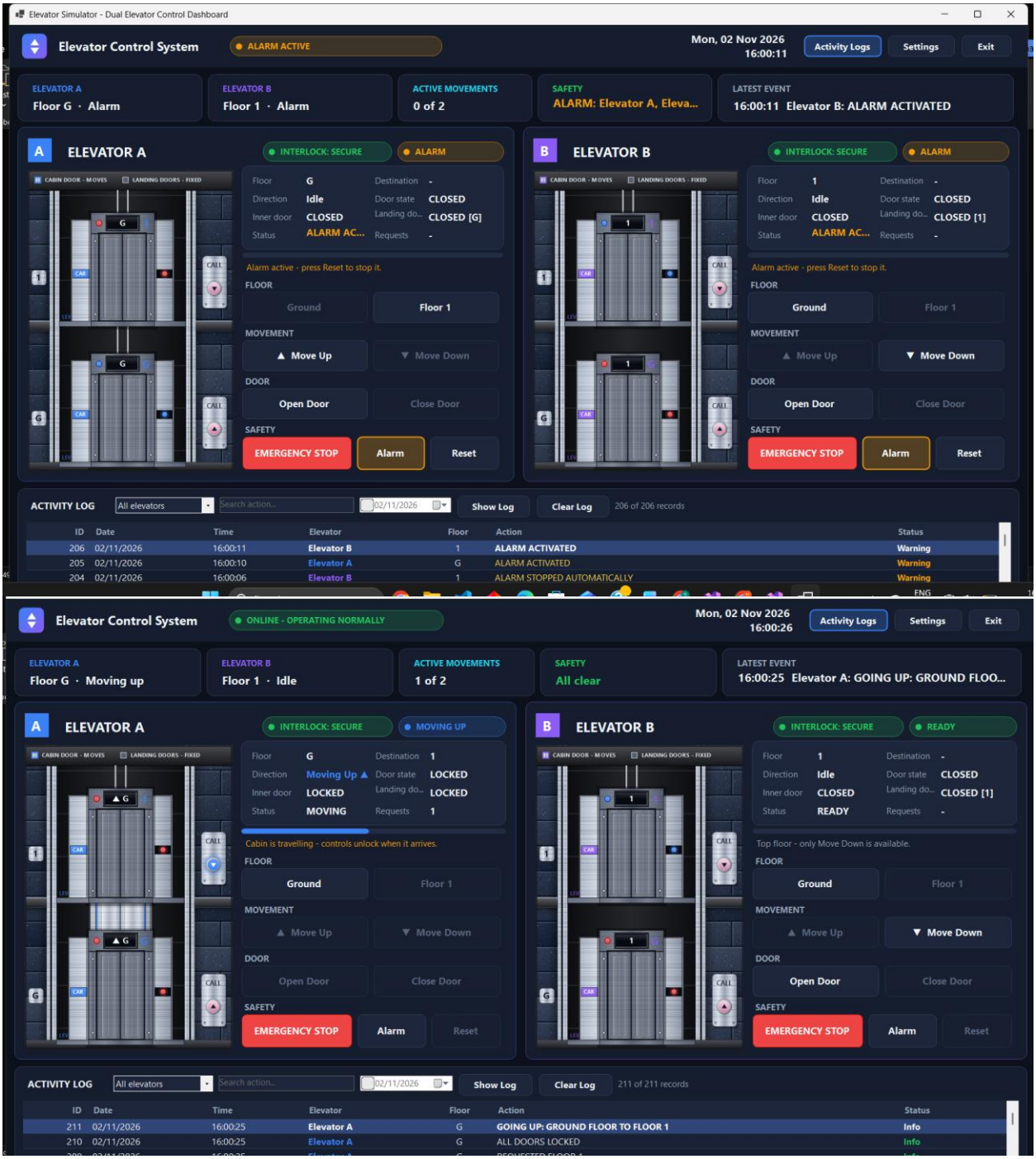

Form1.Designer.cs
Form1.Dashboard.cs
Form1.cs
dbsettings.json
.gitignore
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.Form1
namespace Elevator_with_time
partial class Form1
{
private System.ComponentModel.IContainer components = null;
protected override void Dispose(bool disposing)
{
if (disposing && (components != null))
{
components.Dispose();
}
base.Dispose(disposing);
}
Windows Form Designer generated code
private System.Windows.Forms.Panel _centerPanel;
private System.Windows.Forms.TableLayoutPanel _grid;
private ElevatorPanel _panelA;
private ElevatorPanel _panelB;
private LogView _logPanel;
private OverviewView _overview;
12 references
private TopBarView _topBar;
}
43 %
No issues found
Output

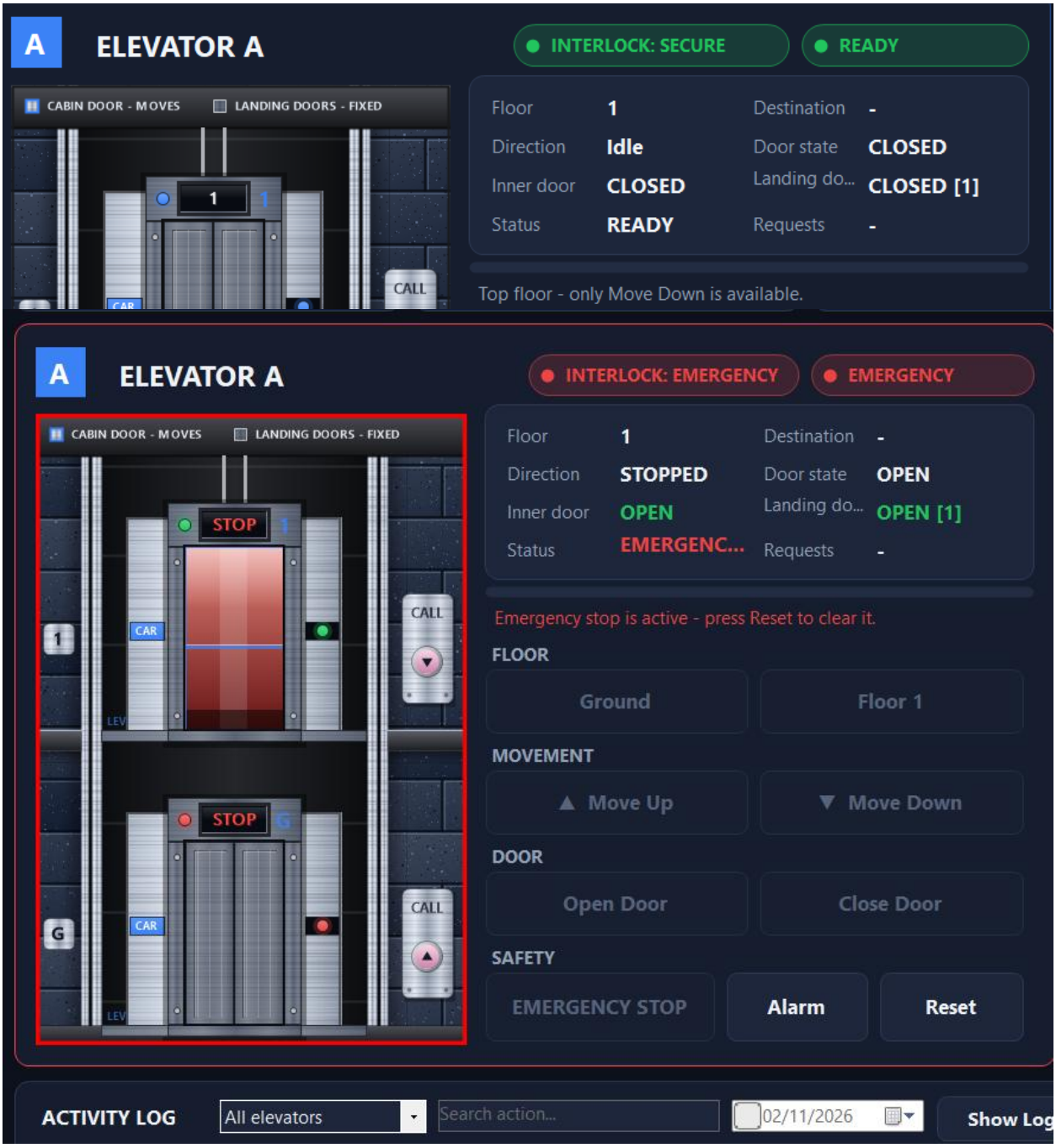

ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
Elevator_with_time.csproj
Elevator.cs
Elevator_with_time
Elevator_with_time.Program
namespace Elevator_with_time
{
0 references
internal static class Program
{
[STAThread]
0 references
static void Main()
{
ApplicationConfiguration.Initialize();
Application.Run(new Form1());
}
}
}

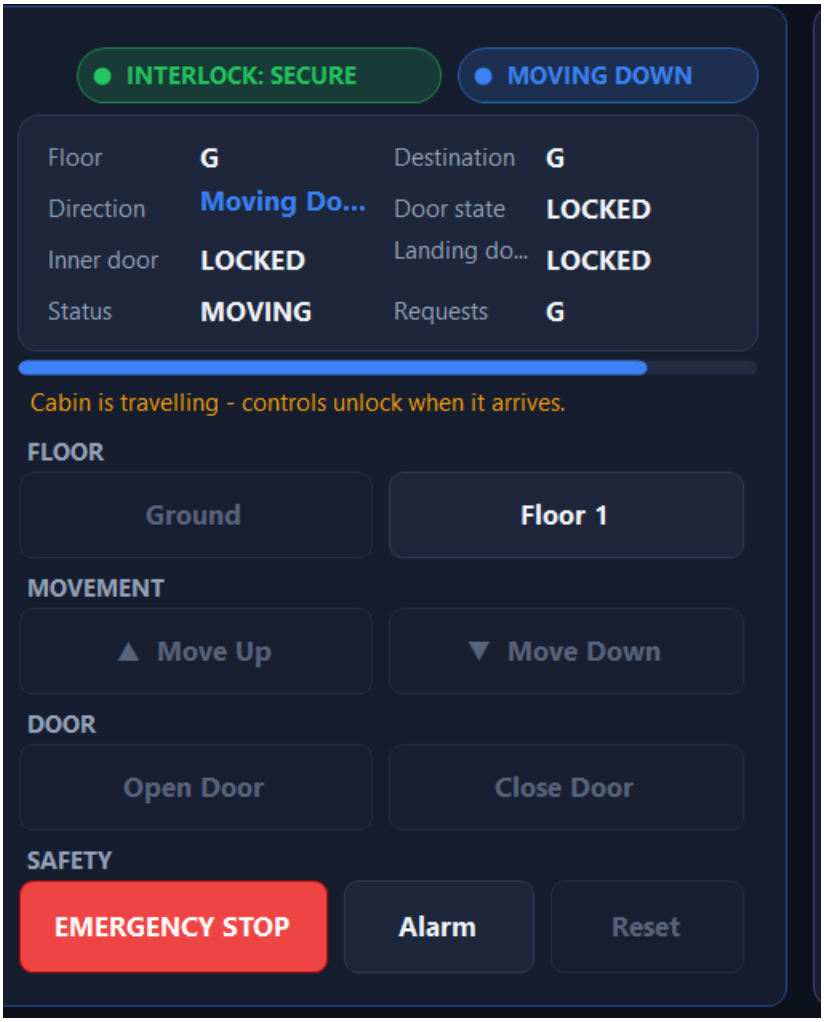

Sections.cs*
ElevatorPanel.cs*
Controls.cs
elevator_db.sql
LogRepository.cs
Program.cs
ElevatorController
Elevator_with_time.csproj
Elevator.cs
Split preview
Open preview
43 %
No issues found
Output
Error List
Ready

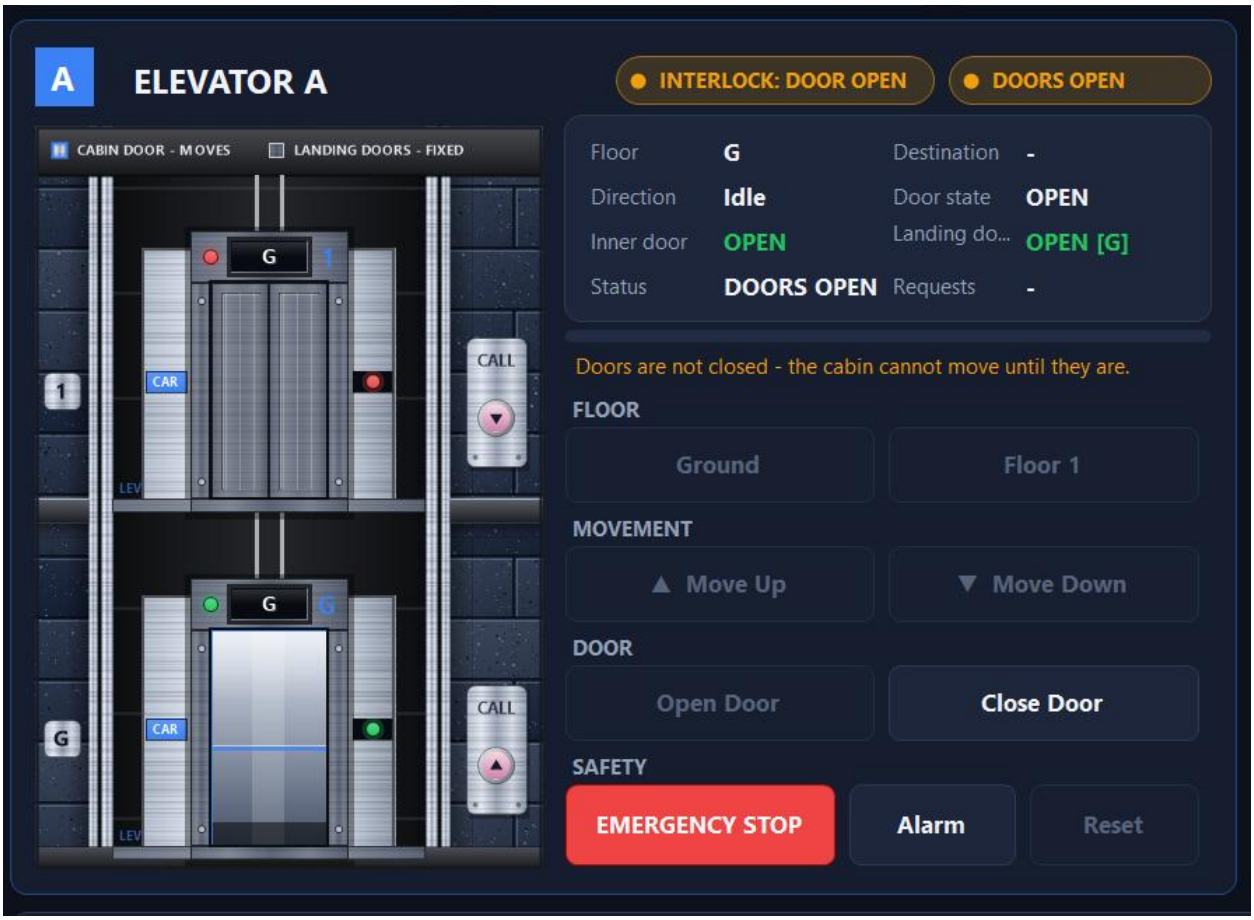
Protoype Screenshot with testing

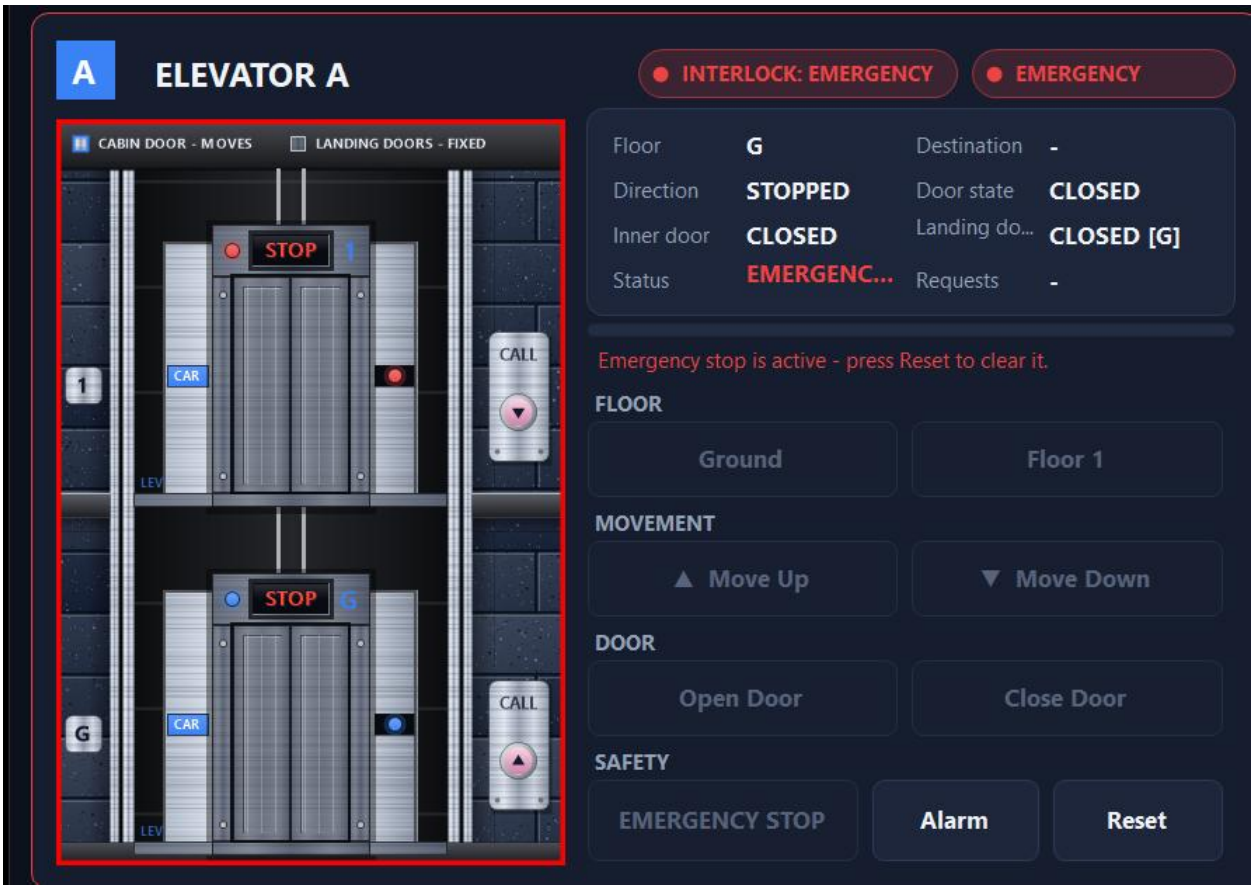



Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ALARM ACTIVE
Mon, 02 Nov 2026
16:00:11
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Alarm
ELEVATOR B
Floor 1 · Alarm
ACTIVE MOVEMENTS
0 of 2
SAFETY
ALARM: Elevator A, Eleva...
LATEST EVENT
16:00:11 Elevator B: ALARM ACTIVATED
A
ELEVATOR A
INTERLOCK: SECURE
ALARM
Floor G
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [G]
Status ALARM AC...
Requests -
Alarm active - press Reset to stop it.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ELEVATOR B
INTERLOCK: SECURE
ALARM
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [1]
Status ALARM AC...
Requests -
Alarm active - press Reset to stop it.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
206 of 206 records
ID Date Time Elevator Floor Action Status
206 02/11/2026 16:00:11 Elevator B 1 ALARM ACTIVATED Warning
205 02/11/2026 16:00:10 Elevator A G ALARM ACTIVATED Warning
204 02/11/2026 16:00:06 Elevator B 1 ALARM STOPPED AUTOMATICALLY Warning
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
16:00:26
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Moving up
ELEVATOR B
Floor 1 · Idle
ACTIVE MOVEMENTS
1 of 2
SAFETY
All clear
LATEST EVENT
16:00:25 Elevator A: GOING UP: GROUND FLOO...
A
ELEVATOR A
INTERLOCK: SECURE
MOVING UP
Floor G
Destination 1
Direction Moving Up ▲
Door state LOCKED
Inner door LOCKED
Landing do... LOCKED
Status MOVING
Requests 1
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ELEVATOR B
INTERLOCK: SECURE
READY
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
211 of 211 records
ID Date Time Elevator Floor Action Status
211 02/11/2026 16:00:25 Elevator A G GOING UP: GROUND FLOOR TO FLOOR 1 Info
210 02/11/2026 16:00:25 Elevator A G ALL DOORS LOCKED Info

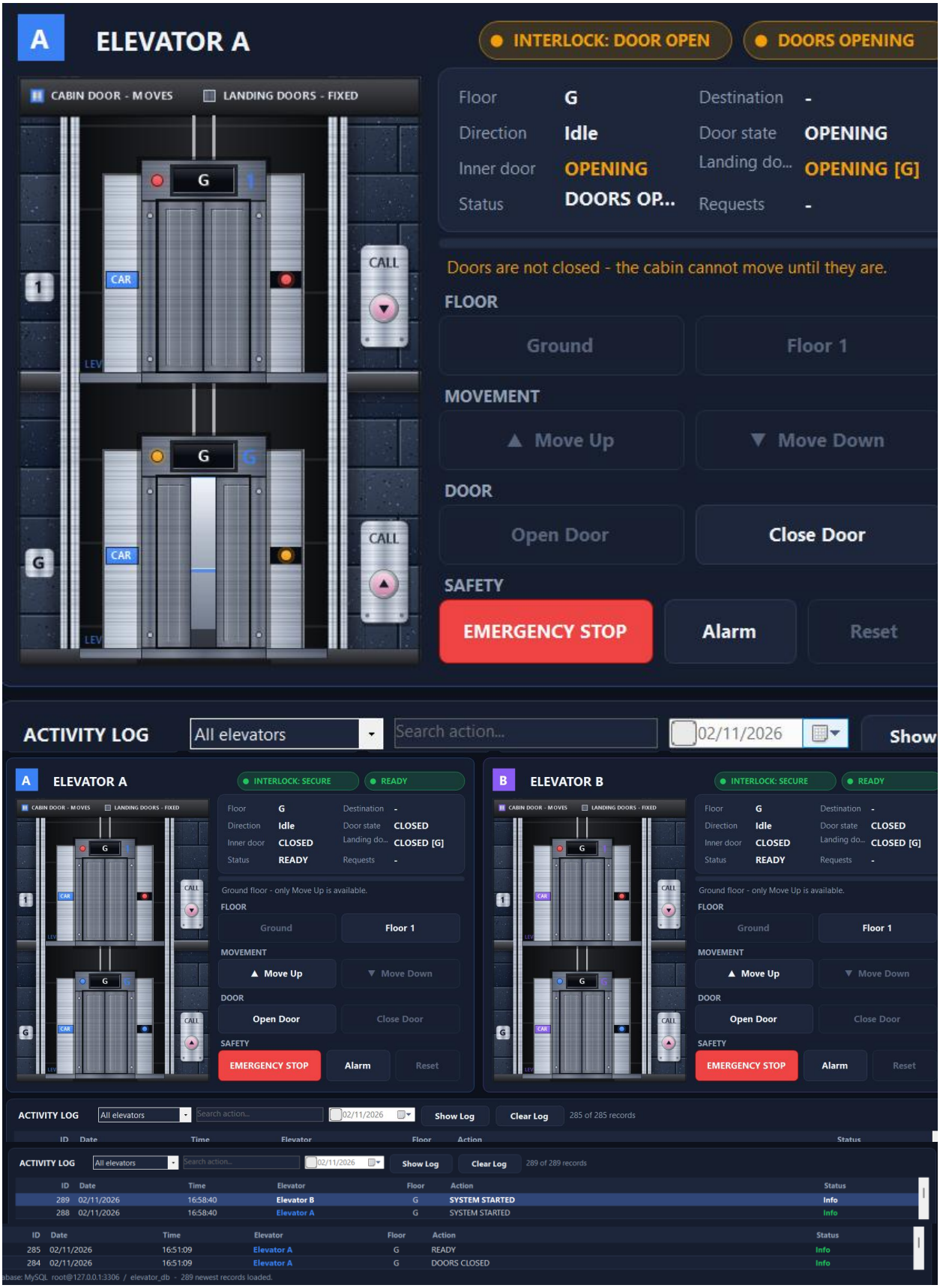

A
ELEVATOR A
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CALL
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
A
ELEVATOR A
INTERLOCK: EMERGENCY
EMERGENCY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
STOP
CAR
CALL
Floor 1
Destination -
Direction STOPPED
Door state OPEN
Inner door OPEN
Landing do... OPEN [1]
Status EMERGENC...
Requests -
Emergency stop is active - press Reset to clear it.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log

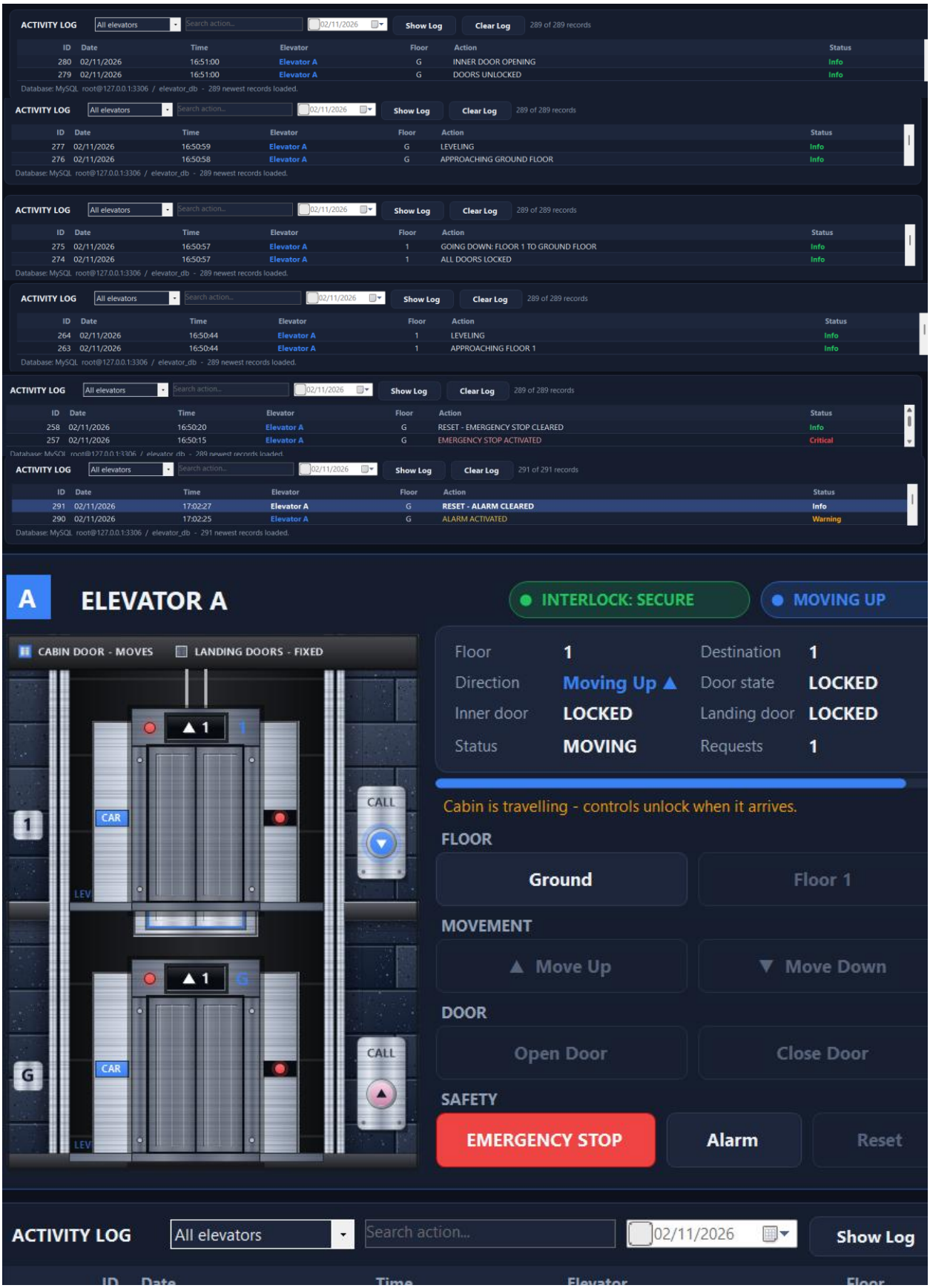

INTERLOCK: SECURE
MOVING DOWN
Floor
G
Destination
G
Direction
Moving Do...
Door state
LOCKED
Inner door
LOCKED
Landing do...
LOCKED
Status
MOVING
Requests
G
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

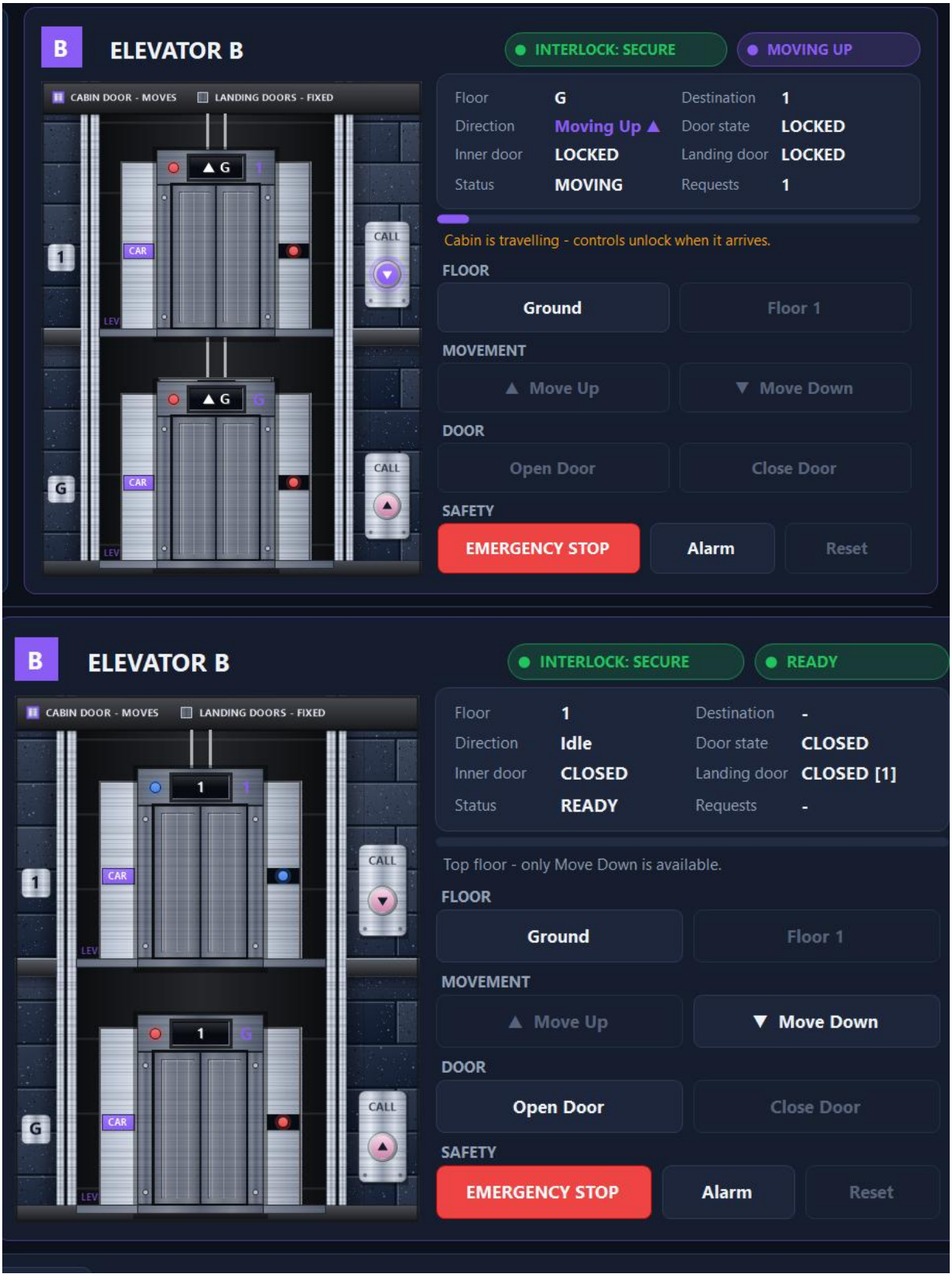

A
ELEVATOR A
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
G
1
CAR
CALL
G
CAR
CALL
Floor G
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing do... OPEN [G]
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

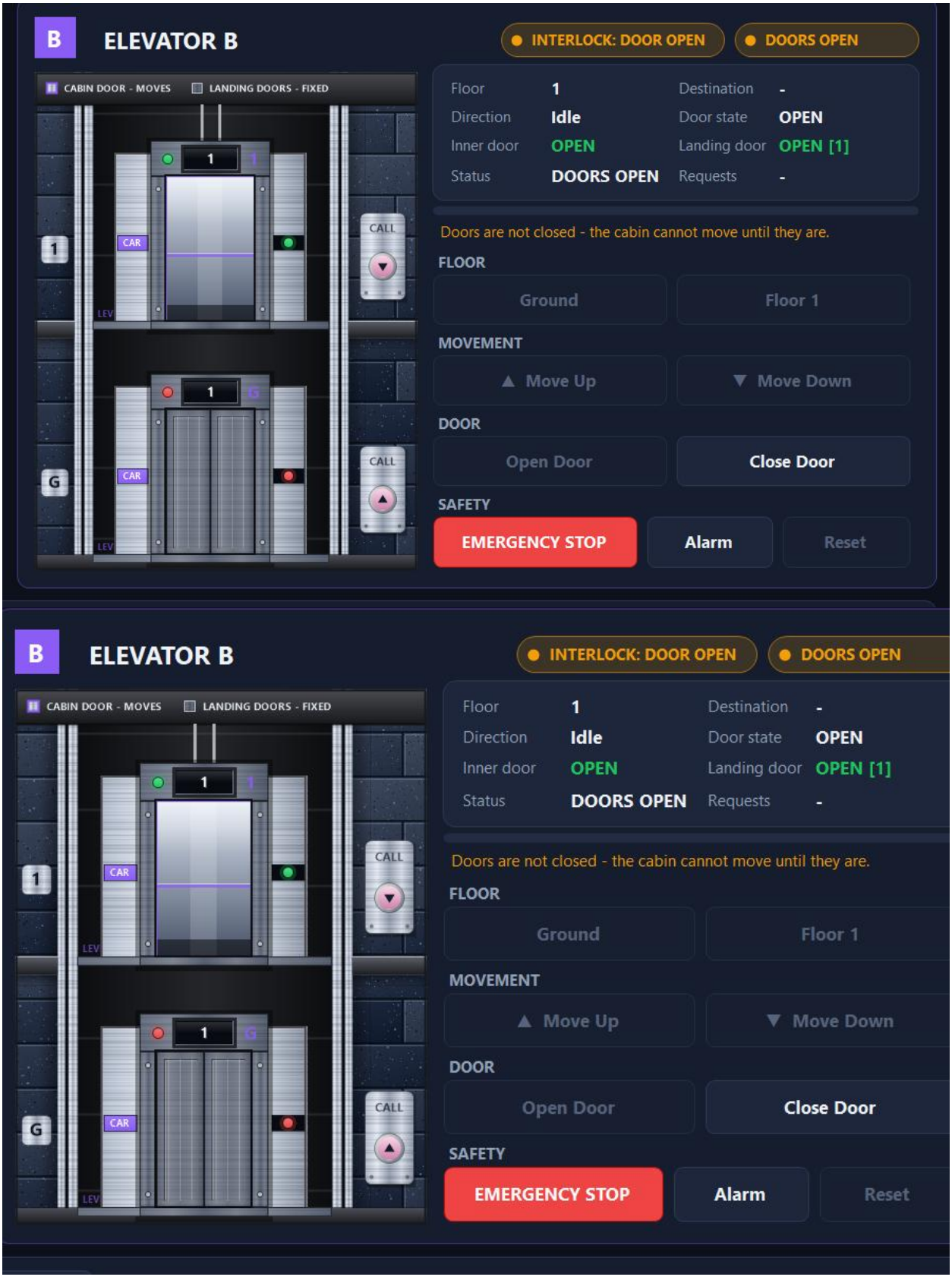

A
ELEVATOR A
INTERLOCK: EMERGENCY
EMERGENCY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
STOP
CAR
CALL
1
STOP
CAR
CALL
G
Floor G
Destination -
Direction STOPPED
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [G]
Status EMERGENC...
Requests -
Emergency stop is active - press Reset to clear it.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

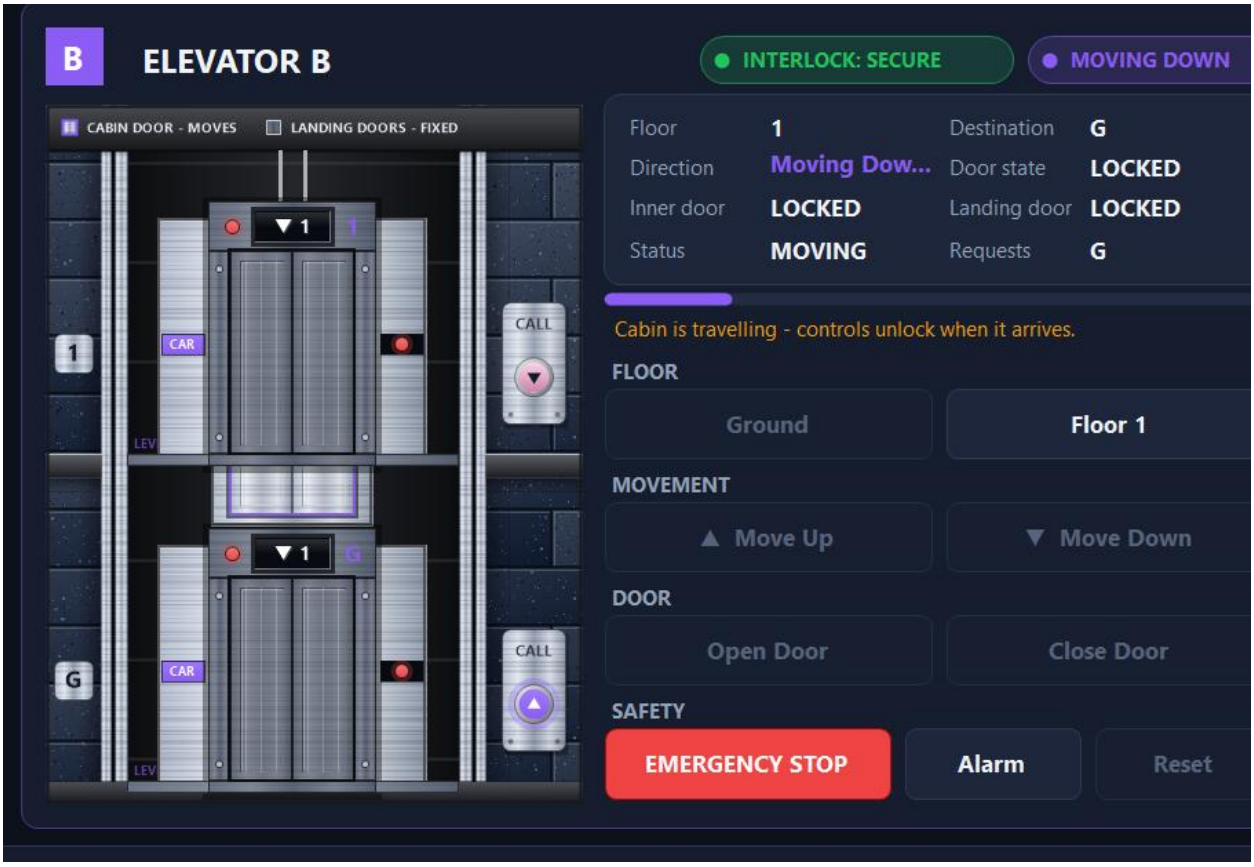

A
ELEVATOR A
INTERLOCK: SECURE
MOVING UP
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Destination 1
Direction Moving Up ▲
Door state LOCKED
Inner door LOCKED
Landing do... LOCKED
Status MOVING
Requests 1
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log

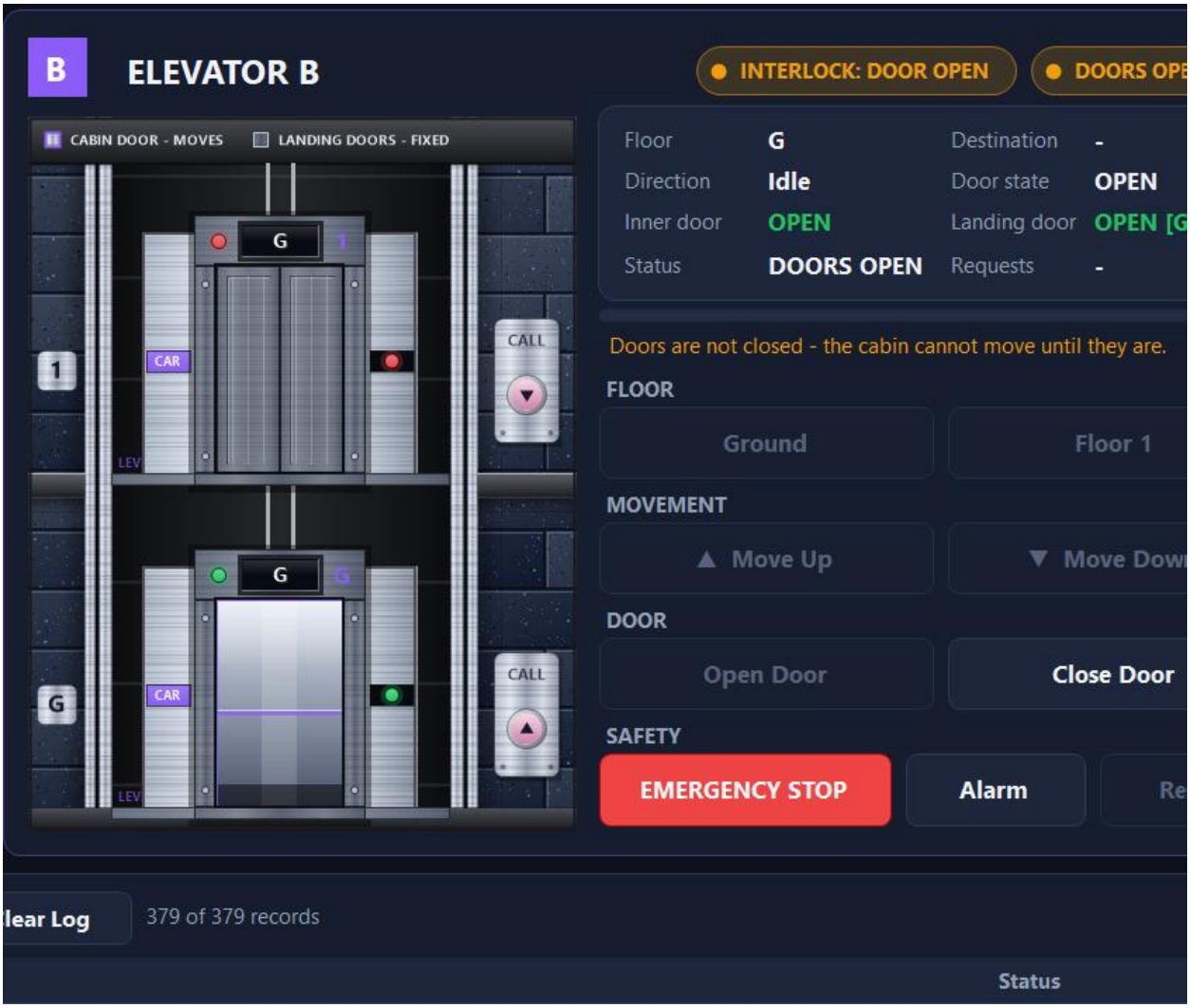

A
ELEVATOR A
INTERLOCK: DOOR OPEN
DOORS OPENING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor G
Direction Idle
Inner door OPENING
Status DOORS OP...
Destination -
Door state OPENING
Landing do... OPENING [G]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show
B
ELEVATOR B
INTERLOCK: SECURE
READY
Ground floor - only Move Up is available.
Show Log
Clear Log
285 of 285 records
289 of 289 records
ID Date Time Elevator Floor Action Status
289 02/11/2026 16:58:40 Elevator B G SYSTEM STARTED Info
288 02/11/2026 16:58:40 Elevator A G SYSTEM STARTED Info
285 02/11/2026 16:51:09 Elevator A G READY Info
284 02/11/2026 16:51:09 Elevator A G DOORS CLOSED Info

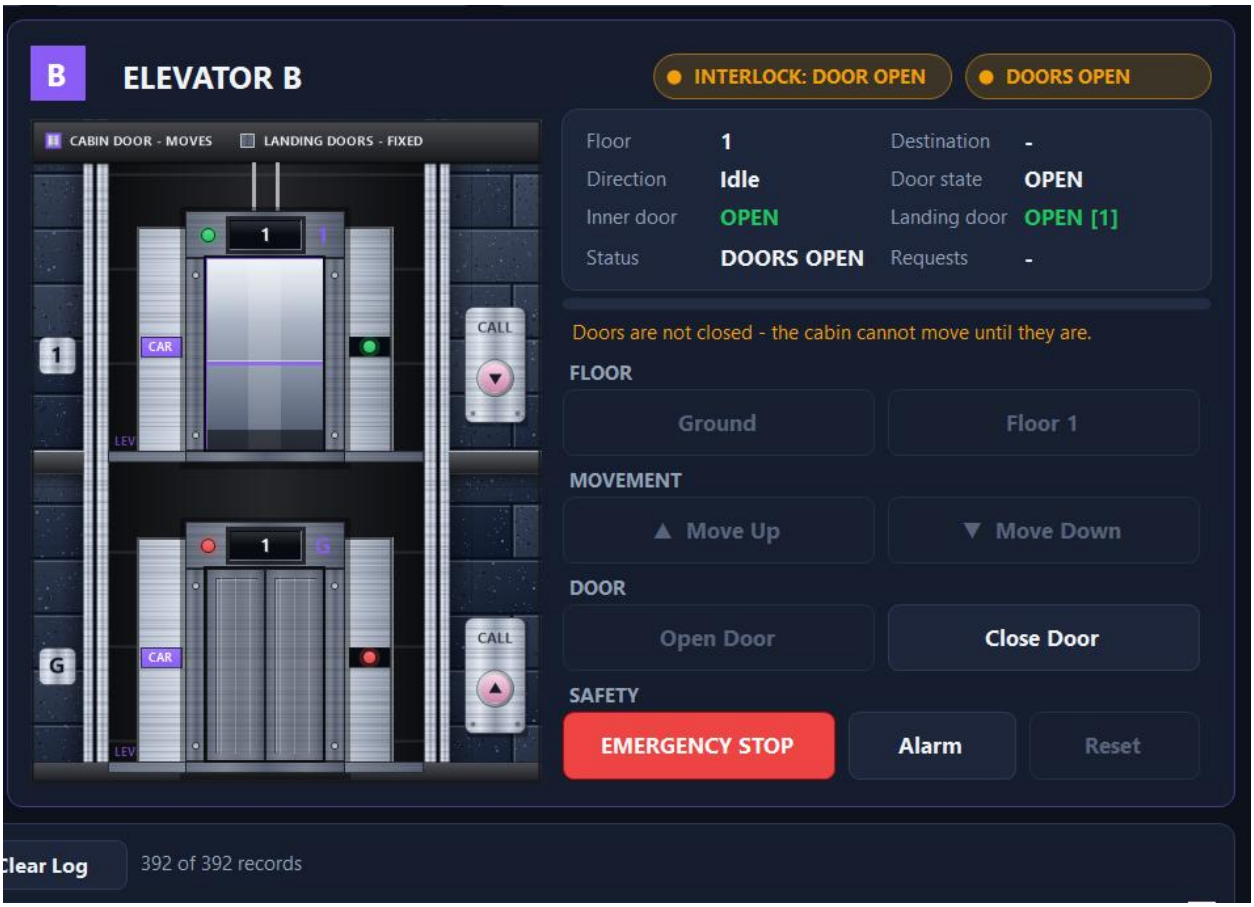

ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
289 of 289 records
ID Date Time Elevator Floor Action Status
280 02/11/2026 16:51:00 Elevator A G INNER DOOR OPENING Info
279 02/11/2026 16:51:00 Elevator A G DOORS UNLOCKED Info
Database: MySQL root@127.0.0.1:3306 / elevator_db - 289 newest records loaded.
277 02/11/2026 16:50:59 Elevator A G LEVELING Info
276 02/11/2026 16:50:58 Elevator A G APPROACHING GROUND FLOOR Info
275 02/11/2026 16:50:57 Elevator A 1 GOING DOWN: FLOOR 1 TO GROUND FLOOR Info
274 02/11/2026 16:50:57 Elevator A 1 ALL DOORS LOCKED Info
264 02/11/2026 16:50:44 Elevator A 1 LEVELING Info
263 02/11/2026 16:50:44 Elevator A 1 APPROACHING FLOOR 1 Info
258 02/11/2026 16:50:20 Elevator A G RESET - EMERGENCY STOP CLEARED Info
257 02/11/2026 16:50:15 Elevator A G EMERGENCY STOP ACTIVATED Critical
291 of 291 records
291 02/11/2026 17:02:27 Elevator A G RESET - ALARM CLEARED Info
290 02/11/2026 17:02:25 Elevator A G ALARM ACTIVATED Warning
291 newest records loaded.
A
ELEVATOR A
INTERLOCK: SECURE
MOVING UP
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Destination 1
Direction Moving Up ▲
Door state LOCKED
Inner door LOCKED
Landing door LOCKED
Status MOVING
Requests 1
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

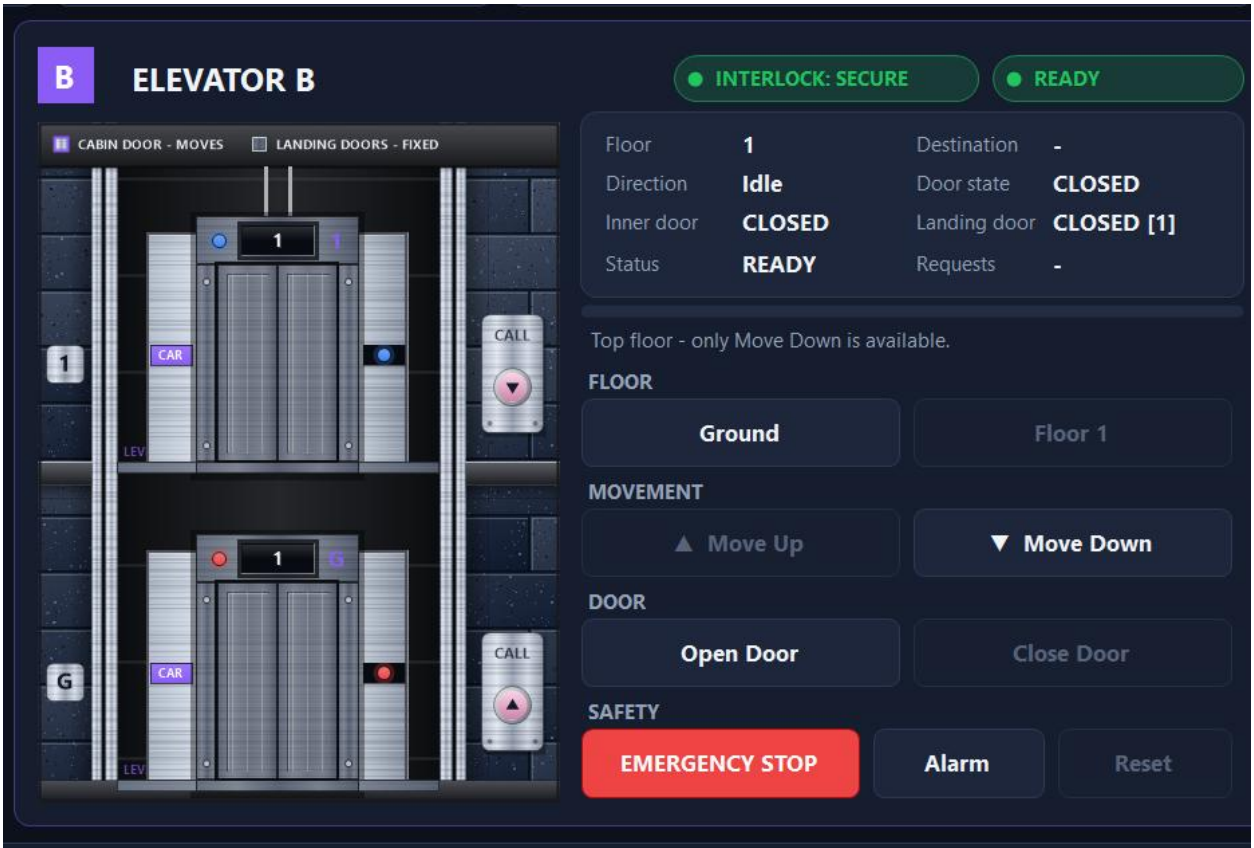

B
ELEVATOR B
INTERLOCK: SECURE
MOVING UP
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor G
Destination 1
Direction Moving Up ▲
Door state LOCKED
Inner door LOCKED
Landing door LOCKED
Status MOVING
Requests 1
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ELEVATOR B
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

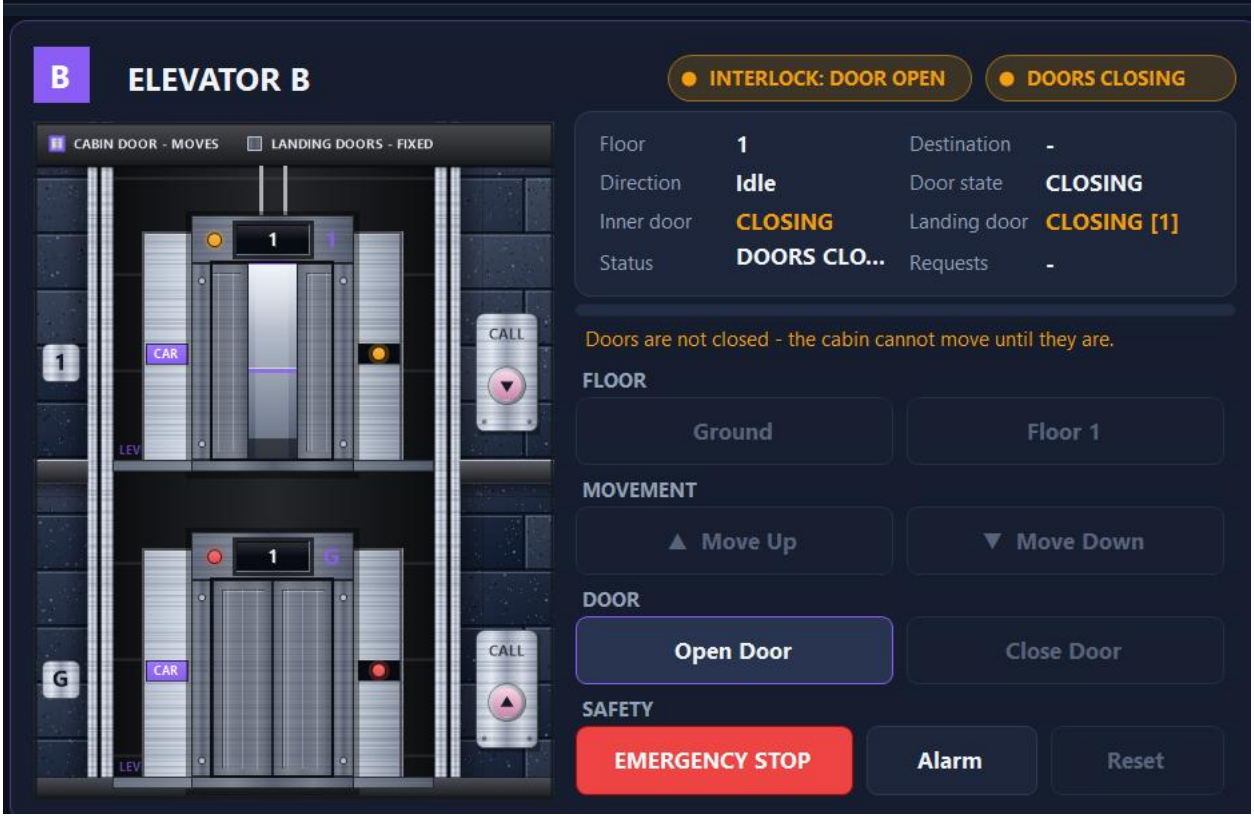

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing door OPEN [1]
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
LEV

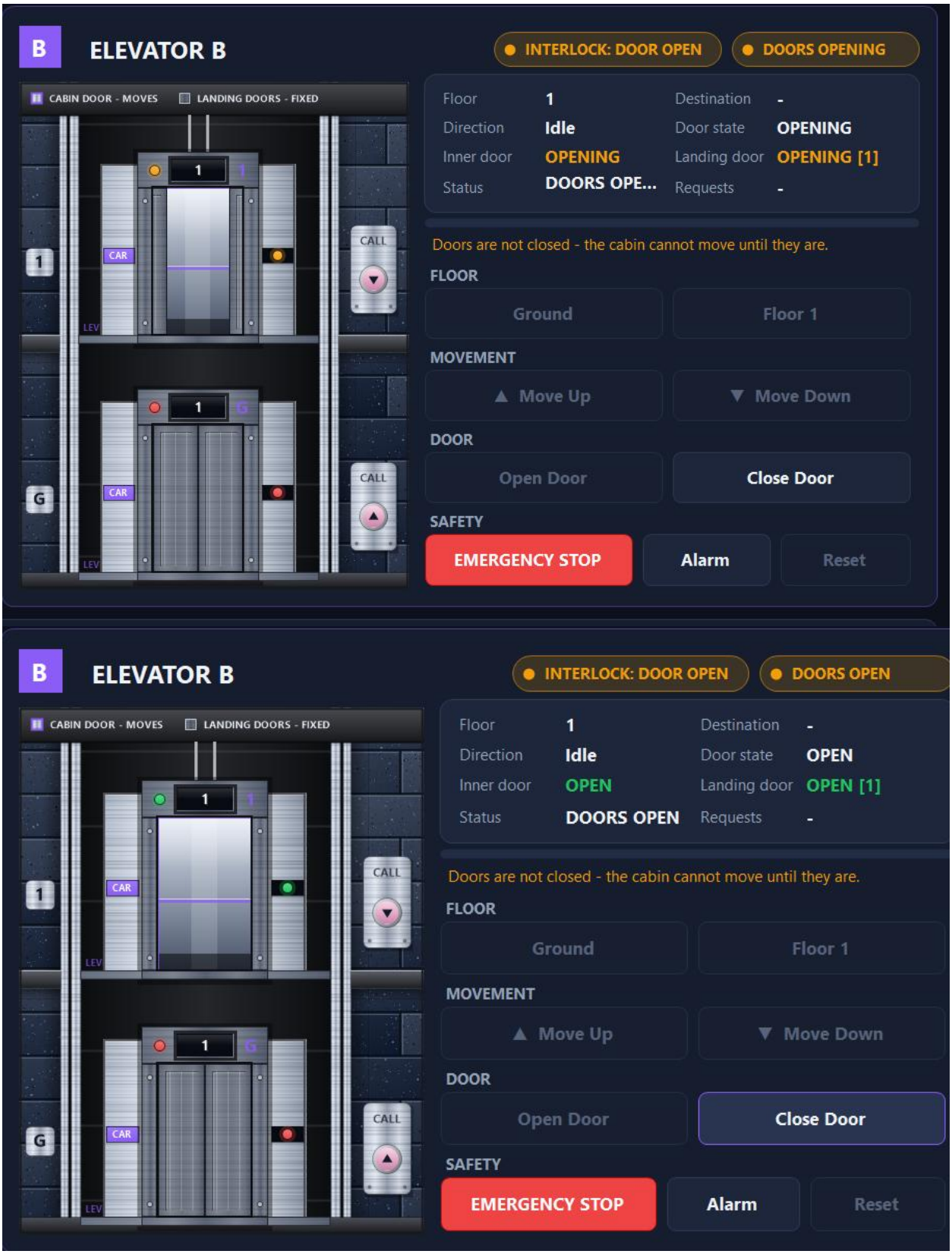

B
ELEVATOR B
INTERLOCK: SECURE
MOVING DOWN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Direction Moving Dow...
Inner door LOCKED
Status MOVING
Destination G
Door state LOCKED
Landing door LOCKED
Requests G
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR

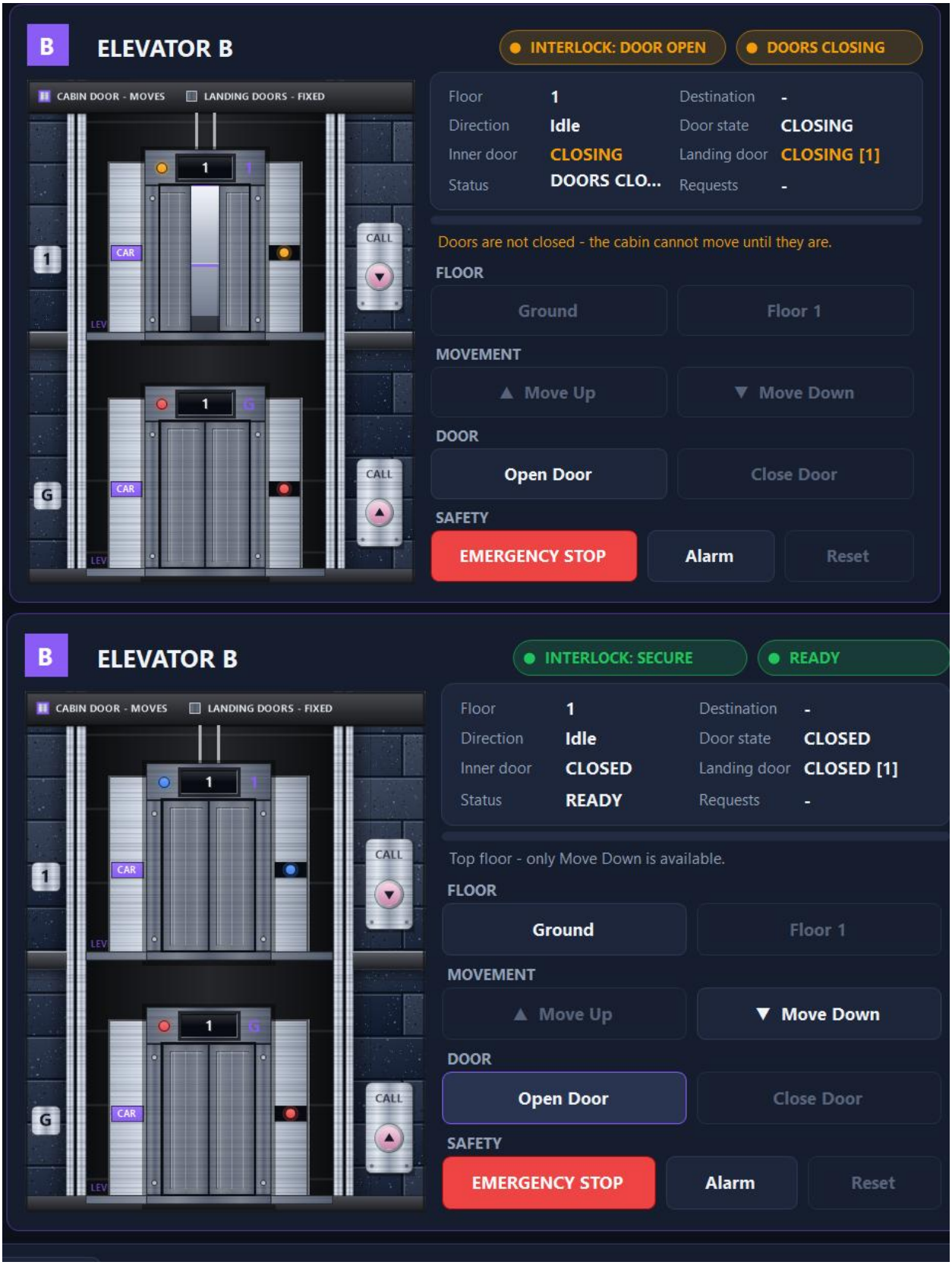

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPE
CABIN DOOR - MOVES
LANDING DOORS - FIXED
G
1
CAR
CALL
LEV
G
G
CAR
CALL
LEV
Floor G
Destination -
Direction Idle
Door state OPEN
Inner door OPEN
Landing door OPEN [G
Status DOORS OPEN
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
lear Log
379 of 379 records
Status

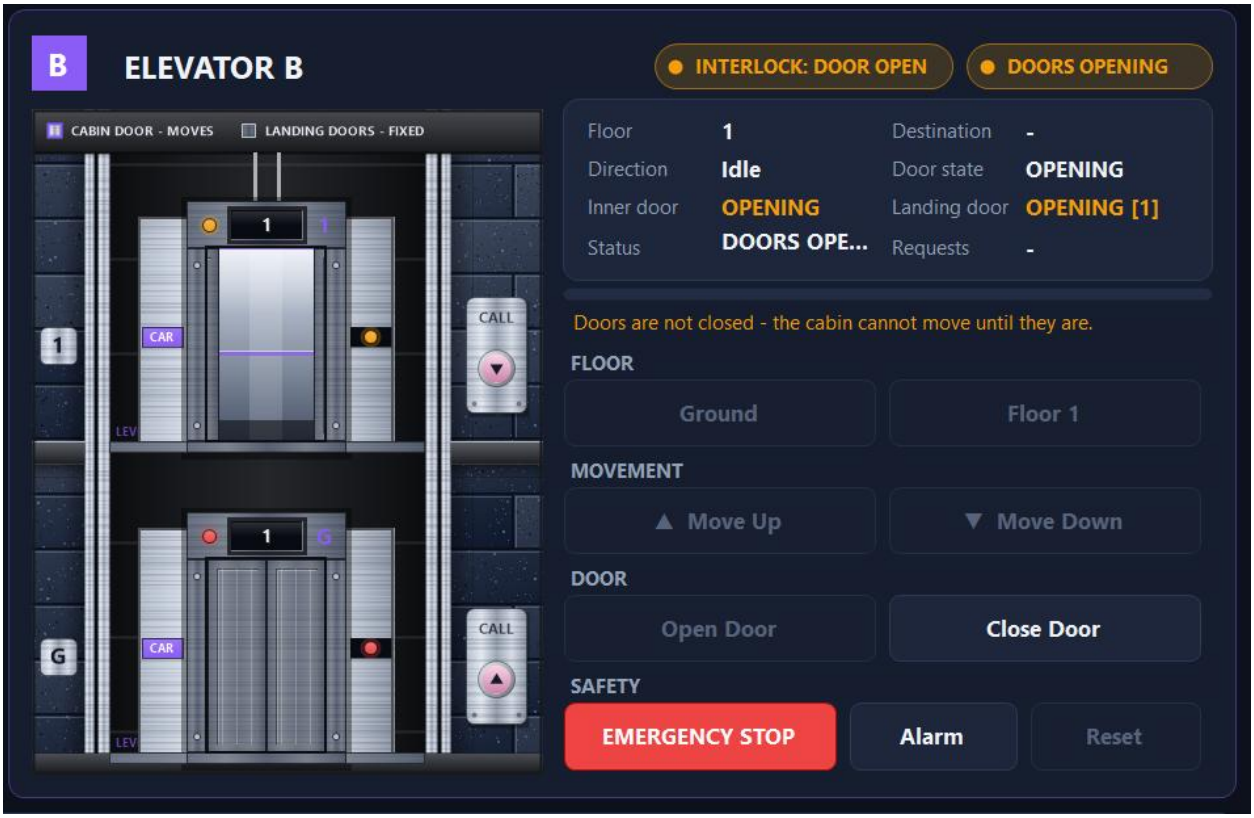

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Direction Idle
Inner door OPEN
Status DOORS OPEN
Destination -
Door state OPEN
Landing door OPEN [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
Clear Log
392 of 392 records

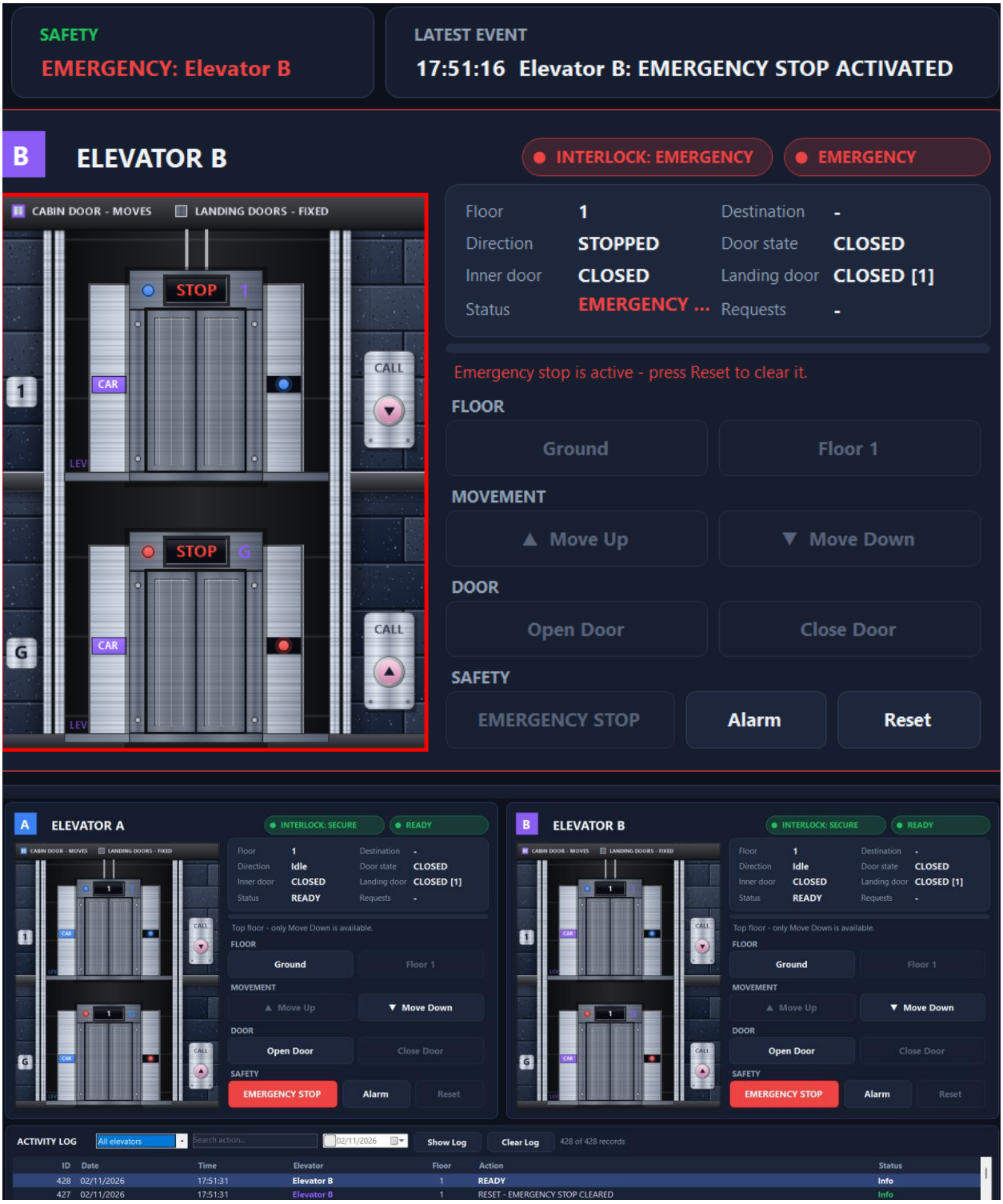

B
ELEVATOR B
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
LEV

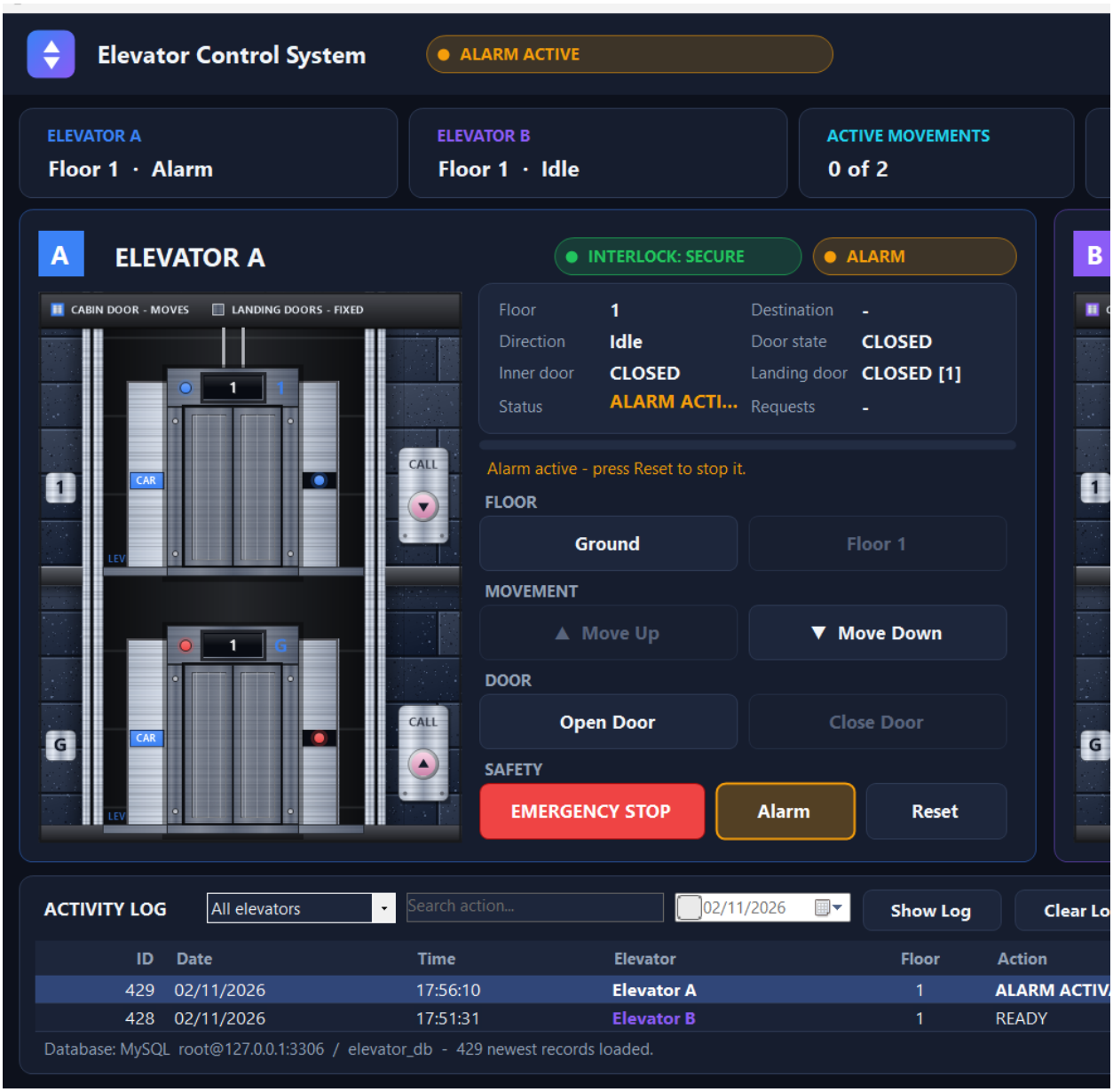

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS CLOSING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSING
Inner door CLOSING
Landing door CLOSING [1]
Status DOORS CLO...
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
LEV

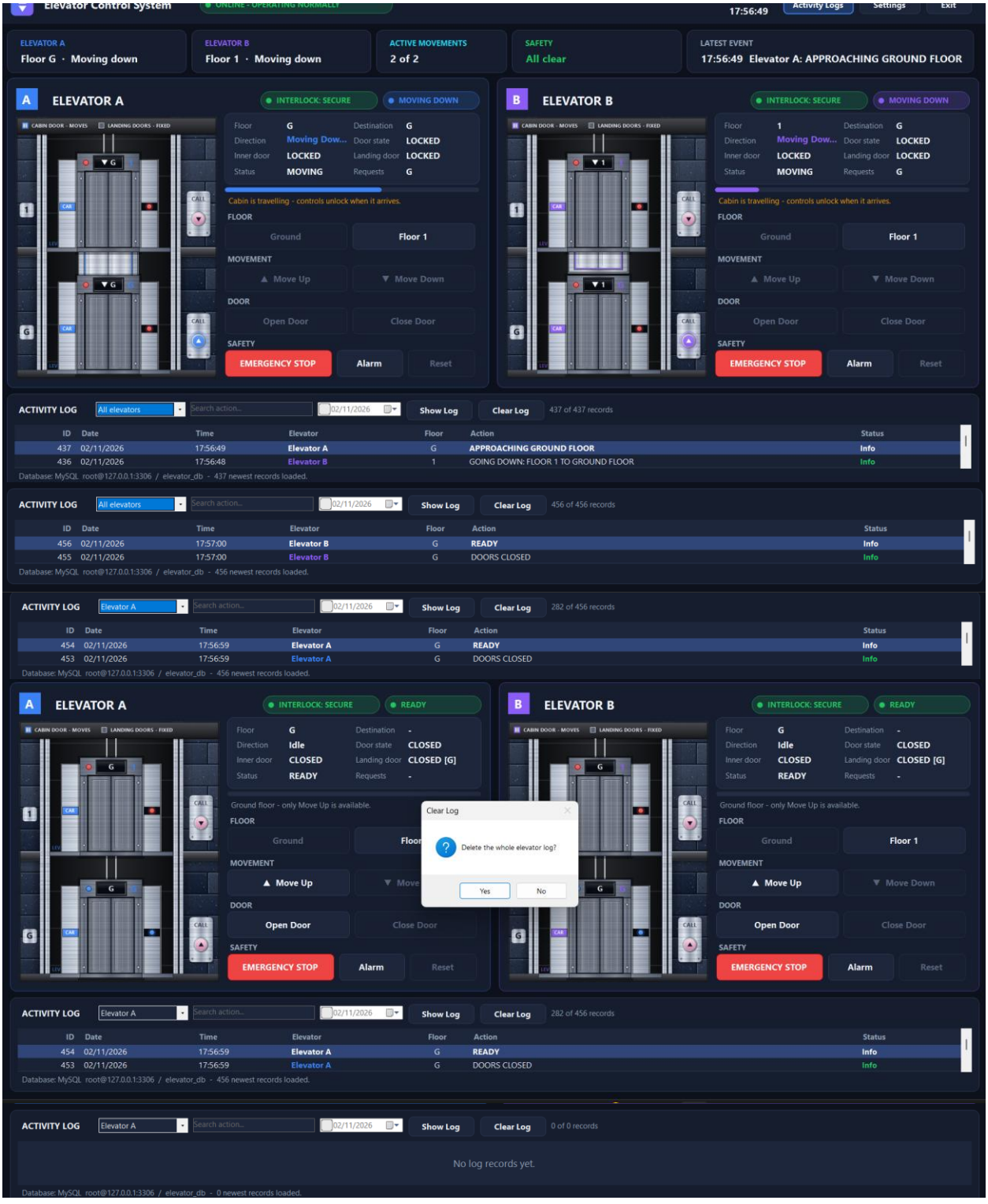

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPENING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Direction Idle
Inner door OPENING
Status DOORS OPE...
Destination -
Door state OPENING
Landing door OPENING [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
LEV
B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPEN
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Direction Idle
Inner door OPEN
Status DOORS OPEN
Destination -
Door state OPEN
Landing door OPEN [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
LEV

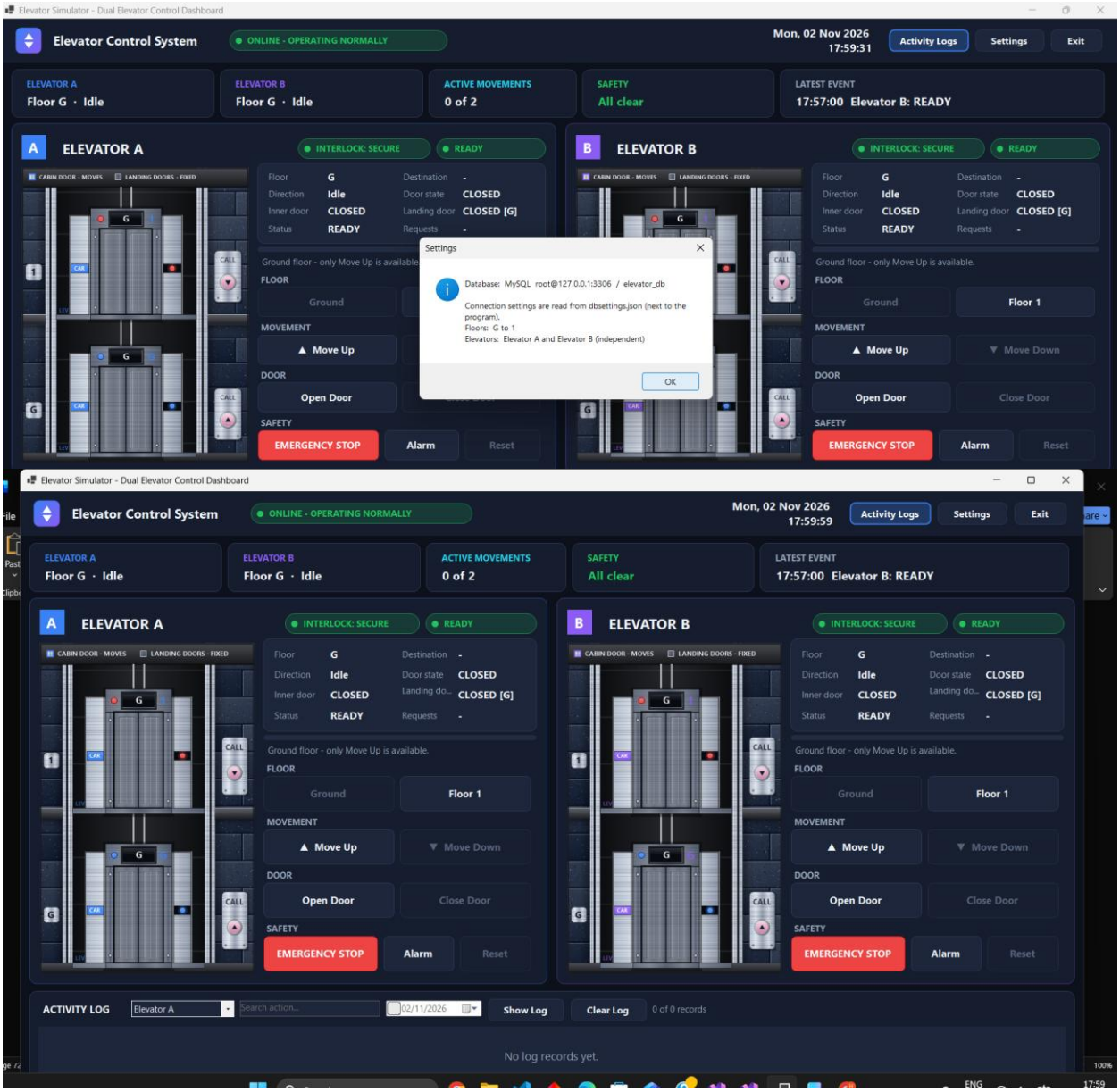

B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS CLOSING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSING
Inner door CLOSING
Landing door CLOSING [1]
Status DOORS CLO...
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR
B
ELEVATOR B
INTERLOCK: SECURE
READY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
Floor 1
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
CALL
CAR


B
ELEVATOR B
INTERLOCK: DOOR OPEN
DOORS OPENING
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Direction Idle
Inner door OPENING
Status DOORS OPE...
Destination -
Door state OPENING
Landing door OPENING [1]
Requests -
Doors are not closed - the cabin cannot move until they are.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset

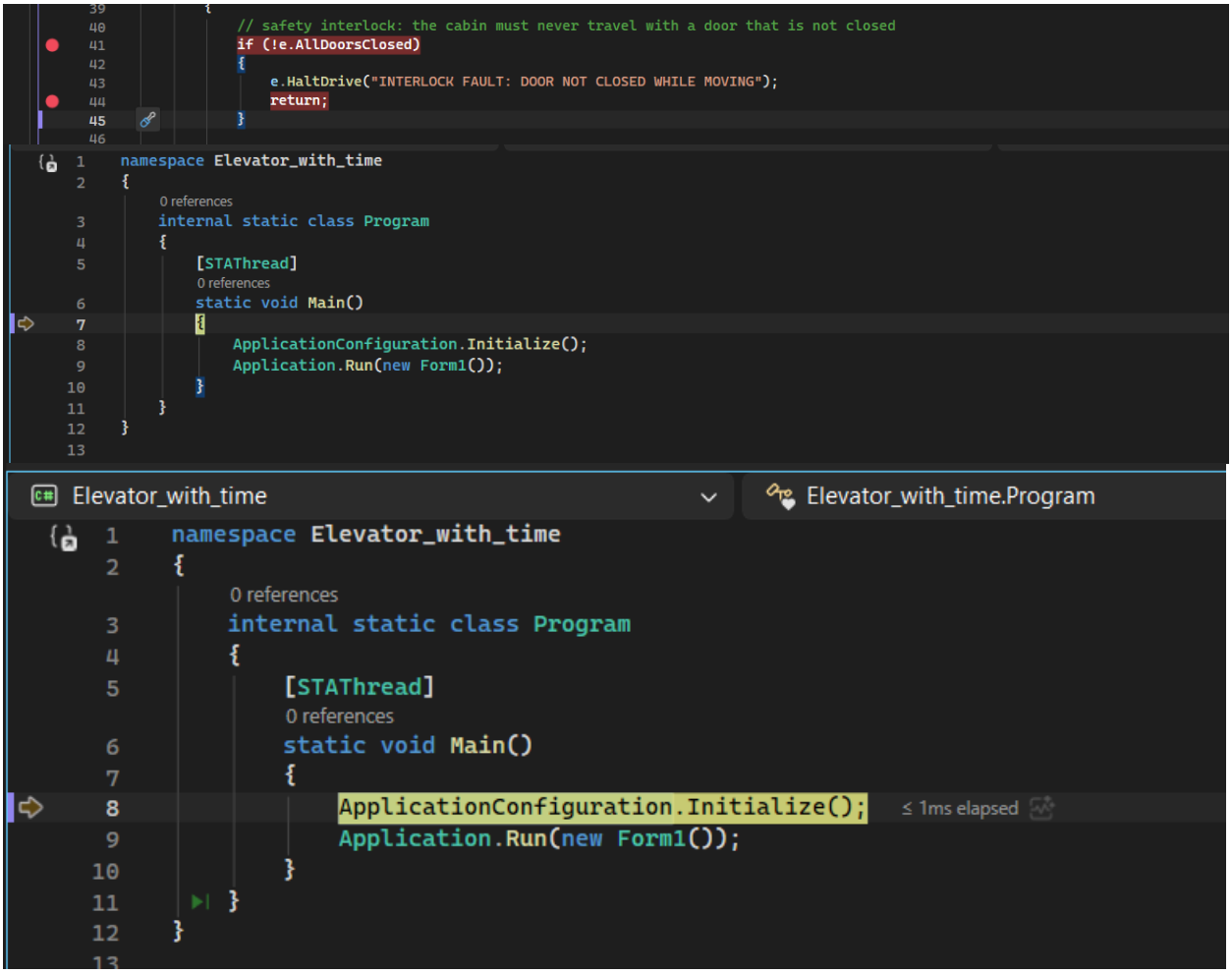

SAFETY
EMERGENCY: Elevator B
LATEST EVENT
17:51:16 Elevator B: EMERGENCY STOP ACTIVATED
B
ELEVATOR B
INTERLOCK: EMERGENCY
EMERGENCY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
STOP
CALL
CAR
Floor 1
Direction STOPPED
Inner door CLOSED
Status EMERGENCY ...
Destination -
Door state CLOSED
Landing door CLOSED [1]
Requests -
Emergency stop is active - press Reset to clear it.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
A
ELEVATOR A
INTERLOCK: SECURE
READY
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [1]
Status READY
Requests -
Top floor - only Move Down is available.
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
428 of 428 records
ID Date Time Elevator Floor Action Status
428 02/11/2026 17:51:31 Elevator B 1 READY Info
427 02/11/2026 17:51:31 Elevator B 1 RESET - EMERGENCY STOP CLEARED Info

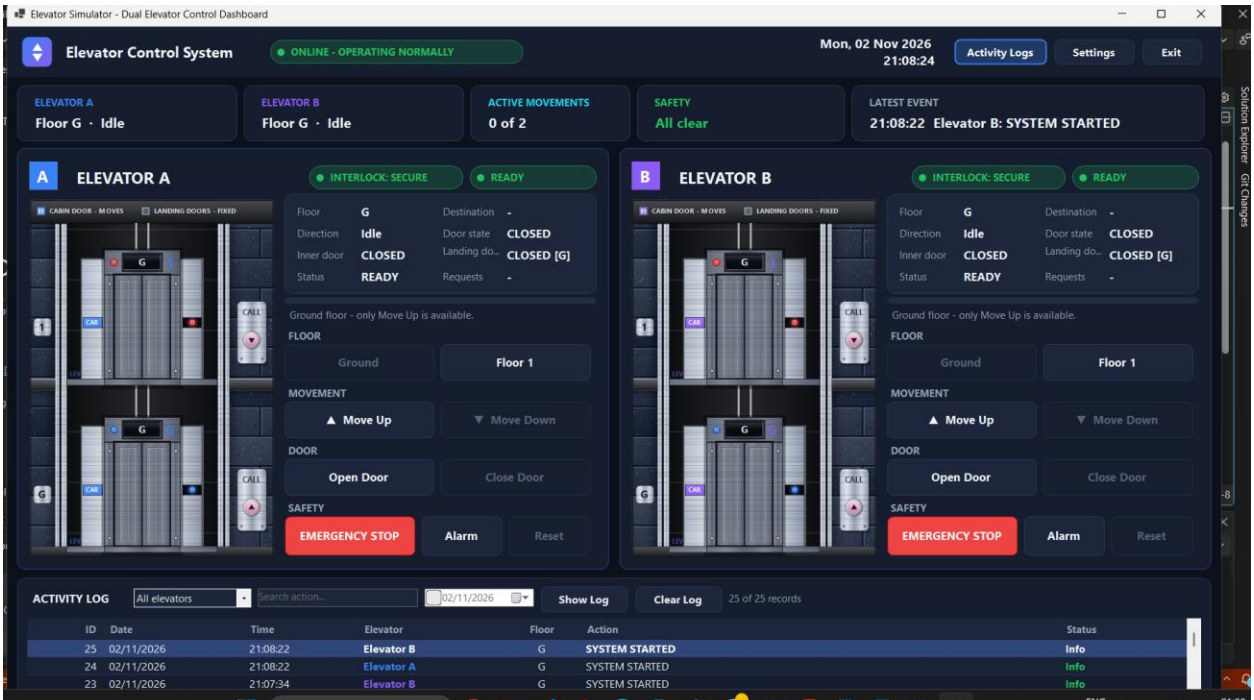

Elevator Control System
ALARM ACTIVE
ELEVATOR A
Floor 1 · Alarm
ELEVATOR B
Floor 1 · Idle
ACTIVE MOVEMENTS
0 of 2
A
ELEVATOR A
INTERLOCK: SECURE
ALARM
CABIN DOOR - MOVES
LANDING DOORS - FIXED
CAR
CALL
Floor 1
Direction Idle
Inner door CLOSED
Status ALARM ACTI...
Destination -
Door state CLOSED
Landing door CLOSED [1]
Requests -
Alarm active - press Reset to stop it.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Lo
ID Date Time Elevator Floor Action
429 02/11/2026 17:56:10 Elevator A 1 ALARM ACTIV
428 02/11/2026 17:51:31 Elevator B 1 READY
Database: MySQL root@127.0.0.1:3306 / elevator_db - 429 newest records loaded.


Elevator Control System
17:56:49
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Moving down
ELEVATOR B
Floor 1 · Moving down
ACTIVE MOVEMENTS
2 of 2
SAFETY
All clear
LATEST EVENT
17:56:49 Elevator A: APPROACHING GROUND FLOOR
A ELEVATOR A
INTERLOCK: SECURE
MOVING DOWN
Floor G
Destination G
Direction Moving Dow...
Door state LOCKED
Inner door LOCKED
Landing door LOCKED
Status MOVING
Requests G
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B ELEVATOR B
INTERLOCK: SECURE
MOVING DOWN
Floor 1
Destination G
Direction Moving Dow...
Door state LOCKED
Inner door LOCKED
Landing door LOCKED
Status MOVING
Requests G
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
437 of 437 records
ID Date Time Elevator Floor Action Status
437 02/11/2026 17:56:49 Elevator A G APPROACHING GROUND FLOOR Info
436 02/11/2026 17:56:48 Elevator B 1 GOING DOWN: FLOOR 1 TO GROUND FLOOR Info
Database: MySQL root@127.0.0.1:3306 / elevator_db - 437 newest records loaded.
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
456 of 456 records
ID Date Time Elevator Floor Action Status
456 02/11/2026 17:57:00 Elevator B G READY Info
455 02/11/2026 17:57:00 Elevator B G DOORS CLOSED Info
Database: MySQL root@127.0.0.1:3306 / elevator_db - 456 newest records loaded.
ACTIVITY LOG
Elevator A
Search action...
02/11/2026
Show Log
Clear Log
282 of 456 records
ID Date Time Elevator Floor Action Status
454 02/11/2026 17:56:59 Elevator A G READY Info
453 02/11/2026 17:56:59 Elevator A G DOORS CLOSED Info
Database: MySQL root@127.0.0.1:3306 / elevator_db - 456 newest records loaded.
A ELEVATOR A
INTERLOCK: SECURE
READY
Floor G
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [G]
Status READY
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
MOVEMENT
Move Up
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
Clear Log
Delete the whole elevator log?
Yes
No
B ELEVATOR B
INTERLOCK: SECURE
READY
Floor G
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing door CLOSED [G]
Status READY
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
Elevator A
Search action...
02/11/2026
Show Log
Clear Log
282 of 456 records
ID Date Time Elevator Floor Action Status
454 02/11/2026 17:56:59 Elevator A G READY Info
453 02/11/2026 17:56:59 Elevator A G DOORS CLOSED Info
Database: MySQL root@127.0.0.1:3306 / elevator_db - 456 newest records loaded.
ACTIVITY LOG
Elevator A
Search action...
02/11/2026
Show Log
Clear Log
0 of 0 records
No log records yet.
Database: MySQL root@127.0.0.1:3306 / elevator_db - 0 newest records loaded.


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
17:59:31
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Idle
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
17:57:00 Elevator B: READY
ELEVATOR A
INTERLOCK: SECURE
READY
ELEVATOR B
Settings
Database: MySQL root@127.0.0.1:3306 / elevator_db
Connection settings are read from dbsettings.json (next to the program).
Floors: G to 1
Elevators: Elevator A and Elevator B (independent)
OK
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
Mon, 02 Nov 2026
17:59:59
ACTIVITY LOG
Elevator A
Search action...
02/11/2026
Show Log
Clear Log
0 of 0 records
No log records yet.

WhiteBox testing



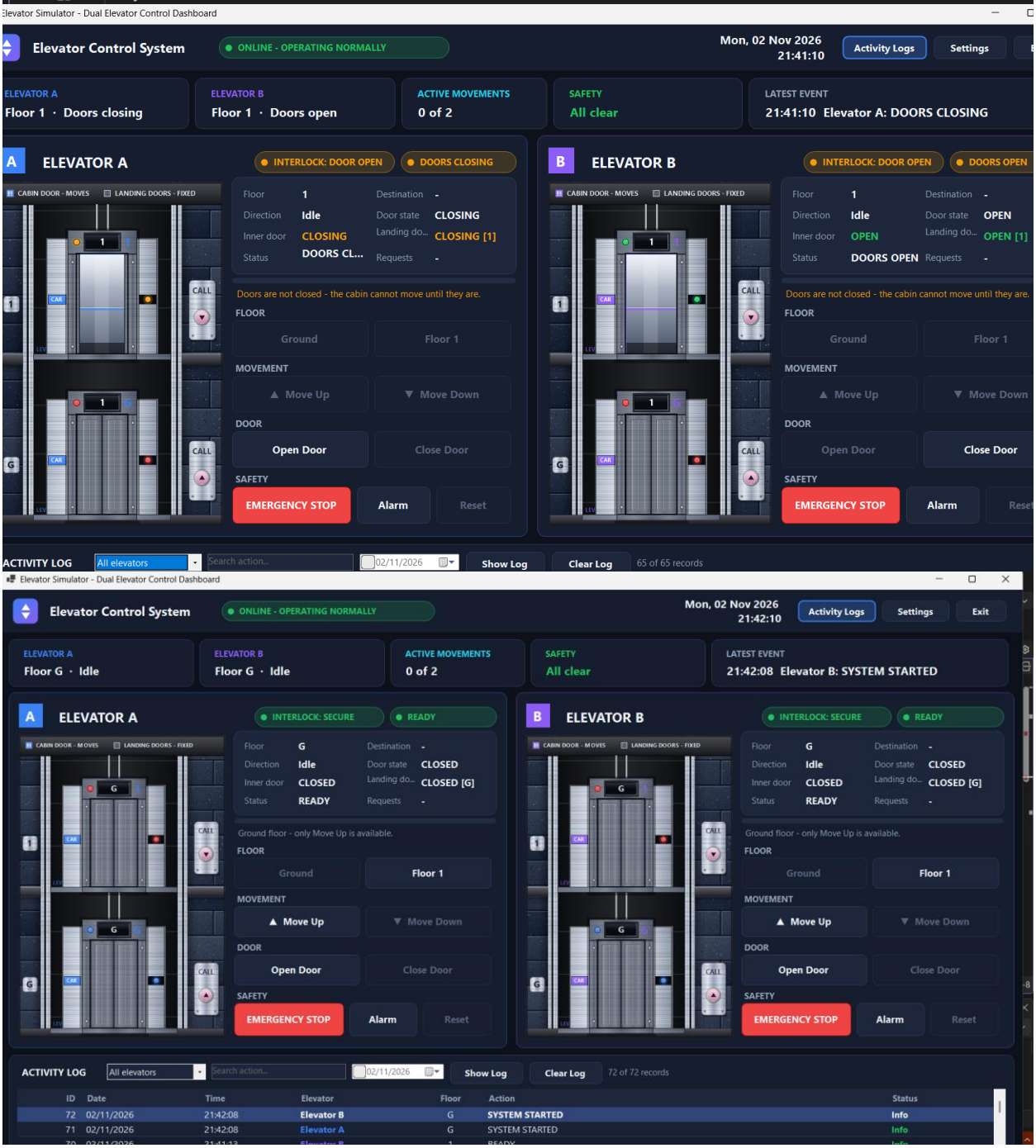


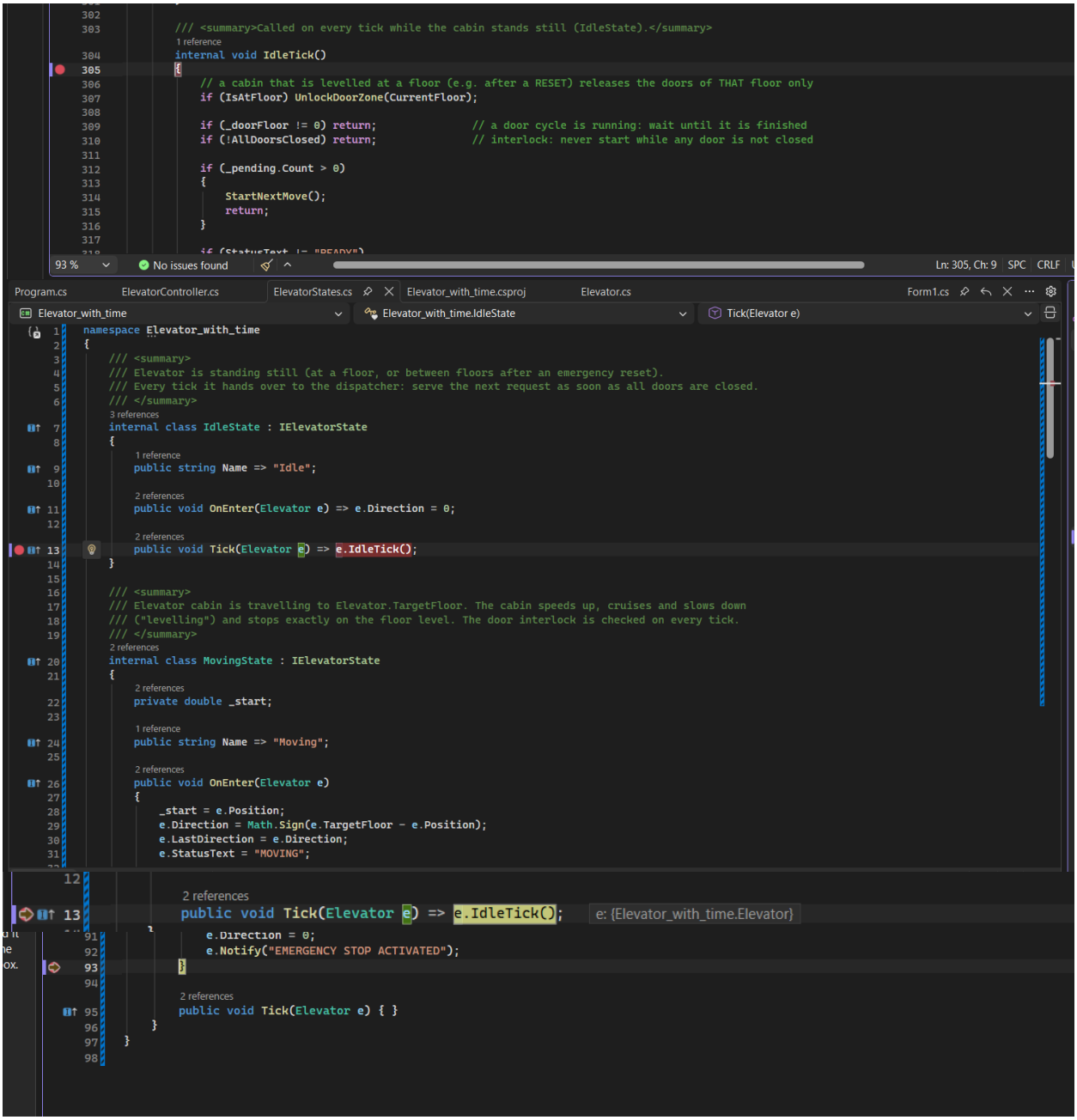

```
Elevator_with_time
Elevator_with_time.Program
Main()
1  namespace Elevator_with_time
2  {
       0 references
3      internal static class Program
4      {
5          [STAThread]
           0 references
6          static void Main()
7          {
8              ApplicationConfiguration.Initialize();
9              Application.Run(new Form1());   ≤ 7ms elapsed
10         }
11     }
12 }
13
```

```
39             {
40                 // safety interlock: the cabin must never travel with a door that is not closed
41                 if (!e.AllDoorsClosed)
42                 {
43                     e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
44                     return;
45                 }
46
47                 double diff = e.TargetFloor - e.Position;
48                 double dist = Math.Abs(diff);
6          static void Main()
7          {
8              ApplicationConfiguration.Initialize();
9              Application.Run(new Form1());
10         }
11     }
12 }
```



```
12          {
13              int floor = Elevator.NextFloorAbove;
14              if (floor <= Elevator_with_time.Elevator.MaxFloor) Elevator.RequestFloor(floor);
15          }
16
17          /// <summary>DOWN: request the next floor below the cabin (nothing happens at the ground floor).</summary>
            namespace Elevator_with_time
              public class ElevatorController
18          {
19              int floor = Elevator.NextFloorBelow;
20              if (floor >= Elevator_with_time.Elevator.MinFloor) Elevator.RequestFloor(floor);
21          }
22      }
23  }
```





```
/// <summary>Called on every tick while the cabin stands still (IdleState).</summary>
1 reference
internal void IdleTick()
{
    // a cabin that is levelled at a floor (e.g. after a RESET) releases the doors of THAT floor only
    if (IsAtFloor) UnlockDoorZone(CurrentFloor);

    if (_doorFloor != 0) return;              // a door cycle is running: wait until it is finished
    if (!AllDoorsClosed) return;              // interlock: never start while any door is not closed

    if (_pending.Count > 0)
    {
        StartNextMove();
        return;
    }
```

93 % · No issues found · Ln: 305, Ch: 9 · SPC · CRLF

Program.cs | ElevatorController.cs | ElevatorStates.cs | Elevator_with_time.csproj | Elevator.cs | Form1.cs

Elevator_with_time · Elevator_with_time.IdleState · Tick(Elevator e)

```
namespace Elevator_with_time
{
    /// <summary>
    /// Elevator is standing still (at a floor, or between floors after an emergency reset).
    /// Every tick it hands over to the dispatcher: serve the next request as soon as all doors are closed.
    /// </summary>
    3 references
    internal class IdleState : IElevatorState
    {
        1 reference
        public string Name => "Idle";

        2 references
        public void OnEnter(Elevator e) => e.Direction = 0;

        2 references
        public void Tick(Elevator e) => e.IdleTick();
    }

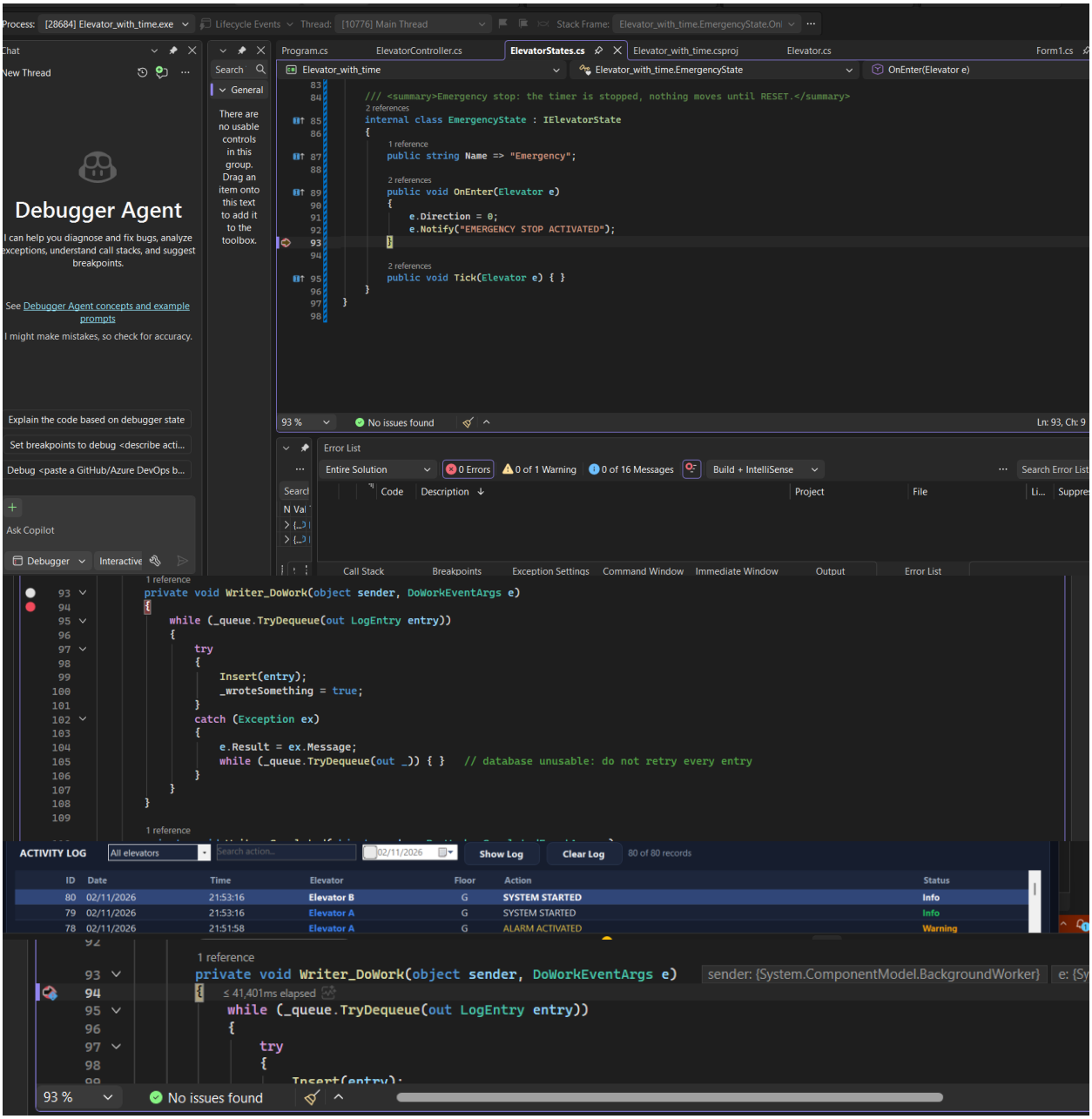
    /// <summary>
    /// Elevator cabin is travelling to Elevator.TargetFloor. The cabin speeds up, cruises and slows down
    /// ("levelling") and stops exactly on the floor level. The door interlock is checked on every tick.
    /// </summary>
    2 references
    internal class MovingState : IElevatorState
    {
        2 references
        private double _start;

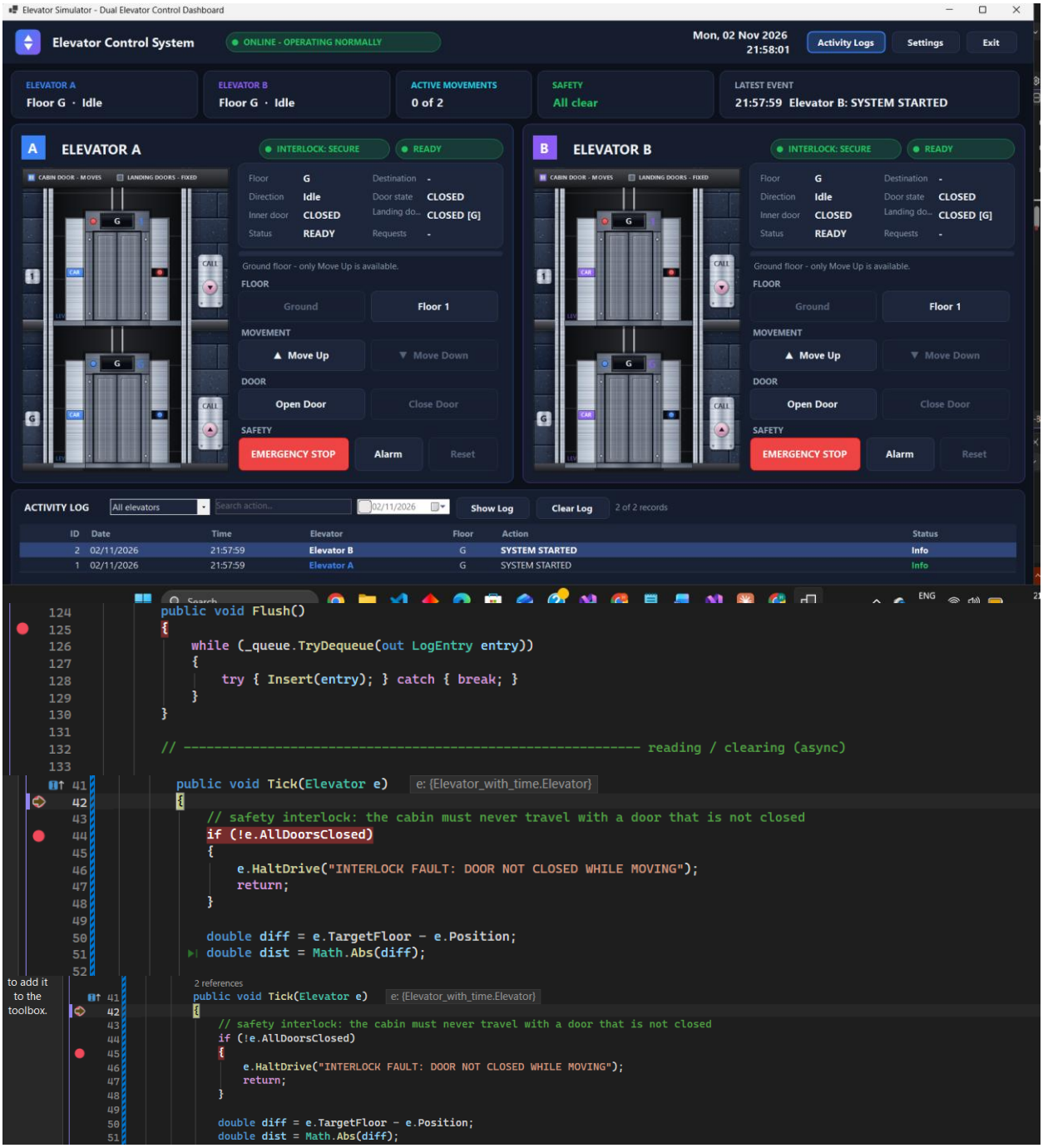
        1 reference
        public string Name => "Moving";

        2 references
        public void OnEnter(Elevator e)
        {
            _start = e.Position;
            e.Direction = Math.Sign(e.TargetFloor - e.Position);
            e.LastDirection = e.Direction;
            e.StatusText = "MOVING";
```

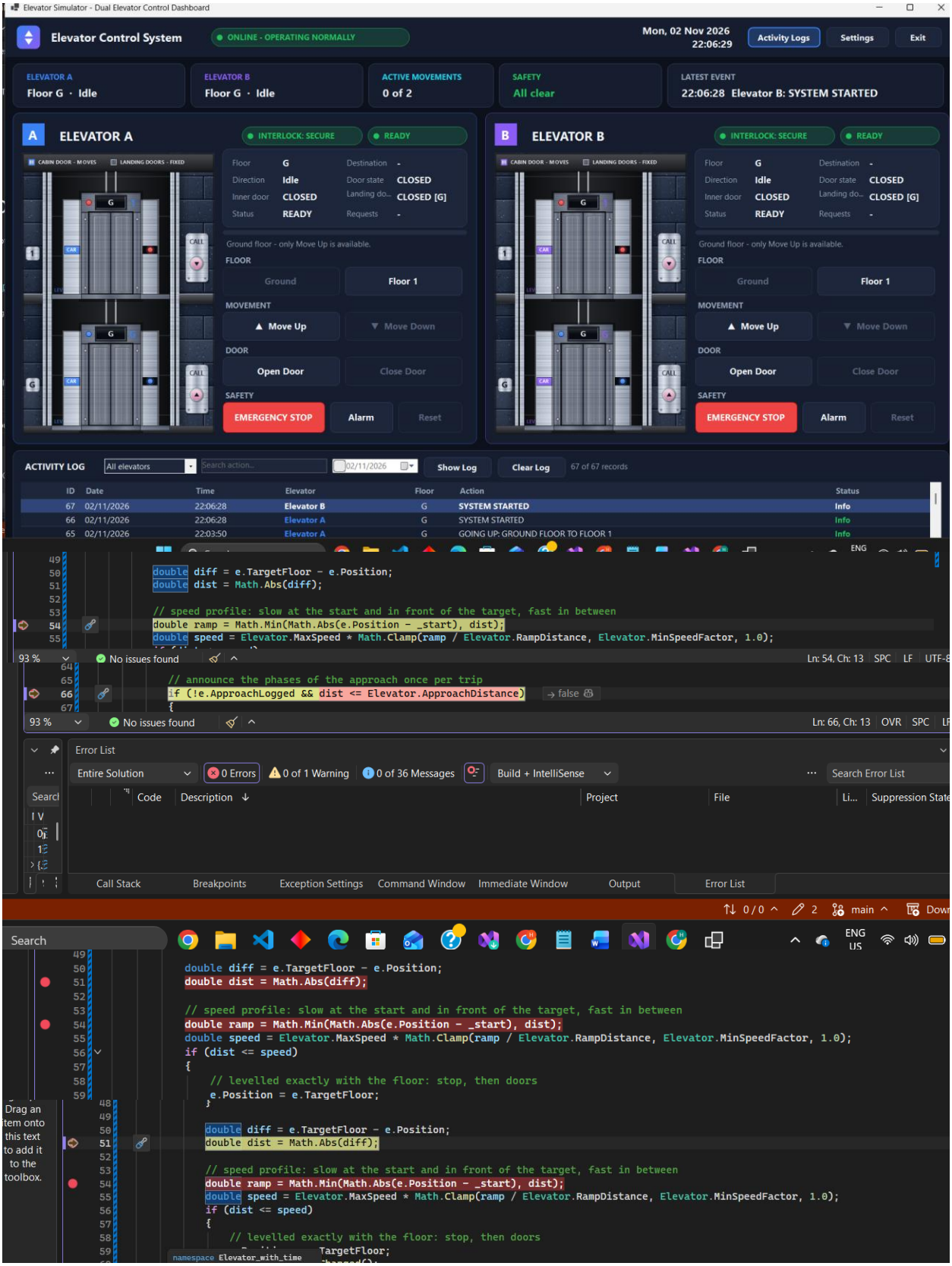
```
        2 references
        public void Tick(Elevator e) => e.IdleTick();    e: {Elevator_with_time.Elevator}
            e.Direction = 0;
            e.Notify("EMERGENCY STOP ACTIVATED");
        }

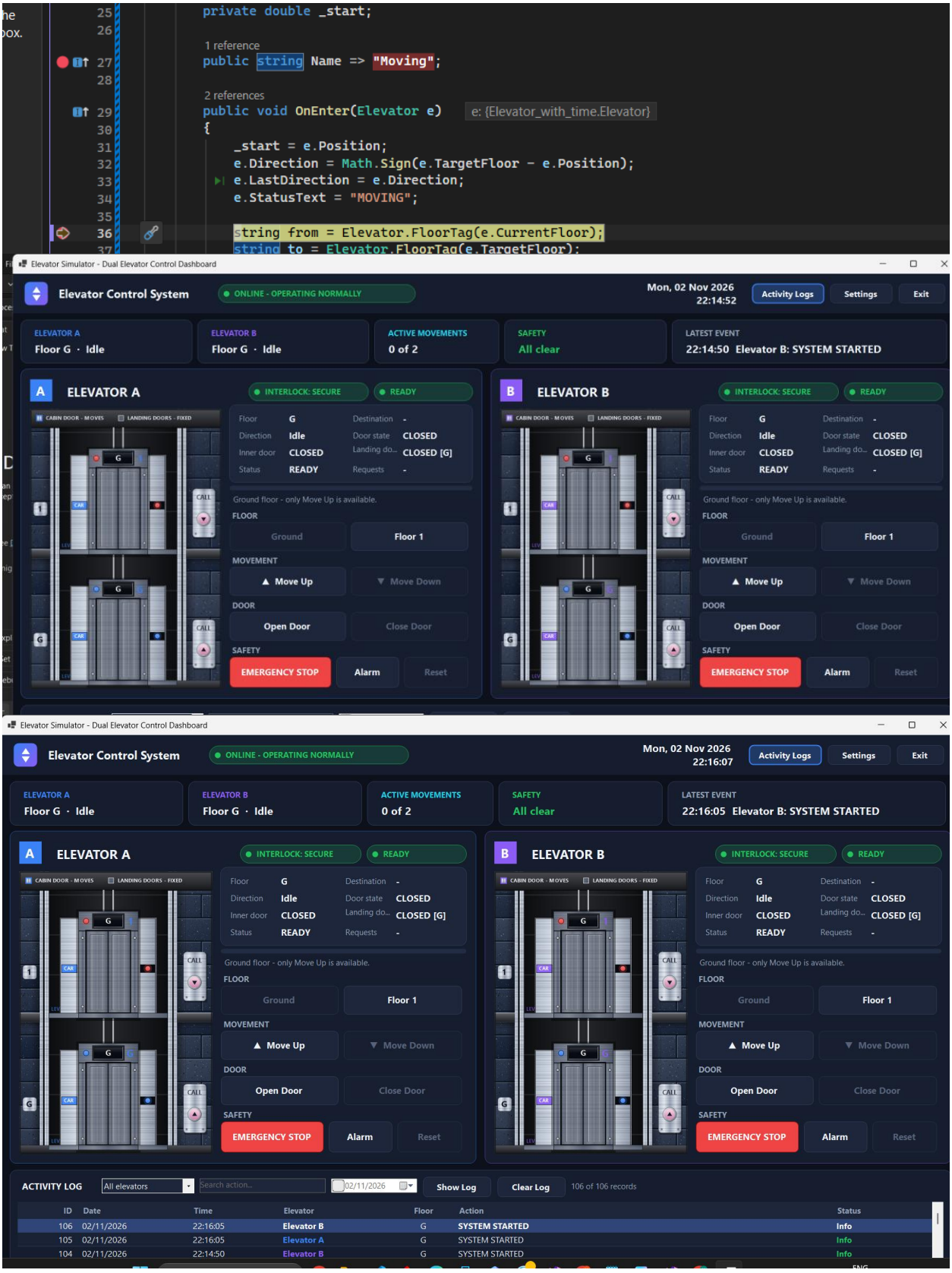
        2 references
        public void Tick(Elevator e) { }
    }
}
```

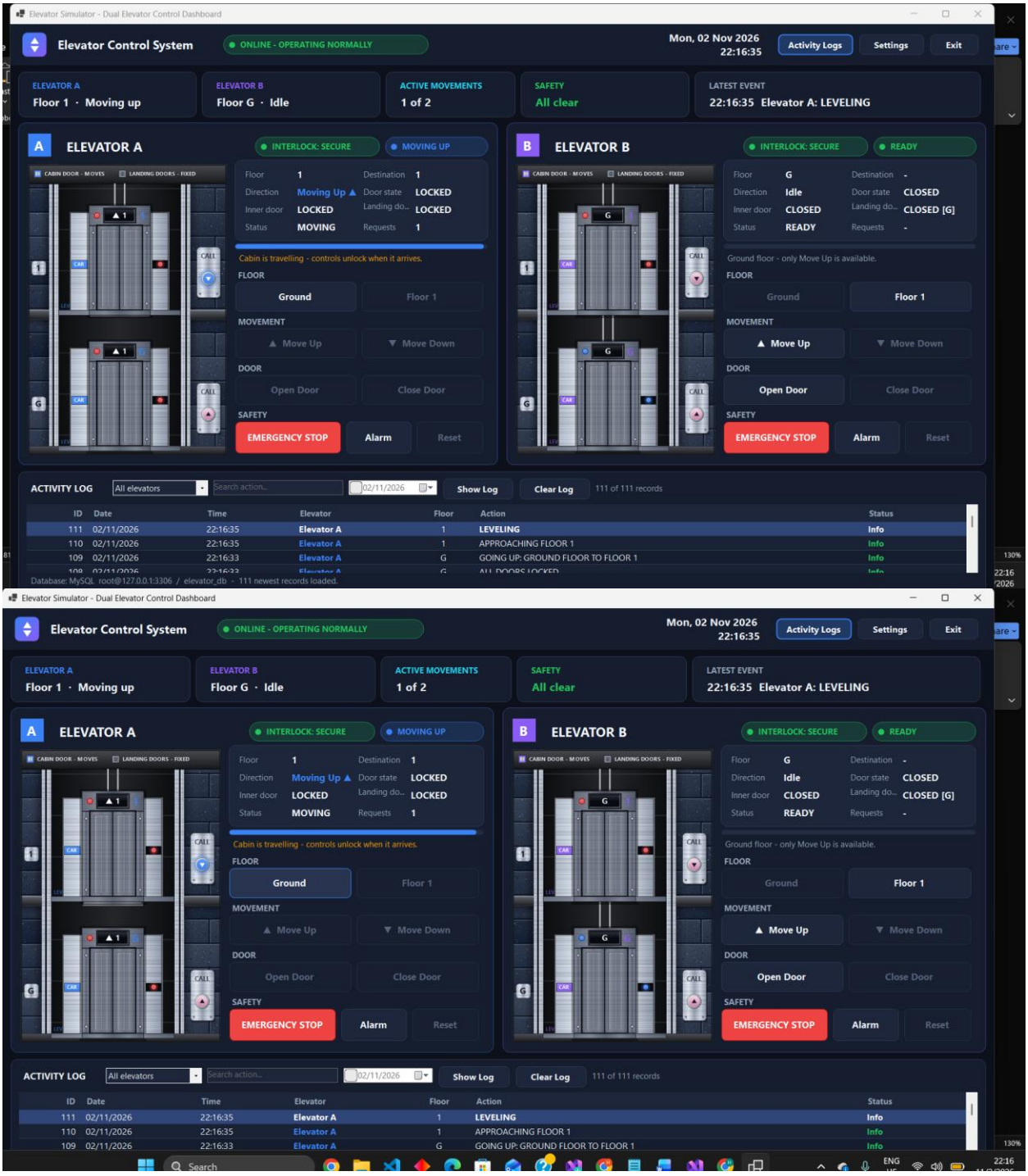

Debugger Agent
I can help you diagnose and fix bugs, analyze exceptions, understand call stacks, and suggest breakpoints.
ACTIVITY LOG
Show Log
Clear Log
No issues found

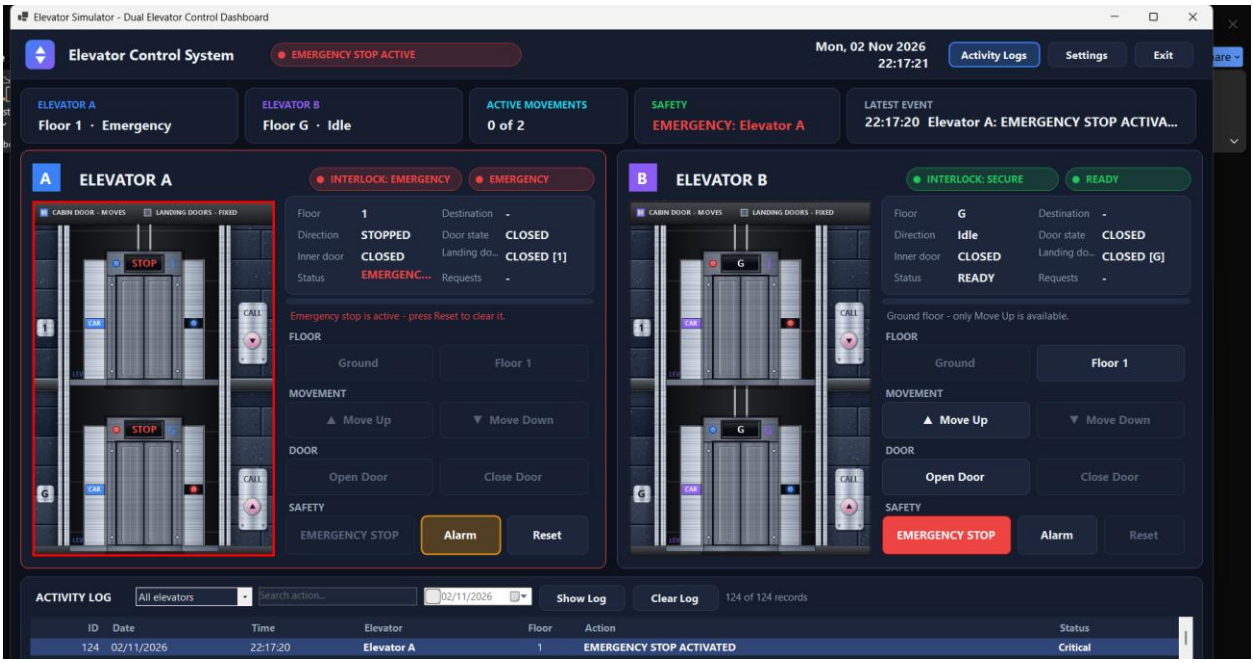


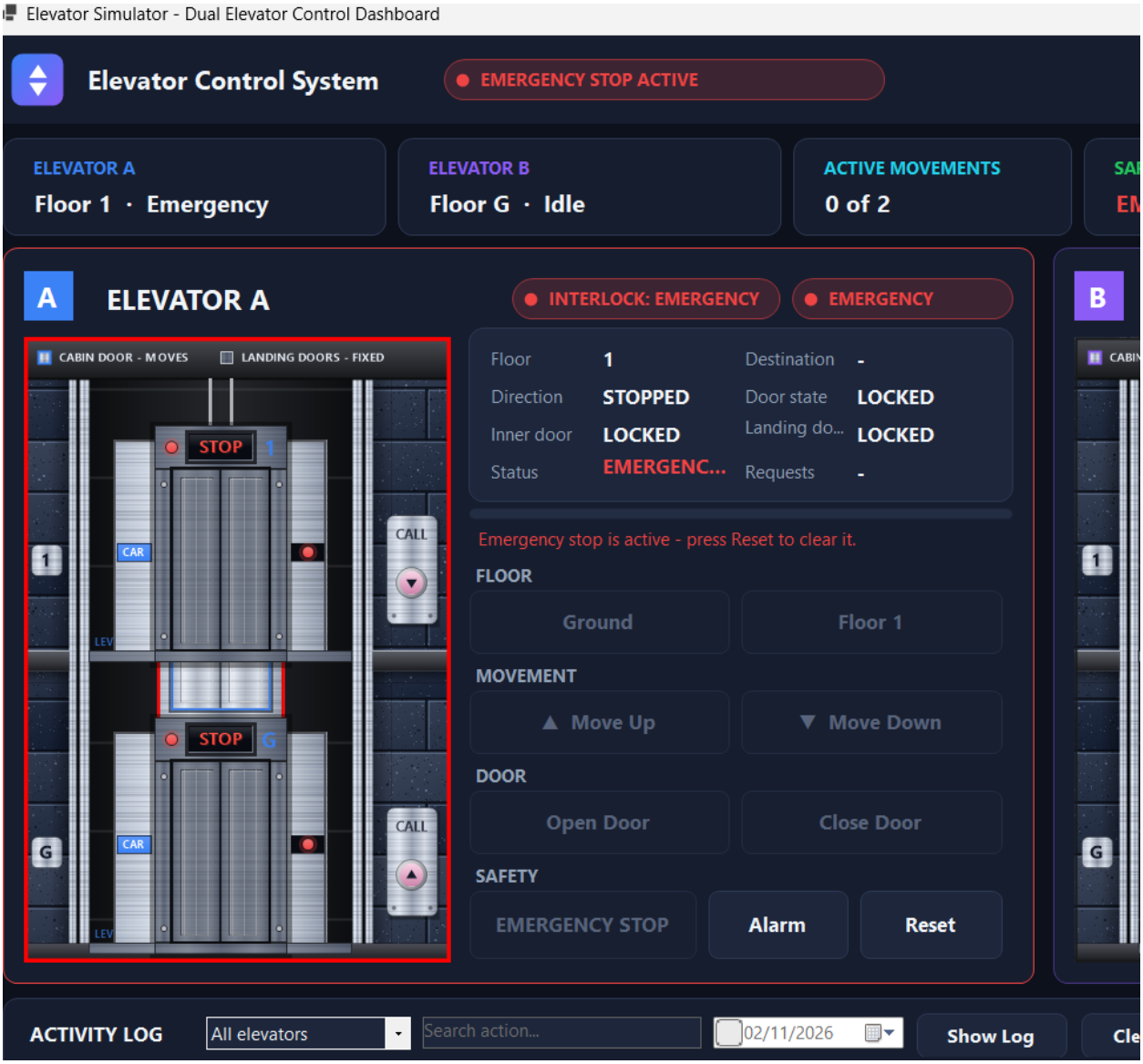

```
124         public void Flush()
125         {
126             while (_queue.TryDequeue(out LogEntry entry))
127             {
128                 try { Insert(entry); } catch { break; }
129             }
130         }
131
132         // ------------------------------------------------------------ reading / clearing (async)
133
41          public void Tick(Elevator e)    e: {Elevator_with_time.Elevator}
42          {
43              // safety interlock: the cabin must never travel with a door that is not closed
44              if (!e.AllDoorsClosed)
45              {
46                  e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
47                  return;
48              }
49
50              double diff = e.TargetFloor - e.Position;
51              double dist = Math.Abs(diff);
52
to add it
to the
toolbox.
2 references
41          public void Tick(Elevator e)    e: {Elevator_with_time.Elevator}
42          {
43              // safety interlock: the cabin must never travel with a door that is not closed
44              if (!e.AllDoorsClosed)
45              {
46                  e.HaltDrive("INTERLOCK FAULT: DOOR NOT CLOSED WHILE MOVING");
47                  return;
48              }
49
50              double diff = e.TargetFloor - e.Position;
51              double dist = Math.Abs(diff);
```


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
22:06:29
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Idle
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
22:06:28 Elevator B: SYSTEM STARTED
ELEVATOR A
INTERLOCK: SECURE
READY
ELEVATOR B
INTERLOCK: SECURE
READY
Floor G
Direction Idle
Inner door CLOSED
Status READY
Destination -
Door state CLOSED
Landing do... CLOSED [G]
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
67 of 67 records
ID Date Time Elevator Floor Action Status
67 02/11/2026 22:06:28 Elevator B G SYSTEM STARTED Info
66 02/11/2026 22:06:28 Elevator A G SYSTEM STARTED Info
65 02/11/2026 22:03:50 Elevator A G GOING UP: GROUND FLOOR TO FLOOR 1 Info
double diff = e.TargetFloor - e.Position;
double dist = Math.Abs(diff);
// speed profile: slow at the start and in front of the target, fast in between
double ramp = Math.Min(Math.Abs(e.Position - _start), dist);
double speed = Elevator.MaxSpeed * Math.Clamp(ramp / Elevator.RampDistance, Elevator.MinSpeedFactor, 1.0);
No issues found
Ln: 54, Ch: 13 SPC LF UTF-8
// announce the phases of the approach once per trip
if (!e.ApproachLogged && dist <= Elevator.ApproachDistance)
false
No issues found
Ln: 66, Ch: 13 OVR SPC
Error List
Entire Solution
0 Errors
0 of 1 Warning
0 of 36 Messages
Build + IntelliSense
Search Error List
Code Description Project File Suppression State
Call Stack Breakpoints Exception Settings Command Window Immediate Window Output Error List
Search
Drag an item onto this text to add it to the toolbox.
namespace Elevator_with_time


private double _start;
1 reference
public string Name => "Moving";
2 references
public void OnEnter(Elevator e)
e: {Elevator_with_time.Elevator}
{
_start = e.Position;
e.Direction = Math.Sign(e.TargetFloor - e.Position);
e.LastDirection = e.Direction;
e.StatusText = "MOVING";
string from = Elevator.FloorTag(e.CurrentFloor);
string to = Elevator.FloorTag(e.TargetFloor);
Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
22:14:52
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Idle
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
22:14:50 Elevator B: SYSTEM STARTED
ELEVATOR A
INTERLOCK: SECURE
READY
ELEVATOR B
Floor G
Direction Idle
Inner door CLOSED
Status READY
Destination -
Door state CLOSED
Landing do... CLOSED [G]
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
22:16:07
Activity Logs
Settings
Exit
ELEVATOR A
Floor G · Idle
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
All clear
LATEST EVENT
22:16:05 Elevator B: SYSTEM STARTED
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
106 of 106 records
ID Date Time Elevator Floor Action Status
106 02/11/2026 22:16:05 Elevator B G SYSTEM STARTED Info
105 02/11/2026 22:16:05 Elevator A G SYSTEM STARTED Info
104 02/11/2026 22:14:50 Elevator B G SYSTEM STARTED Info


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
ONLINE - OPERATING NORMALLY
Mon, 02 Nov 2026
22:16:35
Activity Logs
Settings
Exit
ELEVATOR A
Floor 1 · Moving up
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
1 of 2
SAFETY
All clear
LATEST EVENT
22:16:35 Elevator A: LEVELING
ELEVATOR A
INTERLOCK: SECURE
MOVING UP
Floor 1
Destination 1
Direction Moving Up
Door state LOCKED
Inner door LOCKED
Landing do... LOCKED
Status MOVING
Requests 1
Cabin is travelling - controls unlock when it arrives.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ELEVATOR B
INTERLOCK: SECURE
READY
Floor G
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [G]
Status READY
Requests -
Ground floor - only Move Up is available.
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
111 of 111 records
ID Date Time Elevator Floor Action Status
111 02/11/2026 22:16:35 Elevator A 1 LEVELING Info
110 02/11/2026 22:16:35 Elevator A 1 APPROACHING FLOOR 1 Info
109 02/11/2026 22:16:33 Elevator A G GOING UP: GROUND FLOOR TO FLOOR 1 Info
Database: MySQL root@127.0.0.1:3306 / elevator_db - 111 newest records loaded.


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
EMERGENCY STOP ACTIVE
Mon, 02 Nov 2026
22:17:21
Activity Logs
Settings
Exit
ELEVATOR A
Floor 1 · Emergency
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
SAFETY
EMERGENCY: Elevator A
LATEST EVENT
22:17:20 Elevator A: EMERGENCY STOP ACTIVA...
A
ELEVATOR A
INTERLOCK: EMERGENCY
EMERGENCY
Floor 1
Destination -
Direction STOPPED
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [1]
Status EMERGENC...
Requests -
Emergency stop is active - press Reset to clear it.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ELEVATOR B
INTERLOCK: SECURE
READY
Floor G
Destination -
Direction Idle
Door state CLOSED
Inner door CLOSED
Landing do... CLOSED [G]
Status READY
Requests -
Ground floor - only Move Up is available.
FLOOR
Ground
Floor 1
MOVEMENT
Move Up
Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log
Clear Log
124 of 124 records
ID Date Time Elevator Floor Action Status
124 02/11/2026 22:17:20 Elevator A 1 EMERGENCY STOP ACTIVATED Critical


Elevator Simulator - Dual Elevator Control Dashboard
Elevator Control System
EMERGENCY STOP ACTIVE
ELEVATOR A
Floor 1 · Emergency
ELEVATOR B
Floor G · Idle
ACTIVE MOVEMENTS
0 of 2
A
ELEVATOR A
INTERLOCK: EMERGENCY
EMERGENCY
CABIN DOOR - MOVES
LANDING DOORS - FIXED
STOP
CAR
CALL
STOP
CAR
CALL
Floor 1
Destination -
Direction STOPPED
Door state LOCKED
Inner door LOCKED
Landing do... LOCKED
Status EMERGENC...
Requests -
Emergency stop is active - press Reset to clear it.
FLOOR
Ground
Floor 1
MOVEMENT
▲ Move Up
▼ Move Down
DOOR
Open Door
Close Door
SAFETY
EMERGENCY STOP
Alarm
Reset
B
ACTIVITY LOG
All elevators
Search action...
02/11/2026
Show Log